«Взаимно»

Платформа обратного подбора ИТ-специалистов
Сопроводительная документация решения

Хакатон ЛЦТ-2026, спецтрек ФСП
Сборка документации: 11 октября 2026 (по коммиту de809b3)
Все числа параметров берутся из `config/*.yaml`; результаты валидации получены на синтетической популяции

Состав:

- 1. Архитектура решения
- 2. Механика тестирования
- 3. Механика подбора и логика категоризации
- 4. Процедура валидации
- 5. Схемы интеграции с ФСП / ФСП ID
- 6. Описание API
- 7. Перечень библиотек с версиями
- 8. Инструкция по сборке и запуску
- Дополнительно: ATS, доверие и безопасность, 152-ФЗ, LLM, нагрузка

Содержание

НАВИГАЦИЯ

Карта документации 6
1. Памятка для жюри 6
2. Карта документации 7
3. Дополнительные материалы 8
4. PDF-версии 8
5. Соглашения 9
6. Откуда взяты утверждения 9

ПЕРЕЧЕНЬ ОРГАНИЗАТОРОВ

Требования задачи и где они реализованы 10
0. Кейс на одном экране 10
1. Обратный подбор и контакт 11
2. Категория и тест 13
3. Кабинет кандидата 16
4. Кабинет работодателя 17
5. ФСП ID, достижения, кандидат без истории 17
6. Зарплата, правила видимости, безопасность 18
7. Валидация: собственная и на ваших данных 19
8. Документы, API, запуск, нагрузка 20
9. Ответы организаторов: что учтено 20
10. Сверх кейса 21
11. Чего нет и что частично 25

1. Архитектура 26
1. Принципы 26
2. Функциональная архитектура 26
3. Контекст и контейнеры (C4) 28
4. Модули монолита и их границы 29
5. Данные (ERD) 31
6. Сквозные сценарии (sequence) 33
7. Машины состояний 37
8. Сквозные механизмы 38
9. Статус реализации 39

2. Механика тестирования 41
1. Идея в одном абзаце 41
2. Опрос и классификатор специализации 41
3. Банк: семейства и одноразовые клоны 42
4. Модель ответа и оценка уровня 43
5. Калибровка сложности 44
6. Адаптивный тест (CAT) 45
7. Правила попыток и смены грейда 46
8. Устойчивость к распространению заданий 47
9. Антиподсказчик: «двойник в буфере» и «подтверждение за 5 минут» 48
10. Уровень доверия 50
11. Регулярные короткие задания 50
12. Вакансия → проверочный набор 51
13. LLM пишет генератор, а не задание 51
14. Честные ограничения 52

15. Дизайнеры (UI/UX): по опросу, тест в дорожной карте 52
16. Качество заданий: DIF, дрейф сложности, экспозиция 54

3. Механика подбора и категоризации 56
1. Категория = специализация × грейд 56
2. Конвейер 57
3. Разбор брифа 58
4. Рекомендация категорий и охват вилки 58
5. Формула Score 58
Навыки, наблюдаемые через проверочный набор 60
6. Объяснения: 3 «за» и 1 «против» 61
7. Версии подборки без потери 61
8. Приватность до раскрытия 62
9. Сигнал интереса 63
10. Кандидаты без ФСП 68
11. Банк кандидатов без брифа 69
12. Ограничения 70
13. Открытые данные рынка: что взяли, как, лицензия, что не входит в Score 70

4. Валидация — протокол 72
1. Данные 72
2. Метрики, пороги, бейзлайны 74
3. Что не проверяется этим прогоном (честные ограничения) 76
4. Уточнения процедуры после дымового прогона (пороги не менялись) 77
5. Уточнения после независимого ревью харнесса (после полного прогона; пороги не менялись) 78
6. Добавление по ответам организаторов: test-retest и разделяющая способность по специализациям 78
7. Постфактум-пересмотр по итогам прогона (пороги плана не менялись) 79
8. Расширение отложенной выборки (предрегистрация) 80

4. Валидация — результаты 84
Итоги 84
1. Данные 85
2. Тестирование 86
3. Категоризация по специализации (V-CAT) 96
4. Подбор 97
5. Набор пар для проверки жюри 104
6. Честные ограничения 105
7. Набор жюри: как проверить вручную на своих профилях 106
8. Качество заданий: DIF, дрейф, экспозиция 107
10. Пилот с ИИ-испытуемыми 109
Как воспроизвести 111

5. Интеграция с ФСП и ФСП ID 112
1. Принципы 112
2. Схема интеграции 112
3. ФСП ID (OIDC) 113
4. Предлагаемый контракт API реестра 114
5. Расчёт VSS 118
6. Учёт соревнований и мероприятий 119
7. Согласия, скрытие результатов, приватность 120
8. Несовершеннолетние 121
9. Пошаговый путь к реальной интеграции 121
10. Что нужно согласовать с ФСП 122
11. Панель ФСП: кадровый резерв 122

6. Описание API 125
1. Документация и схема 125
2. Аутентификация 125
3. Формат ошибок и коды 127
4. Ограничение частоты запросов 128
5. Разделы и эндпоинты 129
6. Сквозные соглашения 140
7. Примеры (сценарии curl) 140
8. Статус и то, чего ещё нет 141

7. Библиотеки и версии 142
1. Бэкенд (Python) 142
2. Фронтенд (JavaScript) 145
3. Образы и системные компоненты 146
4. Шрифты 147
5. Иллюстрации и данные 148
6. Замечания по лицензиям 148
7. Как обновить список 149

8. Сборка и запуск 150
1. Требования и ресурсы 150
2. Локальный запуск 150
3. Переменные окружения 153
4. Сборка образов 155
5. Развёртывание на VPS в РФ с HTTPS через Caddy 155
6. Обслуживание 158
7. Диагностика 159
Схема боевого развёртывания 160
8. В разработке 160

ДОПОЛНИТЕЛЬНО

Интеграция с ATS 161
1. Что реально, что эмулировано 161
2. Концепция 161
3. Безопасность 163
4. Pull-API /api/ats/v1 164
5. Адаптер «Хантфлоу» и соответствие полей 164
6. Настройки 165

Доверие и безопасность платформы 167
1. Принципы 167
2. Метка «Проверенная компания» (реализовано) 167
3. Жалобы кандидатов и модерация (реализовано) 168
4. Что уже защищало платформу (используется как есть) 168
5. Автосигналы для модератора (реализовано) 168
6. Что реализовано и что в плане 169
7. Ограничения (честно) 169

Безопасность и персональные данные (152-ФЗ) 170
1. Персональные данные и 152-ФЗ 170
2. Роли и разграничение доступа 173
3. Пароли и токены 174
4. Ограничение частоты запросов 174
5. ПДн кандидата до раскрытия 175
6. LLM-шлюз с маскированием 176

7. Секреты и инфраструктура 176
8. Целостность теста и ключи заданий 177
9. Модель угроз (кратко) 177
10. Известные ограничения и «в разработке» 178

Использование LLM 179
1. Где LLM используется 179
2. Где LLM запрещена и почему 179
3. Какие модели 180
4. Офлайн-использование: «LLM пишет генератор, а не задание» 181
5. Политика ПДн 181
6. Как включить 182
7. Стоимость и ресурсы 182
8. Как проверить 183
9. Результаты на отложенной выборке брифов (GigaChat, 09.10.2026) 183

Нагрузочное тестирование 185
Метод 185
Окружение замеров 185
Результаты (итоговая конфигурация, 4 процесса) 186
Что нашли и исправили 187
Воронка приглашений: пакетная загрузка и постраничность 187
Выводы 188
Что не проверено 188

Цифровой паспорт навыков 189
1. Что используется (стандарты) 189
2. Какие паспорта бывают 189
3. Персональные данные 190
4. Что делает кандидат 190
5. Проверка 190
6. Что настоящее, а что демонстрация 191
7. Настройка и ключ 192
8. Ограничения 192

Инженерные практики 193
Что гоняет CI 193
Тесты 193
Сквозные тесты (Playwright) 194
Типы 194
Статический анализ и стиль 194
Безопасность 195
Детерминизм и конфигурация 195
Миграции 196

Доступность и адаптивность 197
Лицензия инструмента 197
Что проверяется 197
Результаты: было → стало (serious/critical, узлов) 197
Что исправлено 198
Остаётся 198
Адаптивность кабинета работодателя 198

НАЧАЛО · НАВИГАЦИЯ

Документация «Взаимно»

«Взаимно» — платформа обратного подбора ИТ-специалистов с верифицированным профилем ФСП. Кандидат проходит опрос и адаптивный тест на одноразовых заданиях и получает категорию «специализация × грейд»; работодатель описывает потребность текстом с обязательной зарплатной вилкой и получает слепую объяснимую подборку; контакты раскрываются только после взаимного «да» и отдельного согласия кандидата. Достижения ФСП усиливают профиль, но не открывают дверь: кандидат без истории ФСП проходит тот же путь и не штрафуется.

Содержание: 1. Памятка для жюри · 2. Карта документации · 3. Дополнительные материалы · 4. PDF-версии · 5. Соглашения · 6. Откуда взяты утверждения

1. Памятка для жюри

Что читать в первую очередь

0. **Кейс → решение → доказательство** — REQUIREMENTS.md: для каждого требования кейса и каждого ответа организаторов — проблема, решение, живой пример и доказательство, затем «Сверх кейса» и честный список пробелов (15 минут).
1. **Принципы и устройство** — ARCHITECTURE.md, разделы 1 «Принципы» и 2 «Функциональная архитектура» (10 минут).
2. **Как определяется категория и почему ей можно доверять** — TESTING_MECHANISM.md, разделы 1, 8 и 14: идея, устойчивость к распространению заданий и честные пределы.
3. **Как строится подбор и что значит Score** — MATCHING.md, разделы 1–2 и 5: категория, формула `Score = 100 · G · Cov · Cond · Σ(w·x)`, объяснения «3 за, 1 против».
4. **Чем подтверждены утверждения** — VALIDATION.md: таблица итогов в начале (36 из 41 исходных пороговых пунктов выполнены, с двумя добавленными постфактум пунктами — 38 из 43; невыполненные перечислены там же с причинами), протокол с порогами, записанный до прогона, — VALIDATION_PLAN.md.
5. **Что предлагается ФСП** — FSP_INTEGRATION.md: схема интеграции с ФСП ID и реестром, контракт API реестра, расчёт VSS, пошаговый переход на реальные системы.
6. **Переносимые подписанные учётные данные** — CREDENTIALS.md: цифровой паспорт навыков (W3C VC 2.0, Open Badges 3.0, Ed25519, did:web), проверка по QR и по файлу, отзыв, путь к настоящему издателю ФСП.

Как проверить за 10 минут

Минуты	Действие	Где
0–3	Поднять стенд: `docker compose up -d --build`, затем `make demo` (на Windows без make — команды в DEPLOY.md п. 2)	DEPLOY.md
3–5	Открыть http://localhost/jury: одна кнопка создаёт одноразового кандидата; экспресс-тест (до 8 заданий) на том же движке и банке	README репозитория, «Проверьте сами за 5 минут»
5–7	«Попробовать списать»: убедиться, что скопированное в ИИ условие подменено двойником, а на экране истинное	TESTING_MECHANISM.md п. 9
7–9	Войти как демо-работодатель, описать потребность текстом, открыть подборку и панель доказательств (разбор `G · Cov · Cond · Σ`)	MATCHING.md п. 5–6
9–10	Свои данные: страница `/lab/check` (вход жюри → «Лаборатория») — метрики нашего набора пар, своя вакансия против банка, загрузка своего набора; через API — `/api/admin/import`; либо `make validate`	VALIDATION.md п. 7; API.md «Администрирование»

Swagger UI с интерактивной проверкой всех эндпоинтов: http://localhost:8000/api/docs (ReDoc: `/api/redoc`, схема: `/api/openapi.json`). Тесты бэкенда: `cd backend && uv run pytest -q` (более 2400 тестов, включая контрольные примеры из спецификаций; CI — ENGINEERING.md).

Главные числа (источник — сами документы)

- **Тестирование** (VALIDATION.md): верное решение у заявленного порога 0,86; грейд с точностью ±1 — 1,00; средняя длина основной части 12,8 заданий; точный грейд 0,748 при цели 0,75 — не выполнено, и мы это показываем.
- **Подбор** (VALIDATION.md): P@10 по оракулу 0,61 (цель ≥ 0,60); на всех 40 брифах значимо выше базовой сортировки по уровню; отсутствие ФСП никогда не «против» (1,00).
- **Пилот с ИИ-агентами в роли кандидатов** (VALIDATION.md, раздел 10): повтор теста у 12 честных персон — точное совпадение полосы 0,83, ±1 — 1,00; «списывающие» персоны (знают на Junior, заявляют Middle) в 4 прогонах получили «не подтверждён» (выборка мала, один из разделов пилота — пост-хок).
- **Нагрузка** (LOAD_TEST.md): 100 одновременных пользователей — 0 сбоев, p95 запроса 170 мс после оптимизации воронки приглашений (до неё на 100 пользователях было 480 мс).
- **Специализации с тестом**: Backend (Python), Frontend (JS/TS), QA, DevOps; остальные направления — только опрос (`survey_only`).

Численные результаты валидации — симуляция на синтетической популяции с известной истиной: она проверяет процедуру, а не реальных людей. Ограничения сформулированы в каждом разделе.

2. Карта документации

Перечень организаторов к сопроводительной документации и соответствующие файлы. PDF-версии лежат в каталоге pdf/ (см. раздел 4).

№	Требование	Документ	Что внутри	PDF
1	Архитектура	ARCHITECTURE.md	Функциональная и компонентная архитектура, модули монолита и границы, контейнеры, диаграммы C4, ERD, sequence-диаграммы четырёх сценариев, машины состояний, статус реализации	PDF
2	Механика тестирования	TESTING_MECHANISM.md	Семейства заданий и одноразовые клоны, модель IRT, адаптивный выбор заданий, калибровка, смена грейда, устойчивость к утечкам, «Антиподсказчик», регулярные задания	PDF
3	Механика подбора и логика категоризации	MATCHING.md	Категория = специализация × грейд, разбор брифа, формула Score с контрольными примерами, объяснения, версии подборки, приватность до раскрытия, кандидаты без ФСП	PDF
4	Процедура валидации и её результаты	VALIDATION_PLAN.md, VALIDATION.md	Протокол: данные, метрики, пороги и бейзлайны, зафиксированные до прогона; результаты `make validate`, честные ограничения	протокол, результаты
5	Схемы предполагаемой интеграции с ФСП / ФСП ID	FSP_INTEGRATION.md	Схема интеграции с ФСП ID (OIDC) и реестром достижений, предлагаемый контракт API реестра, VSS, согласия, несовершеннолетние, переход на реальные системы	PDF
6	Описание API	API.md	Аутентификация, формат ошибок, ограничение частоты, таблицы всех 242 эндпоинтов по разделам, примеры; ссылки на Swagger UI и ReDoc, получение `openapi.json`	PDF
7	Перечень библиотек с версиями	THIRD_PARTY.md	Python- и JavaScript-пакеты (версии из `uv.lock` и `package-lock.json`), образы, шрифты, лицензии	PDF

№	Требование	Документ	Что внутри	PDF
8	Инструкция по сборке и запуску	DEPLOY.md	Локальный запуск, переменные окружения, образы, боевая схема с HTTPS (Caddy), обслуживание, диагностика; короткий путь — README репозитория	PDF

3. Дополнительные материалы

Тема	Документ	О чём	PDF
ATS	ATS_INTEGRATION.md	Интеграция с системой найма работодателя: pull-API по ключу, исходящие вебхуки с подписью, обратная синхронизация статусов, адаптер «Хантфлоу»	PDF
Доверие и безопасность платформы	TRUST_AND_SAFETY.md	Метка «Проверенная компания», жалобы и модерация, автосигналы, несовершеннолетние	PDF
Безопасность и 152-ФЗ	SECURITY.md	Раздельные согласия и журнал, роли и защита от IDOR, argon2id и токены, rate limit, ПДн до раскрытия, LLM-шлюз с маскированием, известные ограничения	PDF
LLM	LLM.md	Где LLM применяется (разбор брифа и резюме, только добавление с цитатой), где запрещена и почему, провайдеры, ПДн, включение, стоимость	PDF
Нагрузка	LOAD_TEST.md	Метод, результаты на 50 и 100 одновременных пользователей, узкие места и выводы	PDF
Цифровой паспорт навыков	CREDENTIALS.md	Подписанный Verifiable Credential (Ed25519, did:web), QR и проверка по файлу, отзыв, путь к настоящему издателю ФСП	PDF
Инженерия	ENGINEERING.md	CI GitHub Actions, тесты и e2e на Playwright, `mypy --strict` на ядрах, матрица безопасности, миграции	PDF
Доступность	ACCESSIBILITY.md	axe-core (WCAG 2.1 A/AA): было → стало, адаптив	PDF
Кейс → решение → доказательство	REQUIREMENTS.md	Для каждого требования кейса и ответа организаторов: проблема, решение, живой пример, тест или число; «Сверх кейса»; честный список пробелов	PDF

Журнал доработок 9 октября (что исправлено, зачем и как проверено) — WORKLOG.md; в PDF он не входит. Рядом с кодом лежат и другие описания: ../README.md — быстрый старт, демо-аккаунты и сценарий проверки; ../services/fsp-mock/README.md — мок реестра ФСП; ../deploy/keycloak/README.md — realm `fsp` (ФСП ID); `../config/*.yaml` — все веса, пороги, квоты и справочники (в коде магических чисел нет).

4. PDF-версии

Каталог pdf/ содержит:

- Взаимно — документация.pdf — сводный документ: титульный лист, содержание с номерами страниц и закладками, все разделы в порядке перечня организаторов, затем дополнительные материалы; диаграммы отрисованы;
- по одному PDF на каждый документ (номера файлов совпадают с номерами пунктов перечня; 14–16 — паспорт навыков, инженерия, доступность; `00.1` — требования задачи);
- `pdf/diagrams/*.svg` — диаграммы Mermaid (C4, ERD, sequence, состояния) в векторном виде.

Пересборка: `make docs-pdf` (или `python docs/build_pdf.py`). Нужны Python с пакетами `markdown-it-py` и `pypdf`, Node.js, установленные Chrome или Edge и однократно `cd docs/pdf-tools && npm install` (Mermaid и Playwright). Сборка проверяет внутренние ссылки и якоря и падает на битой ссылке или нерисующейся диаграмме.

5. Соглашения

- **«В разработке»** — функция описана, но в коде ещё не построена или построена частично. Таких мест немного: «фабрика семейств» (LLM пишет черновик генератора офлайн), эмбеддинги и колонки `vector`, ночное обновление и вебхук реестра ФСП, общий rate limit для нескольких воркеров. Полный список — ARCHITECTURE.md п. 9 и разделы «Ограничения» документов.
- Диаграммы записаны в Mermaid (C4, ERD, sequence, state) и отображаются на GitHub, GitLab и в большинстве редакторов Markdown; в PDF они уже отрисованы.
- Тексты интерфейса и сообщения API — на русском; идентификаторы, поля API и переменные окружения — на английском (`VZ_*`).
- Все числа параметров берутся из `config/*.yaml`; если число в документе расходится с конфигом, верно значение конфига.
- Данные демо вымышленные: компании, люди, спортсмены и номера ЕКП. Реальных брендов и ФИО нет.
- Валидация идёт на реальном банке заданий и тех же движках, что в проде (VALIDATION.md, VALIDATION_PLAN.md); раздела «люди» в ней нет.

6. Откуда взяты утверждения

Документы написаны по коду и конфигурации репозитория (`backend/app`, `frontend/src`, `services/fsp-mock`, `deploy/keycloak`, `config/*.yaml`, `docker-compose*.yml`, `backend/pyproject.toml`, `backend/uv.lock`, `frontend/package-lock.json`), а списки эндпоинтов и версий библиотек сформированы автоматически из OpenAPI приложения и lock-файлов. Состояние на 09.10.2026 (ветка `claude/gallant-feynman-bwx4u3`). Проверка: `cd backend && uv run pytest -q` (тесты), `uv run python -m eval.validate` (валидация), `uv run python -c "import json; from app.main import app; print(json.dumps(app.openapi(), ensure_ascii=False))"` (схема API).

Кейс → решение → доказательство

Этот документ отвечает на один вопрос: **что требовалось от решения, что мы для этого сделали и чем это можно проверить своими руками**. Формулировки требований кейса и ответов организаторов пересказаны. Для каждого пункта дано четыре вещи: проблема простыми словами, наше решение, **конкретный пример** (экран, запрос с ответом, строка конфигурации или шаг демо-сценария) и доказательство (файл теста, число из валидации, результат пилота).

Как повторить примеры. Поднимите стенд (README, DEPLOY.md), выполните `make demo`, откройте `/jury` — регистрация не нужна: там одноразовый кандидат, вход в демо-компанию «Северный код» и в лабораторию. Логины демо-аккаунтов: `make creds`. Все примеры ниже проверены 9 октября 2026 на этом коде в режиме `VZ_DEMO_MODE=true` на демо-мире (около 300 синтетических кандидатов). Номера вида «К-2118» — псевдонимы: они зависят от секрета стенда, у вас будут другие, а структура ответа та же.

Статусы. «выполнено» — работает и проверено тестами; «частично» — работает, но есть оговорка, она написана тут же; «не сделано» — честно сказано. Сводка пробелов — в разделе 11.

Что такое валидация в этом документе. Все числа «качества» — симуляция на синтетической популяции с известной истиной (VALIDATION.md) плюс пилот с ИИ-агентами в роли кандидатов. Живых людей в проверках не было, и мы это нигде не скрываем.

Содержание: 0. Кейс на одном экране · 1. Обратный подбор и контакт · 2. Категория и тест · 3. Кабинет кандидата · 4. Кабинет работодателя · 5. ФСП · 6. Зарплата, правила видимости, безопасность · 7. Валидация на ваших данных · 8. Документы, API, запуск · 9. Ответы организаторов · 10. Сверх кейса · 11. Чего нет

0. Кейс на одном экране

Проблема кейса	Наш ответ	Главное доказательство	Подробно
Резюме не сопоставимы и им нельзя верить	Уровень задаёт адаптивный тест, а не текст; резюме даёт только серые чипы «заявлено»	Точный грейд по стажу — 0,54, по тесту — 0,75; накрутка навыков в самоописании сдвигает нас на 0,0 позиций, поиск по резюме — на 518	2.1, 1.6
Единый набор заданий утекает, «ИИ под каждого» ненадёжен	Семейства заданий и одноразовые клоны, ключ считается исполнением кода	Утечка 20 кандидатов: ложный Middle 0,10 против 1,00 у фиксированной формы	2.2
Задания решают чат-боты	«Двойник в буфере» подменяет скопированное условие; живая сверка после «да»	Пилот с ИИ-агентами: «списывающие» не получили ложный Middle ни в одном из 4 прогонов (выборка мала)	2.3, 10.1
Работодатель тонет в резюме	Описание потребности текстом с обязательной вилкой → слепая объяснимая подборка	P@10 0,61 против 0,13 у поиска по резюме на тех же брифах	1.1, 1.2
Непонятно, почему именно этот кандидат	Для каждого: три довода «за» и один «против», каждый со ссылкой на доказательство; сумма вкладов равна Score	Σ вкладов = Score в 100 % карточек; довод ссылается на существующее доказательство в 100 %	1.2
Сценарий контакта: кто, когда, на каких условиях	Приглашение с вилкой → «Да / Да, если… / Нет» → контакты только после взаимного «да» и отдельного согласия	`test_contact_privacy.py`, `test_api_contact_lifecycle.py`; в слепой карточке нет ни одного ПДн-ключа	1.3, 1.4
Участие в ФСП не должно быть входным билетом	Достижения ФСП двигают ранг внутри категории; нет истории — F = 0, это никогда не «против»	«Отсутствие ФСП не в “против”» 1,00; экспозиция без ФСП 0,91 при допуске 0,80–1,25	5.2

Проблема кейса	Наш ответ	Главное доказательство	Подробно
Нужна честная проверка на своих данных	Наш набор пар, свой набор, своя вакансия — на странице `/lab/check` и в API; 100 брифов, протокол записан до прогона	38 из 43 порогов выполнены, 5 нет — перечислены с причинами	7
Сдача: документы, API, запуск, нагрузка	19 документов, OpenAPI на 242 операции, Docker Compose, нагрузочный тест	100 пользователей: 24 739 запросов, 0 сбоев, p95 170 мс	8

1. Обратный подбор и контакт

Это ядро кейса: инициатор — работодатель, но решение принимают обе стороны.

1.1. Работодатель описывает потребность текстом, а не листает резюме — выполнено

- **Проблема.** Классический найм — это поток резюме. Нужно, чтобы работодатель описал, кто нужен, а система нашла людей сама, и чтобы без зарплатной вилки ничего не отправлялось.
- **Решение.** Экран `/employer/needs/new`: свободный текст, живая карточка разбора с цитатами из текста, обязательная вилка, необязательный блок «о команде». Разбор — правила и словари; LLM по умолчанию выключен и в ранжировании не участвует.
- **Пример (запрос и ответ).** `POST /api/needs/parse` с текстом «Бэкенд в команду платёжного шлюза. Python, FastAPI, PostgreSQL, опыт от 2 лет. Kafka и Docker будут плюсом. Гибрид, Москва. 220–280 тыс. на руки» возвращает:
  - вилка 220 000–280 000, «на руки», с цитатами «220–280 тыс.» и «на руки»;
  - Python, FastAPI, PostgreSQL — `must`; Kafka и Docker — `nice` с причиной `marker` и цитатой «будут плюсом»;
  - специализация `backend_python` с вероятностью 0,93; грейд Middle 0,69, Senior 0,31 (то есть система сама показывает, что не уверена между ними);
  - `"llm_used": false`.

  Тот же текст без вилки: `salary_check.status = "missing"`, сообщение «Укажите вилку: от и до, в рублях в месяц», а `POST /api/needs` отвечает `422 salary_missing`.
- **Доказательство.** `tests/test_matching_brief.py`, `tests/test_api_needs_rules.py`; разбор брифа на 20 test-брифах: специализация top-1 0,90, грейд точно 0,95, F1 навыков 0,98 (VALIDATION.md п. 4.3); на 60 новых брифах: специализация 0,97, грейд 0,97 (п. 4.1a). Разбор написан правилами, поэтому детерминирован.

1.2. Слепая объяснимая подборка — выполнено

- **Проблема.** «Чёрный ящик» ранжирования нельзя ни проверить, ни оспорить; кроме того, ФИО и бренды прежних работодателей запускают оценку по ярлыку.
- **Решение.** `Score = 100 · G · Cov · Cond · Σ(w·x)` с пятью компонентами: тест T, стек K, ФСП F, свежесть A, условия U (веса в `config/scoring.yaml`: 0,40 / 0,25 / 0,15 / 0,10 / 0,10). Карточка слепая: категория, полоса теста, модули ✓/✓✓, обобщённое ФСП, обезличенный опыт; имя, город, вуз и работодатели скрыты. Обоснование — MATCHING.md п. 8.1.
- **Пример (демо-сценарий).** Войдите через `/jury` → «Работодатель», откройте потребность «Backend Middle в платёжный шлюз» (`/employer/needs/:id`). У первого в подборке из 48 кандидатов `GET /api/needs/{id}/selection` отдаёт:
  - Score 69,3 = T 39,6 + K 19,7 + F 0 + A 0 + U 10,0 (сумма вкладов равна Score);
  - «за»: «Тест: верхние 10 % категории Backend (Python) · Middle, тест сегодня»; «Стек: подтверждено 3 из 3 обязательных — Python, FastAPI, PostgreSQL»; «Ожидания в вашей вилке ✓; гибрид подходит»;

- «против»: «Навык Kafka не подтверждён — проверьте на интервью» с ссылкой на навык брифа;
- `fsp_line`: «Достижения ФСП не привязаны — оценка по тесту и задачам». Первое место занято человеком без ФСП, и это нигде не в «против».

- **Пример (контрольные числа).** Три контрольных набора компонентов в тесте дают Score 83,2, 78,3 и 63,3. Второй (78,3) — без ФСП совсем (F = 0): он обгоняет третий, у которого сильное ФСП, но слабый тест (`test_control_order_strong_test_beats_strong_fsp`).
- **Доказательство.** `tests/test_matching_scoring.py` (контрольные примеры, детерминизм, `test_fake_declared_skills_shift_rank_by_zero`), `tests/test_matching_explain.py`, `tests/test_api_needs_privacy.py`. Валидация объяснений на 1000 карточках: Σ вкладов = Score — 100 %, довод ссылается на существующее доказательство — 100 %, без доказательства довода №1 Score падает — 100 % (VALIDATION.md п. 4.6). Качество выдачи — в разделе 7.

1.3. Приглашение с зарплатой и ответ «Да / Да, если… / Нет» — выполнено

- **Проблема.** Нужен сценарий выхода на контакт, где работодатель инициатор, но без спама и без игры в тёмную по деньгам.
- **Решение.** `POST /api/invitations`: описание, вилка в рублях, способ связи, срок ответа 3/5/7 дней; кандидат отвечает «Да», «Да, если…» (встречное условие: зарплата, формат, дата выхода) или «Нет». Кандидат видит приглашение с вилкой и компанией до ответа.
- **Пример (ошибки отправки).** Без вилки: `400 salary_required`, «Укажите вилку “от–до”: без неё приглашение не отправляется». С вилкой 100 000–400 000: `409 contact_rejected`, «Вилка 100 000–400 000 ₽ шире 2,0× — сузьте её: кандидат должен видеть реальные условия» (мягкое предупреждение с 1,6×, блокировка с 2,0×: `config/contact.yaml → salary`). Там же второе правило: `card_not_opened` — пригласить можно только того, чью карточку вы открыли или закрепили в шортлисте.
- **Пример («Да, если…»).** В демо-мире у «Северного кода» есть приглашение со встречным условием: `counter.salary_from = 325000` при вилке 220 000–280 000, `summary`: «от 325 000 ₽ (ваша вилка 220 000–280 000 ₽)», `within_range: false`, `auto_disclose: true`, доступные действия `counter/accept`, `counter/decline`, `withdraw`, срок ответа на встречное — 3 дня.
- **Доказательство.** `tests/test_contact_invitation.py`, `tests/test_api_contact_lifecycle.py`, `tests/test_api_contact_deadlines.py`, `tests/test_contact_quotas.py`. Квоты (10 в пакете, 20 активных на бриф, 5 новых в неделю кандидату) подобраны симуляцией рынка, раздел 10.4.

1.4. Контакты открываются только после взаимного «да» — выполнено

- **Проблема.** Контакты и персональные данные нельзя отдавать до согласия человека; при этом после «да» стороны должны сразу получить возможность связаться.
- **Решение.** Два типа карточки: слепая (`CandidateBlind`) и раскрытая (`CandidateDisclosed`), правила видимости полей — `config/contact.yaml → visibility`. Раскрытие требует «да» кандидата и отдельного согласия (по умолчанию галочка стоит, снять можно; отозвать — в любой момент).
- **Пример.** `GET /api/needs/{id}/candidates/{ref}` для того же кандидата: до «да» `"view": "blind"`, после принятого приглашения `"view": "disclosed"`, и в ответе добавляется блок `disclosed` с полями `full_name`, `phone`, `email`, `telegram`, `city`, `current_employer`, `university`, `resume`, `fsp_extract`, `skills_passport`. В слепом ответе этих ключей нет ни на какой глубине.
- **Доказательство.** `tests/test_contact_privacy.py` (в слепом DTO нет ключей ПДн), `tests/test_api_contact_privacy.py`, `tests/test_api_needs_privacy.py`. Отзыв: `POST /api/invitations/{id}/revoke-consent`, чат закрывается (`test_chat_closes_when_consent_revoked_while_accepted`).

1.5. Фильтры и уточнения не теряют подборку — выполнено

- **Проблема.** Если работодатель добавил фильтр и «потерял» хороших кандидатов, доверие к системе падает.

- **Решение.** Подборка — неизменяемые версии. Фильтр не удаляет, а прячет: скрытые остаются в группе «Скрыто фильтрами» с причиной. К прежней версии можно вернуться (`POST /api/needs/{id}/rollback`), закрепить кандидата в шортлисте.
- **Пример.** `GET /api/needs/{id}/versions` отдаёт v1, v2…; ответ `selection` содержит `total_n` , `visible_n` , `hidden_n` , `hidden_summary` ; в таблице подборки (`/employer/needs/:id`) — блок «Скрыто фильтрами».
- **Доказательство.** `tests/test_matching_selection.py` , `tests/test_api_needs_flow.py` ; сборка версии: p95 133 мс на 1 865 кандидатов (VALIDATION.md п. 4.7).

1.6. Резюме и самоописание не влияют на категорию и ранг — выполнено

- **Проблема.** Если навыки можно написать в резюме, ранжирование можно «накрутить».
- **Решение.** Заявленные навыки — серые чипы «заявлено» и последний ключ сортировки при равном Score. В K входят только доказательства: тест, задачи, ФСП.
- **Пример.** В `/employer/needs/:id` у кандидата чип «Python» подписан источником («Тест: модуль Python ✓✓»), а Kafka — «нет данных». Заявленное самим кандидатом лежит отдельно (`declared_skills` , состояние `declared`) и рисуется серыми чипами; в карточке приглашения самоописание подписано «заявлено кандидатом».
- **Доказательство.** Накрутка: 50 слабым кандидатам каждого test-брифа дописали 5 навыков брифа, 1 000 случаев. Наш подбор сдвинулся вверх на 0,0 позиций (ни один не попал в топ-10), поиск по резюме (BM25) — на 518,4 позиции, 12,4 % попали в топ-10 (VALIDATION.md п. 4.4). Тесты: `test_fake_declared_skills_shift_rank_by_zero` , `test_self_declared_never_counts_in_k` .

1.7. Неподтверждённый грейд виден со статусом и опущен в выдаче — выполнено

- **Проблема.** Кандидат прошёл опрос, но не прошёл тест (или ещё не проходил). Скрывать такого человека нечестно, смешивать с подтверждёнными — тоже.
- **Решение.** Показываем с бейджем «грейд не подтверждён» и строкой «против» «Категория заявлена, тестом не подтверждена»; ранжируем ниже всех подтверждённых; в охват и перцентили он не входит.
- **Пример.** `POST /api/admin/import` с кандидатом `features: null` попадает в группу «категория по опросу, не подтверждена» и оказывается в хвосте; тот же бриф в `/lab/check` показывает группу отдельно (`n_unconfirmed` в ответе).
- **Доказательство.** `tests/test_api_unconfirmed_visibility.py` (`test_selection_ranks_unconfirmed_after_confirmed_and_explains`), `tests/test_matching_scoring.py::test_unconfirmed_survey_category_goes_below_confirmed` .

1.8. Поиск по банку без брифа — выполнено

- **Проблема.** Работодатель хочет просто посмотреть, кто есть на рынке, без описания потребности.
- **Решение.** `GET /api/bank` и экраны `/employer/bank` (карта талантов «специализация × грейд») и `/employer/bank/list` : специализация, грейд, категория, стек, сфера бизнеса, режим, «С ФСП». Сортировка `0,55·T + 0,25·F + 0,20·A` .
- **Пример.** В демо-мире фильтр «Backend (Python) · Middle» даёт 101 карточку; с включённым «С ФСП» (`has_fsp=true`) остаётся 17.
- **Доказательство.** `tests/test_api_bank_search.py` , `test_bank_sort_without_brief` ; MATCHING.md п. 11.

2. Категория и тест

2.1. Категория = специализация × грейд, определяется обязательным путём — выполнено

- **Проблема.** Нужна сопоставимая категория, которую нельзя получить словами.

- **Решение.** Опрос из четырёх шагов (сфера бизнеса, что делали руками, технологии и стаж, направление) → выбор грейда с брифингом → адаптивный тест → результат с категорией. Грейд присваивает только тест; стаж и ФСП на него не влияют. Справочник — `config/taxonomy.yaml`; с тестом четыре направления: Backend (Python), Frontend (JS/TS), QA, DevOps; остальные (Data/ML, Mobile, ИБ, Go, Java, UI/UX) — «по опросу», в подборке ниже подтверждённых.
- **Пример (демо-сценарий).** Войдите ступенчатым кандидатом `stage1-registered@…` и пройдите `/candidate/survey` → `/candidate/grade` → `/candidate/test/:id` → `/candidate/result/:id`; либо `stage4-category@…` — у него уже «Backend (Python) · Middle», балл 614, интервал 80 % — 535–694, уверенность 98 %, модули Python и SQL с ✓✓. Подсказка специализации по опросу: `POST /api/survey/hint`.
- **Доказательство.** Грейд только из теста: `tests/test_api_testing_rules.py`. Подсказка специализации: macro-F1 0,93 против 0,80 при чистом выборе кандидата, на отложенных профилях 0,92 (VALIDATION.md п. 3; `tests/test_survey_classifier.py`). Точность грейда: решение у заявленного порога верно в 0,86, точный грейд 0,748 (цель 0,75 — **не выполнено на 0,002**), с точностью ±1 — 1,00; грейд «по стажу» из опроса даёт 0,54 (VALIDATION.md п. 2.1).

2.2. Единый набор заданий не годится, «ИИ под каждого» ненадёжен — выполнено

- **Проблема.** Если у всех одно задание, оно утекает. Если генерировать задание нейросетью «под каждого», нельзя гарантировать ни правильность ключа, ни одинаковую сложность.
- **Решение.** Банк из 111 семейств заданий. Семейство — генератор: из одного шаблона получаются клоны с разными данными, ключ (правильный ответ) **вычисляется исполнением кода** (`run_python`, `run_js`, `run_sql`), а не пишется человеком или нейросетью. Каждый клон выдаётся одному человеку. Сложность семейства откалибрована экспертными оценками и ответами. LLM в тесте и ранжировании не участвует (проверяется тестом). Для Frontend есть JS-версии общих семейств: фронтендер не получает заданий на Python.
- **Пример.** `/lab/bank/:familyId` («3 клона рядом»): одно семейство, три разных экземпляра. Песочница жюри `/try` (раздел 2.3) показывает тот же механизм. Для жюри пересечение клонов у пар кандидатов — 0 из 67 974 выданных.
- **Доказательство.** Симуляция утечки: коалиция из 20 кандидатов сливает всё, что видела; ложный Middle у явных Junior — 0,10 против 1,00 у фиксированной формы и 0,77 у адаптивного теста без клонов (VALIDATION.md п. 2.6). Гейты банка — `tests/test_bank_gates.py` и `tests/test_bank_*.py` (банк целиком, включая JS-версии), `tests/test_frontend_no_python.py`, `tests/test_llm_boundaries.py`. Честная оговорка: клоны защищают от слива ответов, но не от слива идеи семейства; на «утечку метода» нужен CUSUM, и его задержка при 50 % знающих — 603 ответа при цели 200 (**не выполнено**, п. 2.7).

2.3. Защита от чат-ботов («Антиподсказчик») — выполнено, с оговоркой

- **Проблема.** Кандидат копирует условие в ИИ и вставляет ответ. Отдельный вопрос — как проверить такую защиту.
- **Решение.** Для каждого клона строится «двойник»: тот же текст с одним изменённым значением данных. Экран всегда показывает истинное условие, а в буфер обмена попадает двойник. Ответ, совпавший с ключом двойника, — улика для методиста. Оговорка: скриншот и ИИ-панель браузера двойник проходят; против них — «Подтверждение за 5 минут» (три вопроса по заданиям кандидата после «да»). Подробности — TESTING_MECHANISM.md п. 9.
- **Пример (живая проверка за минуту).** Откройте `/try`, скопируйте условие задания и вставьте его в любой ИИ или интерпретатор. В нашем прогоне на экране было `queue += [10]`, в буфере — `queue += [13]` (в вашей сессии значения будут другими). Решите версию из буфера и введите её ответ: итог «Поймано двойником: 2 из 4» (скопированы были два Python-задания), а акт сверки подсвечивает единственное изменённое значение (`10 → 13`) и показывает оба ключа рядом: `[72,39,24,7,10]…` по экрану и `[72,39,24,7,13]…` по копии. Те же шаги API: `POST /api/jury/cheat-lab` → `POST …/answers` → `GET …/result`. Песочница помечена `demo_assisted` и в метрики не идёт.

- **Доказательство.** `tests/test_anticheat_*.py` (240 тестов), e2e `frontend/e2e/tests/05-sandbox-twin.spec.ts`, пилот с ИИ-агентами — раздел 10.1. Двойник строится не для всех семейств: у 39 из 111 форма задания допускает безопасный двойник (ответ вводится текстом, язык исполняется офлайн); для остальных защита — клоны и сверка вживую (TESTING_MECHANISM.md п. 9.1).

2.4. Стабильность при повторе и честная смена грейда — выполнено (две цели валидации не достигнуты)

- **Проблема.** Один и тот же человек в разные дни не должен получать разные грейды; тест на ступень ниже не должен «сжигать» попытку; повышение не должно быть автоматическим прыжком.
- **Решение.** Гистерезис (ложное повышение 0,03), пересдача по правилам: повтор без грейда — через 14 дней, смена грейда — не чаще раза в 90 дней, не больше 3 попыток за 90 дней; провал на заявленном грейде открывает тест на ступень ниже сразу и без счётчика попыток; повышение — только отдельным тестом (`config/testing.yaml → rules`).
- **Пример.** Экран `/candidate/qualification` у `stage4-category@…`: «Сменить грейд можно не чаще раза в 90 дней: тест на повышение — через 89 дней» (`GET /api/tests/qualification`: `can_upgrade.code = "change_interval"`, `next_change_in = "через 88 дней"`). Для понижения: `POST /api/tests/qualification/{spec}/downgrade`.
- **Доказательство.** Совпадение решения при пересдаче 0,81 (цель 0,78, выполнено), к 0,551, QWK 0,86 — выполнено; **корреляция оценок 0,86 при цели 0,88 — не выполнено**, и мы объясняем почему: адаптивный тест ищет решение «выше/ниже порога», а не точную оценку каждого (VALIDATION.md п. 2.3, 6). Пилот с ИИ-агентами (раздел 10.1): повтор у 12 честных персон — точное совпадение полосы 0,83, различие не больше одной ступени 1,00. Тесты: `tests/test_api_testing_rules.py`, `tests/test_cat_*.py`. Свои профили: `make jury-check PROFILES=…` (раздел 7).

2.5. Короткий тест — выполнено

- **Проблема.** Длинный тест отпугивает кандидатов.
- **Решение.** Адаптивный тест останавливается, когда уверенность достаточна: от 8 до 20 заданий. Прогресс на экране — «от 8 до 20», а не фиксированный счётчик.
- **Пример.** Уверенный кандидат заканчивает на восьмом задании. В пилоте Middle-персоны ответили на 8 заданий, Junior — в среднем на 13,1 (от 8 до 20).
- **Доказательство.** В симуляции основная часть — в среднем 12,8 задания (цель ≤ 15), с доборами 16,1 (цель ≤ 17), p95 20 (VALIDATION.md п. 2.2).

2.6. Регулярные короткие задания от работодателей — выполнено

- **Проблема.** Профиль устаревает; работодателю нужен способ проверить конкретный подход.
- **Решение.** Тип А — задача из каталога откалиброванных семейств (свой клон, автопроверка), тип В — «предложи подход» с рубрикой из четырёх критериев по 0–2, решение уходит работодателю без имени. Лимиты: не больше 1 предложения в 7 дней кандидату, до 15 минут, отказ не штрафуется, после 3 пропусков — пауза 30 дней. Планировщик раз в час предлагает задания по каденции и напоминает за сутки до срока (`config/scheduler.yaml → task_offers`). Задачи влияют на свежесть профиля и подтверждение навыков, но не меняют грейд.
- **Пример.** `/employer/tasks` — создать задачу из каталога (`GET /api/tasks/catalog`); `/candidate/tasks` — предложение и таймер. Если компаний нет — «тренировочная задача от платформы».
- **Доказательство.** `tests/test_api_tasks_flow.py`, `tests/test_scheduler.py`; TESTING_MECHANISM.md п. 11.

2.7. Из описаний вакансий — задания и тесты — выполнено

- **Проблема.** Нужно показать, как по описанию вакансии строятся проверки, не ломая сопоставимость грейдов.

- **Решение.** Стандартный тест категории **под вакансию не меняется**. Вакансия даёт две вещи: «как мы проверим» (отображение навыков брифа на модули теста) и проверочный набор из 3–5 откалиброванных задач на конкретные навыки, суммарно до 15 минут, для кандидатов из шортлиста.
- **Пример.** `GET /api/needs/{id}/check` для брифа «Python, FastAPI, PostgreSQL, плюсом Kafka и Docker»: «Python ✓ проверено стандартным тестом категории»; «FastAPI ✓ проверяется проверочным набором по вакансии (модуль "FastAPI")»; «Kafka ✗ в стандартный тест категории не входит — проверьте на интервью». `POST /api/tasks/check-sets` собирает набор.
- **Доказательство.** 11 семейств заданий на PostgreSQL, FastAPI, Redis и очереди с ключом-исполнением; на 40 брифах набор поднимает P@10 с 0,66 до 0,71, прирост +0,048 [+0,017; +0,082] (MATCHING.md, раздел «Навыки, наблюдаемые через проверочный набор»; отдельный прогон `eval.checkset_eval`, перепроверен 9 октября); `tests/test_matching_skills.py`, `tests/test_bank_skills.py`.

2.8. Экспресс-тест: жюри проходит тест само за 3–5 минут — выполнено

- **Проблема.** Жюри должно пройти тест сам, но не тратить полчаса.
- **Решение.** `/jury` создаёт одноразового кандидата с готовым опросом; экспресс — до 8 заданий, около 4 минут, тем же движком и банком. Категорию экспресс не присваивает и честно пишет, когда точности не хватило. Режим жюри показывает правильный ответ (`GET /api/demo/jury/hint`); такие сессии помечены `demo_assisted` и в калибровку и метрики не попадают.
- **Пример.** Экспресс-тест с подсказками на Backend Middle закончился после 6 заданий результатом «Экспресс-оценка: Middle или выше · на 97 % уверены»; в ответе `confirmed_grade: null`, `category: null` (категорию даёт только полный тест), разбивка «Junior 3/3, Middle 2/2, Senior 1/1».
- **Доказательство.** `tests/test_cat_express.py`, `tests/test_api_demo_jury_entry.py`, e2e `01-jury-express.spec.ts`.

3. Кабинет кандидата

Требование	Решение и пример	Доказательство	Статус
Профиль вручную или из PDF-резюме с автораспознаванием (ФИО, контакты, стек, грейд, роли, стаж, софт-скиллы)	`/candidate/profile/edit`: загрузка PDF (`POST /api/candidates/me/resume`), черновик полей, кандидат подтверждает каждое (`/resume/confirm`); распознанное помечено «заявлено» и в Score не входит	`tests/test_resume_parser.py`, `tests/test_api_profile_resume.py`	выполнено
Стандартизированный PDF-профиль	`/candidate/profile/pdf` → `GET /api/candidates/me/profile.pdf`; на странице QR со ссылкой `/v/{код}` — публичная сверка файла без входа	`tests/test_pdf_profile.py`	выполнено
Обязательный путь: опрос → грейд → тест → категория	раздел 2.1	`tests/test_api_survey_flow.py`, e2e `02-candidate-journey.spec.ts`	выполнено
Статус, история опросов и тестов, тест на ступень ниже или выше с ограничением частоты	`/candidate/qualification`, раздел 2.4	`tests/test_api_testing_rules.py`	выполнено
Входящие приглашения с вакансией и вилкой; принять, отклонить, «Да, если…»	`/candidate/invitations`, `/candidate/invitations/:id`; в ответе `GET /api/invitations/inbox` приглашение содержит вилку «220 000–280 000 ₽ на руки», компанию, её надёжность, срок «через 5 дней»	`tests/test_api_contact_lifecycle.py`	выполнено
Список вакансий, самостоятельный отклик, статусы	`/candidate/vacancies`; `POST /api/applications`; отклик сразу раскрывает контакты по отдельному согласию	`tests/test_contact_application.py`, `tests/test_api_contact_vacancies.py`	выполнено

Требование	Решение и пример	Доказательство	Статус
Настройки: приватность, согласия на обработку и показ, привязка ФСП ID	`/candidate/privacy`, `/consents`, `/visibility`, `/fsp`; журнал согласий `GET /api/consents`; удаление аккаунта `DELETE /api/candidates/me`	`tests/test_account_erasure.py`, `tests/test_platform_access.py`	выполнено
Кандидат видит только свои приглашения и отклики	Роли и проверка владельца в каждом маршруте; матрица доступа по OpenAPI	`tests/test_security_matrix.py`	выполнено

4. Кабинет работодателя

Требование	Решение и пример	Доказательство	Статус
Профиль компании: описание, направление, контакты	`/employer/company`, `PUT /api/companies/me` (название, сфера, описание, способы связи, сайт, ИНН)	`tests/test_api_company_profile.py`	выполнено
Описание потребности: кто нужен, чем занимается команда, стек, специализация, грейд	`/employer/needs/new`: свободный текст плюс блок «о команде» (размер, роль, контекст стека, о команде); команда не участвует в разборе и Score, но попадает в приглашение	`tests/test_api_needs_features.py`, миграция `0014_need_team`	выполнено
Рекомендованные категории, конкретные кандидаты, обоснование попадания	`GET /api/needs/{id}/categories`: «Backend (Python) · Middle», вероятность 0,64, 47 кандидатов, охват вилки 220–280 тыс.: 32 из 47; панель доказательств	раздел 1.2	выполнено
Поиск по банку: специализация, грейд, категория, стек, ФСП	раздел 1.8	`tests/test_api_bank_search.py`	выполнено
Приглашение с описанием и вилкой, отслеживание статусов	`/employer/invitations` — воронка по статусам, у каждого карточка со сроками; приглашение не привязано к опубликованной вакансии	`tests/test_api_contact_lifecycle.py`	выполнено
Публикация вакансий, редактирование, входящие отклики	`/employer/vacancies`: «Создать вакансию» из потребности или сразу с формы (бриф создаётся автоматически); `POST /api/vacancies`, `PATCH`, `/close`; отклики с причиной отказа	`tests/test_api_contact_vacancies.py`	выполнено
Контакты кандидата скрыты до принятия или самостоятельного отклика	раздел 1.4	`tests/test_contact_privacy.py`	выполнено

5. ФСП ID, достижения, кандидат без истории

5.1. Связь профиля с ФСП ID и совместимость с Keycloak — выполнено на мок-реестре

- **Проблема.** Профиль должен привязываться к участнику ФСП, авторизация — быть совместимой с Keycloak.
- **Решение.** OIDC с PKCE, realm `fsp` в `deploy/keycloak/`, демо-реестр `services/fsp-mock/`; на экранах входа и регистрации есть кнопка «Войти через ФСП ID» (если настоящий ФСП ID недоступен, объясняет, что делать). Первый вход создаёт аккаунт и отправляет код на email.
- **Пример.** `/auth/login` → «Войти через ФСП ID»; либо `/candidate/fsp` → «демо-вход ФСП» (`POST /api/fsp/demo-login`, спортсмены — `GET /api/fsp/demo/athletes`).
- **Доказательство.** `tests/test_api_fsp_link.py`, `tests/test_fsp_provider.py`, `tests/test_fsp_mock.py`. Оговорка: реального реестра ФСП и их Keycloak мы не видели — работает мок и контракт, который мы предлагаем (FSP_INTEGRATION.md п. 4 и 9 — шаги перехода).

5.2. Кандидат без истории ФСП обрабатывается корректно — выполнено

- **Проблема.** Если ФСП даёт преимущество, кандидат без него оказывается в проигрыше — а это большинство людей.

- **Решение.** Без истории F = 0, «против» не попадает никогда; категория, тест и все остальные пути те же. Отдельно измеряем экспозицию.
- **Пример.** Первый в подборке «Backend Middle в платёжный шлюз» (раздел 1.2) — без ФСП, Score 69,3. В `/employer/needs/:id` над ним нет ни одной строки «против» про ФСП.
- **Доказательство.** `test_no_fsp_is_zero_not_penalty`, `test_no_history_gives_zero_f_and_neutral_line`; «Отсутствие ФСП никогда не "против"» — 100 % на 1000 карточек; условная экспозиция кандидатов без ФСП в топ-10 — 0,91, в топ-50 — 0,94 при допуске 0,80–1,25 (VALIDATION.md п. 4.5); доля подтверждённых тестом с ФСП 92,9 %, без ФСП 93,4 % (п. 2.10).

5.3. Как достижения ФСП двигают ранг, а не грейд — выполнено

- **Проблема.** Дисциплины и мероприятия несопоставимы, а явки можно накрутить.
- **Решение.** Вклад = результат (призёр/участник) × место в поле × число стартов (с потолком) × релевантность дисциплины специализации × свежесть (полураспад 24 месяца). Иерархии мероприятий нет: все дисциплины равноправны. Онлайн-явки без призовых мест считаются не больше двух раз. Роль в команде из заявки (`team_function`) показывается серым и в расчёт не входит. Грейд от ФСП не зависит.
- **Пример.** Первое место из 103 на соревновании по продуктовому программированию полгода назад, личный зачёт: VSS 75,4, для брифа Backend вклад в Score 9,0 балла. Двенадцать подряд онлайн-участий без призовых мест дают столько же, сколько два (FSP_INTEGRATION.md п. 5). На экране — «ФСП: Призёр всероссийских · алгоритмическое · 2025 · топ-3 · личный зачёт»; точное место до раскрытия не показывается.
- **Доказательство.** `tests/test_fsp_vss.py` (контрольные примеры, потолок участий, `test_hidden_result_is_excluded_everywhere`, `test_awaiting_protocol_result_is_not_counted`). Модель сверена с открытыми нормами федерации: МСМК в нормах нет и в расчёте отсутствует, сроки разрядов — КМС 3 года, I–III 2 года, МС бессрочно. Веса — экспертные допущения; нужна калибровка вместе с ФСП (FSP_INTEGRATION.md п. 10).

5.4. Схема интеграции с реестром ФСП — выполнено

- **Решение.** FSP_INTEGRATION.md: схема, предлагаемый контракт API реестра, формулы, согласия, несовершеннолетние, 10 шагов перехода на реальные системы. Контракт реализован мок-реестром `services/fsp-mock/` и клиентом `FSPProvider`.
- **Доказательство.** `tests/test_fsp_provider.py` проверяет клиента против мока, включая сбои реестра: при недоступном реестре берётся последний снимок, а сбой не выдаётся за «результатов нет» (`test_fallback_uses_snapshot_when_registry_down`, `test_fallback_outage_is_not_reported_as_no_results`).

6. Зарплата, правила видимости, безопасность

Требование	Решение и пример	Доказательство	Статус
Вилка в рублях обязательна в вакансии и приглашении, подписана «на руки» или «до вычета НДФЛ»	`config/contact.yaml → salary`: шаг 1 000, минимум 30 000, предупреждение с 1,6×, блокировка с 2,0×; в API `salary_type_label: "на руки"`; примеры ошибок — раздел 1.3	`tests/test_contact_invitation.py`, `tests/test_contact_quotas.py`	выполнено
Приглашение содержит предложение, вилку, компанию и способ связи	`GET /api/invitations/inbox`: `salary.text` «220 000–280 000 ₽ на руки», `company` с надёжностью и меткой проверки	`tests/test_api_contact_lifecycle.py`	выполнено
Статусы приглашений и откликов видны обеим сторонам	У работодателя колонки воронки (отправлено, просмотрено, встречное условие, принято…), у кандидата `status_label`, сроки в `timers`	`tests/test_api_contact_deadlines.py`	выполнено

Требование	Решение и пример	Доказательство	Статус
Регистрация по email с подтверждением	`/auth/register`, `/verify-email`; письма в Mailpit `:8025`. В режиме жюри подтверждение отключено для удобства (`VZ_DEMO_AUTO_VERIFY_EMAIL=false` возвращает код)	`tests/test_platform_auth.py`	выполнено; в демо упрощено
Верификация компании и модерация вакансий не обязательны в MVP	Всё равно есть добровольная метка «Проверенная компания» и жалобы: раздел 10.9	`tests/test_trust_verification.py`	выполнено
152-ФЗ: отдельные согласия на обработку и публикацию, журнал, удаление	`/candidate/consents`; `GET /api/consents/documents` — тексты; `DELETE /api/candidates/me`	`tests/test_account_erasure.py`, SECURITY.md	выполнено
Роли, защита от несанкционированного доступа, пароли и токены	argon2id, короткий access и refresh в `HttpOnly`-cookie с ротацией, ограничение частоты; матрица «каждый защищённый маршрут × чужая роль» по схеме OpenAPI	`tests/test_platform_security.py`, `tests/test_security_matrix.py`	выполнено
Кандидаты 15–17 лет с согласием представителя	`config/contact.yaml → minors`: моложе 15 регистрация отклоняется; 15–17 — профиль виден работодателям только после согласия законного представителя (письмо со ссылкой на 7 дней), приглашать можно только на стажировку	`tests/test_minors_guardian.py`	выполнено
LLM открыто и вне оценки	LLM.md: разбор брифа и резюме с цитатой, по умолчанию выключен; ранжирование, тест и ключи — без LLM	`tests/test_llm_boundaries.py` — 0 обращений к модели в тесте и подборе	выполнено

Про слепую карточку: организаторы допускают показ имени и прежних мест работы, мы **сознательно не показываем** их до «да» (меньше предвзятости; основания — MATCHING.md п. 8.1). Вместо них — обезличенный опыт («Backend, 2 г. 3 мес.»).

7. Валидация: собственная и на ваших данных

- **Проблема.** Нужна своя процедура валидации и доказательство, что механика работает; жюри хочет проверить на закрытом наборе.
- **Решение.** Протокол записан до прогона (VALIDATION_PLAN.md: метрики, пороги, бейзлайны, правило решения; хэши файлов брифов), результаты — VALIDATION.md. Синтетическая популяция из 2 000 кандидатов с известным скрытым уровнем, 100 брифов (20 dev, 20 test, 60 предрегистрированных test2 с заморозкой SHA-256), оракул релевантности по скрытой истине, бейзлайны: поиск по резюме (BM25), категория + точечный уровень, случайный порядок, фиксированная форма и «ИИ под каждого» для теста. `make validate` — около 6 минут.
- **Пример (результат, который можно сверить).** На 20 test-брифах P@10: наш сквозной подбор 0,61 [0,49; 0,73], поиск по резюме 0,13 [0,07; 0,19], случайный порядок в категории 0,25. На 60 новых брифах, правило решения записано заранее: 0,54 против 0,36 у «категория + уровень» (разность +0,18, ДИ [0,12; 0,24], значима) и 0,14 у поиска по резюме. Сводка — 38 из 43 пороговых пунктов (из исходных 41 — 36).
- **Пример (на ваших данных, без Swagger).** Войдите через `/jury` → «Лаборатория» → `/lab/check`. Вкладка «Наш набор пар» одной кнопкой даёт метрики нашего набора (20 брифов, 1 706 пар): P@10 0,615 [0,485; 0,725], поиск по резюме 0,13, категория + уровень 0,49; по каждому брифу — топ-10 с компонентами Score и «за/против» для ручной проверки. «Своя вакансия против банка платформы» — вставляете вакансию с вилкой и видите подборку. «Свой набор» — загружаете три таблицы (вакансии, кандидаты, разметка `rel` 0–3) в CSV или JSON; шаблоны скачиваются на странице; методист видит только свои загрузки. Для теста: `make jury-check PROFILES=backend/eval/jury_profiles_example.jsonl` прогоняет ваши профили через боевой движок (на примере из 16 профилей при `--seed 1`: повтор точно 0,81, ±1 1,00, QWK 0,92, r 0,92).

- **Доказательство.** `tests/test_eval_*.py`, `tests/test_api_admin_check.py`, `tests/test_api_admin_access.py`, `tests/test_jury_simple.py`. Протокол пилота — `backend/eval/agent_pilot/PROTOCOL.md`.
- **Честно, что не выполнено (5 из 43).**
  1. Точный грейд 0,748 при цели 0,75 — в пределах выборочной ошибки (на пяти других сидах в среднем 0,758).
  2. Корреляция оценок при пересдаче 0,86 при цели 0,88 — несовместима с бюджетом длины теста.
  3. Задержка детектора утечки CUSUM при 50 % знающих — 603 ответа при цели 200; он остаётся сигналом методисту.
  4. P@10 значимо выше «категория + уровень» на 20 test-брифах — не значимо (разность 0,12, ДИ [-0,04; 0,28]); на всех 40 брифах значимо, на 60 новых — значимо, но абсолютный P@10 0,54 ниже порога 0,60.
  5. Стабильность топ-10 при пересдаче 0,28 при цели 0,70 — цель поставлена некорректно, это показано расчётом; постфактум-метрики выполнены.
- **Статус.** Частично: процедура и числа на синтетике воспроизводимы, но экспертной разметки пар и живого пилота на людях нет. Оракул и генератор писала одна команда, тексты брифов — с ИИ-ассистентом.

8. Документы, API, запуск, нагрузка

Требование	Где и как проверить	Статус
Архитектура: функциональная и компонентная	ARCHITECTURE.md: C4, ERD, sequence-диаграммы четырёх сценариев, машины состояний	выполнено
Механика тестирования, механика подбора и категоризации, валидация	TESTING_MECHANISM.md, MATCHING.md, VALIDATION_PLAN.md, VALIDATION.md	выполнено
Схема интеграции с ФСП / ФСП ID	FSP_INTEGRATION.md	выполнено
API: запросы, ответы, коды ошибок	API.md; Swagger `/api/docs`, ReDoc `/api/redoc`, `/api/openapi.json`. У каждой из 242 операций есть описание и коды 4xx; ошибки в едином формате `{"code", "message", "details"}`, например `salary_required`, `contact_rejected`. Проверка: `tests/test_platform_openapi.py`	выполнено
Библиотеки с версиями	THIRD_PARTY.md из `uv.lock` и `package-lock.json`, лицензии открытые	выполнено
Пошаговый запуск, Docker	DEPLOY.md, `docker-compose.yml`, три Dockerfile; на чистой машине `docker compose up -d --build` и `make demo`. Пул заданий догенерируется около 2,5 минуты: до его конца тест недоступен (в `GET /api/health` поле `pool_ready`). Стенд с HTTPS на VPS одной командой: `sudo bash deploy/bootstrap.sh`	выполнено; `bootstrap.sh` на настоящем VPS не запускали
Производительность при десятках пользователей	LOAD_TEST.md: 100 одновременных пользователей, 3 минуты, 24 739 запросов, 0 сбоев, p50 31 мс, p95 170 мс, p99 300 мс; найдены и исправлены deadlock и N+1 (воронка приглашений: p95 4,9 с → 150 мс). Нагрузка с отдельной машины не проверялась	выполнено
Код открыт, без обфускации, лицензия	`LICENSE` (MIT); стек на открытых лицензиях; единственное исключение — `axe-core` (MPL-2.0) как dev-зависимость e2e, в образы не попадает	выполнено
Развёрнутый прототип по ссылке	**Не развёрнут**: нужен VPS; скрипт готов (`deploy/bootstrap.sh`). Работает локально за 5 минут	не сделано

9. Ответы организаторов: что учтено

Пересказ ответов организаторов (встреча и таблица вопросов) и то, как они отражены. Номера в последнем столбце — разделы этого документа.

Что ответили организаторы	Что сделано	Где
Грейд — только по тесту категории, не по вакансии и не по ФСП; стаж — только атрибут	`Qualification` собирается из теста; ФСП влияет на ранг	2.1, 5.3
Неподтверждённый грейд показывать со статусом и опускать ниже	Хвост выдачи с бейджем	1.7
Кандидат сам выбирает грейд, подсказка допустима	`/candidate/grade`, `POST /api/survey/hint`	2.1
Повышение — отдельным тестом; провал не сжигает попытку; кулдаун на смену	90 дней на смену, 14 на повтор, тест на ступень ниже сразу	2.4
Одинаковые задачи не должны расходиться между кандидатами; защита от ИИ и списывания — большой плюс; песочница и прокторинг	Клоны, «Антиподсказчик», события страницы теста (копирование, вставка), `/try`	2.2, 2.3
LLM допустимы, облачные тоже, использование описать	LLM.md: GigaChat проверен вживую на 62 брифах, прироста нет — честно записано; ПДн маскируются; по умолчанию выключен	6, 10.12
Датасет пар собирает команда, жюри проверит руками; своя процедура, минимума по объёму нет	100 брифов, `/lab/check`, `make jury-check`, импорт	7
Реальный Keycloak не ждут, вход через имитацию ФСП ID — ожидаемый сценарий, регистрация по email тоже	Мок-реестр, realm `fsp`, кнопка на экране входа	5.1
Контакты открываются пригласившему после принятия; отзыв доступа и чат на платформе — плюс	Раскрытие после «да», `revoke-consent`, чат после взаимного «да»	1.4, 10.7
ФСП: дисциплины равны, считать число стартов и результат, не набивать онлайн-явками; срок давности можно ввести	Потолок участий, класс результата, свежесть с полураспадом 24 месяца	5.3
Задачи прошлых соревнований ФСП как дополнительный буст	Не реализовано (дали бы преимущество решавшим)	11
Несколько категорий — плюс; хватит двух специализаций; дизайнеры — приоритет у технических ролей	Несколько квалификаций у одного кандидата, Fullstack как производная; UI/UX — «по опросу»	2.1
Зарплата: подписать гросс/нетто	«на руки» / «до вычета НДФЛ» везде	1.3, 6
Несовершеннолетние 15–17 с согласием родителей — плюс; нужен правовой минимум (152-ФЗ)	Согласие представителя, согласия и журнал	6
Экспресс-тест 3–5 минут; жюри пройдёт тесты само	`/jury`, до 8 заданий	2.8
Проверка компании — добровольная метка	«Проверенная компания» не блокирует сценарий	10.9
Регулярные задания адресно; эталон проверяет система, открытый ответ — работодатель	Тип A (автопроверка) и тип B (рубрика)	2.6
Задания под вакансию — отдельный блок; единый тест на категорию, но не с одинаковым пулом	Проверочный набор, клоны	2.7
ATS: синхронизация вакансий, откликов, статусов; в MVP не нужна, концепция идёт в плюс	Pull-API, вебхуки, адаптер «Хантфлоу» на эмуляторе	10.8
Парсинг допустим только для тестовых данных	Открытые данные рынка без ПДн для словаря и вилок	10.6
Качество подбора важнее скорости; GPU и ограничений по образам нет	p95 сборки версии 133 мс, без локальных моделей	1.5, 8
Питч 10 минут; видео вместо живого стенда допустимо, стенд держать на время оценки	Презентация и видео собираются отдельно; стенд не развёрнут	8, 11
Исходный код открыт, идею после дедлайна не менять, смотрят даты коммитов	Лицензия MIT, история коммитов не переписывалась	8

10. Сверх кейса

Возможности, которых кейс не требовал, но которые делают механику честнее или проверяемее. Для каждой — пример и доказательство; ограничения указаны.

10.1. Пилот с ИИ-агентами: проверка «Антиподсказчика» и повтора теста

- **Что.** Мы не ограничились симуляцией: Claude-агенты играли кандидатов с заданным профилем знаний и проходили настоящий тест через настоящий HTTP-API (регистрация, опрос, выбор грейда, клоны, ответы, результат). 12 честных персон (4 специализации × Junior/Middle/Senior) и 2 «списывающие» (знают на Junior, заявляют Middle и решают каждое задание копией из буфера), по 2 попытки. Протокол и метрики записаны до прогона (`backend/eval/agent_pilot/PROTOCOL.md`).
- **Как устроена защита сейчас.** Двойник ставится на каждое задание сессии, где он посчитан (`twin_rate_per_session: 20`); предрасчёт двойников входит в конвейер пула (сервис `pool` , `make gen-pool`), так что на свежем стенде защита работает с первой сессии; выдача отдаёт сначала клон с готовым двойником.
- **Результат для «списывающих».** В 4 прогонах: статус «не подтверждён», ложного Middle нет; двойник сработал на 4–7 заданиях из 6–8 скопированных. Тест выдачи: `tests/test_api_testing_pool.py::test_issue_prefers_instance_with_ready_twin` . Эти 4 прогона сделаны после доработки выдачи и в предрегистрированные доли не входят; предыдущая версия и её результат записаны в VALIDATION.md п. 10.4 как фактическая запись.
- **Остальные результаты пилота** (VALIDATION.md раздел 10): повтор у честных — точное совпадение полосы 0,83, ±1 — 1,00, взвешенная к 0,83, корреляция баллов 0,97; все Middle-персоны подтвердили заявленный Middle (8 из 8), но по баллу попали в полосу Senior — ИИ «не умеет не знать», это предсказано в протоколе.
- **Оговорки.** Выборка маленькая (12 + 2 персоны), это ИИ, а не люди; **одна ложная тревога на 24 честных прогона при ориентире 0**; скриншот и ИИ-панель браузера двойник не ловит — для них «Подтверждение за 5 минут».

10.2. Цифровой паспорт навыков (W3C VC, QR)

- **Что.** Результат теста и достижение ФСП кандидат выпускает как подписанный файл (Verifiable Credential 2.0 и Open Badges 3.0, подпись Ed25519, издатель `did:web`) — переносится между платформами; проверка по QR и по файлу, отзыв.
- **Пример.** `/candidate/profile/passport` → выпустить «Призёр · продуктовое программирование (хакатон) · 2025» → скачать файл → изменить в нём один символ → загрузить на `/v/` : проверка вместо «Паспорт подлинный: подписан издателем, не изменён, не отозван, срок не истёк» отвечает «Проверка не пройдена: Подпись не совпадает: содержимое файла изменено после выдачи» (шаг `signature` : `ok: false`). То же по API: `POST /api/credentials/me` , `GET …/file` , `POST /api/credentials/verify` .
- **Доказательство.** `tests/test_credentials_core.py` , `tests/test_credentials_api.py::test_tamper_one_byte_turns_red` , `test_revoke_flow_and_status_list` ; CREDENTIALS.md. Издатель демонстрационный: «документ не является официальным сертификатом ФСП»; путь к настоящему издателю описан в документе.

10.3. Панель ФСП «Кадровый резерв»

- **Что.** Для федерации: воронка от соревнований до найма — привязан ФСП ID → прошёл тест → подтверждена категория → попал в подборки → получил приглашение → «Взаимно» → нанят; в разрезе дисциплин и округов, спрос и предложение, медианы оферт. Только агрегаты, k-анонимность не меньше 5 со вторичным подавлением.
- **Пример.** `/jury` → «Панель ФСП» (`/fsp/reserve`). На демо-мире: 45 привязанных против 353 без ФСП; доля дошедших до подборки (от прошедших тест) 90,7 % против 52,8 %, до «Взаимно» (от получивших приглашение) 59,5 % против 33,3 %. Ячейки меньше 5 показываются как «<5». Над цифрами предупреждение: «Демо-данные синтетические … Числа показывают, как панель работает, а не состояние реального кадрового резерва».
- **Доказательство.** `tests/test_fsp_dashboard.py` , `config/fsp_dashboard.yaml` ; выгрузка `GET /api/fsp/dashboard/export.csv` ; FSP_INTEGRATION.md п. 11.

10.4. «Сигнал интереса» и симуляция рынка

- **Что.** Вторая сторона обратной механики: кандидат отмечает компанию (2 сигнала за 30 дней, живёт 60, отзыв слот не возвращает, отдельное согласие). Работодатель видит бейдж «Взаимный интерес» только у карточек, которые сам открыл или закрепил; в Score сигнал не входит. Числа квот выбраны прогоном синтетического рынка.
- **Пример.** `/candidate/vacancies/:id` → «Отправить сигнал интереса (осталось 2 из 2)»; у работодателя в банке `interest=true` оставляет только отметивших, а `interest_first=true` ставит их первыми — Score и порядок остальных не меняются.
- **Доказательство.** `tests/test_interest_signal.py` (`test_employer_sees_signal_only_after_card_opened` , `test_bank_badge_filter_sort_and_score_unchanged`), `tests/test_market_sim.py` . Симуляция (MATCHING.md п. 9.1): квоты меняют заполнение вакансий на нагрузку — 10 приглашений на вакансию вместо 44 (нагрузка на кандидатов ниже в 4,4 раза) ценой 2 п.п. заполнения (88,6 % против 90,6 %); сигнал почти не меняет число совпадений (+0,5 ± 0,5 п.п.), но повышает выигрыш кандидатов (0,79 против 0,64). **Граница, которую мы показываем сами:** при похожих вкусах всех кандидатов сигнал пользы не даёт. Это модель с допущениями, а не измерение живого рынка.

10.5. Мониторинг качества заданий

- **Что.** DIF по Мантелю–Хензелю (справедливость задания для групп ФСП/регион/режим доступности/экспресс), монитор дрейфа сложности, контроль экспозиции семейств. Экран `/lab/quality` .
- **Пример.** Метод проверен на внедрённых смещённых заданиях: из четырёх найдены все (классы B, C); среди 316 пар «настоящее семейство × группа» класс C — 0, класс A — 98 %. На утечке 12 семейств монитор нашёл 12 из 12 без ложных флагов среди остальных.
- **Доказательство.** `tests/test_quality_monitoring.py` , VALIDATION.md раздел 8. Это проверка метода на синтетике; выводов о реальных группах людей нет. Контроль экспозиции Симпсона–Хеттера в продукте выключен: за 8 итераций он сходится не полностью, это написано.

10.6. Открытые данные рынка

- **Что.** Словарь навыков и справочная рыночная вилка из открытых данных портала «Работа в России» (Trudvsem), без ПДн. Вилка — подсказка в форме брифа, в Score не входит.
- **Пример.** `GET /api/needs/market-salary?specialization=backend_python&grade=middle` → `n: 11` , `net.p50: 87000` , источник и дата выгрузки; в поле `note` предупреждение: «Выборка по всей РФ, региональные вакансии занижают уровень относительно Москвы и Санкт-Петербурга». Выборка маленькая (n = 11) — и API это показывает.
- **Доказательство.** `tests/test_market_salary.py` , `config/market.yaml` ; MATCHING.md п. 13.

10.7. Чат после взаимного «да»

- **Что.** Переписка открывается только после раскрытия контактов и закрывается при отзыве согласия; сообщения ограничены по длине и частоте.
- **Пример.** После принятия приглашения кандидат видит экран «Взаимно» с контактом компании и пишет в чат; `GET /api/chat/threads` .
- **Доказательство.** `tests/test_api_chat.py` (`test_chat_closed_before_mutual_yes` , `test_chat_closes_when_consent_revoked_while_accepted`), e2e `04-mutual-chat.spec.ts` .

10.8. ATS и эмулятор «Хантфлоу»

- **Что.** Pull-API `/api/ats/v1` по ключу компании, исходящие вебхуки с подписью HMAC-SHA256 и повторами, обратная синхронизация статусов, адаптер «Хантфлоу» с эмулятором.
- **Пример.** В профиле компании укажите адрес `…/api/demo/webhook-sink` — события приглашений видны на `GET /api/demo/webhook-sink` сразу, с проверкой подписи; журнал адаптера показывает запросы ровно в том виде, в каком они ушли бы в Хантфлоу.

- **Доказательство и оговорка.** `tests/test_api_ats.py`. **Против живого Хантфлоу не запускалось**, повторов боевых запросов к ATS нет (ATS_INTEGRATION.md п. 1).

10.9. Доверие и безопасность платформы

- **Что.** Метка «Проверенная компания» (добровольная, никогда не блокирует сценарий): контрольные суммы ИНН и ОГРН, поиск в учебном реестре, совпадение названия и домена. Жалобы кандидатов на приглашения и вакансии (анонимны для компании). Три жалобы от разных кандидатов за 90 дней снижают квоту приглашений компании вдвое; подтверждённая жалоба на фиктивную вакансию, просьбу о деньгах или сбор резюме снимает метку. Автосигналы для модератора («Много приглашений, ни одного принятого»), которые ничего не блокируют.
- **Пример.** `/employer/company` → ввести ИНН и ОГРН → уровень «Реквизиты проверены»; в приглашении кандидат видит «Не проверена» или метку. Жалобы разбирает методист в `/lab/trust`.
- **Доказательство и оговорка.** `tests/test_trust_verification.py`, `tests/test_trust_complaints.py`, `config/trust.yaml`. Реестр учебный (вымышленные компании), настоящий ФНС не подключён.

10.10. Сравнение кандидатов бок о бок

- **Что.** В подборке работодателя флажок «к сравнению» (до 3), плавающая панель и окно сравнения: только слепая карточка, Score с разложением вкладов, свежесть теста, «нет истории ФСП — не штраф», подсветка лучшего значения, приглашение прямо из колонки.
- **Пример.** `/employer/needs/:id` → отметить трёх кандидатов → «Сравнить». Файлы: `frontend/src/features/employer/compare.ts`, `components/selection/CompareDialog.tsx`.
- **Доказательство и оговорка.** Типы и сборка фронтенда в CI; автотеста интерфейса для этого окна нет.

10.11. Страница `/lab/check`, «Обещание процесса», `bootstrap.sh`

- `/lab/check` — проверка подбора на ваших данных без Swagger: раздел 7.
- **«Обещание процесса».** Работодатель может отметить в приглашении, что кандидата ждёт «Без технического скрининга — результат теста засчитан», «Сразу финальное интервью» или «Без тестового задания»; кандидат видит обещания до ответа. После «да» в сверке «условия совпали?» он может отметить «Обещание не выполнено» — это событие снижает надёжность компании (при низкой надёжности квота приглашений уменьшается вдвое); в Score кандидата не входит. Тест: `tests/test_contact_invitation.py::test_outcome_check_promise_broken_is_a_reliability_minus`, API-тест полного цикла — `tests/test_api_contact_lifecycle.py::test_process_promises_flow_and_promise_broken_check`; неизвестный код — `400 promise_unknown`. Строки — `config/contact.yaml → invitation.promises`.
- `deploy/bootstrap.sh` — стенд одной командой на чистом сервере (Docker, swap, секреты, HTTPS через Caddy на `<ip>.sslip.io`, пул, двойники, демо-мир, проверка health). Проверены синтаксис и `docker compose config`; на настоящем VPS не запускался.

10.12. Прозрачное использование LLM

- **Что.** Шлюз `app/llm/` с режимами `none/replay/record/live`, детерминированным fallback и маскированием ПДн. В контуре оценки (ключи, грейд, Score, ранжирование, вердикты «Антиподсказчика») LLM запрещена архитектурно.
- **Пример.** GigaChat подключён вживую (TLS по корневому сертификату Минцифры), 62 синтетических брифа: F1 навыков 0,975 и «только правила», и «правила + LLM» — **прироста нет**, и документ так и говорит. Кассета позволяет воспроизвести прогон без ключа.
- **Доказательство.** `tests/test_llm_gigachat.py`, `tests/test_llm_boundaries.py`, LLM.md п. 9.

10.13. Инженерия, доступность, безопасность, нагрузка

- **CI** (`.github/workflows/ci.yml`): линт `ruff` , `mypy --strict` на ядрах, `pytest` на PostgreSQL с pgvector тремя шардами, проверка миграций вверх и вниз, фронтенд (`tsc` , `eslint` , сборка, `npm audit`), e2e Playwright, сборка образов, `pip-audit` . Оговорка: после последних слияний мы разбираем таймауты на двухъядерном раннере GitHub; зелёный прогон на текущей ветке до сдачи не подтверждён.
- **Тесты.** Более 2 400 тестов бэкенда (собрано 2 424 командой `pytest --collect-only`), 6 сценариев e2e; контрольные примеры из спецификаций — обязательные тесты.
- **Доступность.** Автоматическая проверка axe-core (WCAG 2.1 A/AA) на 13 экранах: было 115 нарушений контраста и ссылок, стало 0 serious и critical; кабинет работодателя без горизонтального скролла на 390, 768 и 1440 px (ACCESSIBILITY.md). Автоматика ловит около трети проблем, ручная проверка выборочная.
- **Безопасность.** Матрица «защищённый маршрут × чужая роль» по схеме OpenAPI (`tests/test_security_matrix.py`), CSP, argon2id, ротация refresh-токенов, `pip-audit` и `npm audit` без уязвимостей в прод-зависимостях (ENGINEERING.md). Динамический скан (OWASP ZAP) не запускался.
- **Нагрузка.** 100 пользователей: 24 739 запросов, 0 сбоев, p95 170 мс (LOAD_TEST.md).

11. Чего нет и что частично

Список, который мы бы хотели знать сами, будучи жюри.

Что	Статус	Почему и что дальше
Развёрнутый стенд по ссылке	сделано	https://sirius-dev-day.ru — развёрнут `deploy/bootstrap.sh` на VPS (2 vCPU / 4 ГБ, РФ), HTTPS Let's Encrypt; настоящая почта с `no-reply@sirius-dev-day.ru` (SMTP с SSL, MX и SPF домена), доставка проверена на внешний ящик
Проверка на живых людях	не сделано	Вся валидация — синтетика и ИИ-агенты; экспертной разметки пар нет. Оракул и генератор написала одна команда
5 из 43 порогов валидации	не выполнено	Перечислены в разделе 7 с причинами; пороги после прогона не менялись
Реальные ФСП и ATS	эмуляция	Мок-реестр, мок ЕГРЮЛ, эмулятор «Хантфлоу». Контракты и шаги перехода описаны
Защита от скриншота и ИИ-панели браузера	частично	Двойник ловит только копирование текста; против остального — «Подтверждение за 5 минут» (живая сверка), а любой удалённый тест от ИИ-ассистента не защищён полностью. Грейд — решение с уровнем доверия, а не сертификат
Покрытие двойниками	частично	39 из 111 семейств допускают безопасный двойник (ответ текстом, исполняемый язык); для остальных — клоны и сверка
Задачи прошлых соревнований ФСП как буст	не сделано	Дало бы преимущество решавшим
Автоматические тесты интерфейса	частично	6 e2e-сценариев; окно сравнения кандидатов и ряд экранов покрыты только типами и сборкой
CI зелёный на GitHub	сделано	Все 9 заданий зелёные на `main` ; известный обход: завершение процесса pytest сразу после отчёта (сбой pydantic_core при выходе интерпретатора, WORKLOG п. 8)
Подтверждение email в режиме жюри	упрощено	В `DEMO_MODE` код не требуется; в боевом режиме включён
Внутренние ограничения	см. документы	Лимиты запросов хранятся в памяти процесса (при нескольких воркерах мягче), общий rate limit — в плане (ARCHITECTURE.md п. 9)

Архитектура «Взаимно»

Документ описывает, из чего состоит платформа, где проходят границы модулей и как данные движутся между ними. Все утверждения сверены с кодом `backend/app`, `services/fsp-mock`, `deploy/keycloak`, `docker-compose.yml` и `config/*.yaml`. То, что ещё не построено, помечено «в разработке». Инженерные практики — ENGINEERING.md, доступность — ACCESSIBILITY.md, требования задачи и где они реализованы — REQUIREMENTS.md.

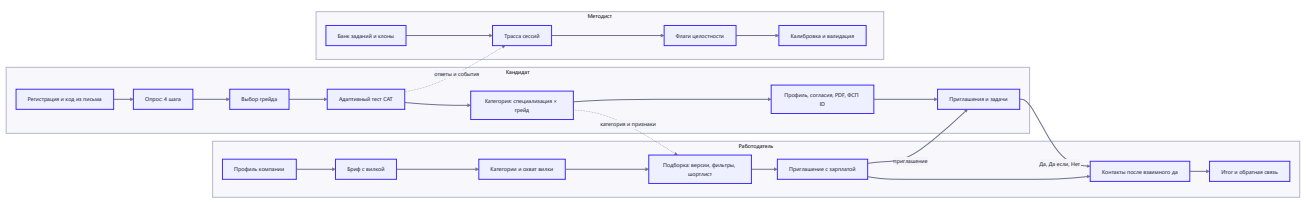
Содержание: 1. Принципы · 2. Функциональная архитектура · 3. Контекст и контейнеры (C4) · 4. Модули монолита · 5. Данные (ERD) · 6. Сквозные сценарии · 7. Состояния · 8. Сквозные механизмы · 9. Статус реализации

1. Принципы

Принцип	Как выполнен
Модульный монолит	Один процесс FastAPI (`app.main:app`), один PostgreSQL. Модули — пакеты Python с однонаправленными зависимостями (п. 4). Дробить на сервисы нет смысла: нагрузка небольшая, а транзакционность между тестом, подбором и контактом нужна
Чистые ядра, тонкие сервисы	Алгоритмы (`app/testing` , `app/matching` , `app/contact` , `app/fsp`) не знают ни о БД, ни о HTTP; зависят только от `app.core` (конфиги, часы). Хранение и права — в `app/services/*` , HTTP — в `app/api/*`
Все числа в YAML	Веса, пороги, квоты, справочники — `config/*.yaml` (8 файлов). В коде магических чисел нет. Версия конфига подбора (`scoring_config_version` = версия + sha256 содержимого) записывается в каждую версию подборки
Детерминизм	Всё случайное — через явный `random.Random(seed)` или `numpy.random.default_rng(seed)` . Время бизнес-логики — только через `app.core.clock.clock` (подменяется в тестах и в демо-режиме)
LLM вне оцениваемого контура	Выдача и ключи заданий, сложность `b` , грейд, Score, ранжирование и объяснения работают без LLM. LLM допустима только через шлюз `app/llm` с маскированием ПДн и детерминированным fallback (разбор брифа и резюме, только «добавляет» поля с цитатой)
Приватность по построению	Работодатель до взаимного «да» получает только слепую проекцию кандидата (`CandidateBlind`); контакты (`CandidateDisclosed`) — при действующей записи `ContactDisclosure` с отдельным согласием
Свой вход, внешний ФСП ID	Регистрация, пароли и сессии — в FastAPI. Keycloak (realm `fsp`) — только внешний провайдер «Войти через ФСП ID»

2. Функциональная архитектура

Три роли пользователей (`candidate` , `employer` , `methodist` ; плюс `admin`) и сквозная цепочка «опрос → тест → категория → подборка → приглашение → взаимное «да» → контакты».



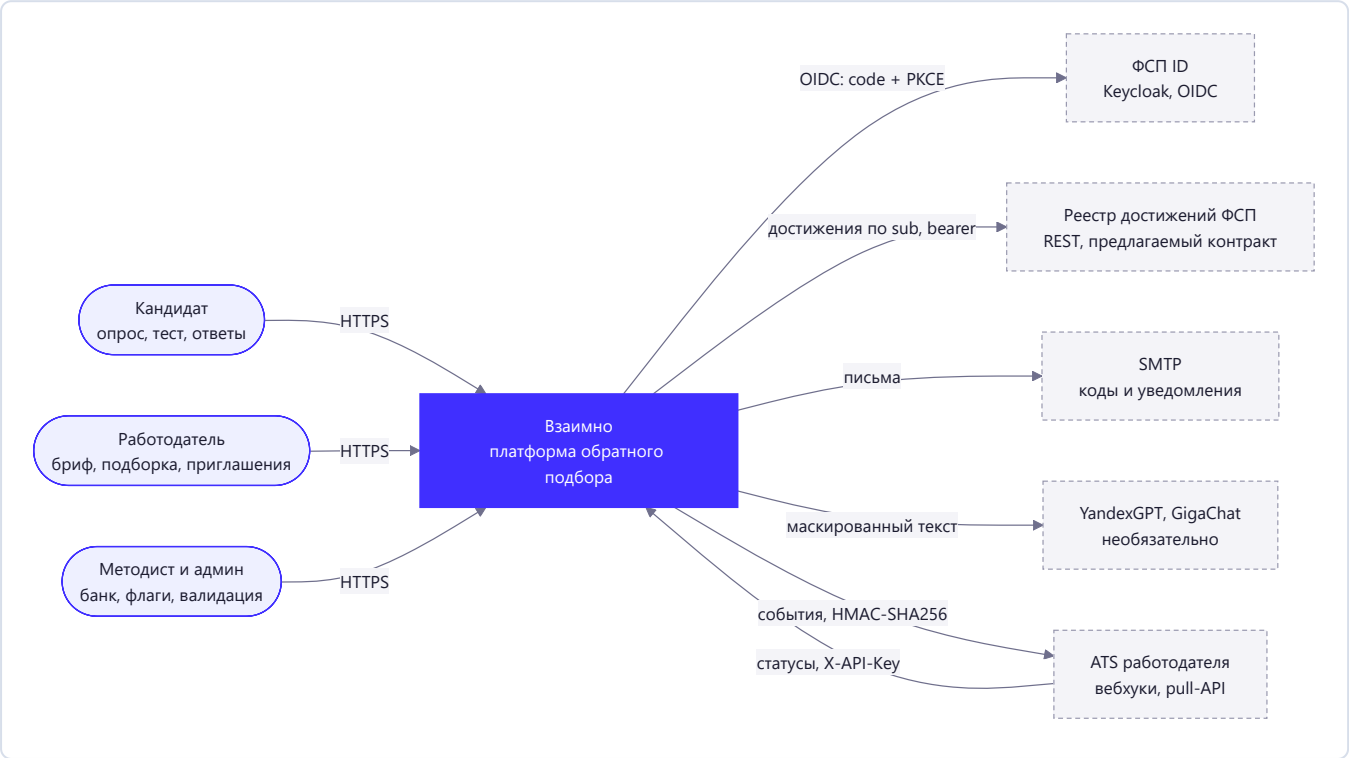
Функции по модулям (раздел «Как устроено» каждой механики — в отдельных документах):

Функция	Кандидат	Работодатель	Методист	Модуль и документ
Опрос и классификатор специализации	`/api/survey/*`	—	—	`app.testing.survey` , TESTING_MECHANISM.md

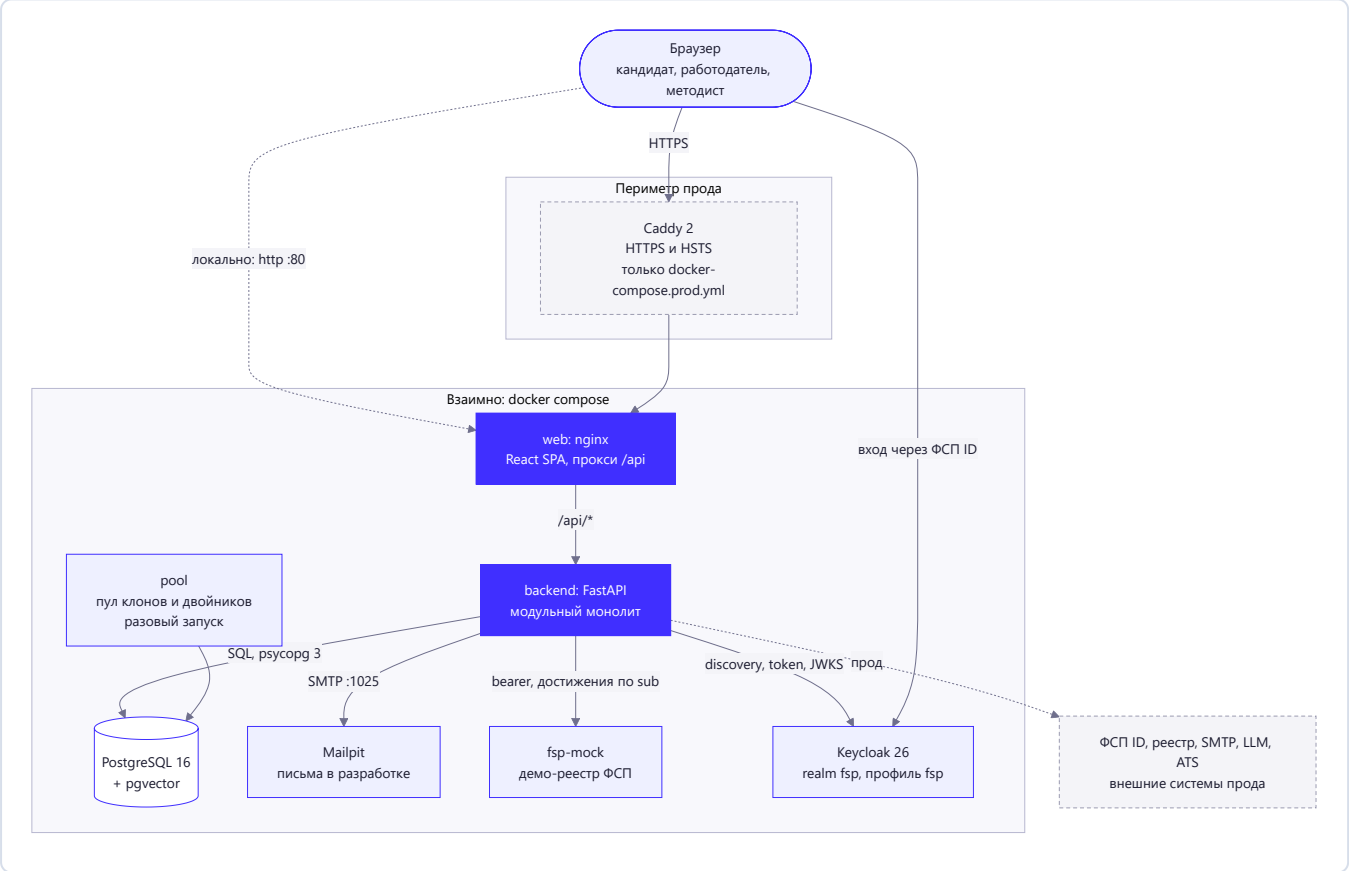
Функция	Кандидат	Работодатель	Методист	Модуль и документ
Адаптивный тест, результат, разбор, смена грейда	`/api/tests/*`	—	трасса, флаги, калибровка (`/api/lab/*`)	`app.testing` , `app.services.testing`
Защита от подсказок («Антиподсказчик»)	флаг видит и оспаривает	не видит	акт сверки	`app.testing.anticheat` , `app.services.anticheat` (TESTING_MECHANISM.md п. 9)
Сигнал интереса	отправляет и отзывает (`/api/interest`)	видит бейдж «Взаимный интерес» только у открытых карточек	—	`app.services.contact` , MATCHING.md п. 9
Цифровой паспорт навыков	выпускает, скачивает, отзывает (`/api/credentials/me`)	проверяет по QR `/v/код` , видит «Паспорт проверен»	—	`app.credentials` , CREDENTIALS.md
Качество заданий (DIF, дрейф, экспозиция)	—	—	`/api/lab/quality`	`app.testing.monitoring` , TESTING_MECHANISM.md п. 16
Панель ФСП «Кадровый резерв»	—	—	`/api/fsp/dashboard` (методист, админ)	`app.services.fsp_dashboard` , FSP_INTEGRATION.md п. 11
Разбор брифа, категории, охват	—	`/api/needs/*`	—	`app.matching` , MATCHING.md
Score, объяснения, версии подборки, банк	«как меня видят»	`/api/needs/*/versions` , `/api/bank`	—	`app.matching` , `app.services.matching`
Приглашение, «Да / Да, если… / Нет», раскрытие контактов, надёжность компании	`/api/invitations/inbox/*`	`/api/invitations/*`	—	`app.contact` , `app.services.contact`
Вакансии и отклики	`/api/vacancies` , `/api/applications`	`/api/vacancies` , `/api/applications`	—	`app.services.contact`
Регулярные задачи и «проверочный набор по вакансии»	`/api/tasks/me` , `/api/tasks/assignments/*`	`/api/tasks/*`	—	`app.services.tasks`
ФСП ID, достижения, VSS	`/api/fsp/*`	видит обобщённую строку	—	`app.fsp` , `app.services.fsp_link` , FSP_INTEGRATION.md
Резюме (PDF), PDF-профиль, сверка по QR	`/api/candidates/me/*` , `/api/v/{code}`	PDF раскрытого кандидата	—	`app.services.resume` , `app.services.pdf` , `app.services.profile`
Согласия, журнал, аудит	`/api/consents` , журнал	—	—	`app.core.consents` , `app.core.audit` , SECURITY.md
Уведомления, вебхуки ATS	`/api/notifications`	`/api/companies/me/webhooks`	—	`app.services.notify` , `app.services.contact.webhooks`
Интеграция с ATS (pull-API, статусы, адаптер «Хантфлоу», журнал), чат после «Взаимно»	`/api/ats/v1` , `/api/chat`	`/api/companies/me/ats*`	`company_api_keys` , `ats_settings` , `ats_log` , `chat_messages`	`app.services.ats` , `app.services.chat` — ATS_INTEGRATION.md
Валидация	—	—	`/api/lab/validation`	`backend/eval` , VALIDATION.md

3. Контекст и контейнеры (C4)

3.1. Уровень 1 — контекст



3.2. Уровень 2 — контейнеры

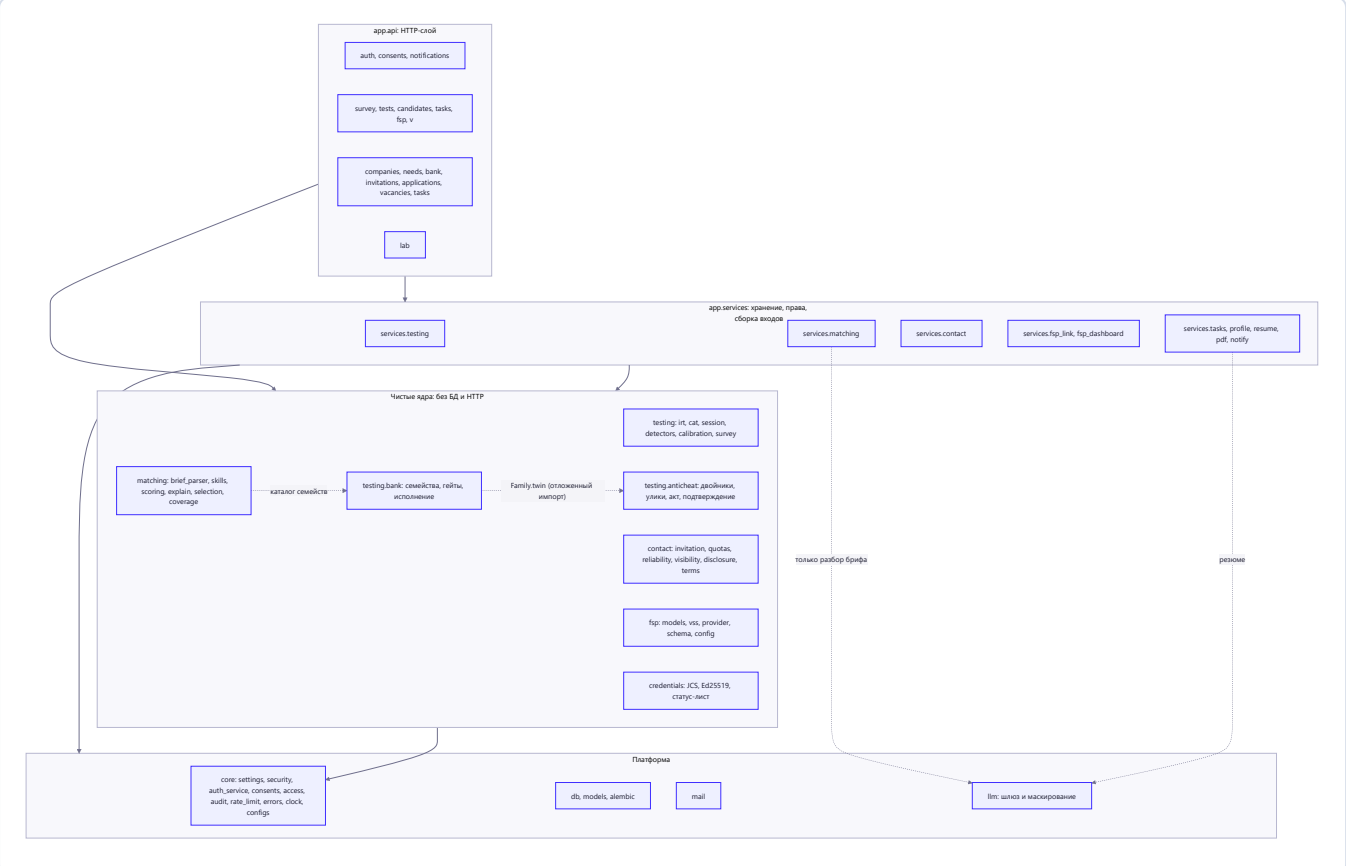


Сервисы `docker-compose.yml` (проект `vzaimno`):

Сервис	Образ / сборка	Порт	Лимит памяти	Назначение
`db`	`pgvector/pgvector:0.8.7-pg16`	5432	512 МБ	PostgreSQL 16; расширение `vector` создаёт миграция `0001`
`backend`	сборка `backend/Dockerfile` (python:3.12-slim, uv, непривилегированный пользователь)	8000	768 МБ	Миграции `alembic upgrade head`, затем `uvicorn app.main:app --no-proxy-headers` (клиентский IP определяет приложение)
`fsp-mock`	сборка `services/fsp-mock` (python:3.12-slim)	8001	256 МБ	Демо-реестр ФСП, 60 синтетических спортсменов
`mailpit`	`axllent/mailpit:v1.31.4`	8025 (веб), 1025 (SMTP)	128 МБ	Письма с кодом видны на http://localhost:8025
`keycloak`	`quay.io/keycloak/keycloak:26.0`, профиль `fsp`	8080	768 МБ	ФСП ID: realm `fsp` с 30 демо-пользователями
`web`	сборка `frontend/Dockerfile` (nginx со статикой Vite, SPA-fallback, прокси `/api` → `backend`)	80	128 МБ	Интерфейс на http://localhost. Для разработки — Vite dev-сервер :5173 (`npm run dev`). Caddy с HTTPS есть только в `docker-compose.prod.yml`: DEPLOY.md
`pool`	образ бэкенда, разовый запуск `scripts/generate_pool.py`	—	1 ГБ	Прегенерация пула клонов заданий и двойников; затем завершается

Демо не зависит от Keycloak и от `fsp-mock`: при `VZ_FSP_PROVIDER=auto` бэкенд берёт данные реестра из встроенного снимка (`app/fsp/data/demo_registry.json`), а вместо входа через Keycloak есть «демо-вход ФСП» с подписанным мок-токеном.

4. Модули монолита и их границы



4.1. Границы по пакетам

Граф зависимостей получен статическим разбором импортов `backend/app`. Ядра (`testing`, `matching`, `contact`, `fsp`) импортируют только `app.core` (и `testing.bank` у подбора для каталога семейств).

Модуль	Пакеты	Что внутри	Чего там нет	Конфиг
testing	`app/testing` (`irt.py`, `cat.py`, `session.py`, `detectors.py`, `calibration.py`, `survey.py`, `simulate.py`) и `app/testing/bank`	Модель ответа и EAP, выбор задания, стоп и решение, машины состояний сессии и квалификации, детекторы и CUSUM, калибровка, классификатор опроса. Банк: контракт `Family`, семейства `families/*`, CI-гейты, офлайн-исполнение ключей	БД, HTTP, LLM. Клон выбирает вызывающий сервис, движок решает «какое семейство и почему»	`testing.yaml`, `taxonomy.yaml`
anticheat	`app/testing/anticheat`	«Двойник в буфере», классификация ответов, агрегирование улик, акт сверки, «Подтверждение за 5 минут», песочница жюри	БД, HTTP, LLM, автоматический бан. Подключается к сервису теста через три хука (`app/services/testing/anticheat_hooks.py`)	`anticheat.yaml`
matching	`app/matching`	Разбор брифа, словарь навыков и граф «навык → модули», Score и его компоненты, объяснения, версии подборки и фильтры, охват и рекомендации категорий, планировщик «проверочного набора»	БД, HTTP; LLM в контур Score не импортируется (проверяет тест `test_ranking_path_never_imports_llm`)	`scoring.yaml`, `skills.yaml`, `fsp.yaml`, `taxonomy.yaml`
contact	`app/contact`	Машина состояний приглашения (таблица `(from, action, actor) → to`), квоты и паузы, надёжность компании R, матрица видимости (`CandidateBlind` / `CandidateDisclosed`), псевдонимы, раскрытие, антифейк-правила, рабочие дни	БД, HTTP	`contact.yaml`
fsp	`app/fsp`	Контракт реестра (`models.py`), расчёт VSS, провайдеры (`HttpFSPProvider`, `InProcessFSPProvider`, `FallbackFSPProvider`), нормализация в схему платформы	Сеть кроме `HttpFSPProvider`; ничего не знает о подборе и Score	`fsp.yaml`
platform	`app/core`, `app/db`, `app/models`, `app/mail`, `app/llm`, `app/api/deps.py`	Настройки `VZ_*`, argon2id и JWT, регистрация и сессии, журнал согласий, контроль владельца записи (IDOR), аудит, rate limit, единый формат ошибок, `Clock`, SMTP, шлюз LLM	Продуктовые правила (они в YAML и в ядрах)	`.env` (`.env.example`)
services	`app/services/*`	Адаптеры «ядро ↔ БД ↔ права»: `testing` (сессии, пул клонов, шифрование ключей, детекторы на данных БД, лаборатория), `matching` (кэш признаков, версии, приватность), `contact`, `fsp_link` (OIDC и синхронизация), `tasks`, `profile`, `resume`, `pdf`, `notify`, `demo` (демо-мир, автоответчик, импорт жюри)	Алгоритмы	—
api	`app/api/*`	19 роутеров под префиксом `/api`, зависимости (текущий пользователь, роли, лимиты), DTO	Бизнес-правил: роутер проверяет роль и владельца, ограничивает частоту и отдаёт DTO	—

4.2. Инварианты, которые проверяют тесты

Инвариант	Где проверяется
LLM не импортируется модулями Score, ранжирования и объяснений	`tests/test_api_needs_review.py::test_ranking_path_never_imports_llm` ; в `app/testing` и `app/services/testing` импортов `app.llm` нет
До взаимного «да» в ответах работодателю нет ни одного ПДн-ключа на любой глубине	`tests/test_contact_privacy.py` , `tests/test_api_contact_privacy.py` , `tests/test_api_needs_privacy.py`
Сумма вкладов компонентов равна Score (± 0,1), 3 контрольных примера Score	`tests/test_matching_scoring.py` , `tests/test_matching_explain.py`
Запись другого пользователя неотличима от несуществующей (404, не 403)	`tests/test_platform_access.py`
Контракт реестра в бэкенде и в `fsp-mock` совпадает байт-в-байт	`tests/test_fsp_mock.py`
Клоны: 200 экземпляров без исключений, детерминизм по seed, ≥ 95 % различных ключей, дистракторы ≠ ключу	`tests/test_bank_gates.py` , `tests/test_bank_*.py`

Всего в `backend/tests` собирается 2297 тестов (`uv run pytest --collect-only -q`). Для тестов API нужен PostgreSQL (`VZ_TEST_DATABASE_URL`).

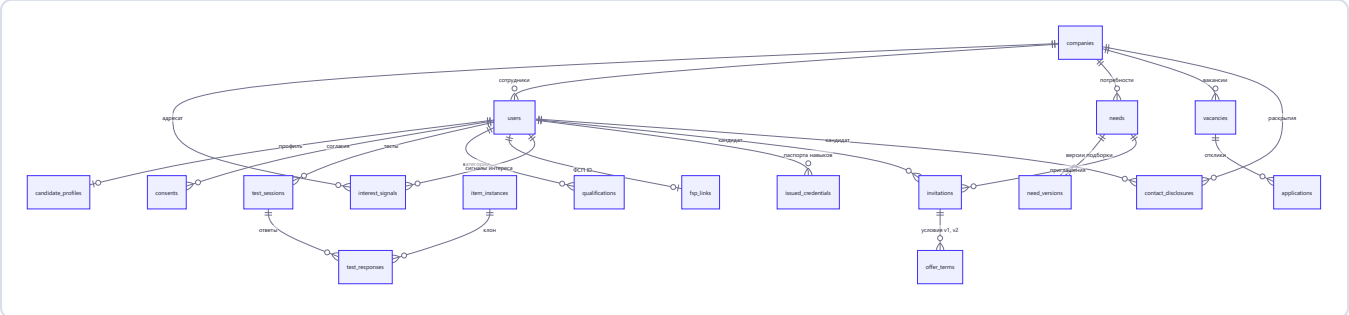
5. Данные (ERD)

Хранилище — PostgreSQL 16. Статусы хранятся строками (`VARCHAR`), не `ENUM` -типами PostgreSQL: новое значение не требует миграции типа. Жёсткие `CHECK` — только там, где ошибка опасна: роль и статус пользователя, цель согласия, статус тестовой сессии, `salary_to >= salary_from` , `email = lower(email)` . Уникальные частичные индексы обеспечивают правила на уровне БД: «одна активная тестовая сессия на аккаунт» (`uq_test_sessions_one_active`), «одно активное приглашение на пару компания–кандидат» (`uq_invitations_active_pair`), «одна корпоративная компания на домен» (`uq_companies_corporate_domain`). Миграции Alembic: `0001` – `0006` (схема, статусы, тестирование, подбор, контакт, профиль и ФСП) и `0007_demo` (таблицы демо-режима и импорта жюри: `demo_personas` , `demo_state` , `jury_batches` , `jury_candidates` , `jury_briefs` , `jury_pairs`).

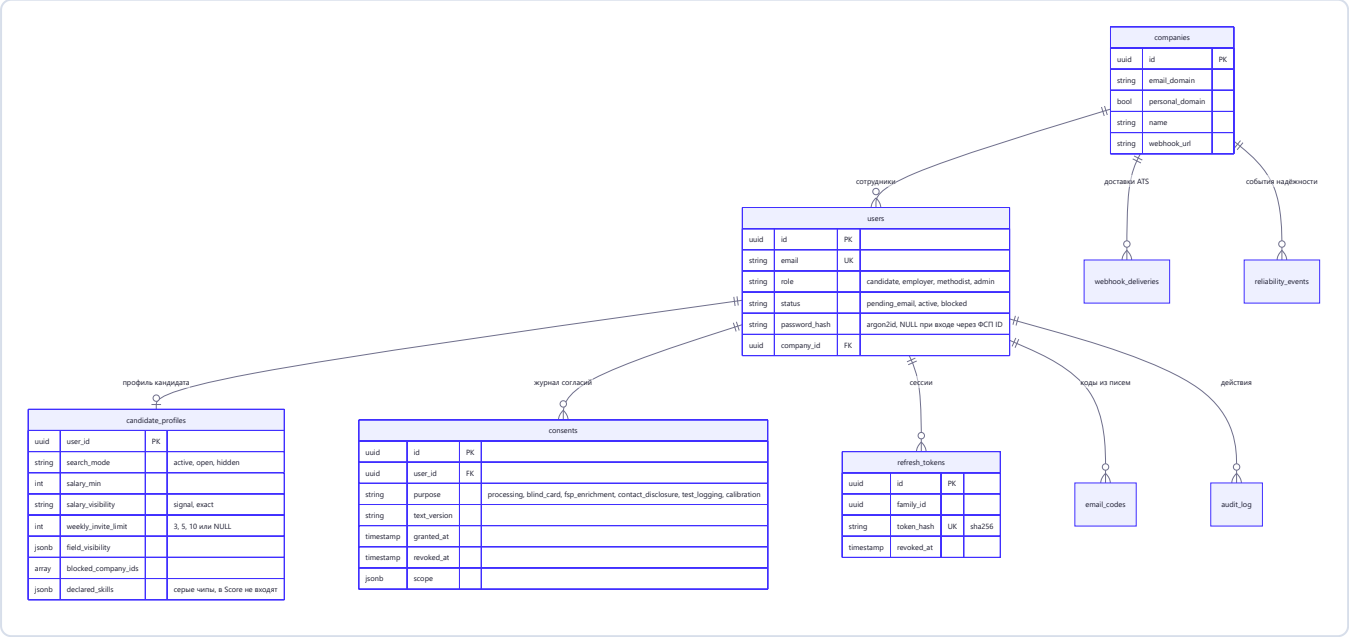
Расширение `pgvector` включено в `0001` , но колонок типа `vector` пока нет: эмбеддинги в рантайме не используются (член `cos(emb, proto)` в разборе брифа по умолчанию равен 0). Это «в разработке».

5.0. Обзор: ключевые сущности

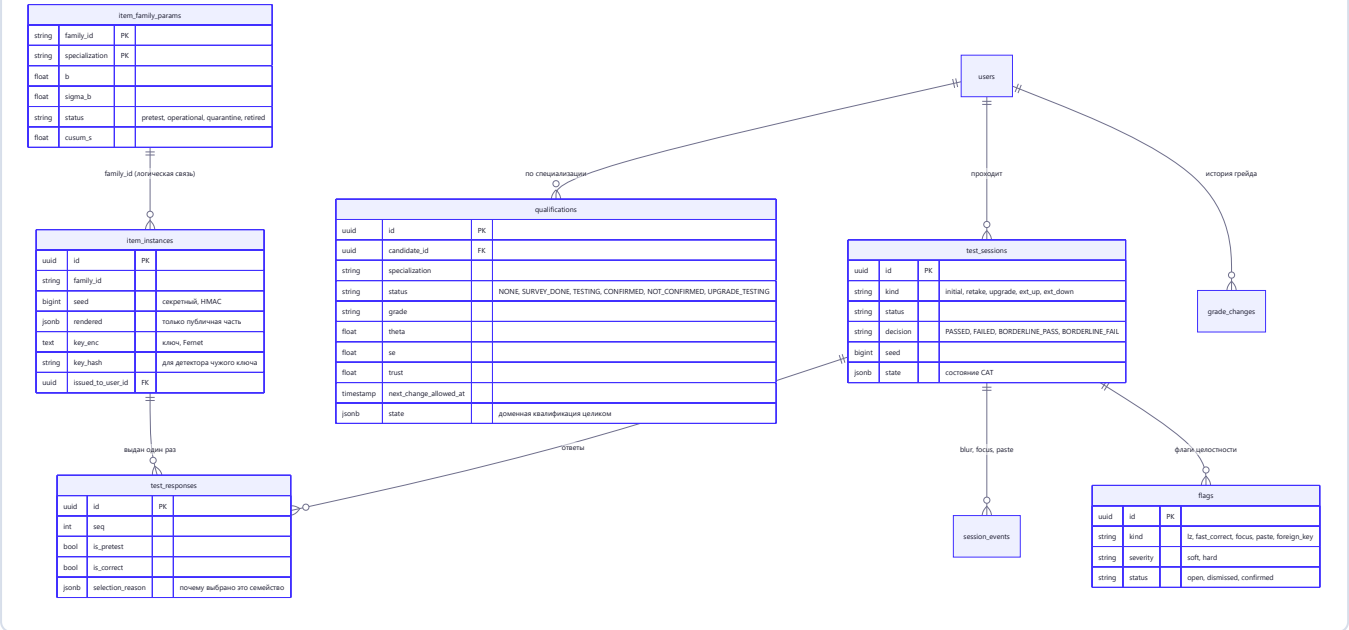
Ниже только основные таблицы и связи; полные ERD по областям — в п. 5.1–5.4.



5.1. Пользователи, согласия, компании

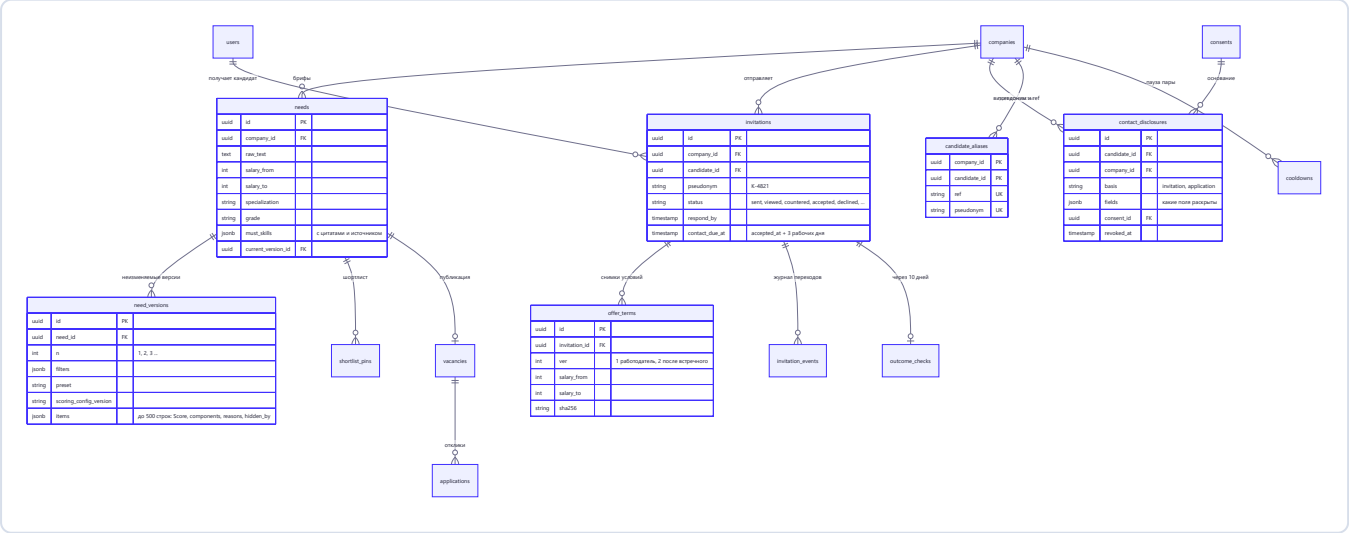


5.2. Тестирование



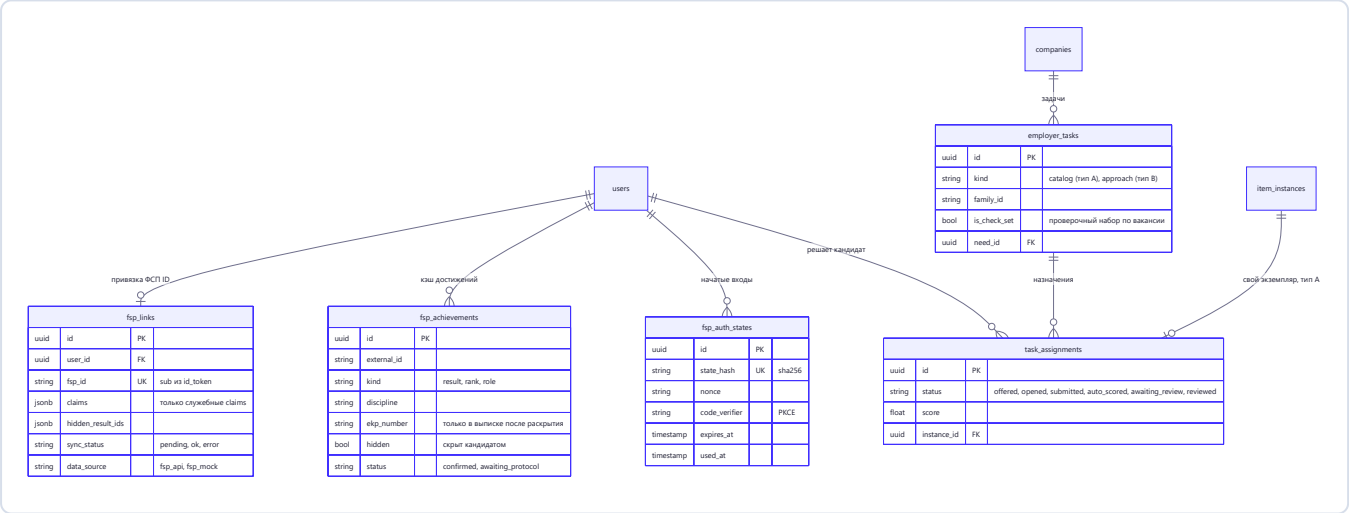
Дополнительно: `drift_events` (срабатывания CUSUM), `calibration_runs` (прогоны MAP-уточнения `b`), `validation_runs` (результаты `eval.validate` для экрана «Лаборатория»), `survey_responses` (ответы опроса с версией). Таблица `candidate_features` — ранняя схема материализованных признаков; подбор читает и пересобирает `matching_features` (п. 5.3).

5.3. Подбор и контакт



Служебные таблицы подбора: `matching_features` (кэш признаков кандидата по категории, `visible` — виден ли в банке), `profile_views` (кто и когда открыл карточку), `impression_log` (показы в подборках для «как меня видят»). Контакт: `application_events`, `feedback`, `notifications`, `reliability_snapshots`.

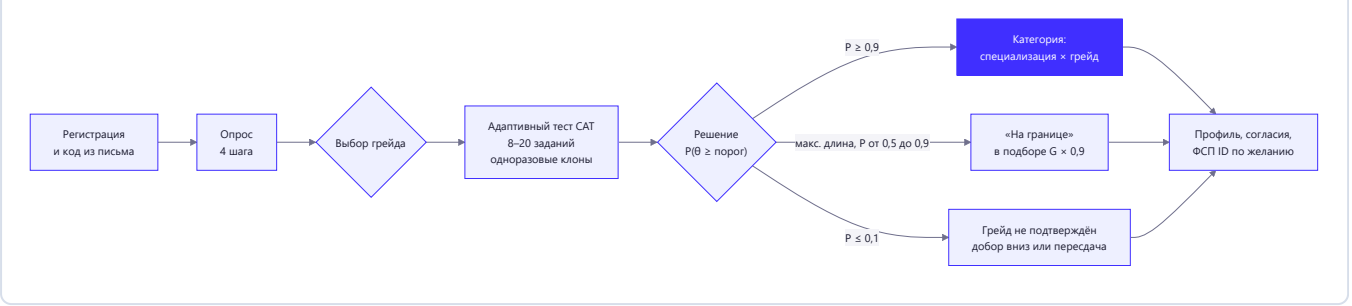
5.4. ФСП и регулярные задачи



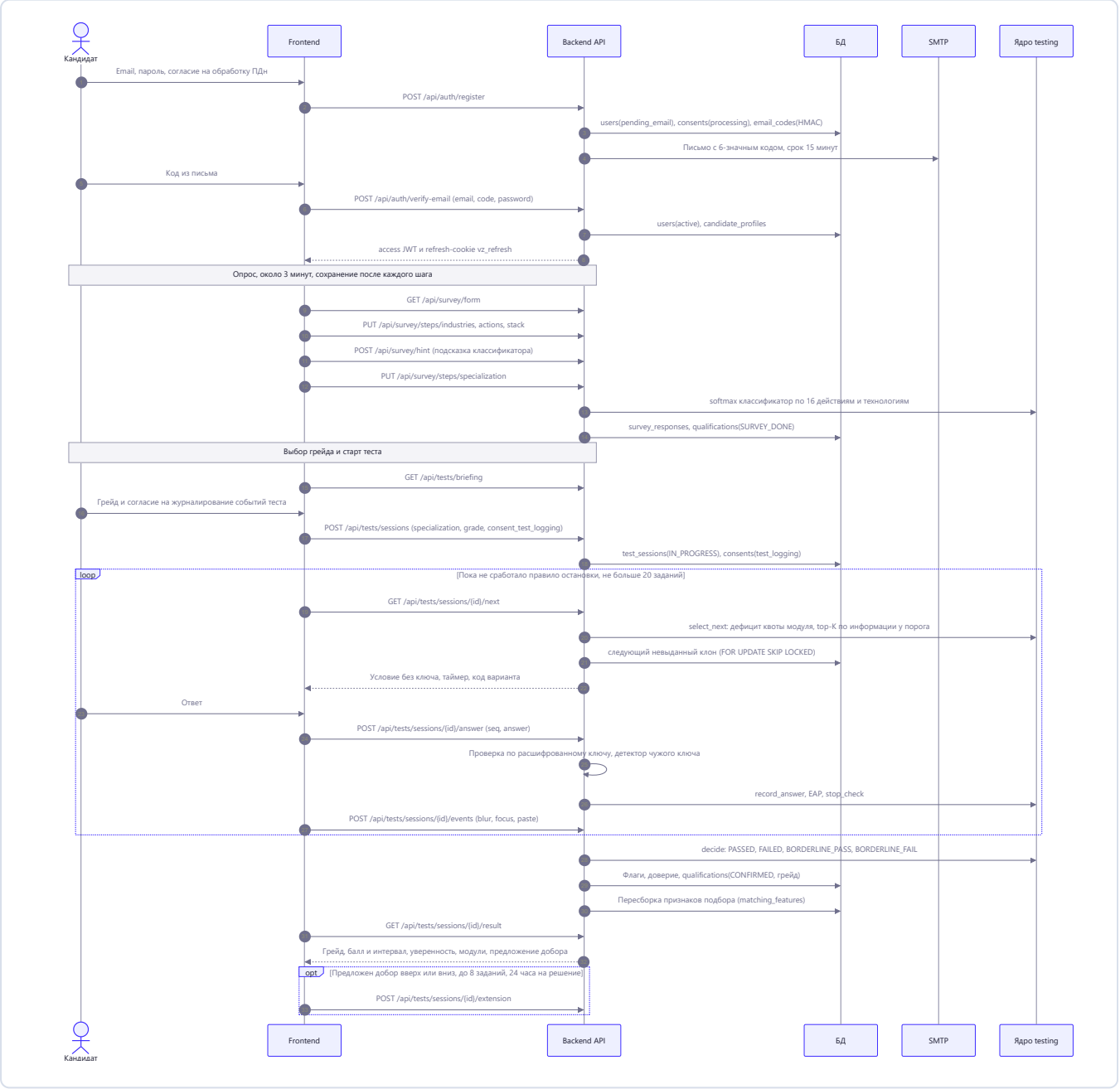
6. Сквозные сценарии (sequence)

Эндпоинты и поля соответствуют OpenAPI (`/api/openapi.json`). Условные обозначения: БД — PostgreSQL; ядро — чистые функции `app.testing`, `app.matching`, `app.contact`, `app.fsp`.

6.1. Путь кандидата: опрос → грейд → CAT → категория

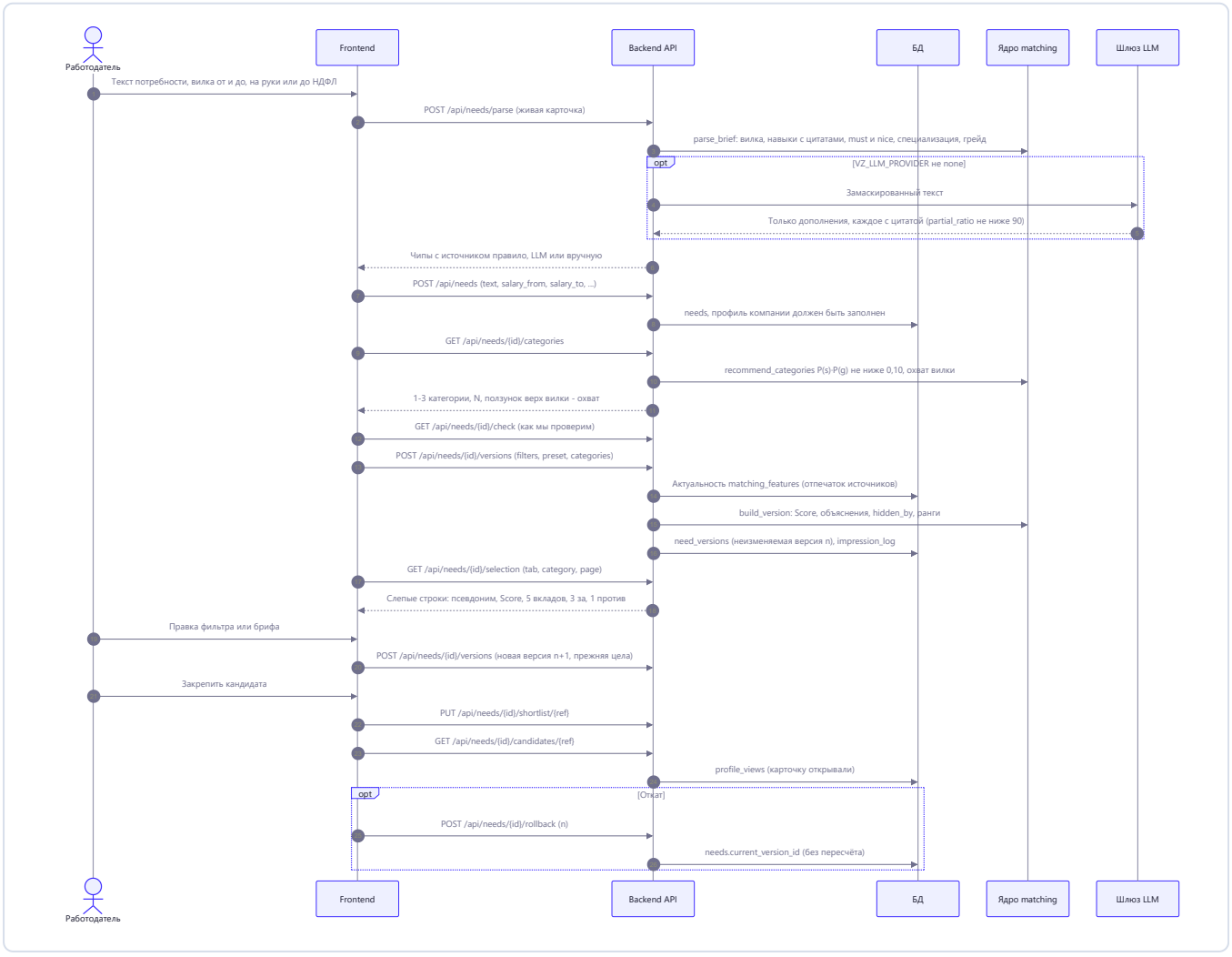


Последовательность вызовов:

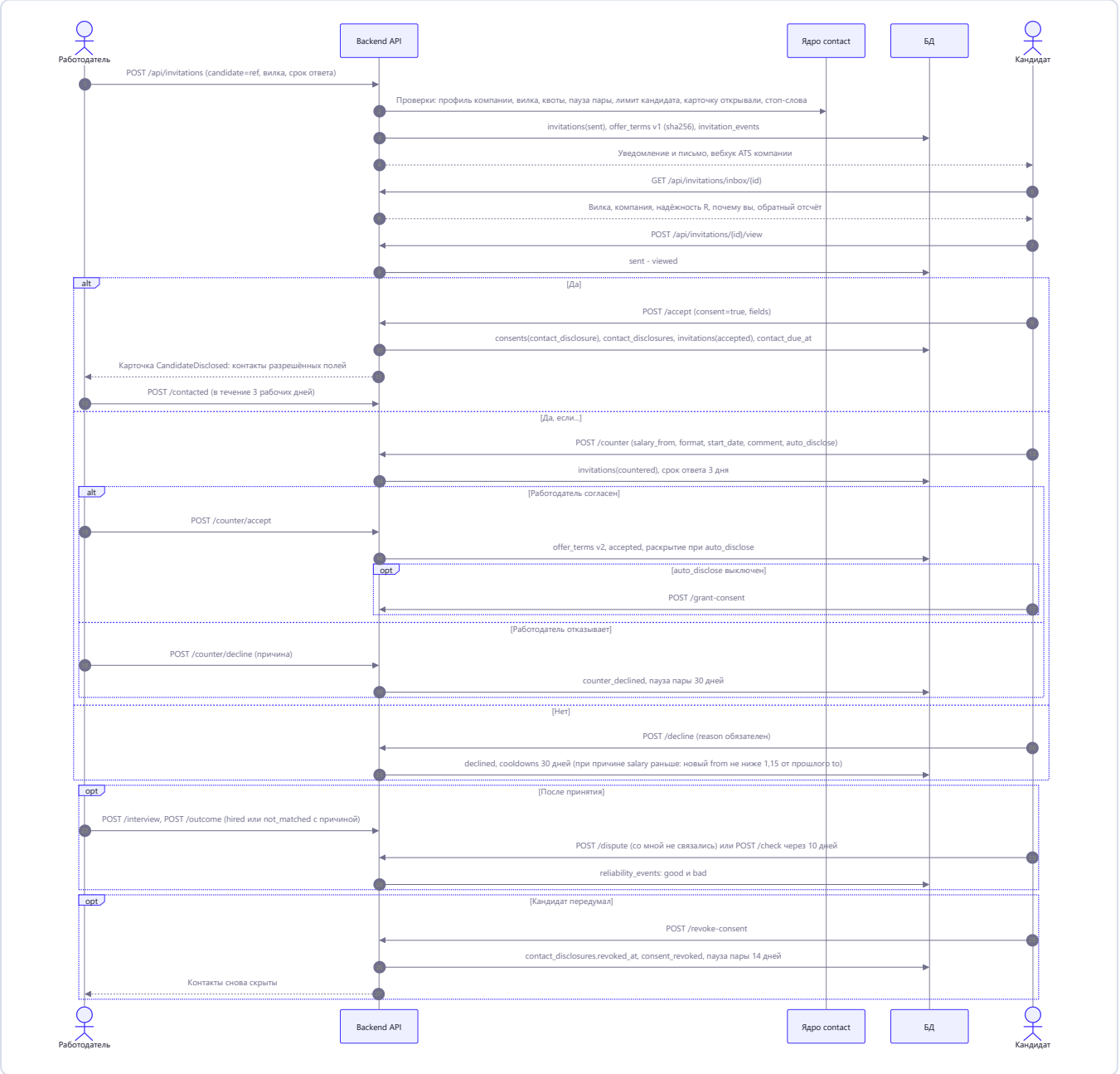


После `CONFIRMED` кандидат виден работодателям (`matching_features.visible`) только при выполнении всех условий: аккаунт активен, есть согласие `blind_card` , режим поиска не «не ищу», сессия не на проверке у методиста, возраст не меньше 16 лет.

6.2. Подбор: бриф → категории → версия подборки



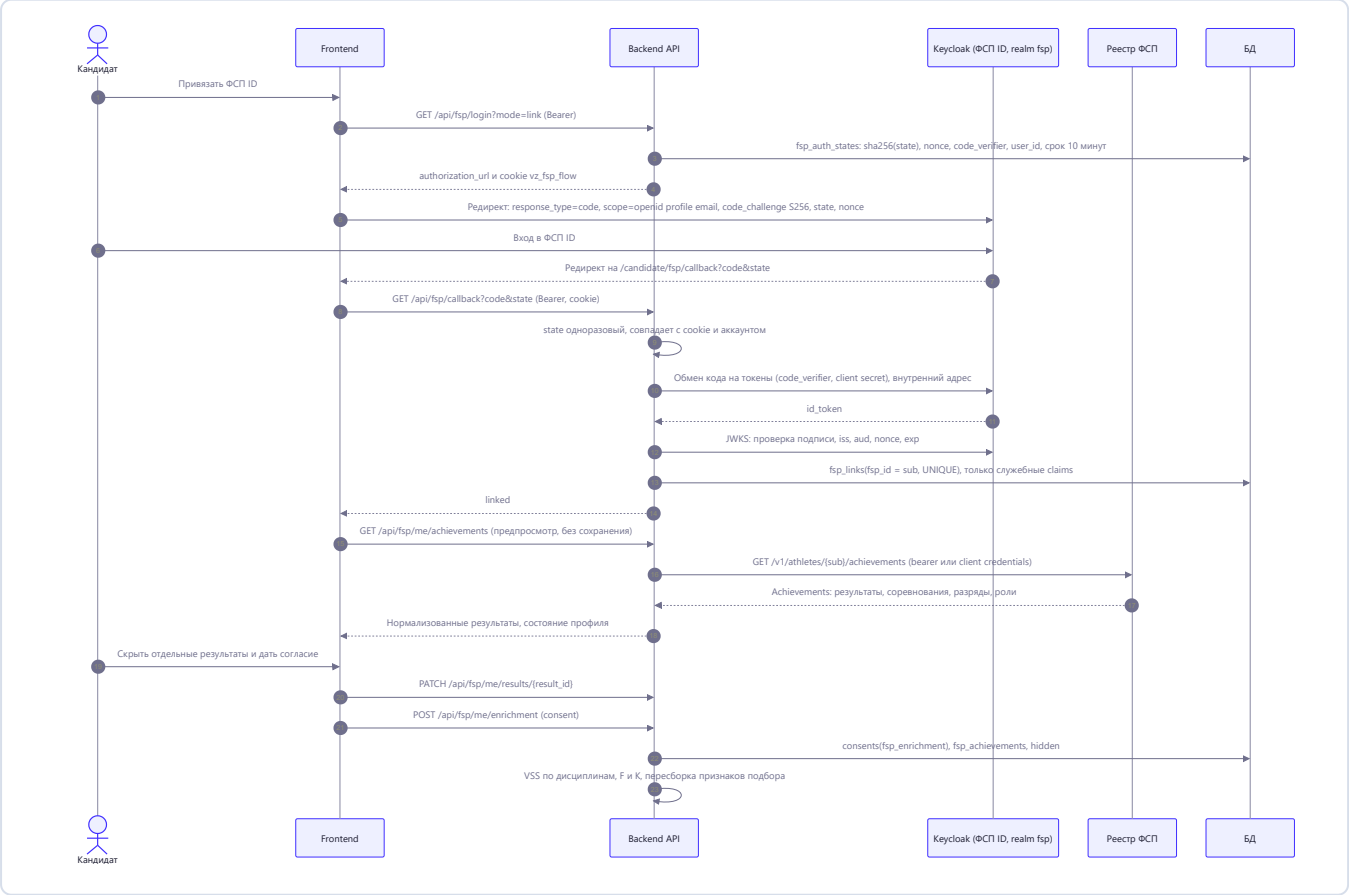
6.3. Приглашение: «Да / Да, если… / Нет» и раскрытие контактов



Пути в блоках — `/api/invitations/{id}/…`. Сроки применяются лениво при чтении (источник истины); `sweep()` лишь делает то же для всех сразу и повторяет неудачные вебхуки.

Обещание процесса. В приглашении работодатель может отметить обязательства по найму (`config/contact.yaml → invitation.promises`: «без технического скрининга», «сразу финальное интервью», «без тестового задания»). Они хранятся флагами приглашения `promise:<код>` — условия и их sha256 не меняются, в Score кандидата обещания не входят. Кандидат видит их чипами до ответа; если в сверке «условия совпали?» он выбирает «Обещание по процессу не выполнено» (`promise_broken`), в журнал надёжности компании пишется `terms_promise_broken` — минус в доле «условия совпали», как и «зарплата ниже» или «вакансии нет».

6.4. Привязка ФСП ID: OIDC + PKCE → реестр

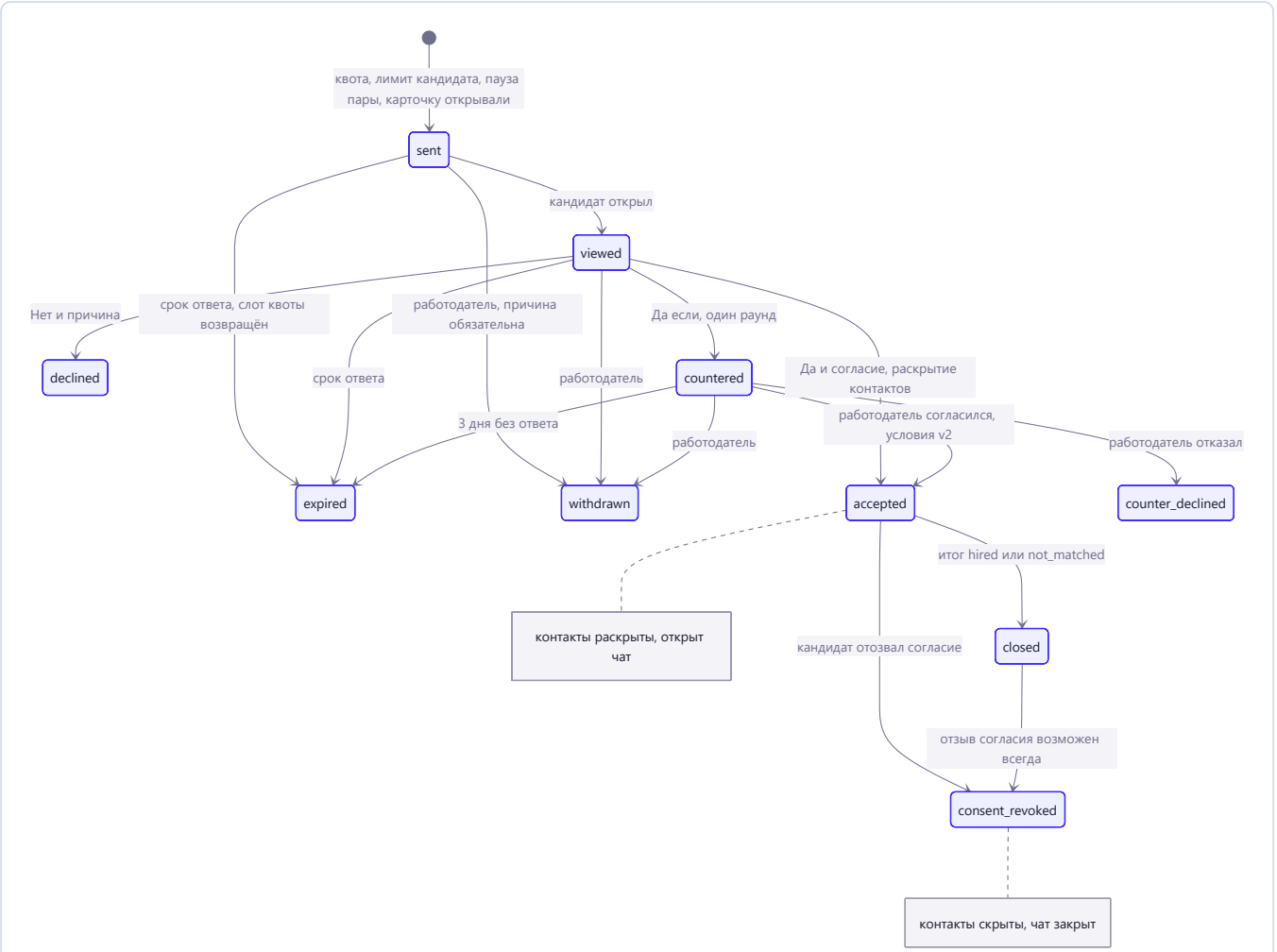


Подробности контракта реестра, расчёта VSS и перехода на настоящий ФСП ID — в FSP_INTEGRATION.md.

7. Машины состояний

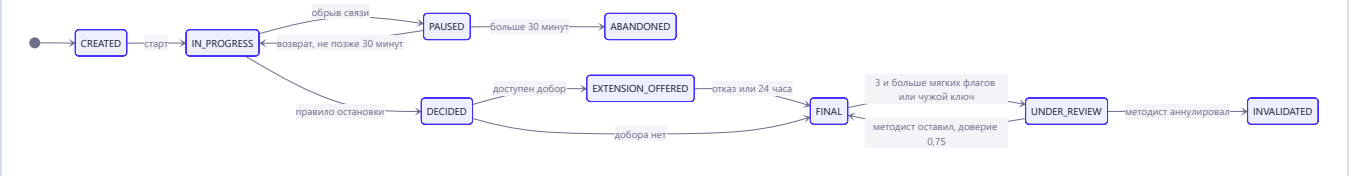
7.1. Приглашение

Таблица переходов `(from, action, actor) → (to, guards, side_effects)` лежит в `app/contact/invitation.py` (`TRANSITIONS`); каждый переход пишется в `invitation_events` . Подписи статусов одинаковы у обеих сторон (`config/contact.yaml → status_labels`).



В `accepted` и `closed` есть «петли» без смены статуса: «связались», «со мной не связались», «условия совпали?», «собеседование», «согласие после встречного».

7.2. Тестовая сессия и квалификация



Квалификация (кандидат × специализация): `NONE → SURVEY_DONE → TESTING → CONFIRMED(g) | NOT_CONFIRMED`, далее `CONFIRMED(g) → UPGRADE_TESTING → CONFIRMED(g+1) | CONFIRMED(g)`. Принудительного понижения нет. Правила попыток и смены грейда — в TESTING_MECHANISM.md.

8. Сквозные механизмы

Механизм	Реализация
Конфигурация	`app/core/configs.py` читает `config/*.yaml` (каталог переопределяется `VZ_CONFIG_DIR`); инфраструктура и секреты — `VZ_*` из окружения (`app/core/settings.py`, `.env.example`). В `VZ_ENV=prod` бэкенд не стартует с dev-секретами, секретами короче 32 символов и без Secure-cookie
Время	`app.core.clock.clock`: заморозка в тестах, смещение «+1 день» в демо-режиме (смещение хранится в `demo_clock` и переживает перезапуск). Подпись access-JWT — по настенным часам

Механизм	Реализация
Сроки и планировщик	Сроки приглашений, откликов и пауз применяются лениво при чтении — это источник истины. **Планировщик** (`app/services/scheduler` , `config/scheduler.yaml`): asyncio-цикл в lifespan приложения (в тестах не запускается, `VZ_SCHEDULER_ENABLED=false` выключает) раз в минуту вызывает `services.contact.sweep()` (напоминания, «истекло», «нет ответа», повтор вебхуков ATS), ночью (00:00 UTC) — калибровку заданий с CUSUM дрейфа (то же, что `POST /api/lab/calibrate`), в 00:30 UTC — чистку по срокам хранения (`config/privacy.yaml`). Между процессами задачу берёт держатель рекомендательной блокировки PostgreSQL; итог суточных задач пишется в `audit_log` (`scheduler.*`) — после перезапуска в ту же ночь повторного прогона нет
Фоновые задачи	`BackgroundTasks` FastAPI: долив пула клонов после выдачи задания (`pool.top_up_background`); периодические задачи — планировщик (строка выше). Отдельной очереди задач нет
Ошибки	Единый формат `{code, message, details}` ; сообщения на русском; 500 перехватывается внутри CORS, чтобы фронтенд мог прочитать тело (см. API.md)
Аудит и согласия	`audit_log` (вход, токены, согласия, раскрытия; без ПДн в payload), журнал `consents` с версией текста и отзывом без удаления
Rate limit	Токен-корзина в памяти процесса (`app/core/rate_limit.py`); для нескольких воркеров корзины нужно вынести в Redis или PostgreSQL. Клиентский IP за Caddy — `X-Forwarded-For` только от доверенных прокси (`VZ_FORWARDED_ALLOW_IPS` , `app/core/netutil.py`)
Удаление аккаунта и сроки хранения	`DELETE /api/candidates/me` , `DELETE /api/companies/me` (`app/services/account/erasure.py`): строка `users` удаляется, каскады БД убирают всё персональное; ночная чистка по срокам (`retention.py` , `config/privacy.yaml`) — подробности в SECURITY.md п. 1.3
Почта	`app/mail` : SMTP (STARTTLS или SSL) или консоль; сбой SMTP не ломает действие пользователя (`send_mail_safe`)
Вебхуки ATS	HMAC-SHA256 в `X-Vzaimno-Signature` , идемпотентный `X-Vzaimno-Event-Id` , повторы с экспоненциальной паузой, защита от SSRF (внутренние адреса запрещены), журнал последних 20 доставок
Метаданные безопасности HTTP	`X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , `Referrer-Policy` , `Cache-Control: no-store` на `/api/auth/*` ; HSTS и TLS — на уровне Caddy

Производительность (замер на синтетической популяции, VALIDATION.md §4.7): сборка версии подборки на пуле 1 864 кандидатов — медиана 83 мс, p95 156 мс при цели ≤ 300 мс. Время подбора растёт линейно с размером пула; кэш признаков (`matching_features`) пересобирается по хукам и по отпечатку источников.

9. Статус реализации

Область	Статус
Бэкенд: аутентификация, согласия, опрос, тестирование, банк (111 семейств в 14 модулях: Backend, Frontend, QA, DevOps и 11 семейств навыков для проверочных наборов), подбор, контакт, ФСП ID и реестр, профиль и резюме, PDF-профиль со сверкой, регулярные задачи, уведомления, вебхуки, лаборатория, валидация	Готово, покрыто тестами
Контракт реестра ФСП, `fsp-mock` , realm Keycloak `fsp`	Готово
Фронтенд	Готово: кандидат (опрос, брифинг, тест, результат, «Мой грейд», приглашения, ФСП, приватность, профиль и PDF), работодатель (потребности, подборка, приглашения, банк, вакансии, задачи, профиль компании), лаборатория, страница жюри и песочница; образ `web` (nginx)
«Антиподсказчик»: двойники, улики, акт сверки, «Подтверждение за 5 минут», песочница	Готово: библиотека `app/testing/anticheat` , сервис `app/services/anticheat` , хуки `anticheat_hooks.py` подключены к выдаче заданий и ответам, клиентский обработчик буфера (`frontend/src/features/anticheat`), лента сверок и акты в лаборатории, песочница жюри

Область	Статус
Банки DevOps и QA	Готово: обе специализации `active` (модули `devops_core` 11 и `qa_testing` 11 семейств плюс общие `git_linux`, `http_api`, `sql`); банк меньше, чем у Backend и Frontend
`/api/admin/*` (импорт данных жюри, ранжирование с компонентами Score, метрики по размеченным парам, валидация) и `/api/demo/*` (часы «+1 день», сброс, автоответчик, режим жюри, приёмник вебхуков; только при `VZ_DEMO_MODE=true`)	Готово; наполнение демо-мира — `make seed` (`backend/scripts/seed_demo.py`)
«Сигнал интереса» и симуляция рынка	Готово: `/api/interest`, бейдж «Взаимный интерес», квоты и недельные лимиты из `config/contact.yaml`; симуляция `eval/market_sim.py` (MATCHING.md п. 9)
Цифровой паспорт навыков	Готово: подписанный Verifiable Credential (Ed25519, `did:web`), QR `/v/код`, отзыв по Bitstring Status List; издатель демонстрационный (CREDENTIALS.md)
Качество заданий: DIF, дрейф, экспозиция	Готово: `app/testing/monitoring.py`, экран «Лаборатория → Качество» (`/api/lab/quality`), `eval.fairness` (TESTING_MECHANISM.md п. 16)
Панель ФСП «Кадровый резерв»	Готово: `/fsp/reserve`, `/api/fsp/dashboard`, k-анонимность и вторичное подавление (FSP_INTEGRATION.md п. 11)
Задания на конкретные навыки в проверочном наборе вакансии; открытые данные рынка	Готово (MATCHING.md, «Навыки, наблюдаемые через проверочный набор» и п. 13)
Вакансия без брифа, банк кандидатов без брифа	Готово: «Создать вакансию» и `GET /api/bank`
Инженерия: CI, e2e, типы, безопасность; доступность	Готово: `.github/workflows/ci.yml`, Playwright, `mypy --strict` на ядрах (ENGINEERING.md); axe: 0 serious и critical (ACCESSIBILITY.md)
Фабрика семейств (LLM пишет черновик генератора офлайн)	В разработке
Эмбеддинги и колонки `vector`	В разработке
ATS: исходящие вебхуки, pull-API, обратная синхронизация статусов, адаптер «Хантфлоу» (эмулятор), чат после «Взаимно»	Готово (ATS_INTEGRATION.md); боевой режим Хантфлоу на живом сервисе не проверялся
Ночная калибровка с CUSUM и периодический `sweep()`	Готово: встроенный планировщик (`app/services/scheduler`, `config/scheduler.yaml`); ручной запуск остался
Удаление аккаунта, сроки хранения ПДн, минимизация ответа реестра ФСП	Готово (`app/services/account`, `config/privacy.yaml`); согласие законного представителя 15–17 лет — письмо с одноразовой ссылкой и подтверждение (SECURITY.md п. 1.4)
Caddy и HTTPS	Готово в `docker-compose.prod.yml` и `deploy/Caddyfile` (в локальный `docker-compose.yml` не входят) — DEPLOY.md

Механика тестирования

Как «Взаимно» определяет уровень кандидата и почему этому уровню можно доверять, даже если задания утекают. Числа параметров — из `config/testing.yaml`, `config/taxonomy.yaml`, `config/anticheat.yaml`; результаты симуляций — из VALIDATION.md. Всё, что не построено, помечено «в разработке».

Содержание: 1. Идея · 2. Опрос · 3. Банк: семейства и клоны · 4. Модель и оценка уровня · 5. Калибровка · 6. Адаптивный тест · 7. Смена грейда · 8. Устойчивость к утечкам · 9. Антиподсказчик · 10. Доверие · 11. Регулярные задания · 12. Вакансия → проверочный набор · 13. LLM пишет генератор · 14. Ограничения

1. Идея в одном абзаце

У каждой тестируемой специализации одна шкала уровня θ («линейка») с порогами грейдов. Задание в банке — не текст, а **семейство**: шаблон, генератор параметров, эталонный решатель и генератор дистракторов. Из семейства порождаются **одноразовые клоны**: у каждого свой код и числа и **свой ключ, полученный исполнением**. Тест адаптивный: следующее семейство выбирается по максимуму информации у порога заявленного грейда с соблюдением квот модулей. Слитый ответ бесполезен: он верен для чужого клона, а ввод чужого ключа сам становится уликой. Код генераторов открыт: стойкость не держится на секретности банка (принцип Керкгоффса) — секретны только seed клонов и ключи в БД.

Цепочка: опрос → выбор грейда → тест → решение → категория «специализация × грейд».

Что тестируется	Статус
Backend (Python): `python_core`, `sql`, `http_api`, `algo`, `git_linux`	Работает
Frontend (JS/TS): `js_core`, `react`, `html_css`, `http_api`, `algo`	Работает
Fullstack	Производная категория: подтверждены обе квалификации, грейд = меньший из двух
QA, DevOps	`active`: тест работает (модули `qa_testing` и `devops_core`), направления включены в `taxonomy.yaml` (DECISIONS 28)
Data/ML, Mobile, ИБ, Backend (Go), Backend (Java/Kotlin)	`survey_only`: опрос есть, теста нет; в дорожной карте

2. Опрос и классификатор специализации

Опрос — 4 шага (`/api/survey/steps/industries | actions | stack | specialization`), около 3 минут, сохранение после каждого шага. Классификатор детерминированный (`app/testing/survey.py`, веса — `config/testing.yaml → survey`, 16 действий «что делали руками за 12 месяцев» и 19 технологий):

```
s_k = Σ_a W[a,k]·freq_a + 1,2·Σ_t V[t,k]·recency_t + 0,8·[выбор = k];     p = softmax(s / 0,5)
```

Если `max p − p(выбор) ≥ 0,2`, показывается подсказка «по ответам вы ближе к …»; решает кандидат. Технологии и резюме дают только серые чипы «заявлено» и **в оценку уровня не входят**. Грейд выбирает сам кандидат (карточки с поведенческими якорями), стаж даёт необязывающую подсказку; при Senior и стаже меньше 24 месяцев — мягкое предупреждение.

3. Банк: семейства и одноразовые клоны

3.1. Контракт семейства (`app/testing/bank/base.py`)

Семейство `Family` задаёт: `id` , `title` , `specs` (общие модули принадлежат нескольким шкалам), `module` , `format` , `target_grade` и `position` (экспертная оценка уровня, см. п. 5), `c` (вероятность угадывания), `time_limit_s` , `enemies` (семейства, подсказывающие друг другу ответ — в одном тесте не встречаются), `is_anchor` и метод `generate(seed) → Instance` . Экземпляр `Instance` содержит публичную часть `Rendered` (условие, код, варианты — **без ключа и тегов дистракторов**), ключ, словарь «неверный ответ → тег заблуждения» и разбор для кандидата после теста.

Форматы (проверяются автоматически, **код кандидата не исполняется**): `predict_output` (что выведет код), `bug_line` (строка с ошибкой и исправление), `sql_result` (результат запроса по показанной таблице), `parsons` (упорядочивание), `complexity` (оценка сложности), `mcq_concept` (концептуальный выбор, не больше 30 % теста).

3.2. Состав банка (Backend, Frontend, QA, DevOps)

111 семейств в четырнадцати модулях: 100 семейств в десяти модулях стандартного теста (в их числе 9 JS-версий общих семейств «Алгоритмы» и `http.ratelimit.token_bucket` для Frontend) и 11 семейств в четырёх модулях проверочного набора по вакансии (таблица ниже); общие модули (`http_api` , `algo` , `sql` , `git_linux`) входят в шкалы нескольких специализаций, и их сложность `b` хранится отдельно на шкале каждой. Специализации с тестом (`status: active` в `config/taxonomy.yaml`): Backend (Python), Frontend (JS/TS), QA и DevOps; банки QA и DevOps (по 11 собственных семейств `qa_testing` и `devops_core`) меньше, чем у Backend и Frontend.

Модуль	Семейств		Формат	Семейств
`python_core`	14		`predict_output`	73
`js_core`	12		`sql_result`	9
`sql`	9		`bug_line`	7
`react`	8		`mcq_concept`	5
`http_api`	9		`parsons`	3
`algo`	16		`complexity`	3
`git_linux`	5			
`html_css`	5			
`devops_core`	11			
`qa_testing`	11			
`redis_core` , `mq_core` , `pg_core` , `fastapi_core`	3 + 3 + 3 + 2			

По экспертному уровню: Стажёр 14, Junior 30, Middle 30, Senior 26 (100 семейств стандартного теста; 11 семейств набора — Junior 2, Middle 6, Senior 3). Модули `redis_core` , `mq_core` , `pg_core` , `fastapi_core` не входят ни в один blueprint: CAT их не выбирает, грейд они не меняют и θ не двигают (`apply_to_theta` пропускает модули вне blueprint); их семейства выдаются только как проверочный набор или тренировочная задача. Каталоги с описанием каждого семейства лежат рядом с кодом: `backend/app/testing/bank/families/*/CATALOG.md` .

3.3. Откуда берутся ключи и дистракторы

Ключ получается **исполнением**, руками не задаётся (`bank/execution.py` , помощники семейств):

Язык и формат	Как считается ключ
Python	`exec` сгенерированного кода в чистом пространстве имён; вывод перехватывается подменой `print` (потокобезопасно); `asyncio` — на цикле с виртуальным временем
JavaScript	`node -e` (нужен Node.js в `PATH` при генерации пула)
SQL	SQLite в памяти (`run_sql`), использование индекса — по `EXPLAIN QUERY PLAN`
Shell и git	Точные модели (chmod, конвейер coreutils, таблица дескрипторов, граф коммитов), сверенные тестами с настоящими bash и git

Дистракторы — тоже результат исполнения: код с типичным заблуждением (мутант) запускается так же, его вывод становится неверным ответом с тегом заблуждения. Теги видит только сервер и кандидат в разборе своего экземпляра после решения. Так «ошибка ключа» исключена конструктивно: в отличие от задания, написанного моделью, ключ не может быть неверным для показанного условия.

CI-гейты семейства (`bank/gates.py`): 200 экземпляров без исключений; детерминизм генерации по seed; не менее 95 % различных ключей у текстовых ответов (для выбора — различность экземпляров); дистрактор ≠ ключу; ключ проходит `check()` ; формулировка на русском; Python-код импортирует только модули из белого списка. Люди ревьют семейство по карточке «3 клона рядом» (`GET /api/lab/bank/{family_id}/clones`).

3.4. Пул клонов и выдача

Правило	Значение
Размер пула	200 экземпляров на семейство; догенерация, когда свободных меньше 50 (`scripts/generate_pool.py` , фоновый долив после ответов). Локальный стенд для жюри работает в «быстром режиме» — 40 на семейство (`VZ_POOL_PER_FAMILY`), порог долива 10; см. ниже
Один клон — один человек	`issue_instance` : следующий невыданный экземпляр, `SELECT … FOR UPDATE SKIP LOCKED` ; выданный экземпляр никогда не возвращается в пул
Секретные seed	seed боевого клона — `HMAC(VZ_APP_SECRET, семейство, номер)` в диапазоне `[2^50, 2^62)` . Код генераторов открыт, а код варианта (`#7F3A` — первые 4 символа `sha256(семейство:seed)`) виден на экране: при последовательных seed 0, 1, 2… кандидат нашёл бы seed по коду и получил бы ключ, запустив открытый генератор. Без секрета перебрать 2^62 значений нельзя
Ключи в БД	`item_instances.key_enc` — Fernet (AES-128-CBC + HMAC), ключ шифрования выводится из `VZ_APP_SECRET` . Дамп БД без секрета не раскрывает ключи пула. Пул, созданный с другим секретом, сервер не расшифрует
Превью для методиста	Клоны «3 рядом» берутся из отдельного диапазона seed (от 10^9), который никогда не пересекается с боевым: показ ключа методисту не раскрывает ключ ни одного кандидата
Что видит кандидат	Условие, таймер, код варианта. Верность ответа во время теста не сообщается; разбор с ключом — только по своему экземпляру и только после решения

Быстрый режим пула (только локальный `docker compose up`). Чтобы жюри не ждало генерацию сотни тысяч строк, локальный compose создаёт 40 клонов на семейство вместо 200 (`VZ_POOL_PER_FAMILY=40`), а тяжёлые семейства с исполнением кода — в нескольких процессах. Это не упрощение продукта: сессия берёт из семейства не больше одного клона, клон по-прежнему выдаётся ровно одному человеку, seed клонов секретные и ключ вычисляется исполнением; пул доливается по мере выдачи (при остатке меньше четверти размера), так что 40 хватает на сотни прохождений, а seed демо доливает его сам. Меняется только запас: при 40 клонах «горячее» семейство чаще упирается в долив, а оценка экспозиции и детектор «чужого ключа» видят меньше клонов за раз. На стенде и в `docker-compose.prod.yml` пул боевой — 200 на семейство.

4. Модель ответа и оценка уровня

Один параметр сложности на семейство и шкалу (модель Раша с угадыванием; `app/testing/irt.py`):

```
P(верно | θ) = c + (1 − c) · σ( A_eff · (θ − b) )          A = 1,5;   σ_inst = 0,15
A_eff = A / √(1 + π · A² · (σ_b² + σ_inst²) / 8)
```

`A_eff` учитывает неуверенность в `b` и разброс сложности клонов внутри семейства. Оценка уровня — EAP на сетке `[−4; 4]` с шагом 0,05; **приор N(−0,25; 1,2²) одинаков для всех** (иначе завысивший заявку получил бы фору на доборе вниз). Выходы: $\hat{\theta}$, стандартная ошибка SE, вероятность `P(θ ≥ порог)`, вероятности по полосам грейдов («облако» уверенности), балл `S = 500 + 100·θ̂` и 80 %-интервал `S ± 128·SE`. В интерфейсе слова «θ» нет: «насколько мы уверены».

Грейд	Порог θ	Балл S	Якорь
Ниже Стажёра	< −2,0	< 300	База не подтверждена
Стажёр	−2,0	300	Решает учебные задачи по подсказкам; базовые понятия знает, без помощи разбирает не всё
Junior	−1,0	400	Выполняет типовые задачи по готовому образцу и разбирается с типичными ошибками; ориентир по опыту — до 1,5 лет, не критерий
Middle	0,0	500	Самостоятельно доводит фичу от постановки до релиза, выбирает подходы и замечает риски; ориентир по опыту — 1,5–4 года, не критерий
Senior	+1,0	600	Проектирует решения и отвечает за качество системы, находит причины сложных сбоев и учит других; ориентир по опыту — от 4 лет, не критерий

Lead вне MVP: управленческие навыки тестом не измеряются.

Свежесть. Без новых тестов и задач неопределённость честно растёт: `SE(t)² = SE² + 0,02 · Δмес`. Актуальность профиля `100·exp(−Δдней/120)`; полосы: свежий (< 30 дней), актуальный (< 120), устаревает (< 240), устарел.

Подбаллы модулей. EAP по заданиям модуля с приором `N(θ̂; 0,6²)` (сжатие к общему уровню). Метка `✓` — при не менее 2 заданиях модуля и `P(θ_k ≥ порог Junior) ≥ 0,8`; `✓✓` — при `P(θ_k ≥ порог заявленного грейда) ≥ 0,8`; `~` — мало данных. В интерфейсе три состояния, а не числа: подбалл по 2–5 заданиям грубый, и это не скрывается. Метки `✓` и `✓✓` — единственное, что идёт в подбор как доказательство навыка из теста.

5. Калибровка сложности

Этап	Что происходит	Статус
Экспертные приоры	Два эксперта задают семейству `target_grade` и `position ∈ [−1; 1]` (модифицированный метод Ангоффа): `b = cut(грейд) + 0,35 · position`; `σ_b = 0,5`, если оценки экспертов расходятся не более чем на 1 шаг `position`, иначе 0,8. Строки `item_family_params` по одной на семейство и шкалу (`seed_family_params`)	Работает
Якоря	Семейства с `is_anchor` задают шкалу: их `b` не обновляется калибровкой	Работает
Фаза 1: MAP-уточнение `b`	`ℓ(b) = Σ[x·ln P + (1−x)·ln(1−P)] − (b − b_prior)²/(2σ_b²)` при известных leave-one-out оценках уровня отвечавших; метод Ньютона до 25 итераций; апостериор становится новым приором; нужно не менее 30 новых ответов на семейство. Выданные грейды не пересчитываются (в сессии хранится `params_version`). Используются только ответы с согласием `calibration`	Работает; запуск `POST /api/lab/calibrate` (ночная задача — в разработке)
Pretest → боевой пул	Перевод `pretest → operational`: не менее 50 ответов, `SE_b ≤ 0,35`, `r_pb ≥ 0,15` по pretest-слотам, нет аномалий дистракторов	Логика готова; pretest-семейств в банке пока нет
Карантин	Семейство из карантина возвращается, если после 100 ответов (перекалибровка с широким приором `σ = 0,8`) `abs(Δb) < 0,3`, иначе выводится из банка	Логика готова
2PL, MML, DIF	Полноценная IRT-калибровка по большим данным	Вне MVP

На синтетике RMSE сложности после 100 ответов — 0,20 при пороге 0,30 (VALIDATION.md §2.8). Это проверка процедуры, не реальных людей.

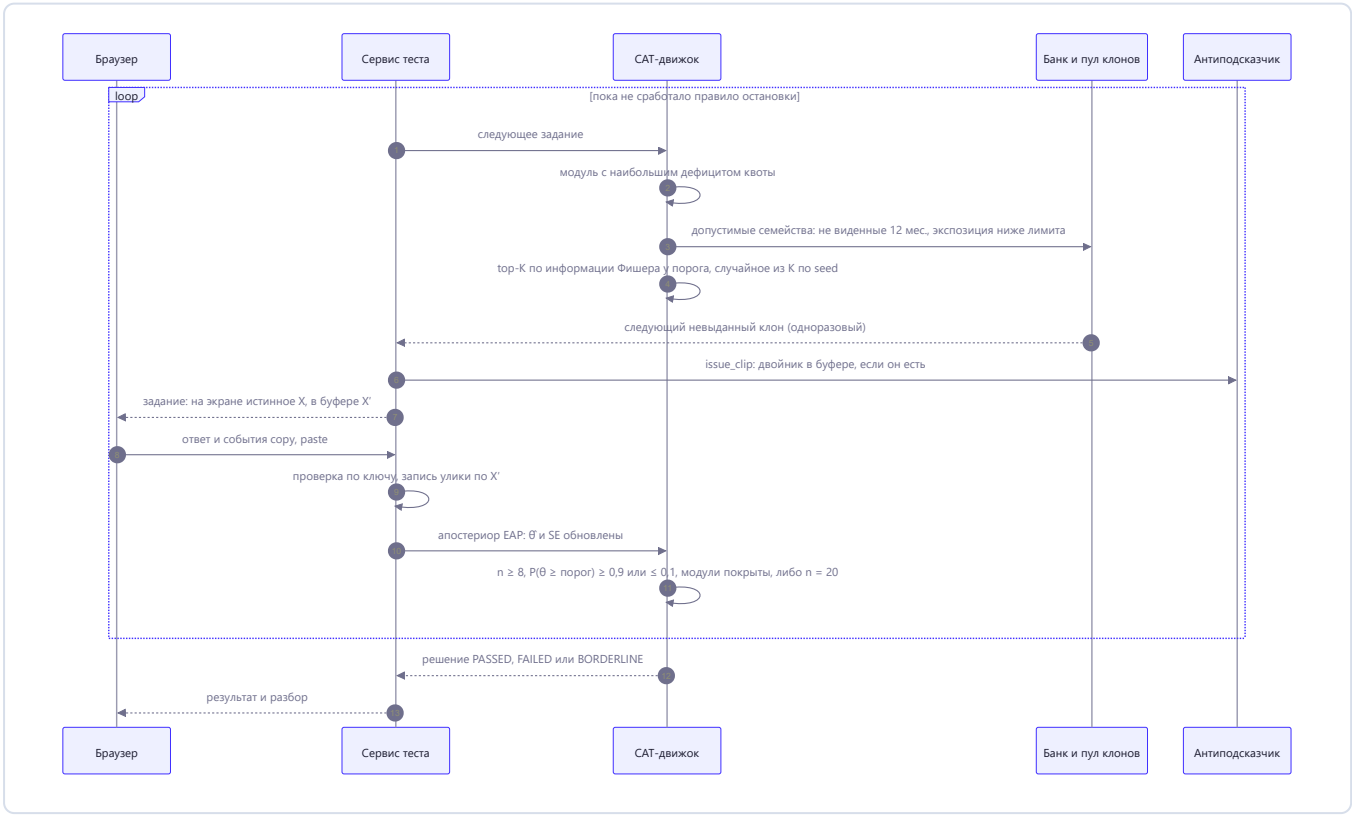
6. Адаптивный тест (CAT)

Движок — `app/testing/cat.py` (чистый), сервис — `app/services/testing/sessions.py`. Параметры — `config/testing.yaml → cat, extension`.

6.1. Выбор задания

1. **Модуль**: с наибольшим дефицитом квоты `q_k·(n+1) − n_k` (квоты — `blueprint` специализации в `taxonomy.yaml`).
2. **Допустимые семейства**: операционные, этой шкалы, не виденные кандидатом за 12 месяцев, не в паре `enemies`, с экспозицией за 30 дней ниже 0,45 (0,25 при банке от 90 семейств; лимит включается, когда за окно набралось не меньше 20 сессий; если допустимых меньше 3, лимит снимается с записью в журнал), со свободными клонами.
3. **Угадывание**: задания с `c > 0` — не больше 30 % теста в любой момент.
4. **Информация у порога, а не у θ̂**: из допустимых берутся top-K по информации Фишера в точке порога заявленного грейда (K = 5 для первых 4 заданий, далее 3), из них выбирается случайное (`random.Random(seed)` сессии).
5. **Pretest-слоты**: 2 задания на позициях 3–14 из пула `pretest` (в пределах 1,0 по `b` от порога), в оценку не входят. Пока pretest-семейств нет, слот занимает зачётное задание.
6. **Клон**: следующий невыданный экземпляр выбранного семейства.

Цикл целиком (клоны и «двойник в буфере» подробно — п. 3.4 и 9):



Причина каждого выбора пишется в `test_responses.selection_reason` (дефициты модулей, top-K с информацией, снятый лимит) — это экран «Трасса» в лаборатории (`GET /api/lab/sessions/{id}/trace`).

6.2. Время, стоп, решение

Правило	Значение
Таймер задания	`time_limit_s` семейства (по умолчанию 90 с) + 5 с запаса; таймаут или `answer = null` — неверно
Пауза	Обрыв связи: `IN_PROGRESS ⇄ PAUSED`; пауза дольше 30 минут — `ABANDONED` (попытка засчитана, результат не публикуется)
Длина	Не меньше 8 и не больше 20 заданий; показанных вместе с pretest — не больше 20
Стоп	`n ≥ 8`, `P(θ ≥ порог) ≥ 0,90` или `≤ 0,10`, и встречены все модули с квотой ≥ 15 %; либо `n = 20`
Решение	`PASSED` (P ≥ 0,9), `FAILED` (P ≤ 0,1); на максимуме длины — `BORDERLINE_PASS` (0,5 ≤ P < 0,9; грейд «на границе», в подборе G × 0,9) и `BORDERLINE_FAIL`
Добор	После решения 24 часа на предложение: вверх (сдал и `P(θ ≥ следующий порог) ≥ 0,30`) или вниз (не сдал); до 8 заданий; прошлые ответы засчитываются (тот же апостериор); повышает только при `P ≥ 0,90`; если ответов уже достаточно (`P ≥ 0,90`), грейд подтверждается одним нажатием без новых заданий

Среднее число заданий в симуляции — 12,8 в основной части и 16,1 с доборами (VALIDATION.md §2.2). «8–20 заданий» — честное обещание кандидату.

7. Правила попыток и смены грейда

Правила — `config/testing.yaml → rules`, реализация — `Qualification` в `app/testing/session.py`.

Ситуация	Правило
Первичный тест и доборы в течение 24 часов	Сменой грейда не считаются (онбординг)
Не прошёл заявленный грейд	Если ответов уже хватает для грейда ниже (P ≥ 0,90) — подтверждение одним нажатием; иначе добор вниз до 8 заданий, прошлые ответы засчитываются
Прошёл уверенно	Предложение добора вверх (Senior сейчас); повышение только при `P ≥ 0,90`
Грейд не подтверждён	Статус `NOT_CONFIRMED`: в банке и подборке показывается **со статусом «грейд не подтверждён» и ниже всех подтверждённых**, а не скрывается; повтор того же грейда через 14 дней, не больше 3 попыток за 90 дней; тест на грейд ниже — сразу (см. ниже)
Повышение позже	Не раньше чем через 90 дней после смены грейда (`next_change_allowed_at`), на одну ступень, на непересекающихся семействах; неудача грейд не снижает, следующая попытка через 30 дней
Добровольное понижение	`POST /api/tests/qualification/{specialization}/downgrade`; считается сменой грейда (вне первых 24 часов — не чаще раза в 90 дней)
Принудительное понижение	Никогда
Сессия на проверке у методиста	Квалификация скрыта из витрины до 7 дней; решение «оставить» (доверие 0,75) или «аннулировать» (откат к состоянию до сессии)
Одна активная сессия на аккаунт	Гарантия на уровне БД (уникальный частичный индекс)

7.1. Правила пересдачи и повышения

Источник — ответы организаторов на вопросы (DECISIONS п. 29). Сроки живут в `config/testing.yaml → rules` и в интерфейсе подставляются из конфига.

Правило	Как работает	Параметр
Повышение — отдельным тестом	Тест на повышение — отдельная сессия на одну ступень вверх, на непересекающихся семействах; доступен не раньше чем через N дней после последней смены грейда: за один день «перепрыгнуть» уровни нельзя	`change_interval_days: 90` (срок ожидания повышения)

Правило	Как работает	Параметр
Провал не сжигает попытку на грейд ниже	Не подтвердился заявленный грейд → кандидат получает тот, который тест подтвердил (добор вниз по тем же ответам). Если подтверждения нет, тест на **более низкий** грейд доступен **сразу**, без ожидания и без лимита; он не расходует пересдачу и не продлевает ожидание	`Qualification.last_failed_grade`, `attempt_free`
Повтор того же грейда — по сроку	После провала и после «дали грейд ниже» тот же грейд можно пересдать через N дней; не больше M попыток за K дней	`retake_days: 14`, `max_attempts: 3`, `attempts_window_days: 90`
Неудачное повышение не понижает	Подтверждённый грейд остаётся; следующая попытка повышения — через N дней	`upgrade_retry_days: 30`
Принудительного понижения нет	Понизить грейд может только сам кандидат	—

Почему так. Организаторы рекомендовали три вещи: не давать менять грейд «изо дня в день» (иначе тест превращается в перебор), не наказывать за честную заявку «на уровень выше» (иначе кандидаты занижают заявку, а подбор получает искажённые грейды) и пересдавать тот же грейд после паузы, а не сразу. Поэтому: (1) повышение — отдельный дорогой шаг с интервалом; (2) провал целевого грейда превращается в подтверждённый нижний грейд, а попытка на нижний не тратится — иначе честная заявка «Middle» стоила бы кандидату двух из трёх попыток; (3) тот же грейд пересдаётся не раньше чем через `retake_days` — за две недели приходит новый клон-набор и новый опыт, а не запомненные ответы. Лимит «3 попытки за 90 дней» защищает банк заданий от перебора и не распространяется на бесплатные попытки на ступень ниже: их число конечно (не больше трёх ступеней вниз) и каждая подтверждается на заданиях, которых кандидат не видел.

Если грейд ниже заявленного подтвердился (добор вниз): ожидание до повтора заявленного грейда — `retake_days`, а не `change_interval_days`: это повтор того же уровня, а не «прыжок».

Кандидату правила показываются на экране брифинга («Пересдача и повышение: когда можно», личная строка «пересдать Middle можно через 12 дней; грейд ниже — сразу»), на экране результата («Когда можно пересдать») и в «Моём грейде». Недоступные грейды на брифинге серые с пометкой, когда станут доступны.

Грейд и стаж. Грейд — уровень навыков, а не годы опыта: якоря в `config/taxonomy.yaml` описывают умения, годы указаны как «ориентир, не критерий» и нужны только для подсказки в опросе. Данные ФСП на грейд не влияют: категорию даёт только тест.

8. Устойчивость к распространению заданий

Угроза: задания и ответы выкладывают в открытый доступ, кандидаты обмениваются решениями. Принцип защиты — не прятать банк, а сделать утечку бесполезной и заметной.

Сценарий	Что происходит	Слой защиты
Кандидату пересылают ответы на «его» задания	Клоны одноразовые: у соседа другие числа и другой ключ. Слитый ответ неверен	Клоны, ключ — результат исполнения
Кандидат вводит слитый ключ чужого клона	Хеш ответа совпадает с `key_hash` клона того же семейства, выданного другому аккаунту — жёсткий флаг, сессия на проверку	Детектор «чужой ключ»
Заготовленные ответы по коду варианта	Seed секретные: по коду `#7F3A` и открытому генератору ключ не восстановить	Секретные seed, Fernet
Сильное завышение результата списыванием	Ответ «не по заданию», аномальное время, паттерн ответов	Person-fit `l_z < −2,33`, «быстро и верно на трудном» (время < 0,25 медианы семейства, `P < 0,4`, не менее 2 раз), события (больше 3 уходов со вкладки дольше 10 с, вставка в поле ответа)

Сценарий	Что происходит	Слой защиты
Массовый слив метода или семейства	Статистика дрейфа сложности растёт	CUSUM (`h = 11`) → семейство в карантин; ≥ 3 «чужих ключа» по семейству за 7 дней → карантин
Перебор банка	Семейство, виденное кандидатом за 12 месяцев, не повторяется; экспозиция семейства за 30 дней ограничена	Экспозиция, неповтор
Пересдача «до нужного результата»	Не больше 3 попыток за 90 дней, повтор через 14 дней, повышение через 90 дней на других семействах	Правила попыток

Результаты симуляции (VALIDATION.md §2.5–2.7; 300 «явных Junior» заявляют Middle, коалиция из k честных кандидатов сливает всё, что видела):

Коалиция	0	5	20	100
Наш тест, осторожный списывающий (отвечает честно)	0,10	0,10	0,10	0,10
Наш тест, наивный (вводит слитый ключ)	0,10	0,00	0,00	0,00
Фиксированная форма из 16 заданий (B0)	0,09	1,00	1,00	1,00
CAT без клонов, 20 экземпляров (B2)	0,10	0,29	0,77	1,00
Наивные, пойманные «чужим ключом»	—	1,00	1,00	1,00

Значения — доля ложных «Middle». Пересечение клонов у пары кандидатов — 0 по построению; два кандидата с одним семейством получили одинаковый по тексту клон в 0,10 % случаев (худшее семейство Backend).

Что защита не закрывает (измерено, а не обещано):

1. **Слив «метода».** Клоны защищают от слива ответов, но не от слива идеи семейства. Если кандидаты знают метод, задание им легче на Δb: при коалиции 20 и 95 % слитых семейств ложный Middle растёт с 0,10 (Δb = 0) до 0,25 (Δb = 0,25) и 0,51 (Δb = 0,5). Массовый слив ловит CUSUM; прод-порог `h = 11` подобран по нулевым потокам (без утечки) на других сидах популяции — ≤ 5 % ложных карантинов на каждом (прежний `h = 6` давал 20–33 %); цена — задержка: при 50 % знающих медиана ≈ 600 ответов и найдено около трети слитых семейств, поэтому цель ≤ 200 ответов не выполнена (VALIDATION.md, 2.7). CUSUM — сигнал методисту, а не автоматическая блокировка.
2. **«Чужой ключ» работает не у всех семейств.** Он активен, если доля коллизий ключей в семействе меньше 2 %: у Backend это 28 из 44 семейств. У заданий с выбором ответа ключи совпадают часто — их защищают только клоны и CUSUM.
3. **Помощь ИИ вне страницы теста.** Скриншот, ИИ-панель браузера, оверлей по экрану, фото телефоном, голосовой помощник не обнаруживаются ни клонами, ни «чужим ключом». От ИИ-ассистента удалённый тест без прокторинга полностью не защищён — как любой удалённый тест. Камеры, прокторинга и сканера процессов нет сознательно: это биометрия и инвазивный контроль (152-ФЗ, обычный браузер). Поэтому **грейд — решение с уровнем доверия, а не сертификат**, а слабое место закрыто живой сверкой — «Подтверждением за 5 минут» (п. 9.3).

9. Антиподсказчик: «двойник в буфере» и «подтверждение за 5 минут»

Библиотека `app/testing/anticheat` (чистый Python: без БД, HTTP и LLM; пороги и тексты — `config/anticheat.yaml`). **Статус: реализовано сквозным образом: библиотека, сервис `app/services/anticheat` , три хука в `app/services/testing/anticheat_hooks.py` (подмена при выдаче задания, запись улик при ответе), клиентский обработчик буфера во фронтенде и лаборатория.** API принимает события `copy` , `cut` , `dragstart` , `self_paste` , `copy_bypass` (`POST /api/tests/sessions/{id}/events`) и отдать данные подмены (`ItemOut.clip`).

9.1. Механизм

Для каждого клона X строится двойник X′: **тот же текст с ровно одним изменённым литералом данных**, а ключ X′ вычисляется тем же исполнителем, что и основной (`run_python` , `run_js` , `run_sql`). На экране и в дереве доступности всегда истинный X. Когда кандидат копирует условие, в буфер обмена попадает X′. Модель, которой вставили X′, честно решает другую задачу и возвращает ответ, которого по экрану получить нельзя. Никаких скрытых инструкций модели нет.

Какие мутации разрешены: только те, что не воспроизводят типичную человеческую ошибку: элемент списка, множитель, значение в строке SQL-таблицы вдали от порога `WHERE` (классы `data_literal` и `table_value` ; форматы `predict_output` и `sql_result` на Python, JavaScript и SQL). Запрещены границы `range` , `<` ↔ `<=` , направление `ORDER BY` , константа-порог — они совпали бы с ошибкой «на единицу» у честного человека.

Гейты генератора (`twins.py` , `gates.py`): ключ X′ ≠ ключу X; не равен ни одному дистрактору; не равен ключу типичной ошибки (`K_err` : исполнение до 40 мутантов Python, 10 JavaScript, 40 SQL); не совпадает с ключом другого клона пула; ровно один изменённый токен, длина ±1, относительное изменение значения 12–80 %; не путаемые на мелком шрифте пары глифов (17/38/56, l/1, O/0 и др.); изменённый токен не в строке, которую переведёт переводчик страницы. Если безопасного двойника нет, клон выдаётся без него.

Покрытие каталога (`python -m app.testing.anticheat.report` , 50 сидов на семейство; 111 семейств): формат `predict_output` или `sql_result` у 87 семейств; по форме двойник возможен у **39** (ответ вводится текстом, язык исполняется офлайн), у остальных 48 двойник невозможен: у 32 язык не исполняется (JSX, HTML, CSS, shell и др.), у 14 ответ выбирается из вариантов (ключ двойника совпал бы с вариантом-дистрактором), у 2 в задании нет кода или данных. Из 39 пригодных валидный двойник на не менее чем 90 % сидов даёт 35 семейств (89,7 % при пороге 85 %; ниже порога `py.mro.super` , `react.batched_updates` , `react.controlled_input` , `react.stale_closure`), 89,1 % экземпляров пригодных семейств получают двойник. То есть двойник защищает около трети каталога (35 из 111 семейств), остальное защищают клоны, детекторы и живая сверка.

9.2. Улики и что видит каждая сторона

Функция `aggregate_session` сводит ответы на задания с двойником в состояние и **предлагаемый** уровень квалификации (решает методист):

Состояние	Условие	Предлагаемый уровень
Наблюдений нет	Нет точных попаданий в ключ X′	«Подтверждена»
Предупреждение	Первое точное попадание (до предупреждения; тост показывается при первом копировании)	Не выше «С оговоркой»
Материал для методиста	Повтор после предупреждения	«С оговоркой» (в режиме доступности — потолок из конфига)
Кандидат на «Требует подтверждения»	Не меньше 2 попаданий, не меньше 1 после предупреждения, не меньше 2 разных классов мутации, вероятность такого числа совпадений у честного кандидата `≤ 10⁻³` , режим доступности не включён	«Требует подтверждения»

Правила учёта: ответ двойника для оценки уровня всегда неверный (это неверный ответ на X); не больше одного попадания на подкласс мутации (защита от коррелированных заблуждений); вставка в собственное поле (`self_paste`) и устаревшая копия (другой `render_id`) в улику не идут. Дополнительные наблюдения (ответ получается при типичной ошибке на X′; в ответе есть значение, которого нет на экране) — только контекст.

Пока вероятность честно попасть на ключ двойника ε не измерена на людях (консервативно принята 0,02), `evidence.calibrated: false` : предлагаемый уровень не применяется, вес улики в доверии 0, а вся сводка — контекст для методиста. Кандидат после закрытия сессии видит только номер задания и фразу «ответ не соответствует показанному условию» с кнопками «Оспорить» и «Пройти подтверждение»; работодатель улик не видит никогда; методист получает **акт сверки** (`build_act`): показанная версия и копия рядом с подсвеченным литералом, оба ключа, ответ кандидата, время копирования и ответа, вероятность такого совпадения у честного

кандидата и чек-лист альтернативных объяснений (вставка в собственное поле, устойчивое заблуждение, путаница глифов, устаревшая копия, режим доступности). Вердикт — фактами, без слов «ИИ» и «списал».

При вставке в собственное поле ответа токен X′ заменяется обратно на X, событие `self_paste` исключает такой ответ из улики: честный кандидат, копирующий литерал в свой черновик, не страдает.

Сколько заданий с двойником. Двойник выдаётся на каждом задании сессии, для которого он посчитан (`evidence.twin_rate_per_session: 20` — максимальная длина теста). Предрасчёт идёт сразу после генерации пула (`make gen-pool`, сервис `pool` в compose: `generate_pool.py`, затем `python -m app.services.anticheat.precompute`), достройка в момент выдачи выключена, чтобы кандидат не ждал. Работу выдачи двойников на каждом задании проверяет пилот с ИИ-испытуемыми (VALIDATION.md, раздел 10). Честному кандидату двойник ничего не стоит — он отвечает по экрану, а порог улик `P(≥ k | H0)` сам учитывает, сколько заданий с двойником было в сессии.

9.3. «Подтверждение за 5 минут» — ответ на скриншот

Двойник ловит только копирование текста. **Скриншот, ИИ-панель браузера и оверлей двойник проходят бесплатно** (во внутреннем пилоте на трёх моделях одного вендора и 14 заданиях канал «текст из буфера» дал 0 верных ответов для экрана, канал «скриншот» — все верные; выборка мала, переносить на другие модели нельзя). Поэтому вторая половина защиты — живая сверка по трассе (`build_kit`): после взаимного «да» работодатель получает комплект из 3 вопросов по заданиям самого кандидата (предпочтительно трудным, решённым верно и быстро, и заданиям со «следом двойника»), в каждом изменён один параметр, ожидаемый ответ вычислен исполнением; рубрика 0–2 и подсказка «что слушать»; до 2 шаблонных вопросов по must-навыкам, которых тест не покрывает (с пометкой «не откалибровано»). Итог «соответствует / выше / ниже» даёт бейдж «Подтверждено вживую». Улик и флагов в комплекте нет. Шаблонные вопросы может перефразировать шлюз LLM, но не оценивать ответы.

Для жюри предусмотрена песочница «Попробуй списать» (`build_sandbox`, `score_sandbox`): 4 задания на Python, JavaScript и SQL; после ответа показывается, что именно заметила система и по какому каналу.

10. Уровень доверия

Флаги сессии	Доверие	Последствие
Нет	1,0	—
1–2 мягких	0,75	В подборе компонент T умножается на доверие; в объяснении — «пониженное доверие к результату теста»
≥ 3 мягких или «чужой ключ»	Проверка методистом, до 7 дней	Квалификация скрыта из витрины; решение «оставить» (0,75) или «аннулировать»

Работодатель флагов не видит: он видит только слова в «против» («тест сдан на границе», «пониженное доверие») и бейдж подтверждения вживую. Кандидат видит свои флаги без источника и может оспорить один раз (`POST /api/tests/flags/{flag_id}/dispute`); оспоренный флаг первым попадает в очередь методиста (`GET /api/lab/flags`).

11. Регулярные короткие задания

	Тип A «задача из каталога»	Тип B «предложи подход»
Что выбирает работодатель	Откалиброванное семейство из каталога (`GET /api/tasks/catalog`)	Свой текст и рубрика из 4 критериев по 0–2
Что получает кандидат	Свой одноразовый клон, автопроверка, серверный таймер	Общий текст; решение уходит работодателю без имени

	Тип A «задача из каталога»	Тип B «предложи подход»
Влияние	Уровень: апостериор с приором `N(θ̂; SE(t)²)` (только по операционным семействам), подтверждение навыка через модуль, свежесть. **Грейд напрямую не меняется**; при `P(θ ≥ следующий порог) ≥ 0,7` — «вам доступен тест на повышение»	Только свежесть (текст общий, мог утечь)
Лимиты	Работодатель: ≤ 5 активных задач, ≤ 15 минут, аудитория «специализация × грейд ±1», отметка «это не рабочая задача компании». Кандидат: не больше 1 предложения в 7 дней, не больше 3 активных, отказ не штрафуется, после 3 пропусков подряд — пауза 30 дней	то же
Если компаний нет	«Тренировочная задача от платформы» из того же каталога	—

Так выполняется «регулярные задания влияют на актуальность профиля»: актуальность и свежесть `A` в подборе считаются от последнего теста или задачи.

12. Вакансия → проверочный набор

Стандартный тест категории **под вакансию не меняется**: иначе грейды стали бы несравнимы. Вакансия даёт две вещи.

1. **«Как мы проверим»** (`GET /api/needs/{need_id}/check`): навыки брифа отображаются на модули теста по графу «навык → модули» (`config/skills.yaml` : 349 навыков, из них 167 привязаны к модулям, 182 «тестом не проверяются»); показываются покрытие, серый список «тестом не проверяется — проверьте на интервью» и веса модулей в компоненте K.
2. **Проверочный набор** (`POST /api/tasks/check-sets` , результаты — `GET /api/tasks/check-sets/{need_id}`): must-навыки → модули → 3–5 откалиброванных семейств типа A суммарно не дольше 15 минут (`plan_check_set`). Каждому кандидату из шортлиста выдаются свои одноразовые клоны как регулярная задача со сроком 3 дня. Результат идёт в подтверждение модуля (K) и свежесть (A); стандартный тест и грейд не затрагиваются. Кандидатам вне шортлиста и тем, кто не принимает обращений компании, набор не уходит. Если must-навыки тестом не проверяются — 409.

Навыки внутри модуля. Стандартный тест меряет модуль целиком (Python, SQL, HTTP …) и не видит Redis, очередей сообщений, особенностей FastAPI и PostgreSQL; резюме в Score не входит. Эти навыки наблюдаются короткими семействами `families/skills/` (поле `Family.observes` = навыки, которые семейство наблюдает; `plan_check_set` берёт семейство под навык и ближе к грейду брифа и ставит на задачу только эти теги). Ключ — исполнение, как у остальных семейств: настоящий FastAPI через `TestClient` , sqlite для SQL-семантики, совпадающей с PostgreSQL (upsert, RETURNING, порядок NULL переписывается по правилам PostgreSQL), эмуляторы Redis и брокеров по документированным правилам. Дистракторы — тот же решатель с заблуждением; «двойник в буфере» строится решателем по показанному тексту. Семейства Docker, Kubernetes, nginx, cron, CI получили `observes` без смены модуля.

Модуль теста, собранный под конкретную вакансию, — вне MVP.

13. LLM пишет генератор, а не задание

В оцениваемом контуре LLM запрещена архитектурно: выдача и ключи заданий, сложность `b` , грейд, Score, ранжирование, объяснения и вердикты «Антиподсказчика» работают без LLM. Причина в данных, а не в принципе: модель, генерирующая задание «под каждого», не гарантирует сопоставимость сложности и ошибается в ключах (в симуляции каждые 5 % битых ключей снижают долю верных решений — VALIDATION.md §2.5; доля битых ключей у реальных YandexGPT/GigaChat не измерена). Решение о человеке принимает человек (ст. 16 152-ФЗ).

Где LLM допустима	Как	Статус
Разбор брифа	Только **добавляет** навыки, пропущенные правилами; каждое поле с цитатой, найденной в тексте (`partial_ratio ≥ 90`), источник «YandexGPT» / «GigaChat»	Работает, выключена по умолчанию
Разбор резюме	То же: ПДн маскируются до вызова, поле без цитаты отбрасывается	Работает
«Фабрика семейств»	Офлайн: LLM пишет **черновик семейства** (шаблон, генератор параметров, эталонный решатель). Ключи считает исполнение, черновик проходит CI-гейты и ревью человека, затем pretest с весом 0 и калибровка по ответам людей	В разработке
Черновик задачи типа B из вакансии, перефразирование вопросов «Подтверждения за 5 минут»	Работодатель правит и подтверждает; оценка — человека	В разработке

Шлюз `app/llm/gateway.py` : цепочка YandexGPT → GigaChat → «без LLM» (детерминированный fallback: результат правил не меняется), общий бюджет 20 с, кэш по `sha256` замаскированного текста. Режимы `VZ_LLM_MODE` : `none` , `live` , `record` (провайдеры плюс запись ответов в JSON-кассету), `replay` (только кассета, без сети и ключей — демо воспроизводимо; в кассете нет текста запроса и ПДн). Контур Score защищён тестом: модули ранжирования и объяснений не импортируют `app.llm` ; в `app/testing` и `app/services/testing` таких импортов нет.

14. Честные ограничения

- Сложность семейств пока **экспертная** (приоры), а не измеренная на людях; калибровка по ответам запускается вручную. Pretest-слоты в банке пока пусты.
- Валидация (VALIDATION.md) — симуляция на синтетической популяции с латентной истиной: она проверяет процедуру, не реальные сложности. Раздела «люди» (пилот, экспертная разметка) в прогоне нет. Два порога из шести основных на момент прогона не выполнены (точный грейд 0,748 при цели 0,75; корреляция оценок при пересдаче 0,86 при цели 0,88).
- Точное совпадение грейда при пересдаче не может быть близким к 1 при 4–5 уровнях и около 15 заданиях; стабильность для пользователя обеспечивает гистерезис (повышение только при `P ≥ 0,90` , смена не чаще раза в 90 дней).
- Не закрыты: слив метода (частично, п. 8), помощь ИИ вне страницы (п. 8 и 9.3), заданий с выбором ответа детектор «чужой ключ» не защищает.
- Поведение `copy` на мобильных браузерах не проверено; если подмена не сработала, это пропуск, а не ложное срабатывание.
- Банки QA и DevOps меньше по объёму, чем Backend/Frontend: доверительные интервалы уровня у них шире.
- В демо-режиме есть «режим жюри» (подсказка с верным ответом): такие сессии помечаются `demo_assisted` и исключаются из метрик, калибровки и медиан времени ответа.

15. Дизайнеры (UI/UX): по опросу, тест в дорожной карте

Статус: специализация `uiux_design` есть в справочнике (`config/taxonomy.yaml` , `status: survey_only`): дизайнер проходит опрос, попадает в лист ожидания, работодатель видит число «ждут теста», но категория «специализация × грейд» **не присваивается**: теста нет, а резюме и портфолио на категорию не влияют (принцип платформы). Приоритет проекта — технические роли; ответы организаторов допускают дизайнеров в каталоге, если есть идея объективной проверки. Ниже — идея, которая укладывается в тот же движок (семейства, одноразовые клоны, IRT/CAT) и **не опирается на субъективную оценку вкуса**.

15.1. Принцип: проверяем то, у чего есть ключ

Эстетику и креативность тестом не измерить, и мы этого не пытаемся: они остаются за портфолио и интервью (в карточке навыки дизайна помечены «тестом не проверяется — посмотрите портфолио»). Объективно

проверяются профессиональные суждения, у которых есть вычислимый или согласованный экспертами ответ:

Модуль (план)	Что проверяет	Формат задания	Как считается ключ
`ux_heuristics`	Эвристическая оценка по Нильсену	Экран-макет (SVG) с **внедрёнными дефектами**: нет статуса загрузки, нет отмены, непонятная иконка, ошибка без подсказки. Кандидат отмечает области и выбирает нарушенную эвристику	Генератор сам внедряет дефекты и знает их координаты и классы. Балл: доля найденных дефектов минус штраф за ложные отметки (F-мера по попаданию в область и классу)
`a11y_calc`	Доступность	«Подберите цвет текста с контрастом не ниже 4,5:1 на этом фоне»; «какие из кнопок меньше 44×44 px»; «в каком порядке пойдёт фокус»	Контраст по формуле WCAG (относительная яркость, отношение `(L1+0,05)/(L2+0,05)`), размеры из геометрии макета. Ответов может быть много — проверяется **вычислением**, а не сверкой со списком
`layout_grid`	Сетка, отступы, адаптивность	Набор блоков и ограничения: сетка 8 px, колонки 12, брейкпоинты. Расставьте блоки или найдите нарушающие сетку	Детерминированная проверка геометрии: кратность отступов, выравнивание по колонкам, отсутствие наложений
`ui_theory`	Закон Фиттса и Хика, иерархия, типографская шкала	Расчётные и ранжирующие задачи: «какой из вариантов меню быстрее по Фиттсу», «упорядочьте элементы по визуальному приоритету», «постройте шкалу с модулем 1,25»	Формулы; для иерархии — согласованный экспертами эталон с несколькими допустимыми порядками
`ux_research`	Чтение результатов исследований	Таблица результатов юзабилити-теста (успех, время, SUS): «какую гипотезу данные поддерживают», «что делать дальше»	Расчёт метрик (SUS, доля успеха с доверительным интервалом) и выбор вывода по заранее заданным правилам

Черновая раскладка стандартного теста категории: `ux_heuristics` 0,30, `a11y_calc` 0,25, `layout_grid` 0,25, `ui_theory` 0,10, `ux_research` 0,10 — в `taxonomy.yaml` как `blueprint`, когда семейства появятся.

15.2. Как это ложится на движок

- **Семейство и клоны.** Макет задаётся параметрами (палитра, размеры, список внедряемых дефектов, seed); клон отличается параметрами, а не текстом, как у существующих семейств (контракт — `app/testing/bank/base.py`). Скриншот одного клона не помогает со вторым: другой дефект в другом месте.
- **Ключ — исполнение.** Правильный ответ считает код генератора (дефекты известны по построению, контраст и размеры вычисляются), а не человек и не LLM. LLM допускается только офлайн как автор черновика семейства, с тем же ревью и калибровкой, что у остальных (п. 13).
- **Рендер без внешних ресурсов.** Макеты рисуются в SVG на стороне платформы; шрифты и иконки самохостятся (локализация данных, SECURITY.md п. 1).
- **Оценка уровня.** Те же IRT-параметры и CAT; грейд — уровень навыков, а не стаж. Задания тегируются навыками, «заявлено» от «проверено» отличается как у остальных специализаций.
- **Антиподсказчик.** Двойник в буфере и подтверждение за 5 минут (п. 9) применимы без изменений: вопросы на выбор и расчёты клонируются.

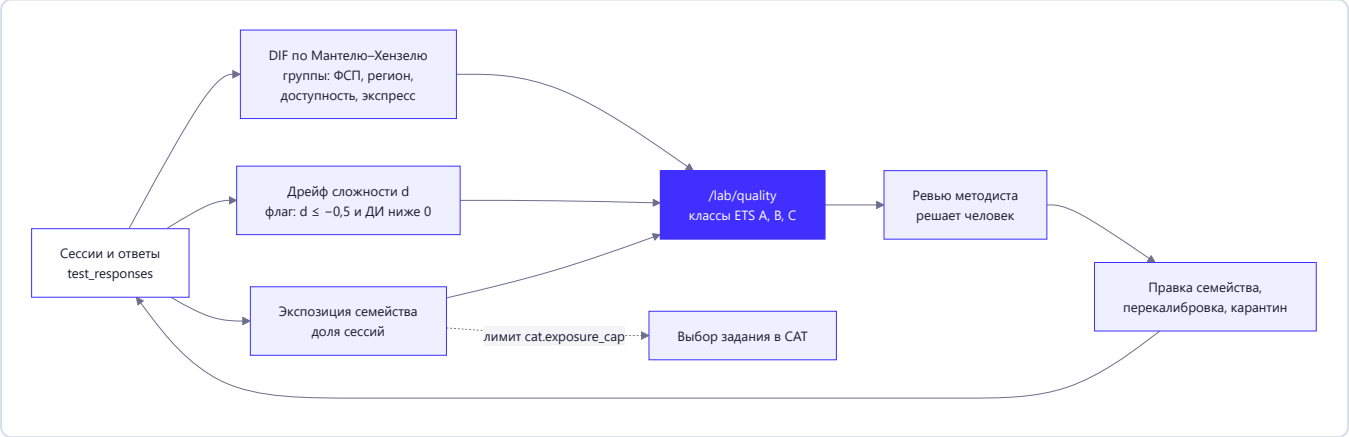
15.3. Что не решено (честно)

- Нужна **калибровка на людях** и экспертная разметка допустимых ответов в задачах на иерархию и эвристики: без неё это гипотеза, не измерение.
- Валидность: умение находить дефекты в макете коррелирует с профессионализмом, но не равно умению проектировать; поэтому категория дизайнера всегда будет сопровождаться пометкой «портфолио смотрите отдельно».
- Оценка трудозатрат: инфраструктура (SVG-рендер, ввод отметок на макете) — основной объём; генераторы семейств — по образцу существующих. До этого `uiux_design` остаётся «по опросу, тест в дорожной карте».

16. Качество заданий: DIF, дрейф сложности, экспозиция

Три монитора психометрического качества. Формулы — `app/testing/monitoring.py` (чистые функции), пороги — `config/testing.yaml` → `quality`, демонстрация на синтетике — `uv run python -m eval.fairness` (результат — раздел 8 VALIDATION.md), экран — «Лаборатория → Качество» (`GET /api/lab/quality`).

Контур мониторинга:



16.1. DIF по Мантелю–Хензелю

Вопрос: при одинаковом уровне задание одинаково трудно для всех групп? Для семейства и пары групп (опорная / фокальная) ответы раскладываются по стратам уровня ($\hat{\theta}$ сессии, шаг 0,5 логита; в адаптивном тесте суммарного балла нет), в каждой страте — таблица 2×2 (A, B — опорная верно/неверно; C, D — фокальная). Оценка общего отношения шансов $\alpha_{MH} = \Sigma(A \cdot D/N) / \Sigma(B \cdot C/N)$; шкала ETS: **$MH\ D\text{-}DIF = -2{,}35 \cdot \ln \alpha_{MH}$**, $\Delta < 0$ — тяжелее для фокальной группы. Дисперсия $\ln \alpha$ — Робинс–Бреслоу–Гринланд; значимость — χ^2 MH с поправкой на непрерывность (порог 3,84).

Классы ETS: **A** — $|\Delta| < 1$ или различие незначимо; **C** — $|\Delta| \geq 1{,}5$ и значимо больше 1 (односторонний тест, $(|\Delta| - 1)/SE > 1{,}645$); **B** — остальное. A — задание годится, B — посмотреть, C — на ревью методисту. Для пар с менее чем 30 ответами в группе класс не присваивается («мало данных»).

Группы — только те, по которым есть законный смысл сравнивать и которые у нас есть: история ФСП (есть/нет), регион (Москва и Санкт-Петербург / остальные), режим доступности, экспресс-версия. Пол и возраст платформа не собирает. Это персональные признаки, поэтому DIF по живым кандидатам в лаборатории **не считается**: экран показывает прогон на синтетике (группы заданы нами, часть заданий намеренно смещена) как доказательство, что метод находит смещение, а не как вывод о людях.

16.2. Монитор дрейфа сложности

Для семейства с калиброванной сложностью b по ответам после калибровки решается уравнение $\Sigma\, P(\theta_j;\, b + d) = \Sigma\, y_j$ (θ_j — апостериорный уровень отвечавшего без этого ответа, как в калибровке, п. 5). d — эмпирическая сложность минус калиброванная; 95% ДИ — по информации Фишера. Флаг **«дрейф»** — $d \leq -0{,}5$ (стало легче: признак слива) и весь ДИ ниже 0; рост сложности даёт лишь «наблюдать» (при массовом сливе он возникает у остальных заданий как артефакт: знающие завышают свой уровень). Не меньше 40 ответов. Оценка консервативна: шум θ_j сжимает d к нулю (на синтетике при сдвиге −1,5 у 80% кандидатов измеряется около −0,7). Монитор дополняет CUSUM (п. 8): CUSUM быстро замечает поток, монитор даёт величину сдвига с доверительным интервалом. Ночная задача `calibration` пишет число семейств с флагом в журнал аудита (`scheduler.calibration` , поле `drift_flags`).

16.3. Экспозиция и контроль Симпсона–Хеттера

Экспозиция семейства — доля решённых сессий шкалы, где оно показано. Одноразовые клоны делают экспозицию конкретного экземпляра ≈ 0, но семейство (идея задачи) видит каждый; поэтому ограничение — на

уровне семейства (`cat.exposure_cap` по 30-дневной экспозиции уже есть в движке).

Контроль Симпсона–Хеттера (`quality.sympson_hetter` , **выключен по умолчанию**): выбранное по информации задание предъявляется с вероятностью K_i, иначе берётся следующее; отклонённое в этой сессии больше не возвращается. K_i подбирают итерациями симуляции: K_i ← min(1, r_max / P(S_i)), где P(S_i) — доля сессий, в которых задание было выбрано до фильтра K. При `enabled: false` выбор не меняется, генератор случайности не расходуется и в трассе нет следов контроля (тест `tests/test_quality_monitoring.py::test_sh_default_off_is_identical`). Результат на синтетике (разд. 8.3 VALIDATION.md) честный: за 8 итераций контроль сходится не полностью, поэтому в продукте он выключен.

Формулу итерации в явном виде мы сверили только по описанию процедуры (assess.com); первоисточник — Sympson & Hetter (1985) — недоступен в открытом доступе, поэтому формула приведена по литературному обзору и подтверждена поведением симуляции (K_i снижает экспозицию, тест на наличие эффекта).

16.4. Источники

- Holland P. W., Thayer D. T. (1988). Differential item performance and the Mantel–Haenszel procedure. B: Wainer H., Braun H. (ред.), *Test Validity*. Erlbaum. — α_MH и шкала дельта.
- Mantel N., Haenszel W. (1959). Statistical aspects of the analysis of data from retrospective studies of disease. *JNCI*, 22. — χ^2 MH.
- Robins J., Breslow N., Greenland S. (1986). Estimators of the Mantel–Haenszel variance consistent in both sparse data and large-strata limiting models. *Biometrics*, 42. — дисперсия ln α_MH.
- Zwick R., Donoghue J. R., Grima A. (1993). Assessment of differential item functioning for performance tasks. *J. Educ. Meas.*, 30. — классы A/B/C в адаптивных тестах; Paek I., Holland P. W. (2015). A note on statistical hypothesis testing based on log-transformation of the MH common odds ratio for DIF classification. *Psychometrika*, 80. — проверка значимости для класса C.
- Sympson J. B., Hetter R. D. (1985). Controlling item-exposure rates in computerized adaptive testing. *Proceedings of the 27th Annual Meeting of the Military Testing Association*; обзор метода — assess.com/sympson-hetter-item-exposure-control.

Механика подбора и категоризации

Как «Взаимно» строит слепую объяснимую подборку: от текста потребности работодателя до версии подборки с разложением Score. Все веса и пороги — в `config/scoring.yaml`, `config/skills.yaml`, `config/taxonomy.yaml`, `config/fsp.yaml`, `config/contact.yaml`; код — `backend/app/matching` (чистые функции) и `backend/app/services/matching` (БД, права, приватность). Результаты симуляций — в VALIDATION.md. Не построенное помечено «в разработке».

Содержание: 1. Категория · 2. Конвейер · 3. Разбор брифа · 4. Категории и охват · 5. Формула Score · 6. Объяснения · 7. Версии подборки · 8. Приватность · 9. Сигнал интереса · 10. Кандидаты без ФСП · 11. Банк без брифа · 12. Ограничения

1. Категория = специализация × грейд

Категория — детерминированная функция `f(специализация, подтверждённый тестом грейд)` (`category_of` в `app/testing/session.py`): код `backend_python:middle`, название «Backend (Python) · Middle». Справочник — `config/taxonomy.yaml`.

Специализация	Статус	Заметка
Backend (Python), Frontend (JS/TS)	`active`	Тест работает; кандидат получает категорию
Fullstack	`derived`	Присваивается, когда подтверждены обе квалификации; грейд — меньший из двух; кандидат виден в обеих категориях и в общем топе один раз
QA, DevOps	`active`	Тест работает (модули `qa_testing` , `devops_core`); кандидат получает категорию
Data/ML, Mobile, ИБ, Backend (Go), Backend (Java/Kotlin)	`survey_only`	Опрос есть, теста нет. Бриф, распознанный как такая категория, получает пометку «категория пока не тестируется. N кандидатов прошли опрос и ждут теста»; подборки по ней нет

Ни резюме, ни самоописание категорию не открывают. Категорию даёт только тест. Резюме и опрос порождают серые чипы «заявлено».

Кандидат виден работодателям (`matching_features.visible`), только если одновременно: аккаунт активен; дано согласие `blind_card` ; режим поиска не «не ищу»; категория подтверждена тестом; сессия не на проверке у методиста; возраст не меньше 16 лет (16–17 лет — только брифы с флагом «стажировка»); компания не в блок-листе кандидата. Отозвав согласие, перейдя в «не ищу» или заблокировав компанию, кандидат пропадает даже из уже собранных версий подборки (места остальных не пересчитываются).

2. Конвейер



Источники признаков кандидата (`app/services/matching/sources.py` , `features.py`): квалификация (θ̂, SE, подбаллы модулей), ФСП (VSS по дисциплинам), регулярные задачи, настройки кандидата (минимум зарплаты, форматы, город, релокация, отрасли), заявленные навыки. Строки кэша `matching_features` хранят **абсолютные**

даты («дней назад» считаются при чтении по `Clock`), пересобираются по хукам (тест, профиль, согласие, ФСП, задачи) и по отпечатку источников; максимальный срок жизни — 7 дней, потому что VSS зависит от времени.

3. Разбор брифа

`POST /api/needs/parse` не сохраняет ничего и работает на правилах за ≤ 300 мс (`app/matching/brief_parser.py`, версия `rules-1`).

Шаг	Как
Нормализация и вилка	«к / тыс / 000» → рубли; «на руки / до НДФЛ» ищется рядом с суммой; число без единиц считается рублями только если оно не меньше 10 000. Проверка: `from ≥ 30 000`, шаг 1000, `to ≥ from`; `to ≤ 1,6·from` — норма, 1,6–2,0× — мягкое предупреждение, больше 2,0× — блокировка
Навыки	Словарь `config/skills.yaml`: 349 навыков с синонимами; Aho-Corasick плюс `rapidfuzz` (≥ 90) для опечаток; у каждого чипа цитата-спан в тексте
must и nice	Маркеры в окне ±6 токенов («желательно», «будет плюсом» → nice; без маркера → must). Не больше 5 must; если больше — первые 4 остаются must, остальные nice
Специализация	`P(s) = softmax((2·cue_s + Σ вес навыков + 1,5·cos(эмбеддинг, прототип)) / 1,5)`. Эмбеддинг-член подключается через `embed_similarity` и по умолчанию равен 0 (модели в рантайме нет — «в разработке»)
Грейд	Явные маркеры («Middle+», «Senior»), «опыт от N лет», слабый априор по середине вилки (медианы на руки: Стажёр 60, Junior 120, Middle 230, Senior 380 тыс.)
Формат и город	Словарь форматов (офис, гибрид, удалённо) и 50 городов; релокация
Уточняющий вопрос	Не больше одного, если `P(spec) < 0,5`, `P(grade) < 0,45` или не хватает вилки/навыков

Каждое поле несёт источник: «правило», «вручную» или имя LLM-провайдера, и цитату для подсветки в тексте. Правки работодателя (`spec`, `grade`, `must`, `nice`, `formats`, `city`) применяются поверх разбора с источником «вручную» (422 `brief_invalid` при недопустимой правке, например больше 5 must).

LLM (по умолчанию выключена, `VZ_LLM_PROVIDER=none`) может только **добавить** навыки, пропущенные правилами, и только с цитатой, найденной в тексте (`partial_ratio ≥ 90`). В ранжировании LLM не участвует (проверяется тестом). Цифры в демо, которые зависят от разбора брифа, считаются движком, а не вручную.

4. Рекомендация категорий и охват вилки

`GET /api/needs/{need_id}/categories` (`app/matching/coverage.py`):

- `P(c = s×g) = P(s)·P(g)`; показываются до 3 категорий с `P ≥ 0,10` (основная — наибольшая); если в основной категории меньше 15 кандидатов, предлагается смежная (вес смежности не ниже 0,8).
- Для каждой категории: число кандидатов N, **ползунок «верх вилки → охват»**, квартили ожиданий P25–P75 (с шагом 10 тыс. руб.), доля с ФСП и доля активных за 30 дней. Подсказка вида «ваша вилка покрывает 11 из 38; при верхе 300 тыс. — 27 из 38» появляется, если вилка охватывает меньше половины.
- **Защита минимума зарплаты отдельного кандидата**: ползунок показывается при N ≥ 10, ступени гистограммы склеиваются, пока в каждой не окажется не меньше 3 человек, квартили округляются до ступени. Подбором верха вилки точный минимум одного человека не вычислить.
- Вкладка «Как мы проверим» (`GET /api/needs/{need_id}/check`): навыки брифа → модули теста; серый список «тестом не проверяется — проверьте на интервью». Подробности — TESTING_MECHANISM.md, п. 12.

5. Формула Score

```
Score(кандидат | бриф) = 100 · G · Cov · Cond · ( wT·T + wK·K + wF·F + wA·A + wU·U )
```

Пресеты весов (`scoring.yaml → presets`): «Баланс» `T 0,40 · K 0,25 · F 0,15 · A 0,10 · U 0,10` (по умолчанию) и «Стек важнее» `T 0,30 · K 0,40 · F 0,10 · A 0,10 · U 0,10` . Ограничения проверяются при загрузке: `wT ≥ 0,30` , `wF ≤ 0,15` , сумма равна 1. Вклад ФСП поэтому не превышает 15 баллов из 100: сильный ФСП не перевешивает большой разрыв в тесте.

Компонент	Формула	Откуда данные
G — категория	`G = adjS · adjG` (· 0,9 при `BORDERLINE_PASS`). Основная категория: `adjS = 1` . Смежные специализации: Backend↔Fullstack и Frontend↔Fullstack 0,8; Backend↔Frontend 0,3; Backend↔Data/ML, Backend↔DevOps, Backend↔Go, Backend↔Java, Go↔Java 0,5; Backend↔QA 0,3; остальные пары 0. Кандидат с бейджем Fullstack для брифов Backend, Frontend и Fullstack получает `adjS = 1` . Грейд: на ступень выше брифа ×0,85, ниже ×0,80 (дальше — степень)	Квалификация
Cond — условия как ограничение	`Cond = (1 − 0,7·(1 − SalFit)) · (1 − 0,7·(1 − FmtFit))` . Условия выполнены → 1,0 и формула прежняя; ожидание выше вилки или неподходящий формат/город уменьшают Score (минимум ×0,09 при полном несоответствии обоих), но не обнуляют: «Да, если…» остаётся возможным. Разложение c_i домножается на Cond, сумма остаётся равной Score. `SalFit` по-прежнему берётся полосами 5% (точный минимум не восстановить)	Настройки кандидата
Cov — покрытие must	0,8, если подтверждено (`v_k ≥ 0,5`) меньше половины must; иначе 1	K
T — тест	`θ_lcb = θ̂ − SE(t)` , где `SE(t)² = SE² + 0,02·Δмес` . `T = доверие · Pct_категории(θ_lcb)` ; при N категории меньше 30 — `Φ((θ_lcb − μ_g)/σ_g)` по калибровочным `μ_g, σ_g` грейда. Доверие 1,0 (нет флагов) или 0,75	Тест
K — стек	`K = Σ w_k·v_k / Σ w_k` , `w = 1,0` (must) или `0,4` (nice). `v_k = max( 1,0·test_k ; 0,9·q·2^(−Дд/90) ; 0,8·m(d,k)·VSS_d/100 )` . `test_k` — максимум по модулям навыка величины `P(θ_m ≥ порог грейда брифа)` , если модуль отмечен `✓` или `✓✓` . Второй член — задачи типа A. Третий — ФСП как доказательство навыка. **Заявленных навыков в `v_k` нет**	Тест, задачи, ФСП
F — ФСП	`F = max_d m(d, специализация брифа) · VSS_d / 100` ; нет истории → **F = 0** (не штраф, в «против» не попадает). `VSS_d` строится из `points = C · Q · Ro · T · V` по лучшим 5 результатам с убывающей отдачей; **мероприятия не ранжируются** (уровень и статус веса не дают, DECISIONS п. 29), класс результата `C` : призёр 1,0 / участник 0,5, участий без призового места учитывается не больше 2. Релевантность дисциплины специализации (`m`) и свежесть (полураспад 24 мес.) — разрешены. ФСП на грейд не влияет	Реестр ФСП
A — свежесть	`A = max_r min(1, q_r·(1 + 0,25·[тег задачи ∈ навыки брифа])) · 2^(−Дд_r/45)` по задачам A и B за 180 дней	Регулярные задачи
U — условия	`U = 0,6·SalFit + 0,4·FmtFit` . `SalFit = 1` , если минимум кандидата ≤ верха вилки, иначе `max(0, 1 − (min/to − 1)/0,25)` с округлением превышения **вверх до полосы 5 %** (значения 1; 0,8; 0,6; 0,4; 0,2; 0), чтобы точный минимум не восстанавливался из вклада. `FmtFit` — максимум по матрице совместимости форматов; офис/гибрид в другом городе без релокации — 0; пересчёт «на руки ↔ до НДФЛ» по ставке 13 %	Настройки кандидата

Когда в брифе нет навыков, K считается по модулям стандартного теста категории брифа (веса `blueprint`).

Зарплата всегда с базисом. Каждая сумма в интерфейсе подписана «на руки» или «до вычета НДФЛ»: вилка брифа (`needs.salary_net`), приглашение и вакансия (`salary_net` / `salary_type`), ожидания кандидата (`candidate_profiles.salary_type`), встречное условие «Да, если…» (в базисе приглашения). Без пометки в тексте брифа берём «на руки» и говорим об этом работодателю. Подписи — `config/contact.yaml → salary.types` ; поле базиса уже есть во всех сущностях, отдельной миграции не требуется.

Чего в Score намеренно нет: близости текста брифа к тексту резюме или профиля, самозаявленных навыков, возраста, пола, фото, вуза, отрасли интереса. Эмбеддинги в ранжировании не участвуют.

Контрольные примеры

Три примера зафиксированы в `tests/test_matching_scoring.py` и обязательны для прохождения (`G = Cov = 1` , пресет «Баланс»):

Кандидат	T	K	F	A	U	Score	Вклады T + K + F + A + U
A, призёр ФСП	0,91	0,80	0,72	0,6	1,0	**83,2**	36,4 + 20,0 + 10,8 + 6,0 + 10,0
B, без ФСП	0,95	0,85	0	0,9	1,0	**78,3**	38,0 + 21,3 + 0 + 9,0 + 10,0
C, сильный ФСП, слабый тест	0,60	0,60	0,95	0,2	0,8	**63,3**	24,0 + 15,0 + 14,3 + 2,0 + 8,0

Так выполняется условие «выше — у кого лучше результаты тестирования **и** есть достижения ФСП»: B без ФСП опережает C с сильным ФСП, а A с обоими преимуществами идёт первым.

Детерминизм

Сортировка: `подтверждённые тестом` → `Score ↓` → `T ↓` → число совпавших заявленных навыков ↓ (только для разрыва ничьих) → `hash(need_id, candidate_id)`. Сравнение Score и T идёт с округлением до 9 знаков (защита от шума float). В каждую версию подборки пишется `scoring_config_version` — версия плюс sha256 **всех** входов формулы (`scoring.yaml` , матрицы ФСП, пороги грейдов и blueprint, граф «навык → модули»): правка любой матрицы не сможет незаметно поменять ранги.

Навыки, наблюдаемые через проверочный набор

Потолок P@10 давали must-навыки, которых стандартный тест внутри модуля не видит: Redis, очереди (Kafka, RabbitMQ, Celery), особенности FastAPI и PostgreSQL. Резюме в Score не входит, поэтому честный путь один: сделать навык **наблюдаемым** короткой задачей из проверочного набора по вакансии (TESTING_MECHANISM.md §12).

- **Граф.** У таких навыков в `config/skills.yaml` есть «свой» модуль (`redis_core` , `mq_core` , `fastapi_core` , `pg_core`), которого нет ни в одном blueprint (`SkillCatalog.check_only_modules`). 11 семейств `families/skills/` с ключом-исполнением и тегом `observes` .
- **План набора** (`plan_check_set`): слоты модулей вне стандартного теста категории идут первыми; семейство выбирается под навык (`observes`) и ближе к грейду брифа; на задачу ставятся только навыки, которые оно наблюдает.
- **«Как мы проверим».** Навык со своим модулем, попавший в слот, получает `via = "check_set"` : «проверяется проверочным набором по вакансии (модуль «…»)». Без слота остаётся прежняя подпись («проверено стандартным тестом» или «проверьте на интервью»).
- **K.** Задача по навыку (`tags` /модуль набора) — прямое свидетельство. Решена (v ≥ 0,5) — значение входит в максимум, как у любого свидетельства (Redis и очереди из «нет данных» становятся «подтверждено задачей»; у PostgreSQL и FastAPI общий модуль теста уже давал подтверждение, поэтому успех ничего не меняет). Не решена — вытесняет вывод по соседнему модулю: чип «Задача … 0/5», состояние «проверено, не подтверждено», навык не засчитывается в Cov. Нет задачи — «не проверено»: чип как раньше. Веса Score не менялись; контрольные примеры 83,2 / 78,3 / 63,3 проходят.
- **Грейд и θ** не затрагиваются: модули набора вне blueprint, `apply_to_theta` их пропускает.

Оценка на синтетической популяции (`python -m eval.checkset_eval`): у кандидатов скрытый уровень таких навыков θ + N(0; 0,6²) (`population.Truth.skill_theta`), оракул считает навык покрытым при уровне ≥ порога грейда − 0,5; набор получают 30 человек из верха выдачи, ответ — σ(2·(уровень − порог)). Результаты — в таблице ниже.

P@10, 40 брифов (test + dev)	оракул со скрытыми уровнями навыков	прежний оракул
Наш подбор без набора («до»)	0.657 [0.573; 0.732]	0.660 [0.573; 0.737]
Наш подбор после набора	0.705 [0.627; 0.778]	0.707 [0.630; 0.780]
B2 (категория + точечная оценка)	0.520 [0.415; 0.622]	0.522 [0.415; 0.623]
Прирост набора, парный бутстреп	+0.048 [+0.017; +0.082]	+0.048 [+0.017; +0.083]

P@10, 40 брифов (test + dev)	оракул со скрытыми уровнями навыков	прежний оракул
После набора минус B2	+0.185 [+0.095; +0.282]	+0.185 [+0.092; +0.280]

Набор получают 30 человек из верха выдачи (`CHECKSET_SHORTLIST`); ответ на задачу — σ(2·(уровень навыка − порог)); при более пологой задаче (крутизна 1,2) прирост +0,045 [+0,012; +0,083]. 95 % ДИ — парный бутстреп по брифам (2000 повторов). Это симуляция: уровни навыков и модель ответа — допущения генератора, а не измерения на людях. Вклад по отдельным навыкам не раскладывали.

6. Объяснения: 3 «за» и 1 «против»

Без LLM (`app/matching/explain.py`):

1. **Разложение**: `c_i = 100·G·Cov·Cond·w_i·x_i` , `Σ c_i = Score` (допуск ±0,1 после округления методом наибольших остатков; проверяется тестом). Работодатель видит полосу из 5 вкладов и множители G, Cov, Cond (с частями по зарплате и формату) в панели доказательств; они же пишутся в снимок версии подборки (`Cond` , `cond_parts`), старые версии без них читаются как Cond = 1.
2. **«За»** — до трёх компонентов, у которых `c_i − медиана_версии(c_i) > 1` , по убыванию превышения.
3. **«Против»** — **первый сработавший** флаг по приоритету (`explain.con_priority`): `salary_above` (ожидания выше вилки) → `format_mismatch` (формат/город) → `category_unconfirmed` → `must_unconfirmed` (обязательный навык не подтверждён) → `borderline` (тест сдан на границе) → `low_trust` → `stale_test` (тест старше 12 месяцев) → `weak_component` (`c_i − m < −3` , **кроме F**) → `nice_unconfirmed` .
4. **Отсутствие ФСП в «против» не попадает никогда.** Для кандидата «по опросу» вместо «ниже медианы» пишется «категория по опросу, не подтверждена».
5. Каждый пункт — шаблон со ссылкой на доказательство: попытку теста, задачу, запись ФСП, настройки, навык брифа. Непроверяемый must формулируется без ссылки на резюме: «Kafka не подтверждена — проверьте на интервью»; заявленное показывается серым чипом. Довод «Стек» ссылается на **все** источники подтверждённых навыков (каждый модуль теста со статусом ✓/✓✓, задача, запись ФСП), а довод «Свежесть» — на все задачи окна: `v_k` и A берут максимум, и если назвать только лучший источник, то после его удаления Score упал бы меньше обещанного превышения над медианой. Это проверяется контрфактом (VALIDATION.md §4.6).
6. **«Почему #2 выше #3»** (`GET /api/needs/{need_id}/versions/{n}/neighbors/{rank}`): разница баллов, главный плюс и главный минус.
7. **Кандидату** в «почему вы» уходят те же «за», переписанные на «вы», без сравнения с другими.

Чипы навыков: зелёный «подтверждено» (с источником «Тест: модуль SQL ✓✓», «Задача … 4/5», «ФСП: алгоритмическое»), жёлтый «слабо», серый «заявлено» или «нет данных». Честность объяснений проверена на симуляции: ссылки на существующие доказательства, `Σ вкладов = Score` — 100 % (VALIDATION.md §4.6).

7. Версии подборки без потери

Свойство	Реализация
Неизменяемость	`POST /api/needs/{need_id}/versions` собирает версию `n` (`need_versions`): снимок брифа, категории, фильтры, пресет, веса, `scoring_config_version` , до 500 строк `items` с Score, компонентами, причинами и `hidden_by[]`
Фильтры не удаляют	Любой фильтр — `has_fsp` , `salary_fit` , `format` , `city` , `must_confirmed` , `skill` , `min_score` , `active` , `industry` , `grade` , `confirmed_only` , `exclude` — заполняет `hidden_by` и причину; отфильтрованные уходят в группу «Скрыто фильтрами (N)». Фильтр «есть ФСП» включает только сам работодатель
Правка = новая версия	Правка брифа, фильтра или пресета создаёт версию `n+1` ; бейджи `новый` , `▲3 / ▼2` — разница с предыдущей (`compare_to`)
Шортлист отдельно	`PUT /api/needs/{need_id}/shortlist/{ref}` , до 30 на бриф; закреплённый кандидат, не прошедший фильтр, остаётся с жёлтой меткой и причиной

Свойство	Реализация
Откат	`POST /api/needs/{need_id}/rollback` переносит указатель на одну из последних 10 версий **без пересчёта**; бриф возвращается к снимку версии, шортлист не меняется
Страницы	Вкладки «По категориям» (до 50 на категорию) и «Общий топ» (кандидат один раз), страницы по 10
Журнал показов	`impression_log` (кандидат, версия, ранг) — основа «как меня видят» и онлайн-метрик воронки

Слабое соответствие (Score ниже 40) — предупреждение при приглашении, не блокировка.

8. Приватность до раскрытия

Работодатель до взаимного «да» (или отклика) работает только со слепой проекцией; границы задаёт белый список `config/contact.yaml → visibility`: поля, которых в матрице нет, не попадают никуда.

Данные	До раскрытия (`CandidateBlind`)	После «да» и согласия (`CandidateDisclosed`)
Идентификатор	Псевдоним «К-4821» = `HMAC(VZ_PSEUDONYM_SECRET, компания ‖ кандидат)` и непрозрачная ссылка `ref`; своя у каждой компании, две компании не сопоставят карточки	То же плюс разрешённые поля
Тест	Полоса «топ-10 / 25 / 50 %» и интервал на линейке с шагом 10 баллов	Точный балл и модули
ФСП	Дисциплина, уровень, год, диапазон места («топ-3», «топ-10»), роль; **без** номера ЕКП, команды, протокола	Выписка с номером ЕКП, местом, командой, ссылкой на протокол
Зарплата	Значок «в вилке ✓ / выше вилки» (по умолчанию) или число, если кандидат так выбрал	То же
Место	Регион (субъект РФ)	Город
ФИО, фото, контакты, резюме, работодатель, вуз	Скрыты (в слепой проекции ПДн-ключи вычищаются на любой глубине)	Только поля, разрешённые кандидатом (`field_visibility`); минимум один канал связи обязателен
Возраст, пол, дата рождения, внутренние id	Никогда	Никогда
Доказательства	Внутренние id попыток теста, задач и записей ФСП наружу не уходят: непрозрачные HMAC-ссылки в рамках компании	То же

Комментарии кандидата, уходящие компании до раскрытия, проверяются на контакты и ссылки (как стоп-слова). Контрактные тесты проверяют, что в ответах работодателю до согласия **0** полей ПДн. Компания видит кандидата, только если открыла карточку (`profile_views`) или закрепила его в шортлисте — иначе пригласить нельзя. Подробнее о защите — SECURITY.md.

8.1. Почему карточка слепая до взаимного «да»

Правила хакатона разрешают показывать работодателю имя и прежних работодателей до согласия кандидата. Мы сознательно этого **не делаем** и ограничиваем показ ровно тем, что нужно для решения «приглашать или нет»:

- **Меньше предвзятости.** Имя, фото, пол, возраст, вуз и громкие бренды в резюме запускают оценку по ярлыку, а не по навыкам. Решение принимается по тому, что измерено: категория «направление × грейд», результат адаптивного теста (полоса, а не точный балл), подтверждённые достижения ФСП и выполненные задачи.
- **Основания в литературе.** Слепые прослушивания оркестров увеличили долю женщин, проходящих отбор (Goldin & Rouse, «Orchestrating Impartiality», American Economic Review, 2000; размер эффекта оспаривался, направление не отвергнуто). В полевом эксперименте с рассылкой одинаковых резюме с «белыми» и «чёрными» именами обратную связь получали заметно чаще первые (Bertrand & Mullainathan, «Are Emily and

Greg More Employable than Lakisha and Jamal?», American Economic Review, 2004). Анонимизация резюме — рекомендуемая практика для таких процессов.

- **Честность для кандидата.** Раскрытие имеет цену: контакты и ПДн уходят только после его «да» и только в разрешённом им составе (152-ФЗ, минимизация).
- **Не «бедная» карточка.** Работодатель видит достаточно для оценки практической применимости: категорию и вилку, модули теста ✓/✓✓, свежесть, задачи, обобщённое ФСП, регион, формат, сферу бизнеса и **обезличенный опыт** — должность и срок («Backend, 2 г. 3 мес.») без названий компаний (`work_history` ; название компании вычищается и из текста должности). Опыт и резюме — «заявлено кандидатом»: они отображаются, но **не влияют** ни на категорию, ни на Score (DECISIONS п. 1).

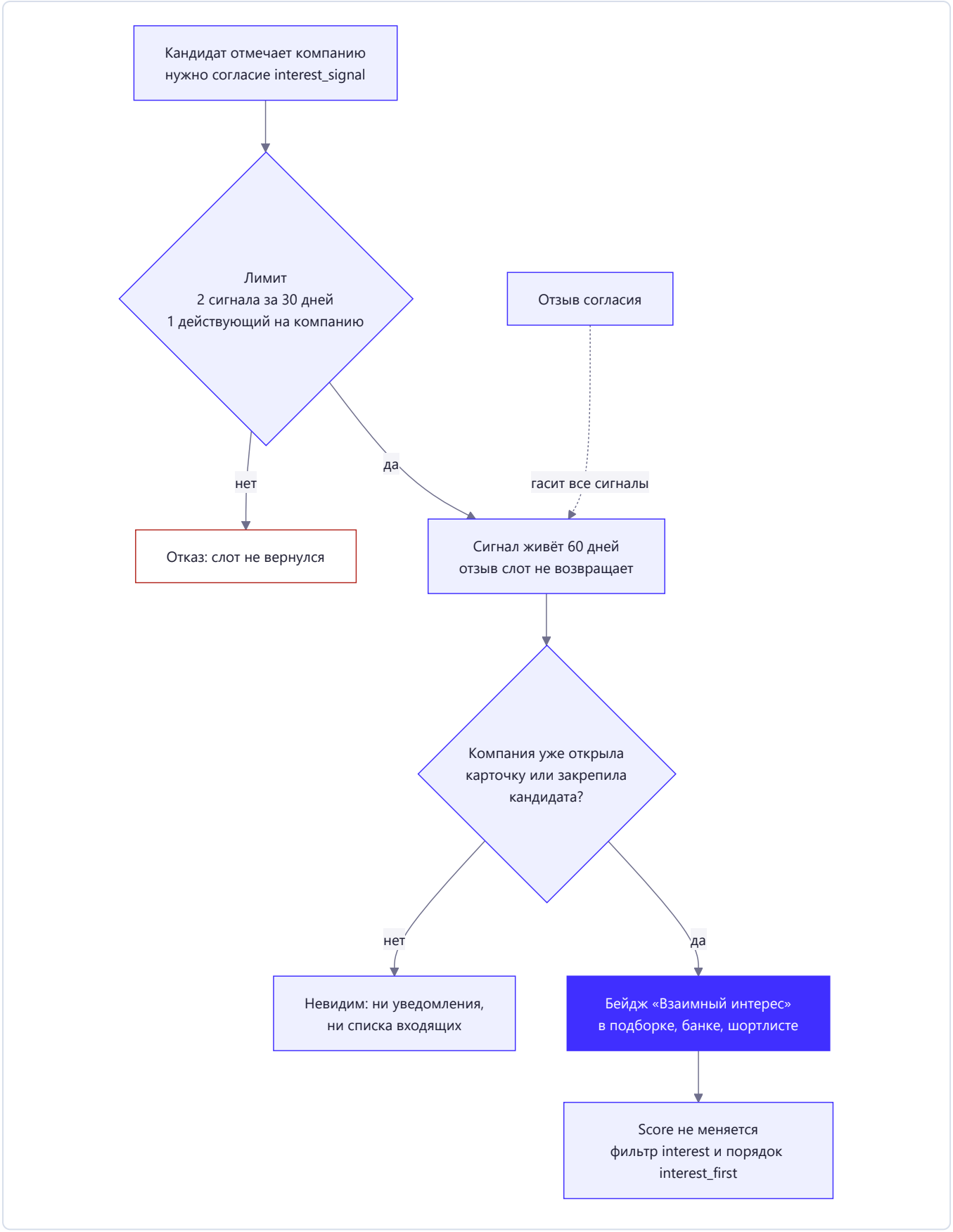
Названия прежних работодателей, ФИО и вуз раскрываются вместе с контактами — по разрешению кандидата после взаимного «да».

Терминология. Направление ИТ («Backend (Python)», «DevOps / SRE» и т. д.) в интерфейсе везде называется «Направление»; фасет интереса кандидата «финтех, ритейл, госсектор…» — «Сфера бизнеса» (в конфиге `industries`). Слово «отрасль» в интерфейсе не используется, чтобы не путать индустрию и IT-направление.

9. Сигнал интереса

Вторая сторона обратной механики (DECISIONS 24): кандидат может отметить компанию, которая ему интересна. **Реализовано** (`POST /api/interest` , `GET /api/interest/me` , `POST /api/interest/{id}/revoke` ; кнопка «Отправить сигнал интереса (осталось 2 из 2)» в карточке вакансии).

Правило в одной схеме:



- **Лимит и срок** (`config/contact.yaml → interest`): 2 сигнала за скользящие 30 дней, каждый виден 60 дней, отзывается; отзыв слот **не возвращает** (иначе сигналы можно «качать»). Один действующий сигнал на компанию.
- **Видимость.** Работодатель не получает уведомления и списка входящих. Бейдж «Взаимный интерес» (подсказка: «Кандидат отметил интерес к вашей компании») появляется в подборке, банке, шортлисте и

карточке **только у тех кандидатов, чью карточку компания уже открыла или закрепила** (DECISIONS 7); проверка пакетная в `visible_for_company`. Остальные компании сигнала не видят. От «закрепить всех, чтобы прочитать сигналы» защищает лимит шортлиста (30 на бриф).

- **Score не меняется.** Сигнал показывается отдельным блоком в панели объяснения («в Score не входит»); в банке по нему можно отфильтровать (`interest=true`) или поставить отметивших первыми (`interest_first=true`) — порядок остальных сохраняется. Тест `test_bank_badge_filter_sort_and_score_unchanged` проверяет, что Score и порядок без флагов те же.
- **Согласие.** Отдельное согласие `interest_signal` (журнал согласий); отзыв согласия гасит все сигналы.
- Для односторонне инициированного контакта у кандидата остаётся **отклик на вакансию** (`POST /api/applications`) с отдельным согласием на передачу контактов; сигнал — не отклик, без интереса работодателя он невидим.

9.1. Симуляция рынка: квоты и сигнал интереса

Чтобы выбрать числа для `config/contact.yaml` (квоты, недельный лимит, два сигнала) и честно показать границы механизма, мы сыграли рынок приглашений «компания — кандидат» на синтетических агентах (`eval/market_sim.py`, детерминированно: `numpy.random.default_rng(seed)`). Это имитация с допущениями, а не измерение живого рынка. Опора — Coles, Kushnir, Niederle, «Preference Signaling in Matching Markets» (AEJ: Micro, 2013; NBER w16185): один дефицитный сигнал интереса помогает работникам и числу совпадений, лучше всего на сбалансированных рынках; и Kushnir, «Harmful Signaling in Matching Markets» (FEEM, 2010): при очень похожих предпочтениях сигналы могут навредить.

Модель в двух строках. Полезность кандидата от компании = общая «престижная» часть (вертикальная) + личная (горизонтальная); доля общей части λ. Работодатель видит Score с шумом, кандидат читает не больше 5 приглашений в неделю, отвечает «Да» выше личного порога. Режимы: (а) без лимитов — 30 приглашений в неделю на вакансию; (б) квоты из `config/contact.yaml`; (в) квоты + сигнал: работодатель, открывший карточки, при выборе, кого пригласить, учитывает известную вероятность «Да» у сигналящих (в Score сигнал не входит).

Политика берётся из `config/contact.yaml`: активных приглашений на бриф ≤ 20, новых за 7 дней ≤ 60, пакет ≤ 10, новых приглашений кандидату ≤ 5 в неделю; сигналов 2 за окно ≈ 4 нед., живёт ≈ 9 нед. Прогонов на ячейку: 60 сидов (одна и та же популяция во всех режимах).

Сбалансированный 1:1.

Предпочтения	Режим	Заполнено вакансий	Недель до первого «Да»	Gini приглашений	Доля топ-10% кандидатов	Впустую	Приглашений на вакансию	Выигрыш кандидатов
Разные вкусы (λ=0,2)	(а) без лимитов	96,2%	1,15	0,30	21%	85%	36,9	1,25
	(б) квоты	95,9%	1,44	0,33	27%	69%	7,7	0,82
	(в) квоты + сигнал	96,2%	1,39	0,32	28%	65%	7,4	1,04
Смешанные (λ=0,5)	(а) без лимитов	90,6%	1,70	0,38	25%	89%	44,2	0,97
	(б) квоты	88,6%	2,30	0,45	34%	80%	10,0	0,64
	(в) квоты + сигнал	89,1%	2,26	0,45	34%	78%	9,8	0,79
Похожие вкусы (λ=0,9)	(а) без лимитов	70,2%	3,47	0,39	24%	94%	68,5	0,51

Предпочтения	Режим	Заполнено вакансий	Недель до первого «Да»	Gini приглашений	Доля топ-10% кандидатов	Впустую	Приглашений на вакансию	Выигрыш кандидатов
	(б) квоты	62,8%	4,15	0,51	34%	88%	10,9	0,39
	(в) квоты + сигнал	62,4%	4,19	0,51	35%	87%	10,8	0,41

Парная разница «квоты + сигнал» − «квоты» (среднее ± 95% ДИ по сидам):

Предпочтения	Δ заполнено	Δ выигрыш кандидатов	Δ недель до первого «Да»
Разные вкусы (λ=0,2)	0,3 ± 0,2 п.п.	0,224 ± 0,013	-0,06 ± 0,02
Смешанные (λ=0,5)	0,5 ± 0,5 п.п.	0,145 ± 0,009	-0,04 ± 0,03
Похожие вкусы (λ=0,9)	-0,3 ± 0,5 п.п.	0,027 ± 0,005	0,04 ± 0,02

Кандидатов вдвое больше.

Предпочтения	Режим	Заполнено вакансий	Недель до первого «Да»	Gini приглашений	Доля топ-10% кандидатов	Впустую	Приглашений на вакансию	Выигрыш кандидатов
Разные вкусы (λ=0,2)	(а) без лимитов	100,0%	1,03	0,39	26%	78%	32,0	0,63
	(б) квоты	100,0%	1,15	0,34	23%	66%	8,7	0,46
	(в) квоты + сигнал	100,0%	1,12	0,34	23%	61%	8,5	0,59
Смешанные (λ=0,5)	(а) без лимитов	99,3%	1,30	0,47	33%	85%	41,2	0,50
	(б) квоты	96,7%	1,76	0,44	31%	79%	11,7	0,36
	(в) квоты + сигнал	96,7%	1,75	0,44	31%	78%	11,6	0,43
Похожие вкусы (λ=0,9)	(а) без лимитов	80,9%	3,00	0,39	27%	95%	89,6	0,27
	(б) квоты	69,7%	3,80	0,60	43%	90%	15,6	0,21
	(в) квоты + сигнал	69,7%	3,80	0,60	43%	89%	15,6	0,22

Парная разница «квоты + сигнал» − «квоты» (среднее ± 95% ДИ по сидам):

Предпочтения	Δ заполнено	Δ выигрыш кандидатов	Δ недель до первого «Да»
Разные вкусы (λ=0,2)	-0,0 ± 0,0 п.п.	0,132 ± 0,007	-0,02 ± 0,01
Смешанные (λ=0,5)	0,0 ± 0,2 п.п.	0,073 ± 0,006	-0,01 ± 0,01
Похожие вкусы (λ=0,9)	0,0 ± 0,4 п.п.	0,012 ± 0,002	-0,00 ± 0,02

Сколько сигналов (сбалансированный рынок).

Сигналов за окно	Смешанные: заполнено	Смешанные: выигрыш канд.	Похожие: заполнено	Похожие: выигрыш канд.
0	88,6%	0,64	62,8%	0,39
1	88,9%	0,74	62,8%	0,40
2	89,1%	0,79	62,4%	0,41
3	89,2%	0,82	62,7%	0,42
5	89,4%	0,84	62,7%	0,43

Размер пакета приглашений (смешанные вкусы, сбалансированный рынок, без сигналов).

Пакет приглашений в неделю	Заполнено	Впустую	Gini	Недель до первого «Да»
3	83,0%	79%	0,44	3,03
5	86,6%	80%	0,44	2,64
10	88,6%	80%	0,45	2,30
20	89,3%	79%	0,44	2,12
40	89,5%	78%	0,43	2,04
без лимита (30)	90,6%	89%	0,38	1,70

Недельный лимит входящих у кандидата (смешанные вкусы).

Лимит в неделю	Заполнено	Отклонено при отправке	Впустую
2	81,2%	85%	78%
3	85,3%	77%	79%
5	88,6%	63%	80%
10	90,1%	39%	79%
20	90,4%	17%	81%

Когда сигнал не помогает. Наивная стратегия — сигнал на лучшую по полезности компанию без оценки шансов:

Предпочтения	Режим	Заполнено	Выигрыш кандидатов	Недель до первого «Да»
Смешанные (λ=0,5)	квоты	88,6%	0,64	2,30
	квоты + сигнал (по шансам)	89,1%	0,79	2,26
	квоты + сигнал (наивно)	88,8%	0,72	2,30
Похожие вкусы (λ=0,9)	квоты	62,8%	0,39	4,15
	квоты + сигнал (по шансам)	62,4%	0,41	4,19
	квоты + сигнал (наивно)	62,8%	0,40	4,18

Что показала симуляция.

- *Квоты — это обмен заполнения на нагрузку.* На сбалансированном рынке со смешанными вкусами рассылка без лимитов заполняет 90.6% вакансий, квоты — 88.6%, но на вакансию уходит 10 приглашений вместо 44 (нагрузка на кандидатов в 4,4 раза ниже), а доля приглашений впустую 80% против 89%. При очень похожих вкусах потеря заполнения заметнее (70.2% против 62.8%). Концентрацию квоты не лечат: Gini 0.45 против 0.38, доля топ-10% кандидатов 34% против 25% (при малом числе приглашений работодатели чаще выбирают лучших по Score). Выигрыш квот — защита кандидата от потока писем, а не равномерность; выигрыш кандидатов от рассылки без лимитов (0,97 против 0,64) — это цена, которую мы платим осознанно.

- *Сигнал возвращает часть потерянного для кандидата.* Против одних квот на сбалансированном рынке сигнал меняет число заполненных вакансий на: разные вкусы +0.3 ± 0.2 п.п., смешанные +0.5 ± 0.5 п.п., похожие -0.3 ± 0.5 п.п. (парное среднее ± 95% ДИ) — то есть почти не меняет. Зато выигрыш кандидатов растёт: смешанные +0.145 ± 0.009 (до 0,79 против 0,64 у одних квот), похожие +0.027 ± 0.005. Читается так: сигнал улучшает **качество** совпадений для кандидата, а не их число; это согласуется с Coles–Kushnir–Niederle (выигрыш на стороне работников, эффект для фирм неоднозначен).
- *Границы (вредный сценарий).* При очень похожих вкусах (λ=0,9) все хотят одних и тех же компаний, сигнал не помогает ни числу совпадений, ни скорости первого «Да»: выигрыш кандидатов +0.027 ± 0.005 против +0.145 ± 0.009 при смешанных. Сигналы уходят в одни и те же места — это тот случай, о котором пишет Kushnir. Заметного вреда, как в его теоретическом примере, в нашей модели не получилось (у работодателя сигнал лишь меняет порядок внутри квоты), поэтому мы заявляем только «нет пользы при общих вкусах», а не «есть вред». Наивная стратегия (сигнал на самую желанную компанию без учёта шансов) выигрыш кандидата снижает по сравнению со стратегией «по шансам» (0,72 против 0,79 при смешанных вкусах), поэтому продукт подсказывает сигнал только по вакансиям, где Score кандидата высок.
- *Допущения, которые могут не держаться на живых данных:* независимые предпочтения кроме λ, одна вакансия на компанию, фиксированные пороги «Да», нет стратегического поведения работодателей, все кандидаты используют сигналы, рынок закрыт на 8 недель. Размеры квот проверены как чувствительность, а не оптимизированы под единственный сценарий.

Решение по настройкам. значения `config/contact.yaml` оставлены без изменений: пакет 10, активных на бриф 20, недельный лимит кандидата 5, сигналов 2 за окно. Пакет: заполнение растёт с размером пакета и выходит на плато (3 — 83.0%, 10 — 88.6%, 40 — 89.5%), колено на 10: дальше +0.8 п.п. ценой 1.1× приглашений. Недельный лимит: 5 даёт 88.6% заполнения против 90.1% при 10, но отсекает больше рассылки (63% против 39% отклонённых при отправке); 5 — осознанный выбор в пользу кандидата (≈1.5 п.п. заполнения), работодателю это видно в квоте и его можно ослабить настройкой кандидата (3/5/10/без ограничения). Сигналы: 0 → 2 → 5 дают выигрыш кандидатов 0.64 → 0.79 → 0.84; два сигнала берут 73% пятисигнального прироста, а дефицитность (смысл сигнала по Coles и др.) сохраняется, поэтому оставлено 2.

10. Кандидаты без ФСП

Требование: отсутствие истории ФСП не должно ухудшать путь кандидата.

Состояние	Что видит работодатель	F	«Против»
ФСП ID не привязан	Серая строка «Достижения ФСП не привязаны — оценка по тесту и задачам»	0	Никогда
Привязан, результатов нет	«Участник ФСП, результатов пока нет»	0	Никогда
Есть только результаты, ждущие протокола	Ничего; у кандидата «Ожидает протокол»	0 до подтверждения	Никогда
Результат скрыт кандидатом	Не участвует ни в F, ни в K, ни в выдаче (иначе — скрытая накрутка)	не учитывается	Никогда

Тот же тест, тот же грейд, та же категория: ФСП влияет только на порядок внутри категории и на подтверждение навыков, но не на грейд. Свой путь наверх у кандидата без ФСП — регулярные задачи (компоненты K и A). Гарантии: тест «100 % профилей без ФСП получают категорию по тем же правилам»; метрика отношения экспозиции (при равном тесте).

Честный результат симуляции (VALIDATION.md §4.5): условное отношение экспозиции кандидатов без ФСП в топ-50 — 0,90 (цель 0,8–1,25, выполнено), в топ-10 — 0,75 (**цель не выполнена**: при равном тесте F поднимает обладателей ФСП над такими же кандидатами без ФСП, и в верхушке из 10 мест это заметно). Вес `wF` вынесен владельцу подбора как открытый вопрос.

11. Банк кандидатов без брифа

`GET /api/bank` — поиск без брифа: специализация, грейд, категория, стек (переключатель «только подтверждённые»), «есть ФСП», отрасль, режим поиска. Сортировка `0,55·T + 0,25·F + 0,20·A`, затем T, затем детерминированный ключ; кандидат показывается один раз — лучшей категорией; фильтры никого не удаляют, а заполняют «Скрыто». Закрепление кандидата в шортлист брифа — `PUT /api/needs/{need_id}/shortlist/{ref}`. Версии и шортлист живут на бэкенде: переход в банк и обратно ничего не теряет.

Кандидаты с неподтверждённым грейдом (ответы организаторов, DECISIONS п. 29): их **показываем со статусом «грейд не подтверждён» и ранжируем ниже всех подтверждённых**, а не скрываем. Это кандидат, который прошёл опрос и заявил категорию, но тест не пройден или не подтвердил грейд, и специализации «только опрос» (Data/ML, Mobile, ИБ, Go, Java). Правила:

- **Откуда берётся заявленная категория.** Грейд, который кандидат сам выбрал для последней попытки теста; иначе специализация из опроса и подсказка грейда опроса. Резюме и самоописание категорию не задают и в Score не входят.
- **Где показываем.** Банк и подборка: хвост после всех подтверждённых (ключ сортировки `confirmed_first`, `config/scoring.yaml → tie_break`), бейдж «грейд не подтверждён» и строка объяснения «Категория заявлена, тестом не подтверждена» (первое «против»). В карточке работодателя подтверждённые и заявленные категории — отдельными списками. Фильтр «Только подтверждённые тестом» убирает хвост в «Скрыто фильтрами».
- **Что не меняется.** Те же жёсткие правила видимости: согласие на слепую карточку, режим «не ищу», возраст, блок-лист, паузы пары. Слепой DTO без ПДн. T = 0, поэтому такой кандидат не обгоняет подтверждённого по баллу и не искажает перцентили; охват и счётчики «N кандидатов» в рекомендациях категорий считаются только по подтверждённым.
- **Несколько категорий.** У кандидата может быть по одной квалификации на каждую специализацию (Backend и QA одновременно): банк находит его по каждой, в списке он один раз — лучшей строкой; «Мой путь» и карточка работодателя показывают все.

Для жюри остаётся и `POST /api/admin/import` с кандидатом `features: null` (группа «категория по опросу, не подтверждена»).

Почему Cond, и какие подходы к ранжированию мы рассмотрели

Условия (зарплата, формат, город) работодатель воспринимает как ограничение: кандидат с ожиданием выше вилки или не готовый к формату брифа для него нерелевантен, как бы ни был силён тест. В аддитивной формуле U давал максимум 10 баллов из 100, и такие кандидаты оставались в топе-10. Мультипликатор Cond — это «gate first, rank second» в мягкой форме (двухслойная схема L1/L2): жёсткий фильтр заменён штрафом, чтобы работодатель видел несоответствие в «против», а не терял кандидата молча. Он же — упрощённое «взаимное» согласование (Reciprocal recommenders): односторонний матч штрафуется произведением, а не складывается с остальным.

Что проверили на настроечных брифах (синтетические брифы, другая популяция — `backend/eval/matching_tune.py`) и отложенной выборке (b01–b40):

Подход	Результат	Решение
Подбор весов (перебор по сетке, цель P@10; вместо градиентного/парного обучения, т. к. брифов мало)	После Cond P@10 почти не зависит от весов (плато 0,59–0,61 при T 0,3–0,6, K 0–0,4, F 0–0,15)	Веса «Баланс» не меняем: нечему подгонять, риск переобучения отсутствует
Нижняя доверительная граница θ (EAP − k·SE)	Уже используется (k = 1, рост SE со временем); абляция «без нижней границы» в VALIDATION.md	Оставлено
Взаимная оценка (гармоническое/ произведение)	Условия как произведение дали основной прирост	Принято как Cond

Подход	Результат	Решение
Фильтр, затем ранжирование	Сила штрафа 0,5–1,0 даёт одинаковый результат (плато)	Мягкий вариант 0,7
Родственные навыки (ESCO-подобная таксономия)	К ухудшает P@10 не из-за синонимов, а потому что тест не различает навыки внутри модуля (см. «Ограничения»)	Не вводили

Ссылки на практики: Reciprocal Recommender Systems (взаимное согласование спроса и предложения), двухслойные ранжировщики в рекрутинге (фильтр + оценка), Rasch-ранжирование по нижней границе (консервативная оценка при неопределённости).

12. Ограничения

- **Что показывает симуляция** (VALIDATION.md §4.1): на 20 test-брифах P@10 0,61 [0,49; 0,72] против 0,47 у базового B2 «категория + точечная оценка уровня» (разница 0,15, ДИ [-0,02; 0,30], на 20 брифах незначима); на всех 40 брифах 0,66 против 0,52, разница 0,14 [0,04; 0,24] значима. Предрегистрированная проверка на 60 новых брифах (test2, VALIDATION_PLAN.md §8, один прогон): 0,54 против 0,35, разница +0,19 [0,13; 0,24] значима; абсолютный P@10 ниже порога 0,60, по специализации Fullstack (12 брифов) разница +0,04 незначима. Случайный порядок в тех же категориях и поиск по самоописанию значимо хуже. Цель ≥ 0,75 пересмотрена до ≥ 0,60 (post hoc, VALIDATION_PLAN.md §1.5): потолок для подбора, который не верит резюме, около 0,76. Первоначальный прогон (0,53 против 0,62 у B2) показал две вещи: оракул не учитывал формат и город, а формула считала условия лишь 10% аддитивного бонуса. Исправлены обе (оракул v2, множитель Cond).
- Must-навыки, которые тест не проверяет (Docker, Kafka, Redis и т. п.), формула не может подтвердить: у неё есть только доказательства (тест, задачи, ФСП), самоописанию она не верит.
- Оракул и генератор популяции написаны одной командой; экспертной разметки пар нет. Это проверка согласованности механики с нашей моделью мира, а не мнения рекрутера.
- Эмбеддинги и колонки `vector` в БД — в разработке.
- Банки QA и DevOps включены (`active`), но меньше по объёму, чем Backend/Frontend; «Сигнал интереса» — в разработке (в коде нет).

13. Открытые данные рынка: что взяли, как, лицензия, что не входит в Score

- **Источник.** Открытые вакансии портала «Работа в России» (`opendata.trudvsem.ru/api/v1/vacancies` , публичный API без авторизации). Выгрузка 2026-10-08: 4 906 вакансий, из них ИТ — 1 990. hh.ru не используем: анонимный `GET api.hh.ru/vacancies` отвечает 403, а условия API требуют регистрации приложения. Парсинг — только для наполнения справочников и тестовых данных (разрешено организаторами), не механика продукта: чужие вакансии не показываются, в рантайме внешних запросов нет.
- **Лицензия и данные.** Данные портала открытые; отдельного текста лицензии на странице API мы не нашли, поэтому храним только производные агрегаты (`data/market/`), сырой кэш удалён, контакты и названия компаний в кэш не попадали. Резюме не используются.
- **Что получилось.** (1) Квантили зарплат p10–p90 по направлению × грейду (9 ячеек с n ≥ 8, плюс 8 по направлению), основание — gross (портал основание не различает; `net = gross·(1 − ндфл)`). (2) Частоты навыков и соседи по NPMI — справочно. (3) Нераспознанные термины → 159 новых алиасов и 4 навыка в `config/skills.yaml` (сленг «постгря», «кубер», «пайтест», «раббит», вендоры сетей и виртуализации, MVC/MVVM, UML/BPMN, GigaChat/RAG и др.).
- **Замер разбора брифа** (`eval/brief_parse_eval.py` , отложенная выборка из 62 брифов `eval/brief_heldout.jsonl` , написана вручную живым языком и сленгом): F1 навыков 0,958 → 0,975 (полнота 0,949 → 0,973), F1 must-навыков 0,837 → 0,842, специализация 0,984, грейд 1,000, вилка 0,952, формат 1,000 —

без регрессий. Честная оговорка: ошибки базовой версии на этой выборке мы видели до расширения словаря, поэтому прирост нужно читать как «словарь закрыл типичные сленговые дыры», а не как независимую оценку; на 40 брифах команды (`eval/briefs`) и в `eval.validate --quick` метрики не изменились (P@10 test 0,415 → 0,415, nDCG@10 0,485 → 0,485).

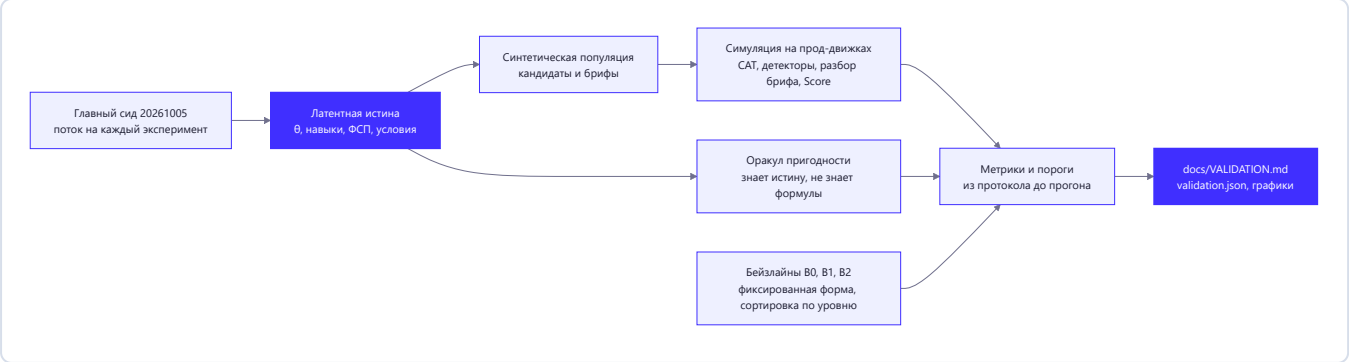
- **Рыночная вилка в форме брифа.** `GET /api/needs/market-salary` → квантили для направления × грейда (если ячейка тонкая — по направлению целиком), в двух основаниях; под полем вилки в форме показываем «Рыночная вилка: p25–p75, медиана, n, источник, дата» в том основании, что выбрал работодатель. Подсказка, а не ограничение.
- **Сигнал доверия** `salary_far_below_market` (`app/trust/flags.py` , `config/trust.yaml`): верхняя граница вилки в пересчёте на gross ниже 0,8 · p25 рынка. Только пометка для очереди модерации.
- **Что не входит в Score.** Рыночные квантили, частоты навыков и соседи навыков не участвуют ни в Score, ни в категории кандидата, ни в ранжировании (в том числе как «частичный зачёт за родственный навык»: по §4 это ухудшало P@10, поэтому карта соседей только справочная).
- **Ограничения данных.** Выборка по всей РФ и смещена в администрирование: devops/сисадмины — 638 из 1 990, разработчиков мало (Python 16, Frontend 16, Go 4, Fullstack 2; для них показываем направление целиком или ничего). Уровень заметно ниже московского — поэтому порог сигнала консервативный. ESCO не подключали: словарь и без него закрывает типичные формулировки, а таксономия ESCO не покрывает рыночный сленг.

План валидации «Взаимно» (протокол до прогона)

Этот документ фиксирует метрики, пороги, бейзлайны и сиды **до** первого прогона. Результаты пишет `docs/VALIDATION.md`, его генерирует команда:

```
cd backend
uv run python -m eval.validate          # полный прогон, < 5 мин
uv run python -m eval.validate --quick  # дымовой прогон, < 1 мин (меньше выборки, те же процедуры)
```

Конвейер валидации:



Правила протокола:

- Используется **тот же код движков, что и в проде**: `app/testing/*` (CAT, EAP, детекторы, калибровка, опрос), реальный банк семейств с экспертными приорами сложности (`app/testing/bank`; на момент записи протокола 69 семейств, на момент последнего прогона — 111, см. VALIDATION.md), `app/matching/*` (разбор брифа, категории, Score, объяснения, версии подборки), `app/fsp/vss.py` (VSS). Формулы в слое валидации не переписываются; модуль `eval/` только генерирует данные, вызывает движки и считает метрики.
- Все случайные величины — через явные генераторы с сидом. Главный сид `20261005`; каждый поток получает свой генератор `random.Random("20261005:<поток>")`, поэтому добавление одного эксперимента не сдвигает остальные.
- Пороги ниже не меняются после прогона. Если метрика не достигает порога, так и пишем в `VALIDATION.md`. Если наш подбор не обгоняет B2 значимо, так и пишем и показываем, где выигрывают объяснимость и устойчивость к накрутке.
- Результаты разделены на «симуляция» (проверяет процедуру) и «люди» (здесь людей нет — см. ограничения).
- В текстах для пользователя (Лаборатория, VALIDATION.md) вместо греческой буквы пишем «уровень».

1. Данные

1.1. Популяция с латентной истиной (`eval/population.py`)

2000 кандидатов (в режиме `--quick` — 300). Скрытая истина и наблюдаемые данные:

Поле	Как порождается
Истинная специализация	Backend (Python) 50%, Frontend (JS/TS) 40%, Fullstack 10%

Поле	Как порождается
Заявленная специализация	У 10% не совпадает с истинной (Backend заявляет Frontend и наоборот)
Истинный грейд	Ниже Стажёра 5% / Стажёр 15% / Junior 30% / Middle 30% / Senior 20%; уровень равномерно внутри полосы грейда (много людей у порогов)
Уровень по модулям	Модули своей специализации: общий уровень + $N(0; 0{,}35^2)$. Модули чужой специализации: общий уровень − 1,5 + $N(0; 0{,}5^2)$
Заявленный грейд	Как в S1: совпадает с истиной в 60%, на ступень выше в 30%, ниже в 10% (`simulate.sample_claimed_grade`)
Тест	Полная CAT-сессия через `simulate.run_simulated_session` на реальном банке: доборы вниз принимаются всегда, вверх — если предложены. Выход: решение, оценка уровня, SE, грейд, «на границе», подбаллы и теги модулей (`cat.module_subscores`)
ФСП	У 35%. Результаты (уровень соревнования, место, размер поля, роль, дата) порождаются из скрытой «силы», коррелированной с уровнем; VSS считает прод-модуль `app/fsp/vss.py` . Целевая корреляция VSS с истинным уровнем ≈ 0,5, фактическая печатается
Регулярные задачи	У 50%: 1–3 задачи, тип A (80%) или B, `q = σ(1,2·(уровень модуля − b)) + шум` , давность 0–180 дней
Самоописание	Истинные навыки + 30% приписанных (несуществующих) навыков. В Score не входит (DECISIONS п.1)
Минимум зарплаты	Центр по заявленному грейду (`scoring.yaml: brief_parser.salary_centers_net`) × lognormal(0; 0,15)
Стаж	lognormal по истинному грейду — для бейзлайна B4 «грейд по стажу»

Одна популяция используется и для тестирования, и для подбора. Тот же генератор пригоден для демо-seed (API описан в docstring `eval/population.py`).

1.2. Наборы для тестирования

- **S1** — вся популяция, честные ответы, точные экспертные b.
- **S2** — ошибка экспертов: истинная сложность = экспертная + $N(0; \sigma^2)$, $\sigma \in \{0; 0{,}3; 0{,}5\}$; неверная крутизна: истинная $A \in \{1{,}0; 2{,}0\}$ при модели 1,5. По 400 кандидатов (quick — 80) на условие.
- **S3** — пересдача: вторая сессия того же человека, все семейства первой сессии исключены (как через 12-месячный запрет повтора). Плюс тест на повышение на грейд выше для проверки гистерезиса.
- **S4** — утечки. 300 «явных Junior» (уровень равномерно в $[-0{,}8; -0{,}3]$) заявляют Middle (Backend). Коалиции из 5 / 20 / 100 честных кандидатов сливают всё, что видели. Утечка метода: тем, кто знает метод семейства, оно легче на $\Delta b \in \{0; 0{,}25; 0{,}5\}$.
- **S5** — дрейф для CUSUM: поток ответов по каждому семейству Backend. Сначала объём, соответствующий 3000 честных сессий, затем ещё 3000 сессий, в которых 20% или 50% проходящих знают метод 12 семейств ($\Delta b = 0{,}5$); контроль — без утечки. Апостериор без задания π_{-j} и ответы берутся из сессий популяции (бутстреп по семейству).
- **Калибровка** — истинная сложность = экспертная + $N(0; 0{,}5^2)$ для неякорных семейств; MAP-уточнение `calibration.calibrate_family` по 0 / 30 / 100 / 300 ответам, 5 повторов.

1.3. V-CAT (категоризация по опросу)

300 синтетических профилей опроса (10 специализаций × 30) и 30 трудных случаев (10 Fullstack, 10 смен профессии, 10 противоречивых опросов). Генератор ответов — своя матрица P(частота действия | специализация) и P(технология | специализация) в `eval/vcat.py` , **не** веса W/V классификатора; 10% шума и 15% «амбициозных» отметок действий соседней специализации. Выбор специализации кандидатом совпадает с истинной в 80%. Классифицирует прод-функция `app/testing/survey.classify` .

1.4. Брифы (`eval/briefs/*.json`)

40 русскоязычных брифов: формальные вакансии, переписка в мессенджере со сленгом и опечатками, короткие запросы, длинные описания с разделами, смесь с английским. У каждого — эталон: специализация, грейд, must и nice (id навыков из `config/skills.yaml`), формат, город, вилка (от, до, на руки / до НДФЛ). **Брифы и эталон**

написаны командой с помощью ИИ-ассистента; посторонних авторов в этом наборе нет. Разбиение 20 dev / 20 test фиксировано в файлах (поле `split`). Веса подбора на этих брифах не подбирались; dev используется только для отладки протокола, все итоговые цифры — по test.

1.5. Оракул релевантности

По скрытой истине кандидата и эталону брифа (концепция §7.1):

```
rel = 0, если смежность(истинная специализация, специализация брифа) < 0,8   (Fullstack ↔
Backend/Frontend = 0,8)
иначе rel = 3, затем
  Δ = |истинный грейд − грейд брифа| (Ниже Стажёра = −1): Δ ≥ 2 → rel = 0; Δ = 1 → −1
  must покрыт ⇔ навык есть в истинном стеке И (навык не тестируется ИЛИ истинный уровень одного из
      его модулей ≥ порог грейда брифа − 0,5)
  покрыто < 50% must → −2; покрыто не всё → −1
  минимум кандидата > 1,1 · верх вилки → −1
  формат/город не подходят (v2, см. ниже) → −1
rel = max(0, rel);  релевантен ⇔ rel ≥ 2
```

Изменение v2 (внесено после первого прогона, до финальных чисел; фиксируем честно). Первый прогон дал P@10 0,53 против 0,62 у B2 «только уровень». Диагностика: внутри правильной категории метка v1 почти не зависела от уровня (AUC θ ≈ 0,52) и зависела от зарплаты и стека; при этом оракул не знал о формате работы и городе, хотя бриф их задаёт (в бриф-эталоне `formats`, `city`). Работодатель не назовёт релевантным кандидата, который хочет только удалёнку при офисе в другом городе, поэтому в метку добавлен пункт «формат/город» (посадка формата ≥ 0,7 по `scoring.yaml → U.format_fit`; город важен только для офиса и гибрида без релокации). Вторая правка — в формуле, а не в метке: условия стали множителем Cond (MATCHING.md §5). Настройка Cond шла на **синтетических брифах и популяции с другим сидом** (`backend/eval/matching_tune.py`), b01–b40 и основная популяция — отложенная выборка. Оракул v1 остаётся доступным (`oracle_rel(..., conditions=False)`); на нём B2 набирает 0,62, наш подбор после Cond — см. VALIDATION.md. **Цель P@10 ≥ 0,75 пересмотрена после наблюдения результатов (post hoc): ≥ 0,60 и значимо выше B0, B1, B3; значимость против B2 — на всех 40 брифах.** Обоснование: потолок для ранжировщика, который не использует резюме и самоописание (правило продукта), — это случайный выбор среди кандидатов, подходящих по всему наблюдаемому (грейд, условия, уровень модулей): ≈ 0,76 на идеальных данных. Стек (Redis, Docker, FastAPI…) тест не различает, а заявленные навыки в Score не входят, поэтому 0,75 достижимо только на пределе и с идеальным разбором брифа.

1.6. Тестовый набор пар для жюри

`eval/pairs_test.jsonl`: для каждого test-брифа — кандидаты из объединения топ-50 всех систем и 10 случайных; строка = (бриф с эталоном, слепые признаки кандидата, rel по оракулу). Схема — `eval/pairs_test.schema.json`, описание — раздел в `VALIDATION.md`.

2. Метрики, пороги, бейзлайны

2.1. Тестирование

Метрики точности и пересдачи считаются по кандидатам, у которых заявленная специализация совпадает с истинной (или истинная — Fullstack). Кандидаты с ошибкой в специализации показываются отдельно.

Пункт	Метрика	Порог
Точность	Верное решение у заявленного порога	≥ 0,85
	Точный грейд / ±1 грейд (после доборов; «не подтверждён» = Ниже Стажёра)	≥ 0,75 / ≥ 0,99
	к и QWK итогового грейда с истинным	показываем

Пункт	Метрика	Порог
Длина	Средняя основная / средняя с доборами / p95 с доборами	≤ 15 / ≤ 17 / ≤ 20
Пересдача (S3)	Совпадение решения / к решения / QWK грейда / корреляция оценок уровня	≥ 0,78 / ≥ 0,55 / ≥ 0,85 / ≥ 0,88
Гистерезис	Ложное повышение: тест на g+1 сдан (P ≥ 0,9), хотя истинный грейд ≤ g	≤ 0,07
Устойчивость к ошибке экспертов (S2)	Точность у порога при σ = 0 / 0,3 / 0,5 и A = 1,0 / 2,0	показываем
Утечка ответов (S4)	Ложный Middle у явных Junior при коалиции 20	≤ 0,15
	Пойман «чужим ключом» (наивный списывающий), если слито ≥ 25% банка	≥ 0,90
Утечка метода	Ложный Middle при Δb = 0 / 0,25 / 0,5	показываем честно
CUSUM (S5)	Найдено слитых семейств, медиана задержки (ответов), доля ложных карантинов; h ∈ {4…8}	при прод-значении h: 50% знающих — медиана ≤ 200 ответов; ложных ≤ 5% (h = 6 до §7, после §7 — 11)
Калибровка	RMSE сложности: эксперты → 30 → 100 → 300 ответов	≤ 0,30 после 100
Дискриминативность	Доля семейств с r_pb ≥ 0,20; ΔP(Middle − Junior) ≥ 0,20; AUC оценки уровня для соседних грейдов; надёжность 1 − mean(SE^2)/var	показываем, флаг r_pb < 0,10
Экземпляры	Пересечение экземпляров у пары кандидатов	0

Бейзлайны тестирования:

- **B0** — фиксированная форма из 16 заданий для всех (по квотам blueprint, самые информативные у порога).
- **B1** — LOFT: случайная форма из 16 заданий по квотам blueprint, без клонов.
- **B2** — наш CAT без клонов: 20 многоразовых экземпляров на семейство.
- **B3** — «LLM под каждого»: CAT, но каждое задание сгенерировано заново; истинная сложность = задуманная + N(0; $0{,}5^2$) **на экземпляр**; доля битых ключей 10% — **допущение**, реальный замер на ответах YandexGPT/GigaChat в этом прогоне не проводился. Чувствительность: 0% / 5% / 20% битых ключей, «идеальный LLM» (σ = 0,3, 0% битых).
- **B4** — грейд по стажу (подсказка опроса `survey.suggest_grade`).

Решение для форм фиксированной длины (B0, B1): «сдал», если P(уровень ≥ порог) ≥ 0,5 — так же, как CAT на максимальной длине (BORDERLINE_PASS засчитывается).

Модель списывания (S4):

- **Наш тест**: экземпляры одноразовые. «Наивный» списывающий вводит слитый ключ того же семейства: для заданий с выбором он угадывает с вероятностью c, для ввода ответа — неверно; флаг «чужой ключ» ставится, если доля коллизий ключей семейства < 2% (доля **измеряется** генерацией 200 экземпляров каждого семейства реального банка). «Осторожный» видит, что код другой, и отвечает честно.
- **B0, B1, B2**: списывающий копирует ответ, только если ему достался тот же экземпляр (идентичный код); иначе отвечает честно. Это консервативно в пользу бейзлайнов.
- **B3**: экземпляры уникальны, слив бесполезен; ложный Middle = честная ошибка B3.

2.2. Категоризация

Метрика	Порог
macro-F1 подсказки классификатора по 300 профилям V-CAT	≥ 0,90
Точность на 30 трудных случаях	показываем
Бейзлайн «выбор кандидата» (без классификатора)	показываем

Метрика	Порог
100% профилей без ФСП получают категорию по тем же правилам	100% (в опросе и тесте нет входов ФСП — проверяется кодом)

2.3. Подбор

Системы (одна и та же популяция, одинаковые брифы):

- **Наш (сквозной)**: разбор брифа → рекомендованные категории (`coverage.recommend_categories`) → версия подборки `selection.build_version` с пресетом «Баланс».
- **Наш (эталонный бриф)**: то же, но поля брифа из эталона — изолирует ранжирование от ошибок парсера.
- **B0** — случайный порядок в тех же категориях (после фильтров).
- **B1** — BM25 по самоописанию по всей популяции, без категорий («как на hh»).
- **B2** — основная категория + сортировка по точечной оценке уровня (минимальное прочтение ТЗ).
- **B3** — косинус TF-IDF «бриф ↔ самоописание» по всей популяции. **Это заменитель эмбеддингов**: модель эмбеддингов в прогоне не используется.
- **Абляции**: без нижней границы (SE = 0), без K, без F, без A, без U (вес снятого компонента передаётся тесту, ограничения wT ≥ 0,30 и wF ≤ 0,15 соблюдаются).

Метрика	Порог
P@10 (главная) по оракулу, test-брифы, сквозной режим; 95% бутстреп-ДИ по брифам	≥ 0,60 (было ≥ 0,75, пересмотрено post hoc, см. §1.5)
P@5, nDCG@10 (выигрыш 2^rel − 1), Recall@50, MRR	показываем
Разница с B2 (парный бутстреп по брифам)	значимо > 0; если нет — пишем
Разбор брифа: специализация top-1 / грейд точно / грейд ±1 / F1 навыков / must-nice / верная категория в топ-3	≥ 0,90 / 0,75 / 0,95 / 0,85 / 0,80 / 0,95
Стабильность при пересимуляции теста (пересдача на других семействах): overlap@10; то же для точечной оценки	≥ 0,70 (**исходная цель признана постфактум некорректной, §7; в «Итогах» остаётся как не выполненная**)
Жаккар топ-10 до и после «добавлен nice-навык»	≥ 0,6
Падение P@10 при сдвиге каждого веса на ±0,05 (компенсация — пропорционально остальным; недопустимые по ограничениям сдвиги пропускаются)	≤ 0,05
Накрутка: 50 самым слабым в выдаче добавлены 5 навыков брифа в самоописание → средний сдвиг ранга вверх	наш ≤ 2 позиций; у BM25 ожидаем ≥ 10
Экспозиция кандидатов без ФСП (DECISIONS п.11): условно, внутри децилей T. Отношение = наблюдаемое число релевантных без ФСП в топ-k / ожидаемое при той же доле попадания в топ-k, что у всех релевантных того же дециля T; k = 10 и 50; безусловное отношение — справочно	0,8–1,25
Честность объяснений: Σ вкладов = Score (± 0,1), каждое «за» и «против» ссылается на существующее доказательство, ФСП никогда не «против», удаление доказательства довода №1 снижает Score не меньше превышения над медианой	100%
p95 сборки версии подборки на всей популяции	≤ 300 мс

Метрики подбора считаются по 20 test-брифам; dev-брифы — справочно. Бутстреп: 2000 повторов, сид потока `bootstrap` .

3. Что не проверяется этим прогоном (честные ограничения)

- Синтетика проверяет процедуру, а не реальные сложности заданий. Ответы порождаются моделью Раша с разбросом клонов; нециклличных DINA-персон и живого пилота в этом прогоне нет.

- Экспертной разметки пар (к между разметчиками, P@5 по экспертам) нет — только оракул по истине генератора. Оракул и генератор писала одна команда.
- Брифы и эталон к ним написаны командой с ИИ-ассистентом; 10 брифов от посторонних авторов, заявленные в концепции, не собраны.
- Генератор V-CAT и веса классификатора W/V составлены одной командой — риск круговой проверки.
- Бейзлайн B3 «LLM под каждого» — симуляция с допущениями (σ = 0,5, 10% битых ключей); реального замера ключей LLM нет. B3 подбора — TF-IDF вместо эмбеддингов.
- Знание «методов» частично завышает результат: клоны защищают от слива ответов, а не от слива идеи семейства.
- От ИИ-ассистента удалённый тест полностью не защищён; грейд — решение с уровнем доверия, а не сертификат.
- Точное совпадение грейда при пересдаче не может быть близким к 1 при 4–5 уровнях и ~12 заданиях; стабильность для пользователя обеспечивает гистерезис.
- В банке (на момент записи протокола 69 семейств, сейчас 111: Backend 44, Frontend 40 с учётом общих модулей, плюс QA, DevOps и семейства навыков) нет pretest-семейств: r_pb считается по боевым заданиям с оценкой уровня без этого задания (LOO), а не по pretest-слотам.
- Fullstack-строки (производная категория) в популяции не порождаются: каждый кандидат проходит один тест.

4. Уточнения процедуры после дымового прогона (пороги не менялись)

Вписаны после первого запуска `--quick`, до полного прогона. Ни одна метрика и ни один порог из раздела 2 не изменены; уточнены только процедуры, которые в плане были описаны неоднозначно.

1. **S2** считается на общих случайных числах: у кандидата во всех условиях один поток `s2:<id>`, меняются только истинные параметры семейств; условие σ = 0, A = 1,5 считается явно тем же способом, а не берётся из основного прогона. Иначе разница условий тонула в шуме выборки.
2. **Веса ±0,05.** Если пропорциональная компенсация выводит вес за ограничение (wF > 0,15 или wT < 0,30), этот вес остаётся на месте, а компенсацию делят остальные; сдвиг пропускается, только если и так нельзя (например, wF + 0,05).
3. **Сквозной режим подбора** повторяет путь работодателя: если парсер задал уточняющий вопрос о специализации или грейде, ответ — вариант из эталона; если парсер не нашёл вилку, работодатель вводит её в обязательной форме (значения эталона). Отдельно показан режим «только парсер, без уточнения» и режим «эталонный бриф».
4. **B2** берёт первую рекомендованную категорию, в которой есть кандидаты (для Fullstack-брифов это смежная категория Backend или Frontend), и сортирует её по точечной оценке уровня.
5. **Набор пар для жюри** — пул из объединения топ-20 систем «наш сквозной», B0, B1, B2, B3 и 10 случайных кандидатов (а не топ-50 всех систем): иначе файл весит ~8 МБ. Для P@10 и P@5 глубины 20 достаточно.
6. **Коллизии ключей** для детектора «чужой ключ» измеряются на 200 экземплярах каждого семейства Backend (сценарий утечек — Backend).
7. **Длина теста**: «показано вместе с pretest» считается по основной части (DECISIONS п.17), доборы — отдельно.
8. Добавлена **описательная диагностика без порогов**: причины нерелевантных мест в топ-10 (наш подбор и B2) и разбивка контрфакта объяснений по компонентам. Она помогает читать результат и ни на что не влияет.
9. Сетка порога CUSUM расширена до h ∈ {4…10} (справочно): порог плана по-прежнему проверяется при прод-значении h = 6.
10. V-CAT дополнен подсказкой владельцу конфигурации опроса: сетка «вес технологий × бонус за выбор», подобранная на чётных профилях и проверенная на нечётных. В прод-конфиг она не вносится и на пороги не влияет.

5. Уточнения после независимого ревью харнесса (после полного прогона; пороги не менялись)

Вписаны после полного прогона по замечаниям ревьюера. Метрики и пороги раздела 2 не изменены; каждое уточнение делает оценку строже или честнее, а не выгоднее.

1. **Утечка ответов, плитка «коалиция 20»** — берётся худшая для нас из двух стратегий списывающего (наивный ввод чужого ключа или осторожный честный ответ), а не наивная: наивный почти всегда проваливается, поэтому его доля ложных Middle занижает риск.
2. **Близнецы по тексту.** id экземпляров в симуляции уникальны по построению выдачи, поэтому «пересечение экземпляров = 0» — свойство выдачи, а не измерение. Дополнительно измеряется вероятность, что два разных экземпляра семейства совпадают по тексту (200 экземпляров каждого семейства Backend), и в модели наивного списывающего слитый ключ верен с этой вероятностью (плюс угадывание c).
3. **Экспозиция без ФСП**: децили T и места считаются по всем кандидатам рекомендованных категорий брифа, а не по ≤ 500 строкам версии подборки (обрезка по Score смещала нижние децили).
4. Добавлена описательная метрика без порога из концепции §7.2, которой не было в плане: точность и полнота тегов ✓ модулей против истинного уровня модуля. Ориентир концепции (≥ 0,90 / ≥ 0,75) показывается справочно.

6. Добавление по ответам организаторов: test-retest и разделяющая способность по специализациям

Организаторы описали ожидаемую логику: **test-retest** (один профиль проходит тест дважды, сравниваются присвоенные грейды) и **разделяющая способность** заданий (различать соседние грейды); симуляция на синтетике допустима, минимального объёма выборки нет, эталон разметки команда задаёт сама. Протокол дополнен отдельным экспериментом (`eval/spec_validation.py` , результаты — VALIDATION.md, 2.11); основные пороги из раздела 2 не менялись.

Что	Как считается
Профили	по 400 латентных профилей на каждую специализацию (Backend, Frontend, QA, DevOps), поровну по четырём грейдам, θ равномерно в полосе грейда, заявка 60/30/10
Сессии	A — основная; B — независимая параллельная (другой сид → другие клоны); C — пересдача по реальным правилам (семейства A исключены, spec-testing §7.1)
Test-retest	точное совпадение грейда, ±1, взвешенный κ, r(θ), совпадение решения сдал/не сдал, доля «переворотов» через каждый порог (в том числе рядом с порогом)
Разделяющая способность	матрица ошибок 4×5, точность решения у каждого порога (всего и при
Дискриминация заданий	r_pb каждого семейства, IRT-крутизна A_eff, прирост доли верных между соседними грейдами
Робастность	чувствительность к разбросу клонов σ_inst ∈ {0; 0,15; 0,30}; парный эксперимент «слитые ключи» (прирост θ); покрытие банка двойниками
Исключение	кандидаты с заявкой ниже истинного грейда не входят в точность: тест на заявленный грейд не выдаёт больше (повышение — отдельный тест)
Воспроизводимость	`random.Random(f"{seed}:...")` ; сид по умолчанию 20261005; число процессов результат не меняет; для жюри — `eval/jury_check.py` на своих профилях

Пороги для этого эксперимента до прогона не фиксировались (ориентиры, назначенные постфактум и потому не засчитываемые как «выполнено»: точное совпадение грейда при ретесте ≥ 0,65, ±1 ≥ 0,98, точность решения у порога ≥ 0,85 по основному разделу 2.1); результаты показаны как есть, слабые места названы в выводах 2.11.

7. Постфактум-пересмотр по итогам прогона (пороги плана не менялись)

Вписан после полного прогона, на котором не были выполнены 8 из 41 пороговых пунктов. Ни один порог из раздела 2 не изменён и не удалён; ниже — что сделано с каждым пунктом и чем это обосновано. Правило: параметры продукта подбираются **на других сидах и популяциях**, чем основной (20261005); основной прогон лишь сообщает результат.

Пункт	Было	Стало	Что сделано
Ложные карантины CUSUM без утечки	0,20 (h = 6)	0,011 (h = 11)	**Изменён параметр продукта** `detectors.cusum_h`: 6 → 11. Подбор — по нулевым потокам трёх других популяций (сиды 301–303, 20 повторов): наименьший h из сетки, дающий ≤ 5% на каждой, — 11 (при h = 10 на одном из сидов 5,3%). Таблица в VALIDATION.md 2.7
macro-F1 подсказки специализации	0,88	0,93	**Изменены веса опроса** `survey.tech_weight` 0,3 → 1,2 и `choice_bonus` 0,5 → 0,8. Сетка подобрана на четырёх других популяциях профилей (сиды 11–14: macro-F1 0,87 → 0,93), проверена на четырёх отложенных (сиды 21–24: 0,87 → 0,92; подсказка верна 0,84 → 0,94; трудные случаи 0,76 → 0,82). Выбрана точка (1,2 / 0,8), а не лучшая по сетке (1,6 / 0,8): она не на краю сетки и в пределах 0,005 от неё. Оговорка: эту же пару раньше предлагала подсказка на чётных профилях основного сида, а генератор профилей писала та же команда — круговая проверка
Без доказательства довода №1 Score падает не меньше превышения	0,96 (34 из 91 по K, 7 из 7 по A не проходили)	1,00	**Исправлено объяснение:** довод «Стек» ссылается на все источники подтверждённых навыков, «Свежесть» — на все задачи окна (`explain._full_skill_evidence`, `scoring.skill_sources`). Причина была в максимуме по источникам: удаление одного названного источника оставляло запасной. Скоринг и ранжирование не менялись
Стабильность топ-10 при пересдаче	0,28 (цель 0,70)	0,28; цель признана некорректной	см. ниже
Точный грейд	0,748	0,748	**Не менялось.** В пределах ошибки выборки (n = 1 810, SE ≈ 0,010); на пяти других сидах (101–105) среднее 0,758. Правило остановки CAT проверено (таблица ниже): выигрыш ≤ 0,3 п.п. ценой +0,8…1,6 задания с доборами — бюджет ≤ 17 нарушается
Пересдача: корреляция оценок уровня	0,86	0,86	**Не менялось.** Среднее по пяти другим сидам 0,857; самое длинное из проверенных правил даёт 0,867 при 17,7 заданиях. CAT оптимизирует решение у порога, а не оценку θ у всех, так что 0,88 несовместимо с бюджетом длины
CUSUM: задержка при 50% знающих	306 (h = 6)	603 (h = 11)	**Не выполнено, цена выбора h.** Задержка ≈ h / (информация на ответ) и ограничена слабым эффектом на один ответ; 200 ответов достижимы ценой 12–31% ложных карантинов. Остаётся сигналом методисту
P@10 значимо выше B2 на 20 test-брифах	нет (ДИ [−0,01; 0,30])	нет	**Не менялось.** Мощность 20 брифов; на 40 брифах (постфактум-срез) значимо. Подгонять формулу под оракул, который награждает то, по чему сортирует B2, не стали

7.1. Стабильность топ-10 при пересдаче: почему исходная цель некорректна

Цель ≥ 0,70 для overlap@10 в плане не имела ни обоснования, ни оценки достижимости (это была оценка концепции). Проверка:

1. **Надёжность внутри категории низкая.** Оценки уровня при пересдаче коррелируют 0,86 по популяции, но внутри категории (одна полоса грейда) всего около 0,25 — размах истинного уровня сужен, а SE теста из ~15 заданий сопоставим с ним (сужение диапазона, Sackett & Yang, 2000; классическая теория тестов: надёжность зависит от разброса истинных значений, Lord & Novick, 1968).
2. **Потолок.** Для ранжирования по одному измерению уровня с такой надёжностью симуляция (двухкомпонентная модель «истина + независимая ошибка», n — размер категорий брифа) даёт overlap@10

≈ 0,09. Достижение 0,70 требовало бы надёжности внутри категории порядка 0,95+, то есть теста в разы длиннее. Это общее свойство рейтингов с ошибкой измерения (Goldstein & Spiegelhalter, 1996: доверительные интервалы мест в таблицах лиг шире различий между соседями).

3. **Наш подбор устойчивее потолка в 3,3 раза** (0,28), потому что Score включает стабильные компоненты (K, A, F, U), а не только оценку теста. Обычный overlap@10 к тому же не поправлен на случайность; индекс Кунчевой@10 (Kuncheva, 2007) = 0,26, RBO (Webber, Moffat, Zobel, 2010; p = 0,9) = 0,23.
4. **Что важно работодателю — качество десятки, а не её состав.** P@10 по оракулу на пересдаче 0,68 против 0,61 в первой сессии: уходят и приходят одинаково подходящие кандидаты. 62% прежней десятки остаётся в топ-50; 12% уходит из категории (грейд при пересдаче изменился).
5. **Попытка принципиального исправления не помогла.** «Равные по уровню» (Score режется на полосы шириной 2/5/8 баллов, внутри полосы порядок по hash — устойчивый тай-брейк): overlap@10 0,28/0,27/0,31, P@10 0,61/0,62/0,63 (без полос 0,28 и 0,61); прирост ≤ 0,03 в пределах шума при 20 брифах. Состав набора определяется тем, кто попал в верхнюю полосу, а попадание зашумлено так же. В продукт не внесено (`eval.matching_eval.rank_fast_banded` — только эксперимент).

Что изменено в протоколе. Исходная строка «overlap@10 ≥ 0,70» в «Итогах» оставлена и показывается невыполненной. Добавлены два постфактум-пункта (в таблице отмечены «постфактум»): P@10 после пересдачи ≥ 0,60 (порог главной метрики P@10, §2.3) и отношение overlap@10 к потолку «одного измерения» ≥ 1,0. Эти пороги назначены после просмотра результата и потому честно помечены; они не заменяют исходный пункт.

Ссылки: Goldstein H., Spiegelhalter D. J. League tables and their limitations: statistical issues in comparisons of institutional performance // JRSS A, 159(3), 1996. Kuncheva L. I. A stability index for feature selection // Proc. AIAP, 2007. Lord F. M., Novick M. R. Statistical Theories of Mental Test Scores, 1968. Sackett P. R., Yang H. Correction for range restriction: an expanded typology // J. Applied Psychology, 85(1), 2000. Webber W., Moffat A., Zobel J. A similarity measure for indefinite rankings // ACM TOIS, 28(4), 2010.

7.2. Правило остановки CAT: чувствительность (сиды 101–105, не основной)

Каждый вариант — отдельный процесс с копией `config/` (`VZ_CONFIG_DIR`), `uv run python -m eval.stop_rule_sweep --seed N` (`backend/eval/stop_rule_sweep.py`). Среднее по пяти популяциям по 2 000 кандидатов:

Вариант (`cat:` в testing.yaml)	Точный грейд	Верное решение	θ-корреляция при пересдаче	QWK	Основная часть	С доборами
сейчас: P ≥ 0,90 / ≤ 0,10, n_min = 8	0,758	0,854	0,857	0,858	13,0	16,1
P ≥ 0,92 / ≤ 0,08	0,759	0,855	0,861	0,861	13,6	17,0
n_min = 10	0,757	0,856	0,865	0,860	14,2	17,0
P ≥ 0,92 / ≤ 0,08 и n_min = 10	0,761	0,855	0,867	0,864	14,7	17,7

Выигрыш в 0,1–0,3 п.п. точного грейда и 0,004–0,010 корреляции — в пределах шума отдельной популяции (SE точного грейда ≈ 0,010), цена — до +1,6 задания с доборами; лимит ≤ 17 нарушается или достигается впритык. Правило остановки не меняли.

8. Расширение отложенной выборки (предрегистрация)

Запись сделана **9 октября 2026, 09:06 (MSK), до написания новых брифов и до любого прогона на них**. Цель — проверить на новых независимых данных ровно один вопрос, на который 20 test-брифов не хватило мощности (§7, последняя строка таблицы): **выше ли P@10 нашего подбора, чем у «только уровень» (B2)**.

8.1. Расчёт мощности (до генерации результатов)

Вход — попарные разности P@10 (наш сквозной − B2) по брифам из `backend/eval/out/validation.json` (`matching.per_brief`):

Выборка	n	Средняя разность	SD разности
test	20	0,145	0,371
все 40 (test + dev)	40	0,1375	0,334

Число брифов для мощности 80% при α = 0,05 (двусторонний), t-приближение для парной разности, $n = ((t_{1-\alpha/2,,n-1} + z_{0{,}8}) \cdot SD / \Delta)^2$: **53** по test (Δ = 0,145, SD = 0,371), **48** по всем 40; для мощности 90% — 70 и 64. Проверка прямой симуляцией (1 500 повторов; выборка из эмпирических разностей с возвращением; критерий — нижняя граница 95% перцентильного бутстрепа > 0, 600 повторов бутстрепа; контроль — t-критерий):

N новых брифов	мощность, разности test	мощность, разности 40 брифов
20 (как сейчас)	0,44	0,51
40	0,73	0,76
50	0,83	0,87
60	**0,89**	**0,91**
70	0,94	0,94

Оговорка: эффект взят из тех же 20 брифов, где он не оказался значимым (оценка может быть завышена «проклятием победителя»); с запасом выбрано **N = 60** вместо минимума 53. Это даёт ≈ 80% мощности и при истинном эффекте около 0,12 (при SD 0,37).

8.2. Выборка

- **N = 60 новых брифов `b41` … `b100` , поле `split = "test2"` .** Написаны командой с ИИ-ассистентом (как b01–b40; посторонних авторов нет — ограничение сохраняется), **без запуска разбора брифа, Score и бейзлайнов на них** и без просмотра результатов на b01–b40 при выборе формулировок, кроме общей статистики состава.
- Состав (как в b01–b40): три специализации, по которым есть банк и популяция, — Backend Python, Frontend JS, Fullstack (четвёртая, QA/DevOps, включается только флагом `--with-qa-devops` и в b01–b40 брифов не имеет; здесь не добавляется); грейды Intern/Junior/Middle/Senior; вилка «на руки» / до НДФЛ / без указания; формат office/hybrid/remote и их сочетания; города из `config/skills.yaml` ; стили formal/chat/short/long/mixed_en (сленг, опечатки, отрицания, «или»); must и nice — id навыков из `config/skills.yaml` . Эталон (`gold`) — по той же схеме, что в b01–b40.
- Оракул релевантности v2 (§1.5), популяция (сид 20261005), конфигурация `config/*.yaml` , код `matching_eval.py` / `metrics.py` и сид бутстрепа **не меняются** после этой записи. Контрольные суммы замороженных до прогона файлов — в §8.4.

8.3. Метрика, критерий, правило решения

- **Единица анализа** — бриф. **Главная метрика** — P@10 по оракулу, сквозной режим (разбор бриф → категории → версия подборки «Баланс»), `relevant ⇔ rel ≥ 2` .
- **Главная гипотеза**: среднее по брифам test2 разности P@10 (наш − B2) > 0. **Тест**: парный перцентильный бутстреп по брифам (`metrics.paired_bootstrap_diff` , 2 000 повторов, поток `paired:test2:ours` , сид 20261005), 95% двусторонний ДИ. **Решение «подтверждено» ⇔** нижняя граница ДИ > 0. Иначе — «не подтверждено», без пересмотра. Контрольный парный t-критерий приводится справочно; при расхождении с бутстрепом решает бутстреп.

- **Вторичные** (не влияют на решение, приводятся все): P@10 и nDCG@10 (выигрыш 2^rel − 1) наших и B0 (случайный в категориях), B1, B2, B3; разность nDCG@10 с B2 и B0 с парным бутстрепом; абсолютный P@10 ≥ 0,60 (порог §2.3) с ДИ; те же числа на объединении test + test2 (80 брифов) и по специализациям (описательно). Поправок на множественные сравнения нет: решение принимается по одной главной проверке, остальное — описательно.
- **Один прогон.** Оценка на test2 выполняется один раз, после заморозки брифов. Если прогон упал по техническим причинам (не из-за качества результата), он повторяется без изменений; все запуски фиксируются.
- **Что не меняется после просмотра результатов:** Score, веса и пороги `config/`, оракул, популяция, состав и тексты брифов test2. Отчёт публикуется **независимо от результата**, в том числе если эффект не подтвердится или окажется меньше прежнего (§7: «Не менялось»).

8.4. Заморозка (до прогона)

Контрольные суммы, зафиксированные 9 октября 2026 в 09:06 (MSK), до написания брифов (коммит `6f3705c`):

Файл	SHA-256
`config/scoring.yaml`	`7cf2eac17154258d6daf916d11cf11cbfa9de4ac999100b4d0f15933a322d22d`
`config/skills.yaml`	`4d303c88e02465bf3f0f56e4c6a570a3687528e4368302bf7bb579a86729f4a2`
`backend/eval/matching_eval.py`	`f6534f794a3a9aedbb72a311fba76c2e08f29284dafe9433639324419fa819dc`
`backend/eval/population.py`	`566060f135919da8ac31784f2195234ee644bc2803285c13b6b6c95958216a87`
`backend/eval/metrics.py`	`f070f5150b26fd36b5842028ed18b80a5797115dfb89077fc8432ac2ee47958e`

Хэш брифов b41–b100 добавляется ниже **до** прогона (§8.5).

8.5. Заморозка брифов test2

Брифы `backend/eval/briefs/b41.json` … `b100.json` (60 шт., `split = "test2"`) написаны и заморожены **9 октября 2026, 09:09 (MSK), до любого запуска разбора брифа, Score или бейзлайнов на них**. Состав: Backend Python 24 (intern 2 / junior 6 / middle 9 / senior 7), Frontend JS 24 (2 / 6 / 9 / 7), Fullstack 12 (junior 3 / middle 5 / senior 4); стили: formal 16, chat 15, short 13, mixed_en 8, long 8; вилка «на руки» / до НДФЛ / без указания; офис, гибрид, удалёнка и их сочетания; 17 городов. Построчные SHA-256 всех 60 файлов — `backend/eval/briefs_test2.sha256` (проверка: `cd backend/eval/briefs && sha256sum -c ../briefs_test2.sha256`).

SHA-256 манифеста `backend/eval/briefs_test2.sha256` (хэш брифов test2): `fa766490b26e3c4e6f38900f3081d6456a15e45dfd25bbcf47fb6efbcbddb1c4`.

> **Поправка о переводе строки (9 октября 2026, после прогона; на план, брифы и числа не влияет).** Построчные хэши брифов и хэши `population.py` и `metrics.py` в §8.4 сняты в рабочей копии Windows, где файлы были с переводом строки CRLF; конфиги в §8.4 сняты с LF. Git хранит и выдаёт все текстовые файлы с LF (`.gitattributes: eol=lf`), поэтому `sha256sum -c` в свежей копии на Linux показывает «не совпадает» при неизменном содержимом. Проверка в коде (`test2_freeze_check`) сверяет по каноническому CRLF: из `backend/` — `uv run python -c "from eval.matching_eval import test2_freeze_check as c; print(c())"` (ожидается `ok: True`, 60 файлов, расхождений 0). Отдельный файл из §8.4 — `python3 -c "import hashlib,sys; b=open(sys.argv[1],'rb').read().replace(b'\r\n',b'\n'); print('LF ', hashlib.sha256(b).hexdigest()); print('CRLF', hashlib.sha256(b.replace(b'\n',b'\r\n')).hexdigest())" <путь>`: одна из двух строк совпадает с таблицей.

Единственная правка кода после заморозки — интеграция test2 в отчёт (§8.6): новые срезы `test2` и `test_all`, исключение test2 из прежних срезов `all` / `checkset`, чтобы ни одно прежнее число не изменилось. Логика

метрик, оракул, популяция и конфиг не трогаются; соответствие проверяется сравнением прежних чисел до/после.

8.6. Результат (вписан после единственного прогона на test2; 9 октября 2026, 09:20 MSK)

Прогон: `uv run python -m eval.validate --workers 4` (сид 20261005), один раз; затем `eval.fairness`, `eval.market_sim`, `eval.checkset_eval` в этом порядке. Заморозка при прогоне подтверждена (`ME.test2_freeze_check()`: хэш манифеста совпал, 60 из 60 файлов совпали). Числа по b01–b40 (test, dev, all, checkset, устойчивость, разбор брифа) не изменились побитно — сравнены с `validation.json` до прогона.

Метрика (test2, 60 брифов)	Наш	B2	B0 (случайный)
P@10	0,540 [0,472; 0,615]	0,352 [0,277; 0,427]	0,167 [0,125; 0,212]
nDCG@10	0,554	0,377	0,203

- **Главная проверка (§8.3): разность P@10 с B2 = +0,188, 95% ДИ [0,128; 0,245], нижняя граница > 0 — гипотеза подтверждена.** Парный t-критерий: t = 6,32, p < 0,001. Разность nDCG@10 с B2 +0,177 [0,132; 0,222]; P@10 против B0 +0,373 [0,305; 0,442].
- Объединение test + test2 (80 брифов): P@10 0,557 против 0,380, разность +0,178 [0,121; 0,238].
- **Неудобное, но записанное:** абсолютный P@10 на test2 (0,54; нижняя граница ДИ 0,47) **ниже порога 0,60** из §2.3; на b01–b40 было 0,61–0,66. По специализациям (описательно): Backend 0,64 против 0,43 (+0,21), Frontend 0,55 против 0,31 (+0,24), **Fullstack 0,32 против 0,28 (+0,04, ДИ [−0,03; 0,13] — преимущество не доказано, 12 брифов)**.
- Эффект на новых данных (+0,188) оказался больше, чем в оценке для расчёта мощности (+0,145); оценка мощности была консервативной, не наоборот.
- Score, конфиг, оракул, популяция, брифы не менялись. Единственные правки кода после заморозки — интеграция test2 в отчёт (`eval/matching_eval.py`: срезы `test2` / `test_all`, `test2_freeze_check`, парный t-критерий, исключение test2 из прежних срезов `all` и `checkset`; `eval/report.py`: раздел 4.1a) и тесты (`tests/test_eval_matching.py`); хэши логики метрик (`metrics.py`), оракула, популяции и конфигов остались прежними. Хэш `matching_eval.py`, указанный в §8.4, после этих правок изменился — это известное и записанное расхождение.

Валидация «Взаимно»: результаты

> Файл сгенерирован командой `cd backend && uv run python -m eval.validate` — режим полный, сид `20261005`, коммит `8b083fe305`, 394 с на 4 процессах. Руками не редактируется. Протокол — VALIDATION_PLAN.md: метрики и пороги в нём записаны до прогона, уточнения процедуры после прогонов вынесены в отдельные разделы 4–5 (пороги не менялись). Все числа в машиночитаемом виде — `../backend/eval/out/validation.json` (его же показывает «Лаборатория» после `--save-db`).

Всё ниже — **симуляция**: она проверяет процедуру на синтетической популяции с известной истиной и работает на тех же движках, что и прод (`app/testing`, банк из 111 семейств с экспертными оценками сложности, `app/matching`, `app/fsp`). Раздела «люди» (пилот, экспертная разметка пар) в этом прогоне нет — см. «Честные ограничения».

Итоги

Раздел	Метрика	Значение	Цель	Статус
Тестирование	Верное решение у заявленного порога	0,86	≥ 0,85	✓ выполнено
Тестирование	Точный грейд	0,748	≥ 0,75	✗ не выполнено
Тестирование	Грейд с точностью ±1	1,00	≥ 0,99	✓ выполнено
Тестирование	Средняя длина основной части, заданий	12,8	≤ 15,0	✓ выполнено
Тестирование	Средняя длина с доборами, заданий	16,1	≤ 17,0	✓ выполнено
Тестирование	p95 длины с доборами, заданий	20,0	≤ 20,0	✓ выполнено
Тестирование	Показано в основной части вместе с pretest (максимум; pretest-семейств в банке пока нет)	20	≤ 20	✓ выполнено
Тестирование	Пересдача: совпадение решения	0,81	≥ 0,78	✓ выполнено
Тестирование	Пересдача: к решения	0,551	≥ 0,55	✓ выполнено
Тестирование	Пересдача: QWK грейда	0,86	≥ 0,85	✓ выполнено
Тестирование	Пересдача: корреляция оценок уровня	0,86	≥ 0,88	✗ не выполнено
Тестирование	Ложное повышение грейда (гистерезис)	0,03	≤ 0,07	✓ выполнено
Тестирование	Общих экземпляров у пар кандидатов (каждый экземпляр выдаётся одному — по построению)	0	0	✓ выполнено
Тестирование	Двум кандидатам с одним семейством выпал одинаковый по тексту экземпляр (Backend, худшее семейство)	0,10%	показываем	справочно
Тестирование	RMSE сложности после 100 ответов	0,20	≤ 0,30	✓ выполнено
Тестирование	Теги ✓ модулей: точность против истинного уровня (ориентир концепции ≥ 0,90)	0,97	показываем	справочно
Тестирование	Теги ✓ модулей: полнота (ориентир концепции ≥ 0,75)	0,76	показываем	справочно
Утечки	Ложный Middle у явных Junior, коалиция 20 (худшая стратегия списывания)	0,10	≤ 0,15	✓ выполнено
Утечки	Наивные списывающие пойманы «чужим ключом» (коалиция 20)	1,00	≥ 0,90	✓ выполнено
Утечки	B0 фиксированная форма: ложный Middle при коалиции 20	1,00	показываем	справочно

Раздел	Метрика	Значение	Цель	Статус
Дрейф	CUSUM, 50% знают метод: медиана задержки (прод-порог h), ответов	603	≤ 200	✗ не выполнено
Дрейф	CUSUM: ложные карантины без утечки (прод-порог h)	0,01	≤ 0,05	✓ выполнено
Категоризация	macro-F1 подсказки специализации (V-CAT)	0,93	≥ 0,90	✓ выполнено
Категоризация	Профили без ФСП получают категорию по тем же правилам	да	да	✓ выполнено
Подбор	P@10 по оракулу, test-брифы, сквозной режим	0,61	≥ 0,60	✓ выполнено
Подбор	P@10 значимо выше B2 (парный бутстреп)	нет	да	✗ не выполнено
Подбор	P@10 значимо выше B2 на всех 40 брифах (постфактум-срез)	да	да	✓ выполнено
Разбор брифа	Специализация top-1	0,90	≥ 0,90	✓ выполнено
Разбор брифа	Грейд точно	0,95	≥ 0,75	✓ выполнено
Разбор брифа	Грейд ±1	1,00	≥ 0,95	✓ выполнено
Разбор брифа	F1 навыков	0,98	≥ 0,85	✓ выполнено
Разбор брифа	must/nice определены верно	0,81	≥ 0,80	✓ выполнено
Разбор брифа	Верная категория в топ-3	0,95	≥ 0,95	✓ выполнено
Устойчивость	Стабильность топ-10 при пересдаче (overlap@10)	0,28	≥ 0,70	✗ не выполнено
Устойчивость	P@10 после пересдачи (постфактум): качество десятки не зависит от шума теста	0,67	≥ 0,60	✓ выполнено
Устойчивость	Стабильность топ-10 / потолок ранжирования по одному измерению уровня при той же надёжности (постфактум)	3,3	≥ 1,0	✓ выполнено
Устойчивость	Жаккар топ-10 после «добавлен nice-навык»	0,95	≥ 0,60	✓ выполнено
Устойчивость	Падение P@10 при сдвиге веса на ±0,05 (максимум)	0,01	≤ 0,05	✓ выполнено
Устойчивость	Накрутка навыков: сдвиг ранга вверх, наш подбор	0,0	≤ 2,0	✓ выполнено
Устойчивость	Накрутка навыков: сдвиг ранга вверх, BM25 (ожидание ≥ 10)	518,4	≥ 10,0	✓ выполнено
Экспозиция	Кандидаты без ФСП: условное отношение экспозиции, топ-10	0,91	0,80–1,25	✓ выполнено
Экспозиция	Кандидаты без ФСП: условное отношение экспозиции, топ-50	0,94	0,80–1,25	✓ выполнено
Объяснения	Σ вкладов = Score	1,00	≥ 1,00	✓ выполнено
Объяснения	Доводы ссылаются на существующие доказательства	1,00	≥ 1,00	✓ выполнено
Объяснения	Отсутствие ФСП никогда не «против»	1,00	≥ 1,00	✓ выполнено
Объяснения	Без доказательства довода №1 Score падает не меньше превышения над медианой	1,00	≥ 1,00	✓ выполнено
Производительность	p95 сборки версии подборки, мс	133	≤ 300	✓ выполнено

Выполнено 38 из 43 пороговых пунктов. Не выполнены: «Точный грейд», «Пересдача: корреляция оценок уровня», «CUSUM, 50% знают метод: медиана задержки (прод-порог h), ответов», «P@10 значимо выше B2 (парный бутстреп)», «Стабильность топ-10 при пересдаче (overlap@10)».

1. Данные

Популяция с латентной истиной (`eval/population.py`). Каждый кандидат порождается своим генератором с сидом, поэтому популяция из n кандидатов — префикс популяции из 2n.

Показатель	Значение
Кандидатов	2 000
Истинная специализация	Backend (Python) 50%, Frontend (JS/TS) 39%, Fullstack 10%
Истинный грейд	Ниже Стажёра 5%, Стажёр 16%, Junior 30%, Middle 31%, Senior 19%
Заявленная специализация не совпадает с истинной	9,5%
Есть история ФСП	35,4%
Корреляция VSS (лучшая дисциплина) с истинным уровнем	0,48 (Спирмен 0,51, n = 708)
Есть регулярные задачи	49,4%
Категория подтверждена тестом	93,2%
Навыков в истинном стеке / приписано в самоописании (в среднем)	14,5 / 4,3

Брифы (`eval/briefs/*.json`): 100 русскоязычных брифов, разбиение 20 dev / 20 test (b01–b40) и 60 test2 (b41–b100, предрегистрированная отложенная выборка, VALIDATION_PLAN §8) зафиксировано в файлах. Стили: переписка со сленгом и опечатками — 26, формальная вакансия — 24, длинное описание с разделами — 15, смесь с английским — 14, короткий запрос — 21. **Тексты и эталонная разметка (специализация, грейд, must/nice, формат, город, вилка) написаны командой с помощью ИИ-ассистента**; посторонних авторов в наборе нет, а история правок эталона до первого коммита не зафиксирована. Все итоговые цифры подбора — по test-брифам; веса на брифах не подбирались.

2. Тестирование

Метрики точности и пересдачи — по кандидатам, чья заявленная специализация совпадает с истинной (n = 1 810). Решение «верное», если «сдал» ⇔ истинный уровень не ниже порога заявленного грейда. «Не подтверждён» считается как «Ниже Стажёра».

2.1. Точность

Система	Верное решение у порога	Точный грейд	±1	к	QWK
Наш CAT (основная часть + доборы)	0,86	0,75	1,00	0,67	0,90
B4: грейд по стажу (подсказка опроса)	—	0,54	0,97	0,38	0,75

Точность решения по тому, как кандидат заявил грейд: верно — 0,84 (n = 1243); на ступень выше — 0,85 (n = 420); на ступень ниже — 1,00 (n = 147).

Матрица «истинный грейд → итоговый грейд» (строки — истина):

Истина \ итог	Ниже Стажёра / не подтверждён	Стажёр	Junior	Middle	Senior
Ниже Стажёра / не подтверждён	62	31	0	0	0
Стажёр	34	204	30	0	0
Junior	0	86	401	54	0
Middle	0	1	105	421	35
Senior	0	0	1	80	265

Точность по специализациям теста (кандидаты с верной заявкой):

Специализация	n	Верное решение	Точный грейд	±1	Подтверждено	Заданий в тесте: среднее / p95
backend_python	1003	0,86	0,73	1,00	94%	15,8 / 20

Специализация	n	Верное решение	Точный грейд	±1	Подтверждено	Заданий в тесте: среднее / p95
frontend_js	807	0,86	0,76	1,00	95%	16,5 / 20

Кандидаты, ошибившиеся со специализацией (проходят чужой тест, n = 190): категорию получают 79%, на уровне своего истинного грейда или выше — 23%. Подсказка опроса (раздел 3) нужна, чтобы таких было меньше.

2.2. Длина теста

Метрика	Значение	Цель
Основная часть, оцениваемых заданий: среднее / p95	12,8 / 20	≤ 15
С доборами, оцениваемых: среднее / p95	16,1 / 20	≤ 17 / ≤ 20
Основная часть, показано вместе с pretest: среднее / p95 / максимум	12,8 / 20 / 20	≤ 20 (DECISIONS п.17)
Pretest-заданий на сессию	0,00	—
Доля основных частей, дошедших до максимума	30%	—

Решения основной части: не сдал, на границе — 203, сдал на границе — 327, не сдал — 367, сдал — 913.

2.3. Пересдача и гистерезис

Вторая сессия того же человека; все семейства первой сессии исключены (n = 1 810).

Метрика	Значение	Цель
Совпадение решения	0,81	≥ 0,78
к решения	0,55	≥ 0,55
Точное совпадение грейда / ±1	0,68 / 0,99	показываем
QWK грейда	0,86	≥ 0,85
Корреляция оценок уровня	0,86	≥ 0,88
Ложное повышение при тесте на грейд выше (гистерезис)	0,03 (n = 400)	≤ 0,07

2.4. Ошибка экспертов в сложности и крутизне (S2)

Истинная сложность = экспертная + $N(0; \sigma^2)$ — ошибка одна на семейство; истинная крутизна A ≠ 1,5 модели. n = 400 кандидатов на условие.

Условие	Верное решение у порога	Точный грейд
σ = 0, A = 1,5 (модель верна)	0,87	0,77
σ = 0,3	0,86	0,76
σ = 0,5	0,85	0,73
истинная крутизна A = 1	0,81	0,69
истинная крутизна A = 2	0,88	0,76

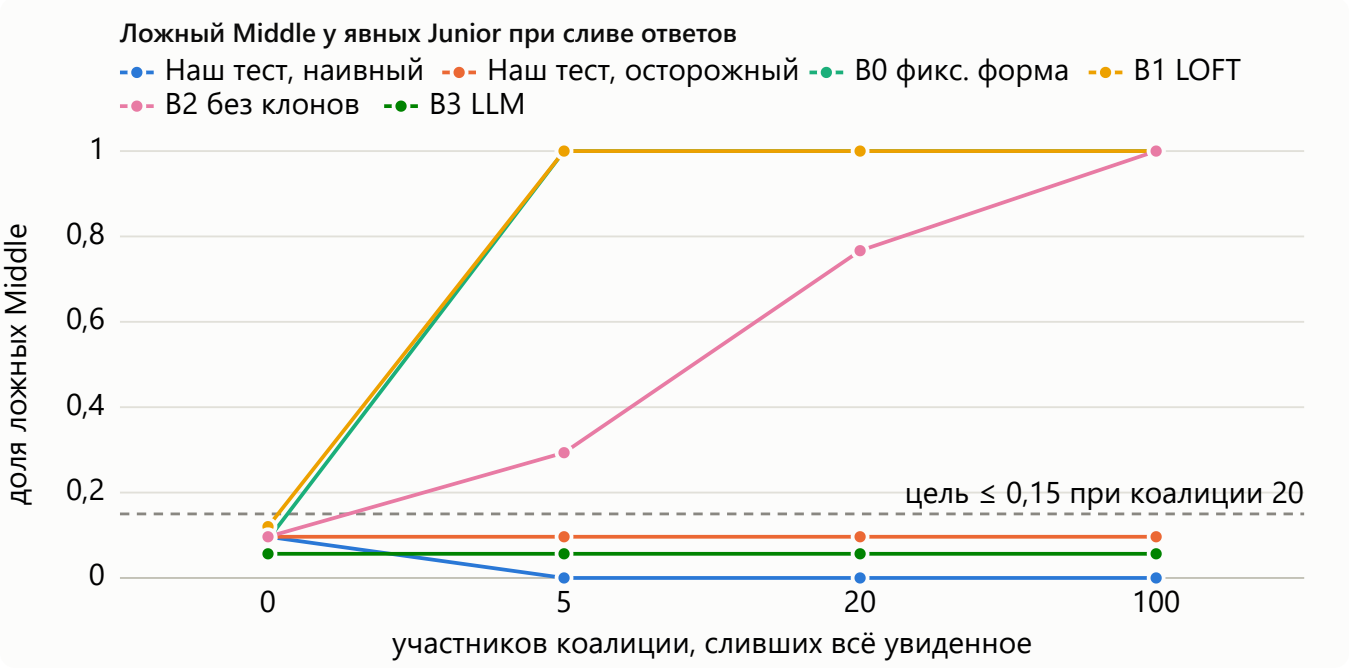
2.5. Бейзлайны тестирования

Подвыборка n = 400. B0 и B1 — формы из 16 заданий («сдал», если вероятность уровня не ниже порога ≥ 0,5). B3 «LLM под каждого»: каждое задание сгенерировано заново, его сложность = задуманная + $N(0; \sigma^2)$ на экземпляр, часть ключей неверна. **Доля битых ключей — допущение, а не замер**: реального прогона заданий через YandexGPT/GigaChat в этом прогоне нет; поэтому показана чувствительность к этой доле.

Система	Верное решение	Точный грейд	Длина	Совпадение решения при пересдаче
Наш CAT (клоны)	0,85	0,73	12,7	0,79
B0 фиксированная форма (16)	0,85	0,69	16	—
B1 LOFT (16)	0,85	0,68	16	0,81
B3 «LLM под каждого» (σ = 0,5, битых ключей 10%)	0,78	0,66	13,5	0,71
B3, битых ключей 0%	0,86	0,76	13,3	—
B3, битых ключей 5%	0,82	0,67	13,3	—
B3, битых ключей 20%	0,63	0,44	13,6	—
B3 «идеальный LLM» (σ = 0,3, 0% битых)	0,87	0,76	13,2	0,78

2.6. Утечки (S4)

300 «явных Junior» (истинный уровень равномерно в [−0,8; −0,3]) заявляют Middle (Backend). Коалиция из k честных кандидатов сливает всё, что видела. Наш тест: экземпляры одноразовые; «наивный» списывающий вводит слитый ключ чужого экземпляра того же семейства, «осторожный» видит, что код другой, и отвечает честно. B0/B1/B2: копирует, только если достался тот же экземпляр (консервативно в пользу бейзлайнов). B3: экземпляры уникальны, слив бесполезен — его доля ложных Middle равна честной ошибке.



Коалиция	0	5	20	100
Слито семейств банка Backend	0%	77%	95%	100%
Наш тест, наивный списывающий	0,10	0,00	0,00	0,00
Наш тест, осторожный	0,10	0,10	0,10	0,10
B0 фиксированная форма	0,09	1,00	1,00	1,00
B1 LOFT	0,12	1,00	1,00	1,00
B2 CAT без клонов (20 экземпляров)	0,10	0,29	0,77	1,00
B3 «LLM под каждого»	0,06	0,06	0,06	0,06
Наивные пойманы «чужим ключом»	—	1,00	1,00	1,00

В «Итогах» для нашего теста берётся худшая из двух стратегий списывающего: наивный ввод чужого ключа почти всегда проваливается, поэтому рациональный списывающий отвечает честно, и его доля ложных Middle равна честной ошибке теста у порога.

Утечка метода (коалиция 20, слито 95% семейств): тем, кто знает идею семейства, оно легче на Δb. Клоны защищают от слива ответов, но не от слива идеи — это видно в таблице.

Δb	Ложный Middle
0,00	0,10
0,25	0,25
0,50	0,51

Детектор «чужой ключ» активен для 28 из 44 семейств Backend (доля коллизий ключей < 2%, измерено на 200 экземплярах каждого семейства реального банка; медиана коллизий 0,00). У семейств с выбором ответа ключи совпадают часто, поэтому для них детектор не работает — их защищают только клоны и CUSUM.

2.7. Дрейф: CUSUM (S5)

Поток ответов по каждому из 44 семейств Backend: сначала объём 3 000 честных сессий, затем столько же, где доля проходящих знает метод 12 семейств (Δb = 0,5). Повторов: 20. Прод-порог h = 11.



Сценарий	h	Найдено слитых семейств	Медиана задержки, ответов	Ложные карантины
без утечки	4	—	—	0,633
без утечки	5	—	—	0,380
без утечки	6	—	—	0,204
без утечки	7	—	—	0,120
без утечки	8	—	—	0,060
без утечки	9	—	—	0,032
без утечки	10	—	—	0,018
без утечки	11	—	—	0,011
без утечки	12	—	—	0,004
20% знают метод	4	0,33	281	0,548

Сценарий	h	Найдено слитых семейств	Медиана задержки, ответов	Ложные карантины
20% знают метод	5	0,33	426	0,306
20% знают метод	6	0,20	441	0,172
20% знают метод	7	0,16	436	0,091
20% знают метод	8	0,12	513	0,057
20% знают метод	9	0,10	520	0,035
20% знают метод	10	0,07	557	0,016
20% знают метод	11	0,07	615	0,008
20% знают метод	12	0,03	648	0,006
50% знают метод	4	0,52	157	0,552
50% знают метод	5	0,70	226	0,312
50% знают метод	6	0,67	306	0,184
50% знают метод	7	0,66	370	0,101
50% знают метод	8	0,57	421	0,049
50% знают метод	9	0,50	461	0,028
50% знают метод	10	0,44	556	0,014
50% знают метод	11	0,35	603	0,008
50% знают метод	12	0,30	703	0,004

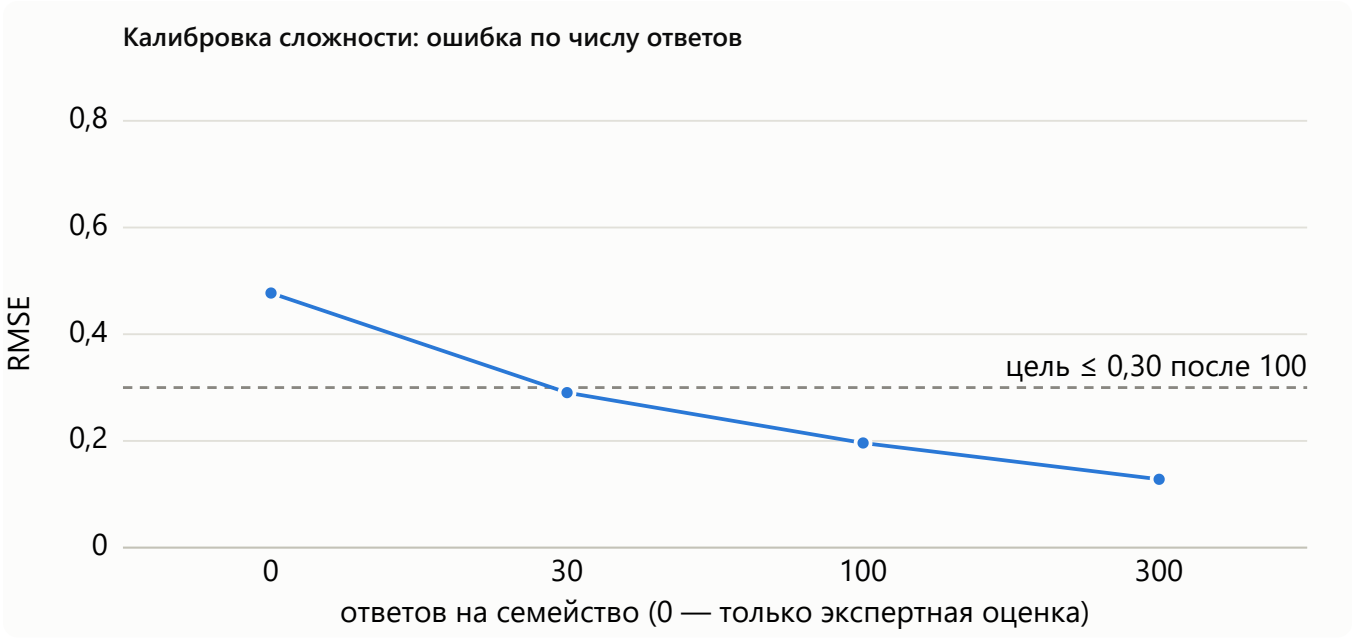
Подбор прод-порога на других популяциях. Порог h выбирается по нулевым потокам (без утечки) на отдельных сидах, а не на основном; основной сид лишь проверяет выбор. Ложные карантины по h:

Популяция	4	5	6	7	8	9	10	11	12
сид 301 (подбор)	0,690	0,449	0,293	0,177	0,108	0,070	0,050	0,027	0,017
сид 302 (подбор)	0,701	0,481	0,314	0,189	0,103	0,064	0,039	0,023	0,017
сид 303 (подбор)	0,693	0,473	0,330	0,201	0,124	0,081	0,053	0,026	0,018
основной сид (проверка)	0,633	0,380	0,204	0,120	0,060	0,032	0,018	0,011	0,004

Наименьший h с ложными ≤ 5% на каждом из сидов подбора — 11; прод-порог h = 11 даёт на основном сиде 1% ложных карантинов (цель ≤ 5%). При 50% знающих такой порог находит 35% слитых семейств с медианой задержки 603 ответа — цель ≤ 200 ответов **не достигнута**: задержка ≈ h / (информация на ответ), а при Δb = 0,5 и половине знающих эффект на один ответ мал (порядка 0,02–0,03 нат); уложиться в 200 ответов можно только ценой ложных карантинов 12–31% (h = 5–7 в таблице) или при вдвое большем трафике на семейство. Выбран контроль ложных карантинов: без утечки CUSUM — сигнал методисту, а не автоматическая блокировка. Порог плана ≤ 200 не меняем.

2.8. Калибровка сложности

Истинная сложность неякорных семейств = экспертная + $N(0; 0{,}5^2)$; MAP-уточнение `calibration.calibrate_family` по n ответам. Семейств: 25, повторов: 5.



Ответов на семейство	0	30	100	300
RMSE	0,48	0,29	0,20	0,13

2.9. Дискриминативность

r_pb — точечно-бисериальная корреляция ответа с оценкой уровня **без этого задания**; pretest-семейств в банке пока нет, поэтому считаем по боевым заданиям. ΔP — доля верных у Middle минус у Junior (по оценке уровня).

Специализация	Семейств	Медиана r_pb	Доля r_pb ≥ 0,20	Флаг r_pb < 0,10	Медиана ΔP	Доля ΔP ≥ 0,20
backend_python	44	0,44	0,98	py.closures.late_binding	0,20	0,46
frontend_js	40	0,46	0,97	нет	0,22	0,60

AUC оценки уровня для соседних грейдов: intern / junior — 0,91, junior / middle — 0,88, middle / senior — 0,90. Надёжность 1 − mean(SE^2)/var: 0,83.

Пересечение экземпляров у пар кандидатов: 0 из 67 974 выданных (тест и пересдача). Это свойство выдачи, а не измерение: симулятор, как и пул в проде, отдаёт каждый экземпляр одному человеку. Измеримый вопрос — увидят ли двое одно и то же задание под разными id: по 200 экземплярам каждого из 44 семейств Backend вероятность, что двум кандидатам с одним семейством выпадет одинаковый по тексту экземпляр, в среднем 0,00%, в худшем семействе (py.complexity.containers) — 0,10%; совпадения есть у 2 семейств. У таких близнецов и ключ общий; модель утечек в 2.6 это учитывает: слитый ключ верен с этой вероятностью (плюс угадывание).

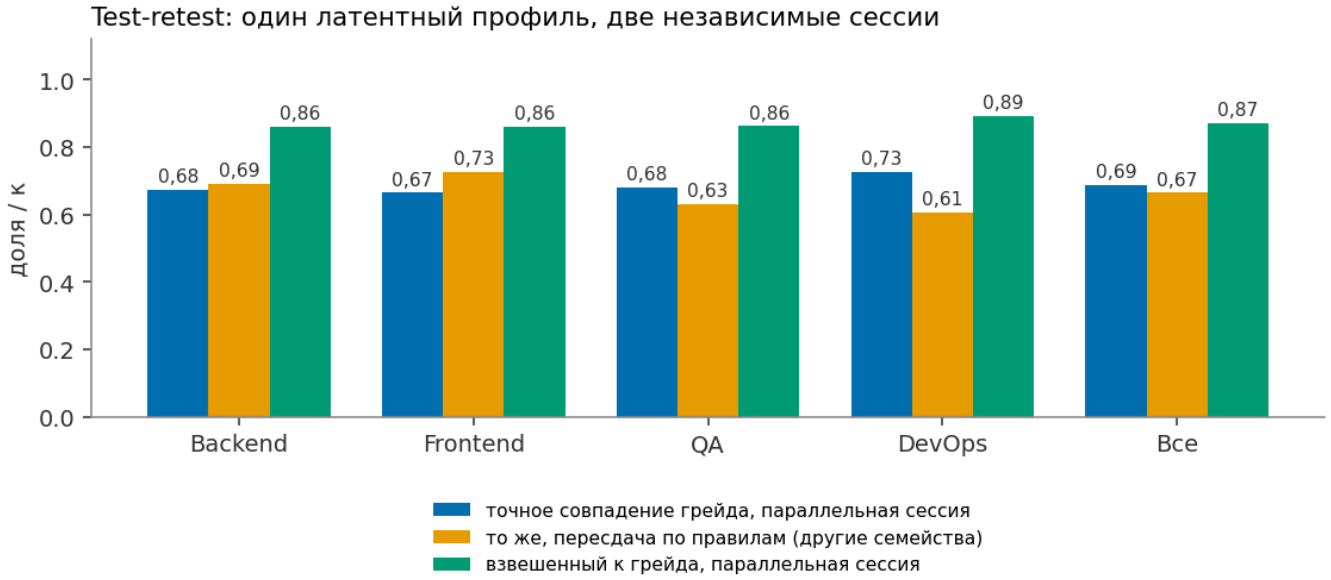
Теги модулей. ✓ утверждает «уровень модуля не ниже порога Junior», ✓✓ — «не ниже порога итогового грейда»; истина — уровень модуля генератора. По 8 570 модулям: точность ✓ 0,97, полнота 0,76 (полнота считается по всем модулям теста, у 10% из них заданий меньше двух — «~»); точность ✓✓ 0,91 (n = 4222). В плане этой метрики не было, поэтому ориентир концепции (≥ 0,90 / ≥ 0,75) показан справочно.

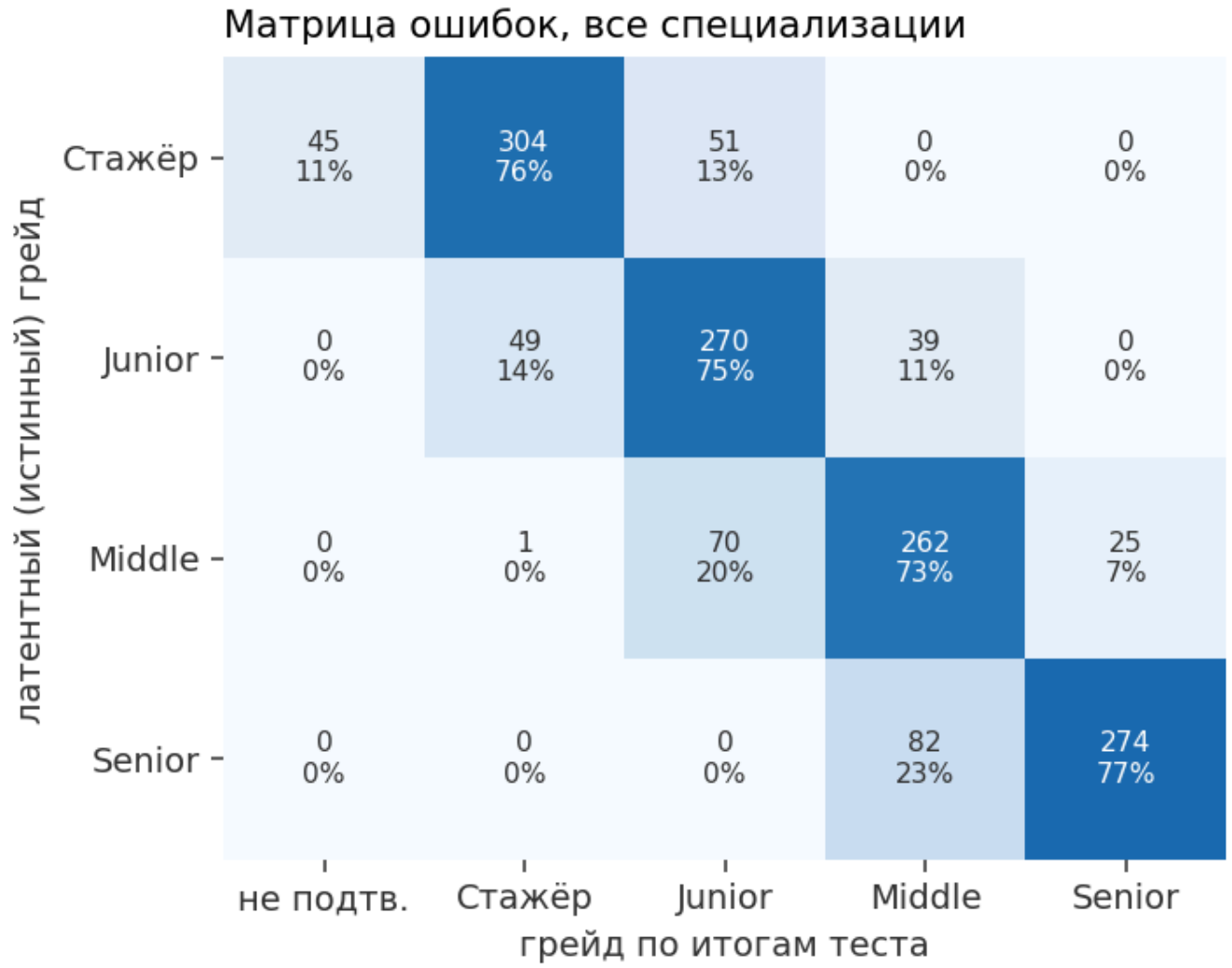
2.10. Кандидаты без истории ФСП

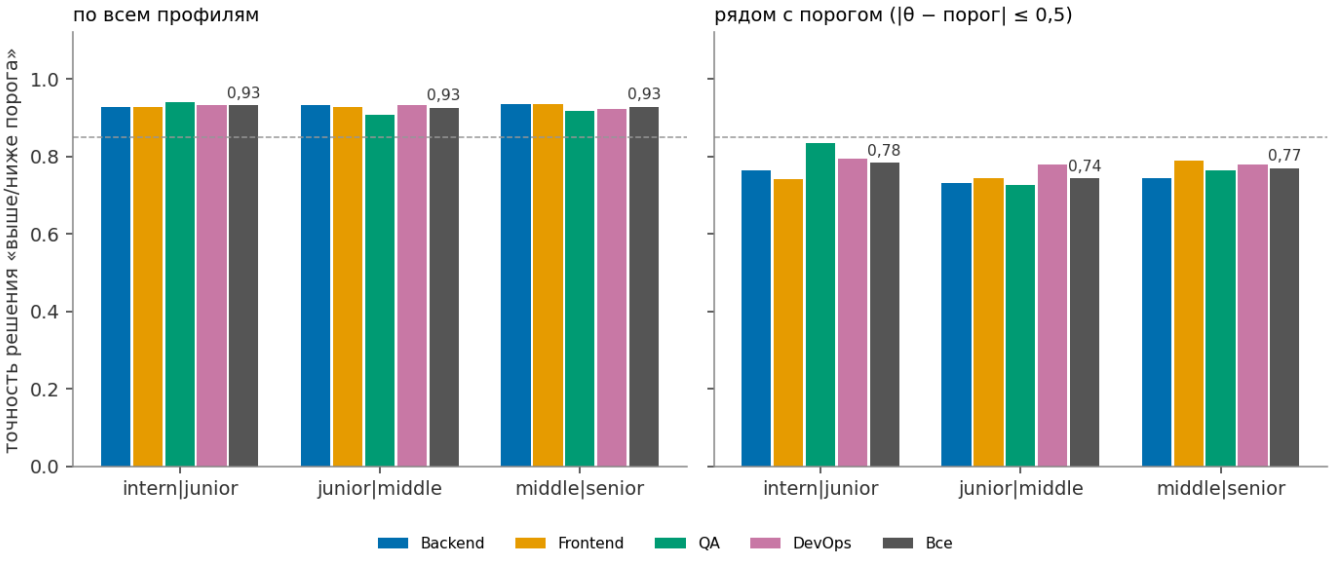
Опрос и тест не принимают на вход ничего из ФСП, поэтому категория присваивается по тем же правилам. Доля подтверждённых тестом: с ФСП — 92,9% (n = 708), без ФСП — 93,4% (n = 1292); разница — только от уровня, который в генераторе коррелирует с ФСП.

2.11. Test-retest и разделяющая способность по специализациям

Отдельный эксперимент на тех же движках (`eval/spec_validation.py`): по 400 латентных профилей на каждую из четырёх специализаций, поровну по грейдам Стажёр / Junior / Middle / Senior, заявка 60% верно / 30% на ступень выше / 10% ниже. Один и тот же профиль проходит три сессии: **A** — основная; **B** — независимая параллельная (другой сид → другие клоны и другая траектория CAT, семейства могут повториться); **C** — пересдача по реальным правилам (spec-testing §7.1: семейства сессии A исключены, то есть другая экспозиция заданий). Истина — латентный уровень профиля; в расчёт точности не входят кандидаты, заявившие грейд ниже истинного: тест на заявленный грейд не может выдать больше, повышение — отдельный тест, это правило продукта, а не ошибка теста.







Test-retest (A↔B — параллельные формы; A↔C — по правилам пересдачи).

Специализация	n	A↔B точно	±1	QWK	r(уровень)	решение сдал/не сдал совпало	к решения	A↔C точно	A↔C QWK	A↔C r(уровень)	общих семейств в C
Backend (Python)	400	68%	99%	0,86	0,87	80%	0,52	69%	0,86	0,85	0,0

Специализация	n	A↔B точно	±1	QWK	r(уровень)	решение сдал/не сдал совпало	к решения	A↔C точно	A↔C QWK	A↔C r(уровень)	общих семейств в C
Frontend (JS/TS)	400	66%	100%	0,86	0,87	79%	0,46	72%	0,88	0,87	0,0
QA	400	68%	100%	0,86	0,86	79%	0,53	63%	0,81	0,79	0,0
DevOps	400	73%	100%	0,89	0,89	80%	0,50	61%	0,80	0,78	0,0
Все вместе	1600	69%	100%	0,87	0,87	80%	0,51	66%	0,84	0,82	0,0

Доля профилей, у которых решение «перевернулось» через порог между сессиями A и B (весь профиль; в скобках — только рядом с порогом, |уровень − порог| ≤ 0,5, где ошибки ожидаемы):

Специализация	Стажёр/Junior	Junior/Middle	Middle/Senior	любой порог	рядом с порогом: грейд разошёлся	SEM по паре сессий
Backend (Python)	9%	11%	10%	32%	37% (n = 299)	0,42
Frontend (JS/TS)	10%	10%	10%	34%	33% (n = 287)	0,42
QA	8%	8%	11%	32%	28% (n = 285)	0,42
DevOps	8%	8%	8%	27%	30% (n = 298)	0,39
Все вместе	9%	9%	10%	31%	32% (n = 1169)	0,41

Разделяющая способность: итог теста против истинного грейда.

Специализация	n	Точный грейд	±1	QWK	Надёжность	SEM (апост.)	SEM у порогов	Заданий	p95
Backend (Python)	371	76%	100%	0,91	0,84	0,45	0,43	16,0	20
Frontend (JS/TS)	369	75%	100%	0,90	0,83	0,47	0,45	16,5	20
QA	360	73%	100%	0,89	0,82	0,47	0,46	17,0	20
DevOps	372	77%	100%	0,91	0,82	0,50	0,48	14,2	15
Все вместе	1472	75%	100%	0,90	0,83	0,47	0,46	15,9	20

Матрица ошибок, все специализации (строки — истинный грейд, столбцы — итог теста):

Истина \ итог	не подтв.	Стажёр	Junior	Middle	Senior
Стажёр	45	302	53	0	0
Junior	0	49	270	39	0
Middle	0	1	70	262	25
Senior	0	0	0	82	274

Точность решения «выше порога / ниже» у каждого порога (всего / рядом с порогом) и AUC оценки уровня между соседними грейдами (ровно «ниже/выше» порога):

Специализация	Стажёр/Junior	Junior/Middle	Middle/Senior
Backend (Python)	93% / 76% / 0,96	93% / 73% / 0,96	94% / 74% / 0,96
Frontend (JS/TS)	92% / 71% / 0,95	93% / 74% / 0,96	94% / 79% / 0,96
QA	94% / 84% / 0,96	91% / 73% / 0,95	92% / 76% / 0,96
DevOps	93% / 79% / 0,97	93% / 78% / 0,98	92% / 78% / 0,96
Все вместе	93% / 77% / 0,96	93% / 74% / 0,96	93% / 77% / 0,96

Информация Фишера, набранная заданиями сессии у порога, и SEM = 1/√I (среднее по профилям):

Специализация	Стажёр/Junior: I / SEM	Junior/Middle: I / SEM	Middle/Senior: I / SEM
Backend (Python)	4,7 / 0,46	4,9 / 0,45	3,5 / 0,53
Frontend (JS/TS)	4,7 / 0,46	4,9 / 0,45	3,3 / 0,55
QA	5,1 / 0,44	4,7 / 0,46	3,2 / 0,56
DevOps	3,9 / 0,51	4,3 / 0,48	3,5 / 0,53
Все вместе	4,6 / 0,47	4,7 / 0,46	3,4 / 0,54

Дискриминация заданий (зачётные задания сессий A; r_pb — ответ против оценки уровня без этого ответа; A_eff — крутизна IRT-кривой из банка; ΔP — прирост доли верных между соседними грейдами):

Специализация	Семейств (n ≥ 25 ответов)	Медиана r_pb	10-й перцентиль	Доля r_pb ≥ 0,20	Медиана A_eff	Медиана ΔP	Флаг r_pb < 0,10
Backend (Python)	43	0,41	0,24	95%	1,35	0,21	нет
Frontend (JS/TS)	39	0,46	0,27	95%	1,35	0,22	нет
QA	25	0,47	0,35	100%	1,35	0,19	нет
DevOps	19	0,52	0,38	100%	1,35	0,21	нет

Эквивалентность клонов. Живых клонов у нас нет, поэтому разброс сложности клона σ_inst (в конфиге 0,15) — допущение. Проверяем чувствительность: как меняются точность и ретест, если клоны на самом деле разнятся меньше или больше (по каждой специализации, доля точного грейда против истины / доля совпадения при ретесте):

Специализация	σ_inst = 0,00	σ_inst = 0,15	σ_inst = 0,30
Backend (Python)	81% / 70%	81% / 68%	79% / 69%
Frontend (JS/TS)	79% / 72%	78% / 71%	83% / 74%
QA	69% / 64%	82% / 78%	75% / 69%
DevOps	75% / 68%	75% / 71%	73% / 63%

Устойчивость к утечке ответов. Коалиция честных участников выкладывает всё, что видела; слабый кандидат (Стажёр/Junior, заявка на ступень выше) вводит слитый ключ чужого клона. Сравнение парное: та же сессия честно и со слитыми ключами. «Осторожный» списывающий, видя другой код, отвечает честно — его прирост равен нулю по построению, поэтому в таблице наивный.

Специализация	n	Коалиция	Банк слит	Прирост оценки уровня [95% ДИ]	Ложный «сдал» честно	со слитыми ключами	Пойман «чужим ключом»	Детектор активен для семейств
Backend (Python)	100	20	80%	-1,76 [-1,91; -1,65]	15%	0%	100%	39 из 55
Frontend (JS/TS)	100	20	98%	-1,72 [-1,85; -1,60]	12%	0%	100%	25 из 41
QA	100	20	100%	-1,83 [-1,95; -1,71]	21%	0%	100%	13 из 27
DevOps	100	20	100%	-1,70 [-1,83; -1,58]	26%	0%	100%	15 из 19

«Двойник в буфере»: покрытие банка. Доля экземпляров, для которых строится двойник (подменённый литерал, ключ двойника ≠ ключа), — детерминированно, без людей и без LLM:

Специализация	Семейств	С двойником	Экземпляров с двойником	Ключ двойника ≠ ключа	Почему нет двойника (экземпляров)
Backend (Python)	55	33	60%	100%	format_unsupported: 48, answer_mode_not_text: 48, language_not_executable: 30, no_candidates: 6, all_candidates_rejected: 1
Frontend (JS/TS)	41	20	48%	100%	format_unsupported: 48, answer_mode_not_text: 48, language_not_executable: 30, executor_failed: 1
QA	27	13	48%	100%	format_unsupported: 30, answer_mode_not_text: 30, language_not_executable: 24
DevOps	19	12	63%	100%	language_not_executable: 24, answer_mode_not_text: 12, format_unsupported: 6

Пилот анти-ассиста (сводка без персональных данных). Пилот 1: 3 модели одного вендора × 2 условия × 2 обычных задания = 12 ответов, верных 12 — правильный ответ ИИ по одному ответу не отличить от сильного человека. Пилот 2: построено 14 заданий с двойником (predict_output — 10, sql_result — 4); ключ двойника ≠ ключа и все гейты пройдены у 14 из 14. **Прогон моделей на этих ловушках не выполнен, поэтому recall двойника не измерен.** Измерен только факт построения двойников (14/14, ключ двойника ≠ ключу) и покрытие банка (ниже, twin). Доля пойманных копипастеров среди живых людей и моделей не измерена; обход скриншотом двойник не ловит.

Выводы по 2.11 (считаются по числам выше).

- Разделяющая способность: точный грейд против истины — 75%, ±1 — 100%, QWK 0,90. Лучше всего DevOps (77%), слабее всего QA (73%). Грубые ошибки (через ступень) редки; ошибаются соседние грейды — у порога.
- Test-retest: точное совпадение грейда между независимыми сессиями 69%, между сессией и пересдачей по правилам 66%; слабее всего Frontend (JS/TS) (66%). Решение «выше/ниже порога» повторяется в 80% случаев. При пересдаче по правилам слабее всего DevOps (61%): в малых банках (QA, DevOps) после исключения семейств первой сессии остаётся меньше подходящих по сложности заданий. к решения около 0,5 — умеренная. Точное совпадение при 4–5 уровнях и ~16 заданиях не может быть близким к 100%; расхождения сосредоточены у порогов.
- Утечка: у наивного списывающего слитые ключи не дают прироста (верхняя граница 95% ДИ прироста оценки уровня по специализациям: -1,58 ≤ 0). Это модельный результат при допущении σ_inst и уникальных клонов; утечку **идеи** семейства он не покрывает (см. 2.6, CUSUM).
- Двойник строится не везде: доля экземпляров с двойником ниже 50% у Frontend (JS/TS), QA — остальные семейства закрыты только клонами и временем ответа.
- Семейств с r_pb < 0,10 нет.
- Всё это симуляция: модель ответа — Раш с экспертными сложностями, а не живые люди; точность против истины отражает процедуру, а не реальную валидность заданий. Живой ретест на людях не проводился.

3. Категоризация по специализации (V-CAT)

300 синтетических профилей опроса (10 специализаций × 30) и 30 трудных случаев. Ответы порождает своя матрица в `eval/vcat.py` (шум 10%, «амбициозные» отметки соседней специализации у 15% профилей, выбор кандидата совпадает с истиной в 80%); классифицирует прод-функция `app.testing.survey.classify`.

Метрика	Подсказка классификатора	Только выбор кандидата
macro-F1	0,93	0,80
Точность	0,93	0,79

Подсказка показывается 16% профилей, верна в 92% случаев; из тех, кто выбрал не свою специализацию, верную подсказку получают 70%.

Специализация	Точность	Полнота	F1
Backend (Go)	1,00	0,87	0,93
Backend (Java/Kotlin)	1,00	1,00	1,00
Backend (Python)	0,88	1,00	0,94
Data Science / ML	1,00	1,00	1,00
DevOps / SRE	0,86	1,00	0,92
Frontend (JS/TS)	0,73	1,00	0,85
Fullstack	1,00	0,67	0,80
Информационная безопасность	1,00	0,90	0,95
Mobile	1,00	0,90	0,95
QA / тестирование	1,00	1,00	1,00

Трудные случаи:

Тип	n	Верная подсказка (top-1)	Подсказка показана
Fullstack	10	0,60	0,00
смена профессии	10	1,00	0,00
противоречивый опрос	10	0,80	0,90

Веса подсказки подбирались не на основном сиде. Прежние веса «технологии / бонус за выбор» (0,3 / 0,5) заменены на (1,2 / 0,8) по сетке на четырёх других популяциях профилей (сиды 11, 12, 13, 14); проверка — на четырёх отложенных (сиды 21, 22, 23, 24), которых в подборе не было, и на основном сиде (таблица выше — уже с новыми весами).

Выборка, веса	Профилей	macro-F1	Подсказка показана	Подсказка верна	Верная подсказка при ошибочном выборе	Трудные случаи: top-1
подбор, прежние веса	1200	0,87	11%	0,79	0,46	0,75
подбор, текущие веса	1200	0,93	14%	0,90	0,68	0,81
отложенные, прежние веса	1200	0,87	13%	0,84	0,47	0,76
отложенные, текущие веса	1200	0,92	17%	0,94	0,71	0,82

Оговорка: генератор профилей писала та же команда, поэтому проверка на отложенных сидах показывает отсутствие подгонки под 300 профилей, а не перенос на живых людей — это нужно перепроверить на реальных опросах. Подсказка ничего не решает: специализацию выбирает кандидат, категорию подтверждает тест.

Сетка «вес технологий × бонус за выбор» на чётных профилях основного сида (проверка на нечётных): текущие (1,2 / 0,8) — macro-F1 на проверочной половине 0,94; лучшие на чётных 1,2 / 0,8 — 0,94.

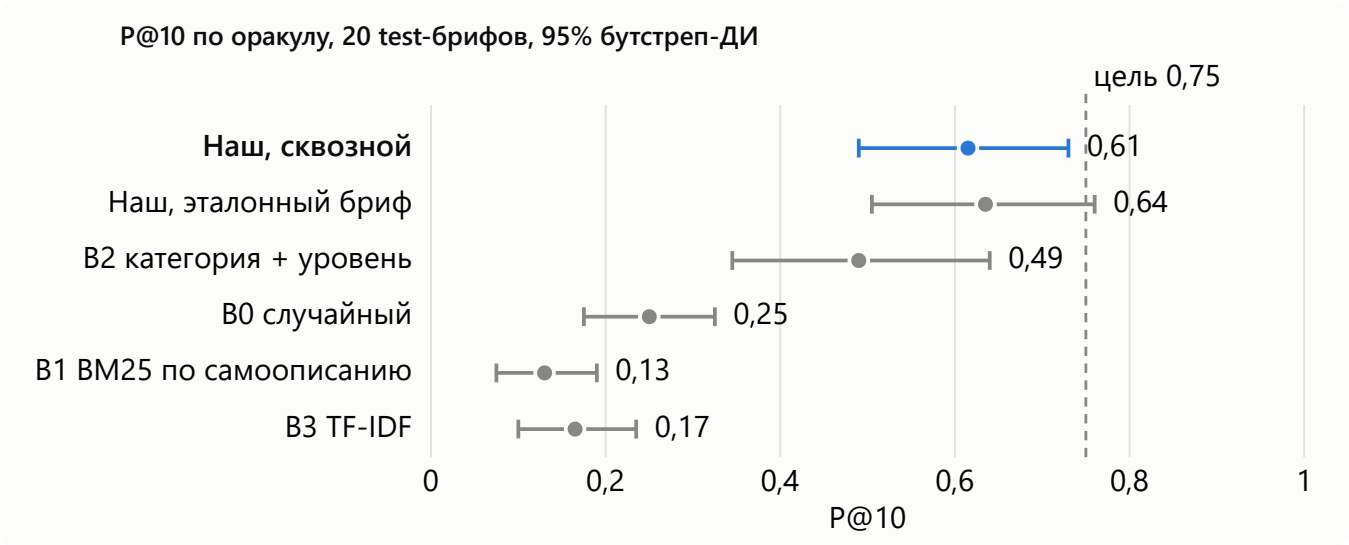
4. Подбор

Оракул релевантности по скрытой истине (VALIDATION_PLAN §1.5): rel = 0 при несмежной специализации; иначе 3, −1 за соседний грейд (≥ 2 ступени → 0), −1 если покрыты не все must, −2 если покрыто меньше половины, −1 если минимум кандидата выше 1,1 · верх вилки. Релевантен ⇔ rel ≥ 2. Навык «покрыт», если он есть в истинном

стеке и (не тестируется или истинный уровень его модуля не ниже порога грейда брифа − 0,5). Формат работы оракул не учитывает.

Популяция — 2 000 кандидатов, из них с подтверждённой категорией 1 865. Сквозной режим: разбор брифа → один уточняющий вопрос, если парсер его задал (отвечает работодатель — вариант из эталона) → обязательная вилка (если парсер её не нашёл, работодатель вводит её в форме) → рекомендованные категории → версия подборки с пресетом «Баланс».

4.1. Главная таблица (20 test-брифов, среднее и 95% бутстреп-ДИ по брифам)



Система	P@10	P@5	nDCG@10	Recall@50	MRR
Наш (сквозной: разбор брифа + 1 уточнение)	0,61 [0,49; 0,73]	0,56 [0,43; 0,68]	0,49 [0,42; 0,56]	0,23 [0,17; 0,29]	0,64 [0,49; 0,79]
Наш (эталонный бриф)	0,64 [0,51; 0,76]	0,58 [0,45; 0,71]	0,51 [0,43; 0,58]	0,24 [0,18; 0,30]	0,67 [0,51; 0,82]
Наш (только парсер, без уточнения)	0,61 [0,48; 0,73]	0,56 [0,43; 0,68]	0,49 [0,42; 0,56]	0,23 [0,17; 0,29]	0,64 [0,49; 0,79]
B2 категория + точечная оценка уровня	0,49 [0,34; 0,64]	0,50 [0,34; 0,67]	0,38 [0,31; 0,48]	0,14 [0,11; 0,16]	0,73 [0,57; 0,89]
B0 случайный порядок в тех же категориях	0,25 [0,17; 0,33]	0,28 [0,20; 0,37]	0,25 [0,18; 0,33]	0,10 [0,08; 0,13]	0,51 [0,36; 0,66]
B1 BM25 по самоописанию (как поиск по резюме)	0,13 [0,07; 0,19]	0,12 [0,06; 0,19]	0,14 [0,10; 0,19]	0,07 [0,05; 0,09]	0,24 [0,14; 0,37]
B3 TF-IDF «бриф ↔ самоописание» (заменитель эмбеддингов)	0,17 [0,10; 0,23]	0,15 [0,09; 0,21]	0,16 [0,12; 0,20]	0,06 [0,05; 0,08]	0,32 [0,21; 0,45]

Потолок P@10 (если поставить всех релевантных наверх): 1,00. Recall@50 ограничен сверху величиной 50 / (число релевантных в популяции), а релевантных у брифа — сотни, поэтому он мал у всех систем.

Разница P@10 с B2 «категория + точечная оценка уровня» (парный бутстреп по брифам):

Система	Разница	95% ДИ	Значимо
Наш (сквозной: разбор брифа + 1 уточнение)	0,12	[-0,04; 0,28]	нет
Наш (эталонный бриф)	0,14	[-0,01; 0,31]	нет
Наш (только парсер, без уточнения)	0,12	[-0,03; 0,28]	нет
B0 случайный порядок в тех же категориях	-0,24	[-0,34; -0,13]	да
B1 BM25 по самоописанию (как поиск по резюме)	-0,36	[-0,53; -0,21]	да

Система	Разница	95% ДИ	Значимо
B3 TF-IDF «бриф ↔ самоописание» (заменитель эмбеддингов)	-0,33	[-0,49; -0,17]	да

Все 40 брифов (test + dev; ни те ни другие не использовались при настройке формулы). P@10 нашего сквозного режима 0,66 [0,57; 0,74], B2 0,54 [0,43; 0,64]; разница с B2 0,12 [0,03; 0,22] — значима. Против случайного порядка в тех же категориях: 0,35 [0,26; 0,44]. Добавленный pooled-срез отмечен как постфактум: заявленная метрика остаётся на 20 test-брифах.

4.1a. Предрегистрированная отложенная выборка test2 (60 брифов b41–b100)

План и правило решения записаны до написания брифов (VALIDATION_PLAN §8, 9 октября 2026, 09:06 MSK); брифы заморожены в 09:09 MSK, до первого запуска разбора и Score на них; оценка выполнена один раз. Заморозка при этом прогоне: хэш манифеста совпадает с записанным в плане (`fa766490b26e3c4e…`, файлов 60, расхождений 0).

Система	P@10 test2	nDCG@10 test2	P@10 test + test2 (80)	nDCG@10 test + test2 (80)
Наш (сквозной: разбор брифа + 1 уточнение)	0,54 [0,47; 0,61]	0,55 [0,50; 0,61]	0,56 [0,49; 0,62]	0,54 [0,49; 0,58]
Наш (эталонный бриф)	0,53 [0,45; 0,61]	0,55 [0,50; 0,61]	0,56 [0,49; 0,62]	0,54 [0,50; 0,58]
Наш (только парсер, без уточнения)	0,53 [0,46; 0,61]	0,54 [0,49; 0,60]	0,55 [0,49; 0,62]	0,53 [0,49; 0,58]
B2 категория + точечная оценка уровня	0,36 [0,28; 0,43]	0,38 [0,33; 0,43]	0,39 [0,33; 0,46]	0,38 [0,34; 0,43]
B0 случайный порядок в тех же категориях	0,18 [0,14; 0,23]	0,22 [0,19; 0,26]	0,20 [0,16; 0,24]	0,23 [0,20; 0,27]
B1 BM25 по самоописанию (как поиск по резюме)	0,14 [0,10; 0,18]	0,15 [0,12; 0,18]	0,14 [0,11; 0,17]	0,15 [0,12; 0,17]
B3 TF-IDF «бриф ↔ самоописание» (заменитель эмбеддингов)	0,13 [0,10; 0,17]	0,14 [0,11; 0,17]	0,14 [0,11; 0,17]	0,15 [0,12; 0,17]

Парный бутстреп по брифам (наш сквозной режим минус сравниваемая система), 2 000 повторов:

Выборка	Разность	Значение	95% ДИ	Значимо
test2	P@10 − B2	0,18	[0,12; 0,24]	да
test2	P@10 − B0 (случайный)	0,35	[0,29; 0,42]	да
test2	nDCG@10 − B2	0,17	[0,12; 0,22]	да
test2	nDCG@10 − B0	0,33	[0,28; 0,38]	да
test + test2	P@10 − B2	0,17	[0,11; 0,23]	да
test + test2	P@10 − B0 (случайный)	0,36	[0,30; 0,41]	да
test + test2	nDCG@10 − B2	0,16	[0,11; 0,20]	да
test + test2	nDCG@10 − B0	0,31	[0,26; 0,35]	да

Справочно: парный t-критерий P@10 наш − B2 на test2: t = 5,92, p < 0,001 (решает бутстреп, §8.3).

Решение по предрегистрированному правилу (нижняя граница ДИ разности P@10 с B2 > 0): подтверждено — разность 0,18 [0,12; 0,24]. Абсолютный P@10 на test2 0,54 [0,47; 0,61]: **ниже порога 0,60 (§2.3), порог на этой выборке не выполнен** — пишем как есть. Ни Score, ни конфиг, ни оракул после просмотра результата не менялись.

Описательно, по специализациям (малые выборки, выводов не делаем):

Специализация	Брифов	P@10 наш	P@10 B2	Разность P@10
backend_python	24	0,64 [0,57; 0,72]	0,43 [0,32; 0,55]	0,21
frontend_js	24	0,55 [0,41; 0,68]	0,33 [0,20; 0,46]	0,23
fullstack	12	0,31 [0,17; 0,44]	0,28 [0,16; 0,39]	0,03

Разбор брифа на test2 (справочно): специализация top-1 0,97, грейд точно 0,97.

По оракулу наш подбор не обгоняет B2 — так и пишем. Оракул видит только грейд, стек и деньги; B2 сортирует основную категорию ровно по тому, что оракул вознаграждает сильнее всего (уровень), а наша формула добавляет то, чего оракул не видит: формат работы, свежесть задач и ФСП. Где формула выигрывает — ниже: устойчивость к накрутке навыков и к пересдаче (4.4), проверяемые объяснения (4.6); где проигрывает — разобрано в диагностике выше и в 4.5.

Откуда нерелевантные места в топ-10 (описательная диагностика после прогона, порогов у неё нет):

Причина по оракулу	Наш подбор	B2
грейд совпал, покрыто меньше половины must	41	32
грейд на ступень выше брифа, покрыты не все must или минимум выше вилки	26	32
грейд на ступень ниже брифа, покрыты не все must или минимум выше вилки	8	4
грейд совпал, покрыты не все must и минимум выше вилки	1	28
другая специализация	1	0
грейд и must подходят, но формат/город или зарплата не подходят	0	4
грейд дальше чем на ступень	0	2
пустые места (в категориях меньше 10 кандидатов)	0	0
всего нерелевантных мест из 200	77	102

Два главных источника: (1) must-навыки, которые тест не проверяет (Docker, Kafka, Redis и т. п.): оракул знает истинный стек, а формула по построению видит только доказательства — тест, задачи, ФСП — и не верит самоописанию; (2) кандидаты на ступень выше брифа: в категорию попадают сильные люди, заявившие грейд ниже своего и не повысившие его, а внутри категории Score растёт вместе с результатом теста. Оракул считает «переросших» релевантными, только если покрыты все must. Это расхождение формулы и оракула, а не ошибка движка; вопрос передан владельцу подбора (штрафовать ли «переросших» внутри категории, как показывать непроверяемые must).

4.2. Абляции

Снятый компонент отдаёт вес тесту (ограничения на веса соблюдены). «Без нижней границы» — ранжирование по точечной оценке уровня (SE = 0, без старения).

Вариант	P@10	nDCG@10	Разница P@10 с полной формулой	95% ДИ
без нижней границы (точечная оценка, SE = 0)	0,60 [0,47; 0,71]	0,49 [0,41; 0,58]	-0,01	[-0,04; 0,01]
без K (стек)	0,60 [0,48; 0,71]	0,48 [0,41; 0,55]	-0,01	[-0,04; 0,01]
без F (ФСП)	0,65 [0,52; 0,76]	0,51 [0,44; 0,58]	0,03	[-0,01; 0,07]
без A (свежесть)	0,60 [0,47; 0,72]	0,47 [0,39; 0,55]	-0,01	[-0,05; 0,01]
без U (условия)	0,60 [0,49; 0,71]	0,49 [0,41; 0,56]	-0,01	[-0,03; 0,01]

4.3. Разбор брифа

Метрика	test	dev (справочно)	Цель
Специализация top-1	0,90	1,00	≥ 0,90
Грейд точно / ±1	0,95 / 1,00	1,00 / 1,00	≥ 0,75 / ≥ 0,95
Навыки: точность / полнота / F1	0,99 / 0,98 / 0,98	0,99 / 0,99 / 0,99	F1 ≥ 0,85
must: F1	0,83	0,86	—
must/nice определены верно (среди найденных)	0,81	0,84	≥ 0,80
Верная категория в топ-3	0,95	1,00	≥ 0,95
Вилка: точно от и до / на руки или до НДФЛ	1,00 / 1,00	1,00 / 1,00	—
Формат точно / город	0,95 / 1,00	1,00 / 1,00	—
Задан уточняющий вопрос	0,00	0,00	—

Ошибки парсера на test-брифах:

Бриф	Стиль	Специализация	Грейд (парсер / эталон)	Пропущены навыки	Лишние	Ошибка must/nice
b05	chat	верно	junior / junior	—	layout	css, responsive
b07	formal	ошибка (backend_python)	middle / middle	—	—	drf
b15	long	верно	senior / senior	highload	—	asyncio, caching, docker, fastapi, kafka, postgresql, redis
b17	chat	верно	junior / junior	—	—	mongodb, postgresql
b21	chat	верно	senior / senior	python	—	—
b23	formal	верно	senior / senior	—	—	webpack
b25	long	ошибка (backend_python)	senior / senior	db_design	—	celery, react, typescript
b29	chat	верно	junior / junior	—	—	jquery, layout
b31	mixed_en	верно	middle / middle	—	—	typescript
b33	formal	верно	junior / intern	—	—	—
b35	long	верно	junior / junior	—	—	css, figma, flexbox, javascript, layout

4.4. Устойчивость

Накрутка навыков. В каждом test-брифе 50 самым слабым кандидатам выдачи добавили в самоописание 5 навыков брифа (1000 случаев). Наш подбор: средний сдвиг вверх 0,0 позиции (максимум 0), ни один не попал в топ-10: заявленные навыки в Score не входят и служат только последним ключом сортировки при равном Score (DECISIONS п.1). BM25 по самоописанию: сдвиг вверх 518,4 позиции, в топ-10 попали 12,4%.

Стабильность при пересдаче (тест пересимулирован на непересекающихся семействах, overlap@10, 20 брифов): наш подбор 0,28, тот же подбор по точечной оценке 0,28, B2 «категория + точечная оценка» 0,06. Исходная цель ≥ 0,70.

Исходный порог не достигнут и, как показывает разбор ниже, **поставлен некорректно** (постфактум-пересмотр, VALIDATION_PLAN §7; сам порог в таблице «Итоги» оставлен и показан как не выполненный). Причина в измерении, а не в формуле: в категории размах истинного уровня — одна полоса грейда, а SE теста из ~15 заданий сопоставим с ней, поэтому надёжность оценки уровня внутри категории — всего 0,25 (по популяции в целом 0,86, но это почти целиком различия между грейдами). Две сессии одного человека честно ранжируют категорию по-разному, и у любого ранжирования, опирающегося на одну оценку теста, состав десятки из сотен кандидатов меняется (Goldstein & Spiegelhalter, 1996, о рейтингах с ошибкой измерения).

Что измеряем вместо «десятка осталась той же»:

Показатель	Значение
overlap@10 нашего подбора	0,28
потолок overlap@10 для ранжирования по одному измерению уровня при надёжности внутри категории 0,25 (симуляция)	0,09
наш overlap@10 / потолок	3,3
индекс Кунчевой@10 (overlap с поправкой на случайность: 0 — как у случайной десятки, 1 — совпадение)	0,26
RBO (p = 0,9, глубина 50; вес позиции убывает, Webber и др., 2010)	0,25
P@10 по оракулу: первая сессия → после пересдачи	0,61 → 0,67
доля прежней десятки, которая остаётся в топ-50 после пересдачи	63%
доля прежней десятки, ушедшей из категории (грейд при пересдаче изменился)	11%

Вывод: состав десятки при пересдаче меняется сильнее, чем хотелось бы, но в 3,3 раза устойчивее потолка «сортировки по оценке теста» (за счёт стабильных K, A, F и U), а **качество** десятки при пересдаче не падает (P@10 по оракулу после пересдачи не ниже, чем в первой сессии), то есть заменяются одинаково пригодные кандидаты. Попытка стабилизировать порядок «полосами» Score (кандидаты внутри полосы ранжируются по hash, «равные по уровню») заметно не стабилизировала (прирост не больше 0,03 — в пределах шума при 20 брифах): ширина 2 — overlap@10 0,29, P@10 0,62; ширина 5 — overlap@10 0,27, P@10 0,62; ширина 8 — overlap@10 0,30, P@10 0,62 (без полос: 0,28 и 0,61); в продукт не внесено. Что делаем в продукте: версия подборки неизменна и меняется только по действию работодателя, грейд защищён гистерезисом.

Добавлен nice-навык: Жаккар топ-10 до и после — 0,95 в среднем, минимум 0,67.

Веса ±0,05 (компенсация пропорционально остальным; если вес упирается в ограничение wF ≤ 0,15 или wT ≥ 0,30, он остаётся на месте). P@10 при весах «Баланс» — 0,61.

Компонент	Сдвиг	P@10	Падение P@10
T	+0,05	0,61	0,01
T	−0,05	0,61	0,01
K	+0,05	0,63	-0,01
K	−0,05	0,61	0,00
F	+0,05	пропущен	—
F	−0,05	0,63	-0,01
A	+0,05	0,61	0,01
A	−0,05	0,61	0,00
U	+0,05	0,62	-0,01
U	−0,05	0,61	0,00

4.5. Экспозиция кандидатов без ФСП (DECISIONS п.11)

Условное отношение: наблюдаемое число релевантных кандидатов без ФСП в топ-k к ожидаемому, если бы они попадали в топ-k так же часто, как все релевантные **того же дециля компонента T** (при равном тесте). 1,0 — паритет; цель 0,8–1,25. Децили считаются по всем кандидатам рекомендованных категорий брифа, а не только по 500 строкам версии. Безусловное отношение (доля без ФСП в топе к их доле среди релевантных) — справочно.

k	Условное	Безусловное	Наблюдаемо / ожидаемо
10	0,91	0,91	73 / 80,2

k	Условное	Безусловное	Наблюдаемо / ожидаемо
50	0,94	0,92	333 / 355,7

Доля кандидатов без ФСП среди релевантных — 65%.

4.6. Честность объяснений

Проверено 1000 карточек (топ-50 каждого test-брифа).

Проверка	Доля
Σ вкладов = Score (± 0,1 после округления)	100,0%
Каждый довод «за» и «против» ссылается на существующее доказательство (попытка теста, модуль, задача, запись ФСП, настройки, навык брифа)	100,0%
Отсутствие ФСП никогда не попадает в «против»	100,0%
«За» даны только компонентам выше медианы версии более чем на 1 балл	100,0%
Без доказательства довода №1 Score падает не меньше его превышения над медианой (994 проверок)	100,0%

Контрфакт по компоненту довода №1: A — 7 из 7; F — 17 из 17; K — 92 из 92; T — 878 из 878. Контрфакт: T — теста нет; F — нет ФСП; K и A — убраны именно те модули, задачи и записи ФСП, на которые ссылается довод. Довод «Стек» ссылается на все источники подтверждённых навыков, «Свежесть» — на все задачи окна (v_k и A берут максимум, поэтому одного лучшего источника мало: при исходной версии, где называлось по одному источнику, этот пункт был 0,96 — 34 из 91 по K и 7 из 7 по A не проходили из-за запасных источников). U — настройки, а не доказательство, не проверяется.

4.7. Производительность

Сборка версии подборки (Score + объяснения + ранги) на пуле из 1 865 подтверждённых кандидатов: медиана 58 мс, p95 133 мс, максимум 378 мс (100 брифов). Цель p95 ≤ 300 мс. Замер — на машине разработчика параллельно с другими разделами прогона.

4.8. По брифам (test)

Бриф	Стиль	Категория эталона	Релевантных	Категории подборки	Вопрос	P@10 наш	P@10 B2	P@10 B1
b01	formal	backend_python · middle	160	backend_python · middle	—	0,8	0,2	0,3
b03	short	frontend_js · middle	326	frontend_js · middle	—	0,9	0,9	0,1
b05	chat	frontend_js · junior	78	frontend_js · junior	—	0,8	0,5	0,2
b07	formal	fullstack · middle	30	backend_python · middle	—	0,6	0,1	0,0
b09	chat	backend_python · junior	77	backend_python · junior; backend_python · middle; backend_python · senior	—	0,8	0,3	0,1
b11	mixed_en	frontend_js · middle	35	frontend_js · middle	—	0,4	0,0	0,1
b13	short	backend_python · middle	244	backend_python · middle	—	0,9	0,5	0,0
b15	long	backend_python · senior	162	backend_python · senior; backend_python · middle	—	0,4	0,9	0,1
b17	chat	fullstack · junior	41	fullstack · junior; backend_python · junior	—	0,0	0,0	0,0

Бриф	Стиль	Категория эталона	Релевантных	Категории подборки	Вопрос	P@10 наш	P@10 B2	P@10 B1
b19	mixed_en	backend_python · middle	275	backend_python · middle; backend_python · senior	—	0,6	0,7	0,5
b21	chat	backend_python · senior	142	backend_python · senior	—	0,6	0,9	0,0
b23	formal	frontend_js · senior	82	frontend_js · senior; frontend_js · middle	—	1,0	0,3	0,0
b25	long	fullstack · senior	93	backend_python · senior; fullstack · senior; backend_python · middle	—	0,3	0,5	0,3
b27	short	backend_python · junior	184	backend_python · junior	—	0,4	0,5	0,3
b29	chat	frontend_js · junior	97	frontend_js · junior; frontend_js · intern	—	0,7	0,8	0,0
b31	mixed_en	fullstack · middle	64	fullstack · middle; frontend_js · middle; backend_python · middle	—	0,0	0,1	0,2
b33	formal	backend_python · intern	132	backend_python · junior; backend_python · intern	—	0,9	0,1	0,1
b35	long	frontend_js · junior	249	frontend_js · junior	—	0,8	1,0	0,0
b37	chat	frontend_js · middle	177	frontend_js · middle; frontend_js · senior	—	0,7	0,5	0,2
b39	short	backend_python · senior	163	backend_python · senior	—	0,7	1,0	0,1

4.9. dev-брифы (справочно)

Система	P@10	nDCG@10
Наш (сквозной: разбор брифа + 1 уточнение)	0,70 [0,59; 0,81]	0,56 [0,47; 0,65]
Наш (эталонный бриф)	0,71 [0,60; 0,82]	0,56 [0,47; 0,66]
Наш (только парсер, без уточнения)	0,70 [0,59; 0,81]	0,56 [0,47; 0,66]
B2 категория + точечная оценка уровня	0,58 [0,42; 0,73]	0,48 [0,37; 0,59]
B0 случайный порядок в тех же категориях	0,37 [0,23; 0,51]	0,29 [0,19; 0,40]
B1 BM25 по самоописанию (как поиск по резюме)	0,23 [0,15; 0,33]	0,19 [0,13; 0,26]
B3 TF-IDF «бриф ↔ самоописание» (заменитель эмбеддингов)	0,20 [0,13; 0,28]	0,19 [0,13; 0,25]

5. Набор пар для проверки жюри

`backend/eval/pairs_test.jsonl` — 1 706 строк: для каждого из 20 test-брифов пул из объединения топ-20 систем (наш сквозной, B0, B1, B2, B3) и 10 случайных кандидатов. Одна строка — одна пара «бриф × кандидат» в JSON. Схема — `backend/eval/pairs_test.schema.json` (JSON Schema 2020-12). Кандидаты вне пула при оценке считаются нерелевантными (обычный пулинг); для P@10 и P@5 глубины 20 хватает.

Поля строки:

- `brief_id`, `brief.text`, `brief.form` (вилка из обязательной формы, если её нет в тексте), `brief.gold` — эталон: специализация, грейд, must, nice, форматы, город, вилка;
- `candidate_id`, `candidate.features` — слепые признаки в форме `app.matching.scoring.CandidateFeatures` (категория, оценка уровня и SE, подбаллы модулей с тегами, задачи, VSS по дисциплинам ФСП, заявленные навыки, минимум зарплаты, форматы, город); `null`, если

категория тестом не подтверждена; `candidate.self_description` — самоописание (в Score не входит, нужно только для B1/B3);

- `rel` — релевантность 0–3 по оракулу (релевантен ⇔ rel ≥ 2); `ranks_top20` — место в топ-20 каждой системы пула.

Как пользоваться: ранжируйте кандидатов каждого брифа своей системой (или нашей — порядок в `ranks_top20`) и посчитайте P@k по полю `rel`. Импорт этого файла и своих пар — `POST /api/admin/import`, метрики — `POST /api/admin/eval/pairs`, ранжирование — `POST /api/admin/eval/rank` (раздел 7). Скрытой истины (истинный уровень, истинный стек) в файле нет — только итоговая метка оракула.

Как набор собран. Популяция с латентной истиной (`eval/population.py`) → 40 брифов (`eval/briefs`, эталон разметки написан командой) → для каждого test-брифа пул из топ-20 пяти систем и 10 случайных кандидатов → `rel` 0–3 даёт оракул по скрытой истине (специализация, грейд, доля must/nice в истинном стеке, формат/город — не учитывается); релевантен rel ≥ 2. **Размер** — см. выше; минимального размера организаторы не требуют. **Ограничение:** разметка машинная, не рекрутерская (см. раздел 6).

6. Честные ограничения

- Синтетика проверяет процедуру, а не реальные сложности заданий. Ответы порождает модель Раша с разбросом клонов; нецикличных DINA-персон и живого пилота в этом прогоне нет, раздела «люди» нет.
- Экспертной разметки пар нет: релевантность — оракул по истине генератора. Оракул и генератор писала одна команда, поэтому P@k проверяет согласованность механики с нашей же моделью мира, а не с мнением рекрутера.
- Брифы и эталон к ним написаны командой с ИИ-ассистентом; 10 брифов от посторонних авторов, заявленные в концепции, не собраны. Разбиение dev/test есть, но авторы одни и те же.
- Генератор V-CAT и веса классификатора опроса составлены одной командой — проверка частично круговая.
- B3 «LLM под каждого» — симуляция с допущениями (разброс сложности σ = 0,5, доля битых ключей задана, а не измерена). B3 подбора — TF-IDF, а не эмбеддинги: модель эмбеддингов в прогоне не используется.
- Знание «методов» частично обходит защиту: клоны закрывают слив ответов, но не слив идеи семейства; это измерено в 2.6 и ловится CUSUM лишь при массовом сливе.
- От ИИ-ассистента удалённый тест не защищён, как любой удалённый тест; грейд — решение с уровнем доверия, а не сертификат.
- Точное совпадение грейда при пересдаче не может быть близким к 1 при 4–5 уровнях и ~15 заданиях; стабильность для пользователя обеспечивает гистерезис.
- В банке нет pretest-семейств: r_pb считается по боевым заданиям с оценкой уровня без этого задания.
- Fullstack как производная категория в популяции не порождается (каждый кандидат проходит один тест), поэтому Fullstack-брифы ищут в смежных категориях Backend и Frontend.
- Test-retest и разделяющая способность (2.11) измерены на симулированных профилях: ответы порождает модель Раша с экспертными сложностями и допущенным разбросом клонов, поэтому цифры — верхняя оценка для живых людей, где к шуму добавятся усталость, подсказки и неидеальные ключи. Ретеста на людях не проводилось.
- Покрытие двойниками неполное (2.11): для части форматов (выбор ответа, не исполняемые языки) двойник не строится; recall двойника на моделях и людях не измерен.
- Формат работы и город оракул не учитывает, а наша формула учитывает (компонент U): часть «ошибок» по оракулу — это кандидаты, которых формула честно опустила из-за формата.

Пороговые пункты, которые не выполнены в этом прогоне, перечислены в «Итогах»; пороги плана после прогона не менялись. По итогам предыдущего прогона менялись параметры продукта, а не пороги — на других сидах, чтобы не подгонять под основной: `cusum_h` (6 → 11, раздел 2.7), веса подсказки опроса (раздел 3) и полнота доказательств довода в объяснениях (4.6). Что не выполнено и почему:

- **Точный грейд.** Разница с целью 0,75 в пределах выборочной ошибки (n = 1 810, SE ≈ 0,010): на пяти других сидах (101–105) среднее 0,758 при том же коде, на основном сиде 0,748. Ужесточение правила остановки (P ≥

0,92 / n_min = 10) даёт +0,0…0,3 п.п. ценой +0,6…1,7 задания основной части (+0,8…1,6 с доборами) — бюджет длины (≤ 17 с доборами) нарушается, поэтому не внесено (`eval/stop_rule_sweep.py` , VALIDATION_PLAN §7).

- **Корреляция оценок уровня при пересдаче.** CAT выбирает задания для решения «выше/ниже порога», а не для оценки уровень у всех: SE у кандидатов, далёких от порога, остаётся большим. На пяти других сидах среднее 0,857; самые длинные варианты правила остановки дают 0,861–0,867 при 17,0–17,7 заданиях. Цель 0,88 несовместима с бюджетом длины ≤ 17.
- **Задержка CUSUM при 50% знающих.** Порог h подобран по ложным карантинам (≤ 5% на отдельных сидах), а задержка ≈ h / (информация на ответ) физически ограничена: при Δb = 0,5 и половине знающих эффект на один ответ мал. Уложиться в 200 ответов можно ценой 12–31% ложных карантинов (h = 5–7). CUSUM остаётся сигналом методисту, а не автоматической блокировкой.
- **P@10 значимо выше B2 на 20 test-брифах.** Разница и её ДИ — в 4.1: при 20 брифах интервал чуть захватывает 0, на всех 40 брифах разница значима. Оракул награждает ровно то, по чему сортирует B2 (уровень), а формат, свежесть и ФСП оракул не видит; подгонять веса под оракул ради значимости не будем. Мощности не хватало, поэтому разность проверена на 60 новых брифах по предрегистрированному протоколу (VALIDATION_PLAN §8): результат — в 4.1a, пункт 20 test-брифов остаётся невыполненным.
- **Стабильность топ-10 при пересдаче (overlap@10 ≥ 0,70).** Цель поставлена некорректно: при надёжности оценки уровня внутри категории порядка 0,25–0,3 даже идеальное ранжирование по одному измерению даёт overlap@10 около 0,1 (раздел 4.4). Постфактум-метрики — P@10 после пересдачи и отношение к потолку — выполнены; исходная цель оставлена невыполненной.

7. Набор жюри: как проверить вручную на своих профилях

Организаторы проверяют воспроизводимость и разделяющую способность на своём закрытом наборе синтетических профилей. Есть два пути.

A. Test-retest и разделяющая способность (тест) — `eval/jury_check.py` . Принимает JSONL или CSV с профилями и прогоняет их через боевой CAT-движок (три сессии на профиль: основная, параллельная, пересдача по правилам).

```
cd backend
uv run python -m eval.jury_check eval/jury_profiles_example.jsonl --seed 1 --out eval/out/jury_check.json
```

Формат строки (CSV — те же имена столбцов, заголовок обязателен):

Поле	Значение	Обязательно
`id`	уникальная строка	да
`spec`	`backend_python` · `frontend_js` · `qa` · `devops`	да
`true_grade`	ваш эталон: `intern` · `junior` · `middle` · `senior`	да
`theta`	латентный уровень на шкале движка (пороги: Стажёр/Junior −1, Junior/Middle 0, Middle/Senior +1); нет — берётся из полосы грейда по сиду	нет
`claimed_grade`	грейд, на который профиль проходит тест; нет — равен `true_grade`	нет

Пример: `backend/eval/jury_profiles_example.jsonl` (16 профилей). Вывод — таблица в консоль и JSON (`eval/out/jury_check.json`): согласие грейдов, κ, r(уровень), доли переворотов, матрица ошибок против `true_grade` , точность и AUC у каждого порога, информация и SEM у порогов, дискриминация заданий и грейды по каждому профилю. Фиксированный `--seed` даёт тот же результат. Неверная строка → код 2 и сообщение с номером строки и полем.

B. Подбор: свои пары «бриф × кандидат» — админ-API. Формат строк — `backend/eval/pairs_test.jsonl` (схема `pairs_test.schema.json` , раздел 5): бриф, слепые признаки кандидата (или `features: null` — без

теста), `rel` 0–3.

1. `POST /api/admin/import` — загрузить пары жюри (данные в банк платформы не попадают); `GET /api/admin/import` — список загрузок.
2. `POST /api/admin/eval/pairs` — P@5, P@10, nDCG@10 с 95% бутстреп-ДИ по брифам (тот же код, что в `make validate`); без `pairs` берётся последний импорт.
3. `POST /api/admin/eval/rank` — ранжирование пула по брифу нашей формулой Score с объяснениями (CSV — `?format=csv`).

Требуется роль администратора или методиста; ключи доступа — в `docs/API.md`.

C. Без Swagger — страница `/lab/check`. Вход жюри на `/jury` → «Лаборатория» (роль методиста). Вкладка «Наш набор пар» одной кнопкой считает P@5, P@10, nDCG@10 с 95% ДИ на нашем наборе и показывает топ-10 по каждому брифу с релевантностью; «Своя вакансия против банка платформы» подбирает кандидатов из банка тем же движком, что у работодателя; «Свой набор» принимает три таблицы (вакансии, кандидаты, разметка `rel` 0–3) в CSV или JSON, шаблоны скачиваются на странице. Методист видит только свои загрузки. Тесты: `tests/test_api_admin_check.py`, `tests/test_jury_simple.py`.

8. Качество заданий: DIF, дрейф, экспозиция

Монитор справедливости и стабильности заданий (`app/testing/monitoring.py`, прогон — `uv run python -m eval.fairness`, seed 20261005). **Это проверка метода на синтетике:** группы задаём мы, в одну группу кладём заведомо смещённые задания, и отчёт обязан их найти. Выводов о реальных людях здесь нет; пол и возраст платформа не собирает, поэтому группы — ФСП (есть/нет), регион (Москва и Санкт-Петербург / остальные), режим доступности, экспресс-версия.

8.1. DIF: Мантель–Хензель, классы ETS

Сессий в анализе: 4000. Критерий сопоставления — уровень сессии в стратах по 0,5 логита; MH D-DIF = −2,35·ln α_MH; A — |Δ| < 1 или различие незначимо, C — |Δ| ≥ 1,5 и значимо больше 1, иначе B. Δ < 0 — задание тяжелее для фокальной группы.

Внедрённые смещённые задания (сдвиг сложности для фокальной группы в логитах):

Группа	Семейство	Сдвиг	Доля верных: опорная / фокальная	MH D-DIF (±SE)	Класс
регионы	`py.format.spec`	0,30	0,65 / 0,54	-1,33 ± 0,35	**B**
регионы	`py.defaults.mutable`	0,60	0,47 / 0,35	-1,59 ± 0,36	**C**
регионы	`http.idempotency.retry_cart`	1,00	0,51 / 0,28	-3,56 ± 0,38	**C**
без истории ФСП	`sql.group.having`	0,60	0,63 / 0,47	-2,34 ± 0,37	**C**

Настоящие семейства банка (без внедрения), все пары «семейство × группа» с достаточным числом ответов (не меньше 30 в каждой группе):

Группа (фокальная)	Проверено пар	A	B	C
без истории ФСП	81	77	4	0
регионы	79	77	2	0
режим доступности	74	74	0	0
экспресс-версия	82	82	0	0
Всего	**316**	**310**	**6**	**0**

Доля класса A среди проверенных: **98%**. При ~316 проверках отдельные B ожидаемы (5% ложных срабатываний на уровне значимости), поэтому в лаборатории B — повод посмотреть, C — повод вывести семейство на ревью.

Различие в уровне между группами при этом не принимается за DIF (impact): средний истинный уровень у кандидатов без ФСП -0,03 против -0,04 с ФСП — МН сравнивает людей одного уровня.

8.2. Монитор дрейфа сложности

По ответам после калибровки оценивается сдвиг сложности Δb (метод моментов при апостериорном уровне отвечавшего без этого ответа, 95% ДИ по информации Фишера). Флаг «дрейф» — Δb ≤ −0,50 (стало легче) и весь ДИ ниже 0 (не меньше 40 ответов). Эксперимент: 12 самых частых семейств «утекли» — 80% кандидатов знают ответы (сложность для них ниже на 1,50); 1912 сессий.

- Найдено утёкших семейств: **12 из 12** (полнота 100%); средний Δb по утёкшим -0,71 при «наивном» ожидании -1,20 (оценка сжата: знающие завышают собственный уровень, и это съедает часть сдвига).
- Ложные флаги среди остальных семейств в прогоне с утечкой: 0 (специфичность 100%).
- Контроль (без утечки): флагов **1** из 44 семейств.

Это дополняет CUSUM (раздел 2.7): CUSUM быстро реагирует на поток, монитор даёт величину сдвига с доверительным интервалом для решения методиста. Ночная задача калибровки пишет число флагов в журнал аудита (`scheduler.calibration`).

8.3. Экспозиция семейств и Симпсон–Хеттер

Специализация backend_python: 44 семейств, целевой максимум r_max = 0,60. Экспозиция — доля сессий, в которых семейство показано (одноразовые клоны делают экспозицию конкретного экземпляра ≈ 0, поэтому контроль нужен на уровне семейства). Контроль Симпсона–Хеттера в движке выключен по умолчанию (`quality.sympson_hetter.enabled`); таблица — симуляция с K_i, подобранными за 8 итераций на одной половине кандидатов и проверенными на другой.

Показатель	Без контроля	Симпсон–Хеттер
Сессий	455	455
Максимум экспозиции	0,68	0,67
Семейств выше r_max	5	2
Доля почти неиспользуемых (<1%)	0,00	0,00
Средняя экспозиция	0,37	0,37
Средняя погрешность (SE) итогового уровня	0,447	0,452
Среднее число заданий	16,27	16,16
Верных решений о пороге	0,818	0,804

Семейств с K_i < 1: 4. Нижняя граница экспозиции — средняя длина теста / размер банка = 0,37: r_max ниже этого числа недостижим, поэтому его выбирают выше. Формула итерации: K_i ← min(1, r_max / P(S_i)), P(S_i) — доля сессий, где задание было выбрано до фильтра K (Sympson & Hetter 1985).

Честно про результат. За несколько итераций на ~450 сессиях контроль сходится не полностью: K_i считаются по P(S_i), а P(S_i) растёт, когда соседние задания «отклоняют» (их место занимают остальные); классическая процедура требует десятков итераций на тысячах сессий. Максимум задают семейства самого большого модуля: квота модуля обязывает показывать их почти каждому. Поэтому контроль в продукте выключен, а `eval.fairness` — способ посчитать K_i под реальный банк перед включением.

10. Пилот с ИИ-испытуемыми

10.1. Что и зачем (предрегистрация)

Модель IRT-симуляции (раздел 2) проверяет движок на синтетических людях, которых мы же и описали. Пилот добавляет сквозное свидетельство: **Claude-агенты играют кандидатов с заданным профилем знаний и проходят настоящий адаптивный тест через настоящий HTTP API** (регистрация, опрос, выбор грейда, одноразовые клоны заданий, решение, экран результата), каждый — дважды, на двух аккаунтах (повторный тест подтверждённого грейда правилами пересдачи не предусмотрен, поэтому вторая попытка идёт со свежего аккаунта и с другими клонами). 12 честных персон (4 специализации × Junior/Middle/Senior) и 2 «списывающих», которые копируют задание и решают его копию из буфера (проверка двойника-ловушки).

Протокол, профили знаний и анализ описаны в `backend/eval/agent_pilot/PROTOCOL.md` и **зафиксированы до первого прогона**. Сессии пилота помечены `demo_assisted`: в калибровку, норму семейств, экспозицию и метрики не входят.

Честная рамка. ИИ-испытуемые не заменяют людей. Языковой модели трудно «не знать»: она склонна отвечать лучше, чем отвечал бы живой Junior, поэтому **ожидаем завышение у младших персон** (и почти нулевое занижение у старших). Пилот не калибрует шкалу и не доказывает валидность на людях — он показывает, что весь путь работает, повтор воспроизводим, а ловушка срабатывает; что получилось, то и публикуем.

Предрегистрированные метрики (ориентиры заданы до прогона и не используются как условие «прошёл / не прошёл»):

Метрика	Что считаем	Ориентир
Согласие повторов	полоса грейда по баллу (0 — ниже Стажёра … 4 — Senior), точное совпадение	≥ 60%
Согласие ±1	различие не более чем на одну полосу	≥ 90%
Каппа Коэна	взвешенная (линейные веса) и обычная по полосам	взвешенная ≥ 0.60
Балл	средний модуль разности баллов повторов, корреляция Пирсона	без порога
Уровень против задуманного	матрица «задуман × получен», смещение (получен − задуман) по уровням	без порога; ожидаем + у Junior
Длина и время	число заданий по уровням, медиана секунд на задание, таймауты	без порога (у агентов время не человеческое)
Ложные тревоги	честные прогоны с уликой-«следом двойника» или флагом	0
Ловля списывающих	доля прогонов «списывающих» с уликой `twin_trace`, флагом или проверкой методиста	≥ 50%

10.2. Результаты

Прогонов: **32**, персон: **14**; пар повторов у честных персон с результатом: **12**.

Повтор теста (test-retest), честные персоны

Метрика	Значение	Ориентир
Точное совпадение полосы	0,83	≥ 0,60
Различие не больше ±1	1,00	≥ 0,90
Каппа Коэна (обычная)	0,68	—
Каппа Коэна (взвешенная)	0,83	≥ 0,60
Совпадение решения «подтверждён / нет»	0,92	—
Средний модуль разности баллов	20,1	—
Корреляция баллов	0,97	—

Персона	Специализация	Задуман	Полоса 1 / 2	Балл 1 / 2	Решение 1 / 2
P01	backend_python	Junior	Junior / Junior	472 / 417	да / да
P02	backend_python	Middle	Senior / Senior	665 / 698	да / да
P03	backend_python	Senior	Senior / Senior	687 / 687	да / да
P04	frontend_js	Junior	Junior / Стажёр	472 / 376	да / нет
P05	frontend_js	Middle	Senior / Middle	601 / 595	да / да
P06	frontend_js	Senior	Senior / Senior	686 / 687	да / да
P07	qa	Junior	Junior / Junior	473 / 483	да / да
P08	qa	Middle	Senior / Senior	669 / 694	да / да
P09	qa	Senior	Senior / Senior	686 / 686	да / да
P10	devops	Junior	Junior / Junior	467 / 464	да / да
P11	devops	Middle	Senior / Senior	607 / 616	да / да
P12	devops	Senior	Senior / Senior	700 / 697	да / да

Полученный уровень против задуманного (полоса по баллу, все попытки честных персон)

Задуман \ получен	ниже Стажёра	Стажёр	Junior	Middle	Senior	смещение	совпало	подтверждён
Junior	0	1	7	0	0	-0,12	0,88	0,88 (n=8)
Middle	0	0	0	1	7	0,88	0,12	1,00 (n=8)
Senior	0	0	0	0	8	0,00	1,00	1,00 (n=8)

Длина теста и время

Уровень	Прогонов	Заданий (среднее, мин–макс)	Доля верных ответов
Junior	8	13,1 (8–20)	0,56
Middle	8	8,0 (8–8)	0,94
Senior	8	9,0 (8–12)	1,00

Ответов: 241, таймаутов: 0, пропусков: 6, медиана времени ответа агента: 4,3 с (у агента это не человеческое время, поэтому метрика справочная).

Ложные тревоги антиподсказчика: 1 из 24 честных прогонов (ориентир: 0).

«Списывающие» персоны

Персона	Заявлено / задуман	Заданий	Скопировано (с двойником)	Улики	Флаги	Статус	Решение	Поймана
C01 (поп. 1)	middle / junior	14	14 (4)	twin_trace: 4	—	FINAL	подтверждён	да
C01 (поп. 2)	middle / junior	13	13 (4)	twin_trace: 4	—	FINAL	подтверждён	да
C02 (поп. 1)	middle / junior	17	12 (3)	correct: 1, twin_trace: 3	—	FINAL	подтверждён	да
C02 (поп. 2)	middle / junior	8	6 (4)	twin_trace: 4	—	EXTENSION_OFFERED	не подтверждён	да

Доля пойманных прогонов: **1,00** (ориентир ≥ 0,50). «Поймана» значит, что улика записана в акт сверки для методиста; на решение теста улика не влияет (вес 0 до калибровки ε). **Завышенную категорию (подтверждён**

заявленный Middle при задуманном Junior) получили 3 из 4 прогонов — двойник доставался только первым четырём заданиям сессии, а дальше копирование в ИИ проходило без последствий.

10.3. Как читать

Результаты — на малой выборке (12 честных персон, 2 списывающих) и на агентах, а не на людях: доверительные интервалы не строились, выводов о валидности на людях делать нельзя. Если ориентир не достигнут, это записано как есть. Пилот повторяется командами из `PROTOCOL.md`; сырые прогоны лежат в `backend/var/pilot_runs/` (в git не входят).

10.4. Повтор «списывающих» после исправления (пост-хок)

Не входит в предрегистрированные доли выше. По итогам 10.2 изменена выдача двойников: двойник ставится на каждое задание, для которого он посчитан (`evidence.twin_rate_per_session: 20` вместо 4), а предрасчёт двойников встроен в конвейер пула. Те же персоны и инструкции, свежие аккаунты и клоны; честных персон это изменение не касается (они отвечают по экрану) и они не перепрогонялись. В пилоте двойники достраиваются в момент выдачи (запас 20 с, PROTOCOL.md §2), на стенде — предрасчётом в конвейере пула и выдачей клона с готовым двойником.

Персона	Заявлено / задуман	Заданий	Скопировано (с двойником)	Улики	Флаги	Статус	Решение	Поймана
C01 (поп. 3)	middle / junior	8	8 (6)	twin_trace: 6	—	EXTENSION_OFFERED	не подтверждён	да
C01 (поп. 4)	middle / junior	8	8 (7)	twin_trace: 7	—	EXTENSION_OFFERED	не подтверждён	да
C02 (поп. 3)	middle / junior	8	6 (4)	twin_trace: 4	—	EXTENSION_OFFERED	не подтверждён	да
C02 (поп. 4)	middle / junior	8	6 (5)	twin_trace: 5	—	EXTENSION_OFFERED	не подтверждён	да

Завышенную категорию получили **0 из 4** прогонов (до исправления — 3 из 4).

Как воспроизвести

```
cd backend
uv run python -m eval.validate            # полный прогон: этот файл, eval/out/validation.json, графики, набор пар
uv run python -m eval.validate --quick    # дымовой прогон на меньших выборках (eval/out/validation_quick.json)
uv run python -m eval.spec_validation --quick   # только test-retest и разделяющая способность (eval/out/spec_validation.json)
uv run python -m eval.jury_check eval/jury_profiles_example.jsonl   # проверка на своих профилях (раздел 7)
uv run python -m eval.validate --save-db  # дополнительно записать ValidationRun в базу
uv run python -m eval.validate --save-db --from-json eval/out/validation.json  # залить готовый результат без прогона
```

Время по разделам, с: population 14,7, matching 75,8, leak 37,7, hysteresis 4,4, cusum 9,1, calibration 0,3, vcat 0,5, cusum_calib:301 66,4, cusum_calib:302 66,4, cusum_calib:303 66,7, s2:sigma:0.0 5,0, s2:sigma:0.3 5,4, s2:sigma:0.5 6,0, s2:a:1.0 5,8, s2:a:2.0 4,9, baselines:0 12,2, baselines:1 6,6, baselines:2 5,7, baselines:3 6,1, baselines:4 10,2, spec_validation 274,3.

Предложение ФСП: интеграция с реестром и ФСП ID

Документ описывает, как «Взаимно» получает данные Федерации спортивного программирования, как их использует и как перейти от демо-стенда к реальным системам ФСП. Он рассчитан на то, чтобы Федерация могла взять его как есть: **контракт API реестра** (разделы 3–4) — предложение, по которому мы уже написали рабочий мок и клиент; **расчёт VSS** (раздел 5) — прозрачная формула, все веса которой лежат в `config/fsp.yaml` и пересматриваются вместе с экспертами ФСП.

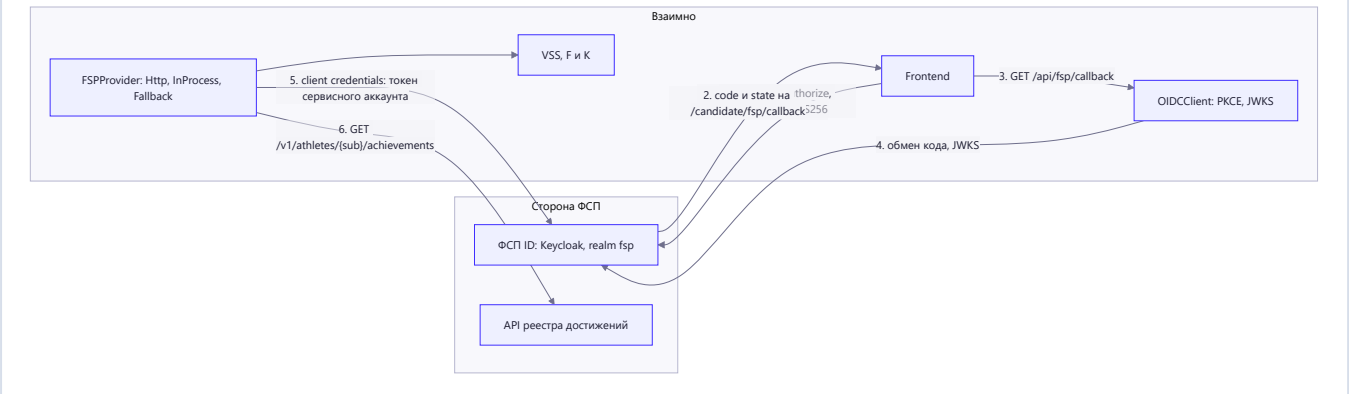
Источники истины: контракт — `backend/app/fsp/models.py` (копия в `services/fsp-mock/fsp_mock/contract.py` синхронизируется командой `python -m fsp_mock sync-contract`, расхождение ловит тест `test_fsp_mock.py`); realm — `deploy/keycloak/realm-fsp.json`; веса — `config/fsp.yaml`; клиент — `backend/app/fsp`, `backend/app/services/fsp_link`.

Содержание: 1. Принципы · 2. Схема · 3. ФСП ID · 4. API реестра · 5. VSS · 6. Соревнования и мероприятия · 7. Согласия · 8. Несовершеннолетние · 9. Путь к реальной интеграции · 10. Вопросы к ФСП

1. Принципы

Принцип	Что это значит
ФСП усиливает профиль, но не открывает дверь	Достижения влияют на **порядок внутри категории** (компонент F, не больше 15 баллов из 100) и на подтверждение навыков. Грейд и категорию даёт только тест
Отсутствие истории — не штраф	Нет привязки, нет результатов, результат ждёт протокола: F = 0, в «против» не попадает никогда
Согласие и контроль у спортсмена	Данные реестра сохраняются только при отдельном согласии; результаты можно скрыть; привязку можно снять с удалением данных
Минимум данных работодателю	До взаимного «да» — обобщённая строка («Призёр всероссийских · продуктовое программирование (хакатон) · 2025 · топ-3 · в команде»), без ФИО, номера ЕКП, команды и ссылки на протокол: пара «ЕКП + место» по публичным протоколам однозначно выдаёт человека
Проверяемость	После раскрытия — выписка с номером ЕКП, местом из N, командой и ссылкой на протокол
Замена без переписывания кода	Переход на реальные системы ФСП — смена переменных окружения и реализация одного HTTP-контракта

2. Схема интеграции

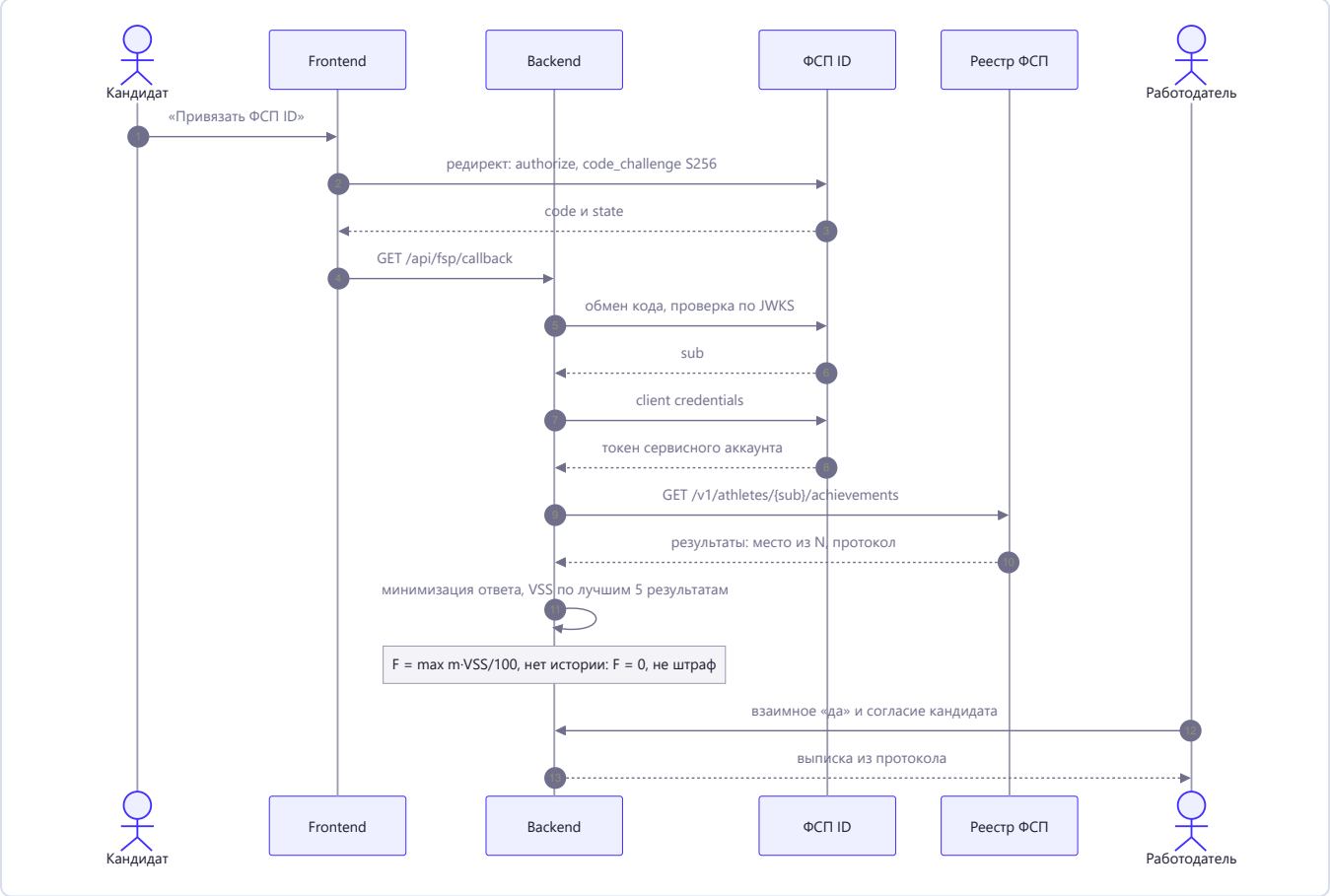


Две независимые части:

1. **ФСП ID** отвечает на вопрос «кто это»: даёт стабильный `sub` и (для нового аккаунта) email. Своя регистрация и сессии «Взаимно» от него не зависят.
2. **Реестр достижений** отвечает на вопрос «что у него в протоколах»: по `sub` отдаёт результаты, соревнования, разряды и роли.

Пока у ФСП нет публичного API реестра (протоколы публикуются сканами PDF), на стенде работают: Keycloak с тем же realm `fsp` и сервис `fsp-mock` (60 вымышленных спортсменов, 25 соревнований, фамилии из ИТ-терминов, в номере ЕКП после года стоит «99» — признак синтетики). Если мок или Keycloak не подняты, демо не ломается: данные берутся из встроенного снимка (`VZ_FSP_PROVIDER=auto`), а вместо входа есть «демо-вход ФСП» с подписанным мок-токеном (в интерфейсе везде пометка «Демо-реестр ФСП»).

Последовательность: привязка, достижения, VSS и выписка:



3. ФСП ID (OIDC)

3.1. Клиент в realm `fsp`

Параметр	Значение
Realm	`fsp` (на стенде — свой Keycloak 26 из `deploy/keycloak/realm-fsp.json` ; в проде — `https://sso.fsp-russia.ru/realms/fsp`)
Client ID	`vzaimno` (`VZ_FSP_OIDC_CLIENT_ID`)
Тип клиента	Конфиденциальный (client secret), `publicClient: false`
Поток	Authorization code (`standardFlowEnabled`) + **PKCE S256** (`pkce.code.challenge.method = S256`); прямой доступ по паролю выключен (`directAccessGrantsEnabled: false`)

Параметр	Значение
Redirect URI	`https://<домен>/candidate/fsp/callback` — страница фронтенда (`VZ_FSP_OIDC_REDIRECT_URI` , маршрут есть в `routes.tsx`), она вызывает `GET /api/fsp/callback` с `code` и `state` и привязывает ФСП ID. Локально — `http://localhost:5173/candidate/fsp/callback` . В демо-realm дополнительно разрешён реальный адрес бэкенда `…/api/fsp/callback` : переход браузера на него (заголовок `Sec-Fetch-Mode: navigate`) бэкенд перенаправляет (303) на страницу фронтенда с теми же `code` / `state` (и `error` , если ФСП ID отказал) — так работает и client, у которого зарегистрирован именно этот адрес. Старых путей `/auth/fsp/callback` и `/api/auth/fsp/callback` нет
Scopes	`openid profile email` (`VZ_FSP_SCOPES`)
Сервисный аккаунт	Включён (`serviceAccountsEnabled`): grant `client_credentials` для чтения API реестра
Защита от перебора	В демо-realm включена `bruteForceProtected` ; срок жизни access-токена 300 с

3.2. Какие claims нужны и что мы с ними делаем

Claim	Нужен	Использование
`sub`	Да	Ключ связи: `fsp_links.fsp_id` (UNIQUE: один ФСП ID — один аккаунт) и ключ запроса к реестру `GET /v1/athletes/{sub}…` . **Должен быть стабилен**
`email`	Только при первом входе с лендинга	Адрес нового аккаунта; на него отправляется код подтверждения (аккаунт создаётся в `pending_email`). Без `email` — ошибка `fsp_email_missing` . Email из ФСП ID **не даёт права** на чужой аккаунт: если такой адрес уже зарегистрирован, вход отклоняется (409), привязать ФСП ID можно в профиле
`iss` , `aud` , `nonce` , `exp` , `iat`	Да	Проверка id_token: подпись по JWKS (допустимы RS/PS/ES-алгоритмы, `none` и HS* запрещены), `iss` равен realm, `aud` содержит наш client, `nonce` совпадает с начатым входом, срок действия с запасом 30 с. При неизвестном `kid` JWKS перечитывается один раз (ротация ключей)
`name` , `given_name` , `family_name` , `preferred_username` , `email_verified`	Нет	В привязке хранятся только служебные claims (`iss` , `sub` , `aud` , `azp` , `auth_time` , `preferred_username` , `email_verified`): имя и email из ФСП ID платформе для подбора не нужны

3.3. Как защищён вход

- `state` и `nonce` — случайные 256/192 бит; в БД хранится только `sha256(state)` , запись одноразовая (`used_at`), живёт 10 минут (`VZ_FSP_STATE_TTL_S`).
- Вход привязан к браузеру: ответ `GET /api/fsp/login` ставит httpOnly-cookie `vz_fsp_flow` (путь `/api/fsp`) со `state` ; завершить вход можно только из того же браузера. Привязка (`mode=link`) завершается только аккаунтом, который её начал.
- Бэкенд ходит за токеном по внутреннему адресу realm (`VZ_FSP_OIDC_INTERNAL_URL`), браузер — по публичному (`VZ_FSP_OIDC_ISSUER`); `iss` в id_token принимается и публичный, и внутренний. С настоящим SSO ФСП оба адреса совпадают.
- Лимиты: начать вход, завершить вход, демо-вход — 30/300 с, 30/300 с, 20/300 с с одного IP; «обновить достижения» — 6 раз за 10 минут на аккаунт.

4. Предлагаемый контракт API реестра

Контракт — OpenAPI 3.1, версия `1.0` (`CONTRACT_VERSION`). Лишние поля в ответах клиент игнорирует (`extra="ignore"`), поэтому реестр может расширять схему, не ломая клиентов. Получить схему из мока:

```
cd services/fsp-mock
pip install -r requirements.txt
uvicorn fsp_mock.main:app --port 8001        # документация: http://localhost:8001/docs
# схема файлом:
```

```
python -c "import json; from fsp_mock.main import app; print(json.dumps(app.openapi(),
ensure_ascii=False, indent=1))" > fsp-registry-openapi.json
```

4.1. Эндпоинты

Все `/v1/*` требуют заголовок `Authorization: Bearer <токен>`; ответы — JSON в кодировке UTF-8; ошибки — `{"detail": "текст на русском"}`.

Метод и путь	Ответ	Назначение	Коды
`GET /v1/athletes/{sub}`	`Athlete`	Профиль спортсмена: ФИО, дата рождения, регион. `sub` — из ФСП ID	200, 401, 404, 422
`GET /v1/athletes/{sub}/achievements`	`Achievements`	Результаты (вместе с соревнованиями, на которые они ссылаются), разряды, роли судьи и тренера. Пустые списки — спортсмен есть, результатов нет	200, 401, 404, 422
`GET /v1/competitions/{ekp}`	`CompetitionProtocol`	Карточка соревнования и выписка результатов по возрастанию места. Нужен для сверки протокола; платформа пока вызывает его только через провайдер, в потоках подбора не использует	200, 401, 404, 422
`GET /protocols/{ekp}`	HTML	Публичная страница протокола — значение `Competition.protocol_url`; токен не нужен (в демо)	200, 404
`GET /health`	`{"status", "contract_version"}`	Проверка работоспособности	200

Семантика ошибок для клиента: `404` — спортсмена нет в реестре (для платформы это «результатов нет», F = 0); `401/403` — отказ в доступе (`FSPAuthError`); сетевая ошибка, таймаут и `5xx` — «реестр недоступен» (`FSPUnavailable`): прежние данные кандидата **не обнуляются**, синхронизация получает статус `error`, а в режиме `auto` используется резервный снимок. Различие важно: сбой сети не должен выглядеть как «у спортсмена нет результатов».

Требования к данным: `athlete_sub` во всех записях ответа равен `sub` из пути (чужие записи платформа отбрасывает и пишет предупреждение); каждый `Result.ekp` присутствует в `Achievements.competitions`; `ekp` — ровно 16 цифр; записи про одного спортсмена стабильно идентифицируются полем `id` (на нём держатся «скрыть результат» и дедупликация).

4.2. Аутентификация клиента

Режим	Как	Где
Демо	Статический bearer-токен (`FSP_MOCK_TOKENS` в моке, `VZ_FSP_REGISTRY_TOKEN` в бэкенде), сравнение за постоянное время	`fsp-mock`
Прод	**OAuth 2.0 client credentials** сервисного аккаунта того же client в realm `fsp`: `POST VZ_FSP_TOKEN_URL` (`grant_type=client_credentials` , `client_id` , `client_secret` , необязательный `scope` из `VZ_FSP_REGISTRY_SCOPE`). Токен кэшируется до истечения с запасом 30 с; при `401` обновляется один раз. Если задан `VZ_FSP_TOKEN_URL` , статический токен не используется	`ClientCredentialsAuth`

Доступ к реестру идёт от имени платформы, а не пользователя: платформа обязана запрашивать только `sub`, который получила от ФСП ID при входе этого пользователя, и только при действующем согласии.

4.3. Модели

Типы полей — по `backend/app/fsp/models.py`. Колонка «Как используем» показывает, где поле влияет на результат.

`Athlete` — спортсмен

Поле	Тип	Описание	Как используем
`sub`	строка	Идентификатор = claim `sub` ФСП ID	Ключ связи
`last_name`, `first_name`, `middle_name`	строки	ФИО	Только кандидату в его собственном предпросмотре (фамилия и инициалы). Работодателю не передаются. Не участвуют в ранжировании
`birth_date`	дата	Дата рождения	Только год рождения, для политики по несовершеннолетним (раздел 8); наружу не отдаётся
`region_code`, `region_name`	строки	Субъект РФ	Не используется в ранжировании
`organization`	строка	Региональное отделение или клуб	Не используется

`Competition` — соревнование из ЕКП

Поле	Тип	Описание	Как используем
`ekp`	16 цифр	Номер в Едином календарном плане	Ключ; в выписке после раскрытия
`title`	строка	Название	Строка результата
`discipline_code`	`^\d{10}[А-ЯA-Z]$`	Код дисциплины ВРВС	Дисциплина → короткий код (`algo`, `product`, `bas`, `robotics`, `infosec`); подпись в интерфейсе — из `config/fsp.yaml`, для `1010021811Я` — «Продуктовое программирование (хакатон)», как дисциплину называет ФСП
`level`	enum	`international`, `all_russian`, `interregional`, `regional`, `municipal`	Вес уровня L
`status_type`	enum	`championship`, `cup`, `first` (первенство), `other_sport`, `physical_culture_event`, `student`	Вес уровня L (для всероссийских: чемпионат, первенство, кубок) и подпись
`age_group`	enum	`adults`, `juniors_u26`, `youth_u19`, `kids_u15`	Передаётся и хранится, в вес не входит
`stage`	enum	`qualifier`, `final`, `single`	Вес L для отбора; группировка отбор+финал
`parent_ekp`	16 цифр или пусто	Для отборочного этапа — ЕКП финала	Отбор и финал считаются **одним результатом** (берётся лучший)
`format`	enum	`personal`, `team`	Размер поля N: число команд при командном зачёте
`online`	bool	Проводилось онлайн	Не используется
`start_date`, `end_date`	даты	Даты	Свежесть T считается от `end_date`
`region_code`, `city`	строки	Место проведения	Не используется
`participants_count`, `teams_count`	числа ≥ 1	Участники и команды	Размер поля N (`field_size`)
`max_score`	число > 0 или пусто	Максимально возможный балл	Поправка Q по сырому баллу
`protocol_status`	enum	`published`, `pending` (протокол ещё не сдан, до ~2 недель)	`pending` → результат «ожидает протокол», в оценку не идёт
`protocol_url`	строка	Ссылка на протокол	Только в выписке после раскрытия

`Result` — результат спортсмена или его команды

Поле	Тип	Описание	Как используем
`id`	строка	Идентификатор результата	«Скрыть / показать»
`ekp`	16 цифр	Соревнование	Склейка с `Competition`
`athlete_sub`	строка	Спортсмен	Проверка владельца
`place`, `place_to`	числа ≥ 1 или пусто	Место или диапазон мест (5–8 в сетке на выбывание); пусто — участие без места	Качество места Q; для диапазона берётся середина, бонус — по худшему месту
`field_size`	число ≥ 1	N — из скольких участников (команд) занято место	Q
`score`	число ≥ 0 или пусто	Итоговый балл	Q (вместе с `max_score`)
`role`	enum	`personal`, `captain`, `participant`, `reserve`	Вес роли Ro
`team`	`TeamRef` или пусто	`id`, `name`, `members[]` (`athlete_sub`, `role`, `team_function`)	Название команды — только в выписке после раскрытия
`team_function`	строка	Функция в команде из заявки: `backend`, `frontend`, `fullstack`, `mobile`, `ml`, `devops`, `design`, `pm`, `analyst`	**Только как заявленное**: в протоколах ФСП роли в команде нет, вклад в командный хакатон по ним честно не выделить. В VSS, F, K, обобщённую строку и паспорт навыков не входит; работодатель видит её отдельно серым — «роль backend — по заявке участника»
`status`	enum	`confirmed`, `awaiting_protocol`	Подтверждён ли результат

`Rank` — разряд или звание. Источники: нормы ЕВСК по спортивному программированию — Приложение № 24 к приказу Минспорта России № 450 от 22.04.2024 (PDF на fsp-russia.ru); сроки действия разрядов — Положение о ЕВСК (приказ Минспорта России № 173 от 03.03.2025), п. 76. Вид спорта «спортивное программирование» — код ВРВС **1010001311Я** (`config/fsp.yaml → sport`): он печатается в выписке из протокола и в паспорте навыков.

Поле	Тип	Описание	Как используем
`id`, `athlete_sub`	строки	Идентификаторы	—
`code`	enum	`MS`, `KMS`, `I`, `II`, `III`, `I_y`, `II_y`, `III_y` (и `MSMK` — см. справа)	Вес разряда Rk. Нормы ЕВСК по спортивному программированию — МС, КМС, I–III и юношеские I–III; звания МСМК в них нет: код `MSMK` оставлен в контракте «на будущее», но веса не имеет, в VSS не входит и в интерфейсе не показывается
`discipline_code`	код ВРВС или пусто	Дисциплина результата, за который присвоен	Привязка к дисциплине; пусто — ко всем дисциплинам с результатами; код вне справочника в VSS не идёт
`assigned_at`, `valid_until`	даты	Присвоение и срок действия: КМС — 3 года, I–III и юношеские — 2 года, МС — бессрочно (пусто)	Действующий разряд ×1, истёкший — со свежестью от даты окончания. Если реестр не прислал `valid_until` для КМС или I–III, срок досчитывается от `assigned_at` по `config/fsp.yaml → vss.rank_validity_years`
`order_no`, `assigned_by`, `basis_ekp`, `status`	строки / enum (`active`, `expired`)	Приказ, кто присвоил, основание	Бейдж «КМС до 2027»; в оценку не входят

`RoleBadge` — судья или тренер: `kind` (`judge` , `coach`), `position` , `category` (`VK` , `1K` , `2K` , `3K` , `YuSS`), `discipline_code` , `since` . **Только бейдж** («судья-эксперт (1К)»), в оценку силы не входит.

`Achievements` — ответ `/achievements` : `athlete_sub` , `as_of` (момент, на который реестр отдал данные: показывается как «данные реестра на …» на странице сверки и в PDF), `source` (`fsp_api` , `fsp_mock` , `diploma`), `results[]` , `competitions[]` (все соревнования из результатов), `ranks[]` , `roles[]` .

`CompetitionProtocol` — `competition` и `results[]` . `RegistrySnapshot` — полная выгрузка (`contract_version` , `data_as_of` , все списки): начальная синхронизация и встроенный резервный провайдер. `ErrorResponse` — `{detail}` .

4.4. Когда и как платформа обращается к реестру

Момент	Что происходит
Предпросмотр (до согласия)	Запрос «на лету»; ответ **не сохраняется**
Согласие на обогащение	Синхронизация: ответ сохраняется (`fsp_links.achievements`), нормализуется в `fsp_achievements` , пересчитывается VSS, признаки подбора помечаются устаревшими
Кнопка «обновить достижения»	Повторная синхронизация, не чаще 6 раз за 10 минут на аккаунт
Вход с привязанным ФСП ID	Синхронизация, если согласие действует
Ночное обновление и вебхук реестра `achievement.created`	В разработке; при вебхуке реестру нужно только отправлять `sub` изменившегося спортсмена

Таймаут запроса — 5 с (`VZ_FSP_TIMEOUT_S`); редиректы не выполняются. Нагрузка на реестр — по одному–двум запросам на кандидата при привязке и по запросу «обновить»; кэширования ответов на стороне реестра не требуется.

5. Расчёт VSS

VSS («подтверждённая сила») считается **по каждой дисциплине** из подтверждённых, не скрытых результатов и разрядов (`app/fsp/vss.py` , веса — `config/fsp.yaml → vss`). Время передаётся явно из `Clock` .

```
points(r)  = L · C · Q · Ro · T · V                результат r (L = 1 для всех мероприятий, C — класс результата)
A_d        = Σ_{i ≤ 5} p_(i) · 0,6^(i−1)           лучшие 5 результатов по убыванию (отбор и финал одного соревнования — один, лучший;
                                                   участий без призового места учитывается не больше 2)
VSS_d      = 100 · ( 1 − exp( −(A_d + 0,5·Rk) / 0,6 ) )      разряд без результатов в реестре: VSS_d ≤ 50
F          = max_d  m(d, специализация брифа) · VSS_d / 100   → компонент Score (вес 0,15 → не более 15 баллов)
K          : v_k ≥ 0,8 · m(d, модуль навыка) · VSS_d / 100    ФСП как доказательство навыка
```

Множитель	Что это	Значения
L — уровень мероприятия	**Равноправие** (ответы организаторов, DECISIONS п. 29): все дисциплины и мероприятия равны, «чемпионат России выше хакатона» не утверждаем	`1,0` для всех уровней и статусов; правила `vss.level_rules` оставлены в конфиге, чтобы эксперты ФСП при желании могли завести вес, не трогая код
C — класс результата	Призёр (худшее место ≤ 3) или участник (участник, участник отбора, финалист без призового места). **Защита от накрутки**: профиль нельзя раздуть онлайн-явками	призёр `1,0` ; участник `0,5` ; в `A_d` идут не больше **2** участий без призового места (`vss.result_class`)
Q — место с поправкой на размер поля	`q = 1 − (место−1)/max(N−1; 1)` ; при сыром балле `q' = 0,5·q + 0,5·балл/макс` ; `F(N) = min(1; ln(1+N)/ln 51)` ; `Q = max(0,1; min(1; q^1,5 · F(N) + бонус))`	бонус за подиум: 1-е место +0,15, 2-е +0,10, 3-е +0,05. **Подтверждённое участие без места или в хвосте — не меньше** `Q_min = 0,1` : сам факт участия не обнуляется

Множитель	Что это	Значения
Ro — роль		личный зачёт 1,0; капитан 0,9; участник команды 0,85; запасной 0,4
T — свежесть	`2^(−Δмес/24)`	полураспад 24 месяца — экспертное допущение «результат заметно устаревает за два года»; это **не** срок действия разряда (разряды действуют 2–3 года, МС — бессрочно, см. Rk)
V — источник		`fsp_api` 1,0; `fsp_mock` 1,0 (демо-реестр стоит на месте API); `diploma` 0,5
Rk — разряд	Вес лучшего действующего разряда; истёкший — со свежестью от даты окончания срока (КМС — 3 года, I–III и юношеские — 2 года, МС — бессрочно)	МС 0,9; КМС 0,7; I 0,5; II 0,35; III 0,25; юношеские 0,1. МСМК в нормах ЕВСК по спортивному программированию нет — веса нет

Пример (дата отсчёта 05.10.2026). Спортсмен занял 1-е место из 103 на соревновании по продуктовому программированию 05.04.2026 в личном зачёте: `L = 1,0`, `C = 1,0` (призёр), `Q = 1,0` (поле больше 51, подиумный бонус обрезается единицей), `Ro = 1,0`, `T = 2^(−6/24) = 0,841`, `V = 1,0`; `points = 0,841`; `A = 0,841`; `VSS = 100·(1 − e^(−0,841/0,6)) = 75,4`. Для брифа Backend при `m(product, backend_python) = 0,8` получается `F = 0,8·0,754 = 0,603`, то есть вклад в Score `100 · 0,15 · 0,603 = 9,0` балла. Для сравнения: 15-е место из 60 на региональном соревновании год назад в команде (участник, `C = 0,5`) даёт `VSS = 28,4`; участие в студенческом муниципальном соревновании без места 60 дней назад — `VSS = 6,5`; двенадцать подряд онлайн-участий без призовых мест дают столько же, сколько два (потолок участий); один разряд КМС без результатов — `VSS = 44,2` (с пометкой «оценка по разряду»).

Что учитывается, а что нет (ответы организаторов, DECISIONS п. 29). Учитываем: результат (класс «призёр / участник» и место в поле), число стартов (убывающая отдача `0,6^(i−1)` и потолок участий), релевантность дисциплины специализации кандидата (`m(d, s)`: ИБ-соревнования — для ИБ-специалиста) и свежесть (полураспад 24 месяца — допущение, не срок действия разряда). **Не** ранжируем мероприятия между собой: уровень или статус соревнования веса не даёт. Прежняя шкала уровней (всероссийский 0,9, региональный 0,3 и т. д.) отменена: она утверждала иерархию, которой организаторы не признают. **ФСП никогда не влияет на грейд** — грейд определяет только тест; ФСП входит лишь в Score (F) и в доказательство навыков (K), а в модулях `app/testing` и `app/services/testing` данных ФСП нет вовсе.

Матрицы `m(d, s)` (дисциплина → специализация) и `m(d, модуль)` лежат в `config/fsp.yaml` (`spec_relevance`, `module_relevance`): например, алгоритмическое → Backend 0,8, Frontend 0,4; продуктовое → Fullstack 0,9, Backend и Frontend 0,8; ИБ → DevOps 0,7. Все веса — экспертные допущения: **их нужно калибровать вместе с экспертами ФСП**. Обобщение места для работодателя до раскрытия — полосы «топ-3», «топ-10», «топ-10 %», «топ-25 %», «топ-50 %»; 1-е место тоже показывается как «Призёр», точное место — только в выписке.

6. Учёт соревнований и мероприятий

Вид записи в реестре	Как учитывается
Результат на официальном соревновании ЕКП (чемпионат, первенство, кубок)	Вес результата (класс «призёр / участник», место в поле, раздел 5); уровень и статус мероприятия веса **не** дают; попадает в VSS
Отбор и финал одного соревнования	Связаны `parent_ekp`; считаются **одним** результатом — берётся лучший (иначе отбор и финал удваивали бы вклад)
Другие официальные соревнования, физкультурные мероприятия, студенческие соревнования (`other_sport`, `physical_culture_event`, `student`)	Считаются так же, как любые другие: по результату, числу стартов и свежести; веса за уровень или статус нет
Командный зачёт	Размер поля N — число команд; вклад умножается на вес роли по протоколу (капитан 0,9, участник 0,85, запасной 0,4). Функция в команде (`team_function`) — из заявки, в протоколе её нет: на оценку не влияет, работодателю показывается отдельно серым «по заявке участника»

Вид записи в реестре	Как учитывается
Онлайн-формат и возрастные группы	Поля `online` и `age_group` передаются и хранятся, в вес не входят; накрутку онлайн-явками гасит потолок участий без призового места (не больше 2 в `A_d`, вес участника `0,5`)
Результат без места	Подтверждённое участие: `Q = Q_min = 0,1`
Результат с ожидающимся протоколом	`status = awaiting_protocol` или `protocol_status = pending` → «ожидает протокол»: в VSS не идёт, у кандидата подпись «Ожидает протокол», учтётся после публикации
Разряд	Бейдж (например, «КМС до 2027») и слагаемое `0,5·Rk` в VSS. Только разряды норм ЕВСК по спортивному программированию (МС, КМС, I–III, юношеские)
Судья, тренер	**Только бейдж**, в Score не входят
Дисциплина не из справочника ВРВС (другой вид спорта)	В VSS не идёт

Состояние профиля, которое видит кандидат и от которого зависит подпись для работодателя: `not_linked`, `linked_no_results`, `awaiting_protocol`, `linked`. Отдельное состояние экрана — `registry_unavailable`: реестр не ответил (сеть, отказ в доступе, по адресу `VZ_FSP_REGISTRY_URL` отвечает чужой сервис); оно не равно «результатов нет», причина — в `sync_error`, привязка и согласие сохраняются, F = 0 без штрафа. HTTP-клиент считает «спортсмена нет в реестре» только `404` с телом контракта `{"detail": "…"}`; любой другой `404` (другой сервис на порту, прокси) — недоступность. Внешние соревнования вне ЕКП (например, Кубок Федерации CTF, Codeforces) в текущей модели не учитываются — это вопрос к ФСП (раздел 10).

7. Согласия, скрытие результатов, приватность

Что	Как устроено
Согласие `fsp_enrichment`	Отдельная запись журнала согласий с версией текста (`POST /api/fsp/me/enrichment`). Без него данные реестра только показываются в предпросмотре и **не сохраняются**, на подбор не влияют
Скрытие результатов	`PATCH /api/fsp/me/results/{result_id}` `{hidden}`. Выбор живёт и до согласия (предпросмотр → скрыть → согласие). Скрытый результат не участвует ни в F, ни в K, ни в выдаче работодателю: иначе получилась бы скрытая накрутка
Отзыв согласия в журнале	Данные реестра удаляются (в том числе сохранённый ответ), привязка остаётся
Отвязка	`DELETE /api/fsp/me`: удаляются привязка, `fsp_id` и все данные ФСП, согласие отзывается (запись в журнале)
Что сохраняется при согласии	Только используемые поля ответа реестра (`app/services/fsp_link/minimize.py`, `config/privacy.yaml → fsp_registry`): в `fsp_links.athlete` — `sub` и год рождения (политика по несовершеннолетним), в `fsp_links.achievements` — результаты, соревнования и разряды без номера приказа и без `sub` других участников команд (остаётся размер состава и роли), плюс нормализованные записи `fsp_achievements`. ФИО, дата рождения целиком, регион и организация не сохраняются; старые полные строки дочищает ночная задача. Работодателю ФИО из реестра не передаётся никогда
Что видит работодатель до раскрытия	Обобщённо: дисциплина, уровень, год, полоса места, роль, бейджи; без ЕКП, команды, протокола, ФИО
После взаимного «да» и согласия	Выписка из протокола (моноширинный текст): номер ЕКП, место из N, команда, ссылка на протокол, «источник: Реестр ФСП, на 05.10.2026». Разрешённые кандидатом поля профиля
Защита от «предзахвата» аккаунта	Аккаунт, созданный входом через ФСП ID, ждёт подтверждения email; если владелец email зарегистрируется по паролю сам, чужая привязка ФСП ID удаляется при первом обращении. Согласие, отмеченное при таком входе, применяется только после подтверждения email
Демо-вход	Открывает только аккаунты, созданные входом через ФСП ID (аккаунт с паролем, к которому привязали демо-спортсмена, входит по паролю). Выключается `VZ_FSP_DEMO_LOGIN_ENABLED=false`

8. Несовершеннолетние

В реестре есть несовершеннолетние спортсмены (в демо-реестре 7 из 60). Политика платформы (`config/contact.yaml → minors`):

- **моложе 15 лет** — регистрация отклоняется, в выдачу работодателям не попадают;
- **15–17 лет** — профиль виден работодателям только после согласия законного представителя, приглашать можно только на стажировку или лёгкий труд (флаг «стажировка»);
- возраст считается по году рождения из реестра, **реестр важнее заявленного кандидатом** (иначе несовершеннолетний обошёл бы защиту, указав год рождения до привязки); без года рождения кандидат с флагом `is_minor` скрывается осторожно в пользу защиты;
- возраст и пол работодателю не показываются и в Score не участвуют никогда.

Согласие законного представителя кандидатов 15–17 лет запрашивается платформой сама (письмо представителю с одноразовой ссылкой, `POST /api/candidates/me/guardian`, SECURITY.md п. 1.4). Подтверждение родства документом остаётся организационным вопросом и **требует решения ФСП** (раздел 10).

9. Пошаговый путь к реальной интеграции

Код платформы при переходе не меняется: меняются переменные окружения и (на стороне ФСП) публикуется HTTP-контракт реестра.

#	Шаг	Что сделать	Проверка
1	Зарегистрировать client в realm `fsp` ФСП	Конфиденциальный, standard flow + PKCE S256, redirect URI `https://<домен>/candidate/fsp/callback` (страница фронтенда), scopes `openid profile email`; claims `sub`, `email`, имя уже есть в кабинете ФСП	`GET <issuer>/.well-known/openid-configuration` отвечает; в JWKS есть ключи RS/PS/ES
2	Сменить issuer и учётные данные	`VZ_FSP_OIDC_ISSUER=https://sso.fsp-russia.ru/realms/fsp`; `VZ_FSP_OIDC_CLIENT_ID`, `VZ_FSP_OIDC_CLIENT_SECRET` (только из секретов окружения), `VZ_FSP_OIDC_REDIRECT_URI`; **убрать** `VZ_FSP_OIDC_INTERNAL_URL` (публичный и внутренний адрес совпадают); `VZ_FSP_KEYCLOAK_PUBLIC_URL` и профиль compose `fsp` больше не нужны	`GET /api/fsp/availability` → `fsp_id_available: true`, `registry_label: "Реестр ФСП"`
3	Получить доступ к реестру	Сервисный аккаунт того же client (client credentials) с правом чтения реестра: `VZ_FSP_TOKEN_URL=https://sso.fsp-russia.ru/realms/fsp/protocol/openid-connect/token`, при необходимости `VZ_FSP_REGISTRY_SCOPE`; `VZ_FSP_REGISTRY_TOKEN` не задавать	Токен выдаётся, `GET /v1/athletes/{sub}` отвечает 200 или 404, не 401
4	Опубликовать API реестра по контракту	Реализовать эндпоинты раздела 4.1 (минимум — `athletes/{sub}` и `achievements`), использовать мок как эталон; прогнать контрактные тесты на ответах реального реестра (`Achievements.model_validate`)	Тест соответствия моделям; поля `ekp` — 16 цифр, `discipline_code` — коды ВРВС
5	Переключить `FSPProvider`	`VZ_FSP_REGISTRY_URL=<адрес API реестра>`, `VZ_FSP_PROVIDER=http` (в режиме `auto` сбой реестра для `sub`, которого нет в демо-снимке, всё равно даёт «недоступно», но на проде резервный демо-реестр не нужен)	`GET /api/fsp/me/achievements` у тестового спортсмена возвращает его результаты; остановка реестра даёт `sync_status = error`, а не «результатов нет»
6	Отключить демо	`VZ_FSP_DEMO_LOGIN_ENABLED=false`; остановить `fsp-mock` и Keycloak стенда	Кнопки «демо-вход ФСП» нет; `/api/fsp/demo-login` → 403 `fsp_demo_disabled`
7	Учесть задержку протоколов	Результаты до сдачи протокола приходят как `awaiting_protocol` / `protocol_status = pending` и в оценку не идут	В интерфейсе «Ожидает протокол»; после публикации учитывается при следующей синхронизации

#	Шаг	Что сделать	Проверка
8	Привязки	`sub` у настоящих спортсменов другие: демо-пользователей realm в проде нет, привязка `fsp_id` создаётся заново при первом входе. Сопоставление исторических данных по ФИО и дате рождения — только с согласия человека, **вне MVP**	—
9	Согласовать открытые вопросы	Раздел 10: состав данных, веса, несовершеннолетние, внешние соревнования	Протокол согласования; правка `config/fsp.yaml`

Дальнейшее развитие (вне MVP): вебхук реестра `achievement.created`, брокеринг ФСП ID через собственный Keycloak платформы (если платформа переедет на него целиком), загрузка диплома как источника с весом `diploma` 0,5.

Достижения из реестра можно выдавать кандидату как подписанные переносимые учётные данные (W3C VC / Open Badges 3.0): демонстрационный вариант и роль ФСП как настоящего издателя описаны в CREDENTIALS.md, раздел 6.

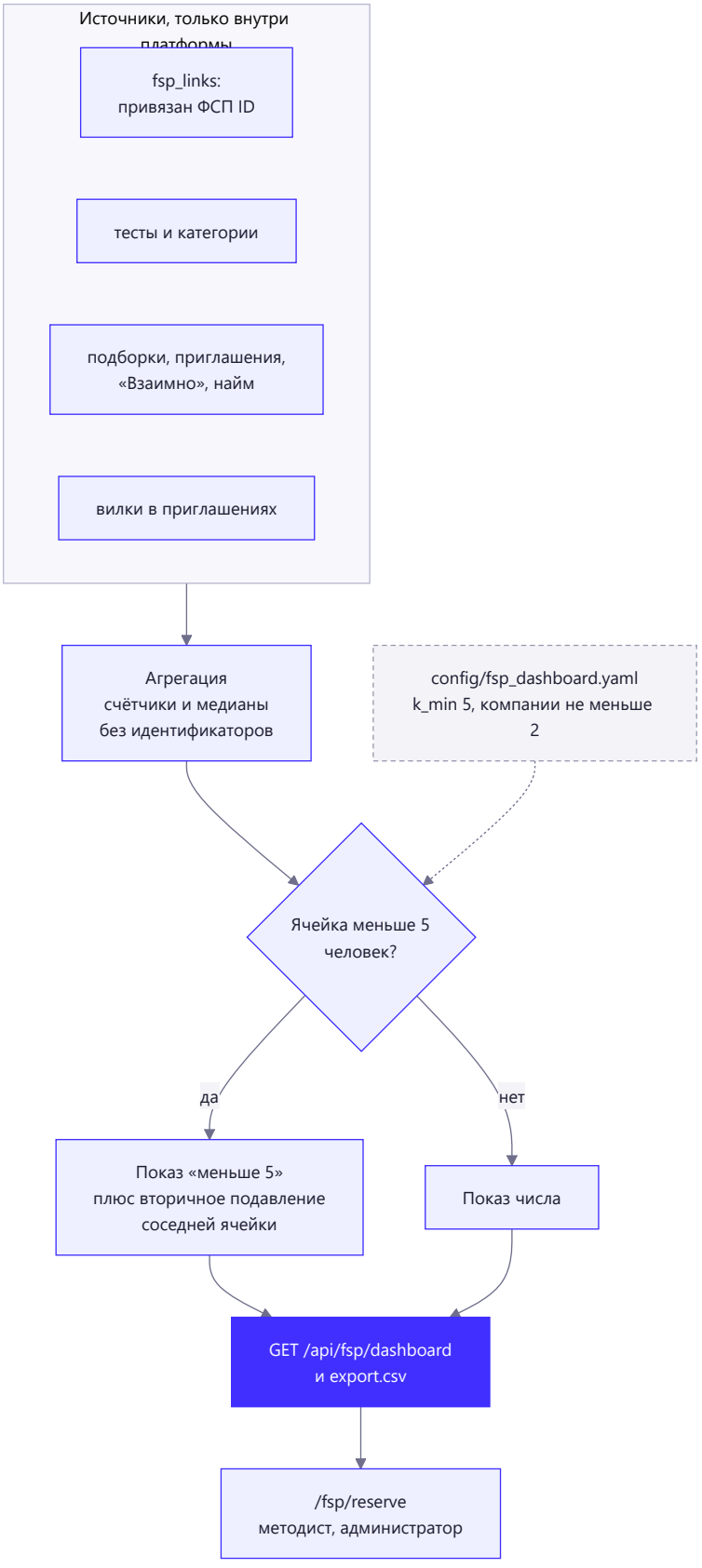
10. Что нужно согласовать с ФСП

1. **Состав данных реестра.** Будут ли в реальной интеграции место из N, сырой балл, роль и состав команды, разряд со сроком действия, номер ЕКП? Это определяет схему и формулу VSS (Q, Ro, Rk).
2. **Модель доступа.** Сервисный аккаунт (client credentials, как в предложении) или токен пользователя после входа через ФСП ID? Стабилен ли `sub`; есть ли тестовый client в realm `fsp`?
3. **Публичность и скрытие.** Можно ли спортсмену скрывать отдельные результаты; публичны ли достижения по умолчанию? Предложение: можно, скрытые не участвуют в оценке.
4. **Несовершеннолетние.** Показывать ли их работодателям; допустимы ли только стажировки с 16 лет; нужно ли согласие законного представителя?
5. **Вес ФСП в ранжировании.** Устраивает ли «не более 15 баллов из 100, не перевешивает большой разрыв в тесте, отсутствие ФСП не штраф»? В симуляции при равном тесте обладатели ФСП несколько выше в топ-10 (VALIDATION.md §4.5): возможно, вес стоит пересмотреть.
6. **Матрица «дисциплина → специализация»** и учёт роли внутри команды продуктового программирования: сейчас роль есть только в заявке участника, поэтому платформа показывает её как заявленную и в оценке не использует. Если ФСП начнёт фиксировать функцию в команде в протоколе, её можно будет учитывать.
7. **Внешние соревнования** (Кубок Федерации CTF, Codeforces): учитывать в F или только официальные ЕКП?
8. **Анонимность до согласия.** Платформа скрывает не только контакты, но и имя: протоколы публичны, и пара «ЕКП + место» однозначно выдаёт человека. Не противоречит ли это видению ФСП?

11. Панель ФСП: кадровый резерв

Что получает ФСП. Кабинет `/fsp/reserve` (API `GET /api/fsp/dashboard`, выгрузка `/api/fsp/dashboard/export.csv`): воронка «участник соревнований (привязан ФСП ID) → прошёл тест → подтверждённая категория → попал в подборки → получил приглашение → «Взаимно» → нанят» рядом с остальными кандидатами; разрезы по пяти дисциплинам, федеральным округам, регионам и «специализация × грейд»; динамика по неделям; число компаний, приглашавших и нанимавших спортсменов; медианы предложенной зарплаты «ФСП / остальные» по грейдам (на руки и до НДФЛ, ставка в конфиге); «где не хватает кадров» — спрос компаний по категориям против спортсменов ФСП с подтверждённой категорией. Это инструмент, которого у федерации сейчас нет: основа программы трудоустройства.

Поток данных панели:



Приватность.

- В ответе только счётчики и медианы: ни идентификаторов, ни имён, контактов, ФСП ID, ни названий компаний.
- k-анонимность: ячейка, где меньше 5 человек, отдаётся как «<5» (ноль не скрывается). Для взаимоисключающих строк (округа, регионы) действует вторичное подавление: если скрыта одна ячейка,

скрывается и наименьшая соседняя, а мелкие регионы сливаются в «прочие», иначе скрытое число вычиталось бы из итога. Спрос работодателей считается в компаниях и скрыт, пока их меньше 2. Доля от предыдущей стадии не показывается, если скрыта сама ячейка или знаменатель.

- Доступ: роли «методист» и «администратор» (в демо — кнопка «Панель ФСП» на странице жюри). Для реальной ФСП предлагается отдельная роль аналитика и договор о том, какие агрегаты ей положены.
- Все пороги и справочники — `config/fsp_dashboard.yaml`. Отсутствие истории ФСП — не штраф: сравнение описательное и в Score не участвует.

Честность демо. Данные синтетические, реестр ФСП — мок; на странице об этом сказано. Чтобы k-анонимность не превратила панель в сплошные «<5», `scripts/seed_demo.py` и `POST /api/demo/reset` добавляют воспроизводимую «демо-историю подбора» (`app/services/demo/fsp_reserve.py`): 8 вымышленных компаний-архивов и ~160 приглашений за 12 недель. Эффекта «ФСП сильнее» в историю не заложено — исходы и вилки для обеих групп одинаковы.

API «Взаимно»: обзор

REST/JSON-API бэкенда. Исчерпывающее описание (поля, схемы, примеры ошибок) — в интерактивной документации и в `openapi.json`; этот документ объясняет, как с API работать: аутентификация, формат ошибок, разделы и сквозные соглашения. Названия эндпоинтов и полей сверены с OpenAPI, сформированным из кода (`app.main:app`, версия API `0.1.0`).

Содержание: 1. Документация и схема · 2. Аутентификация · 3. Формат ошибок · 4. Ограничение частоты · 5. Разделы · 6. Соглашения · 7. Примеры · 8. Статус

1. Документация и схема

Что	Адрес
Swagger UI (интерактивно, «Authorize» принимает access-токен)	`/api/docs`
ReDoc	`/api/redoc`
Схема OpenAPI 3.1 (JSON)	`/api/openapi.json`
Готовность и жизнь процесса	`/api/health`, `/api/health/live`

Все пути — под префиксом `/api`. Локально бэкенд слушает `http://localhost:8000` (`docker compose up -d`), фронтенд (Vite dev-сервер, порт 5173; в compose — контейнер `web` с nginx на порту 80) проксирует `/api` на него.

Как получить `openapi.json`:

```
# 1. С запущенного сервера
curl -s http://localhost:8000/api/openapi.json -o openapi.json

# 2. Без сервера и базы данных (схема строится из кода)
cd backend
uv run python -c "import json; from app.main import app; print(json.dumps(app.openapi(),
ensure_ascii=False, indent=1))" > openapi.json

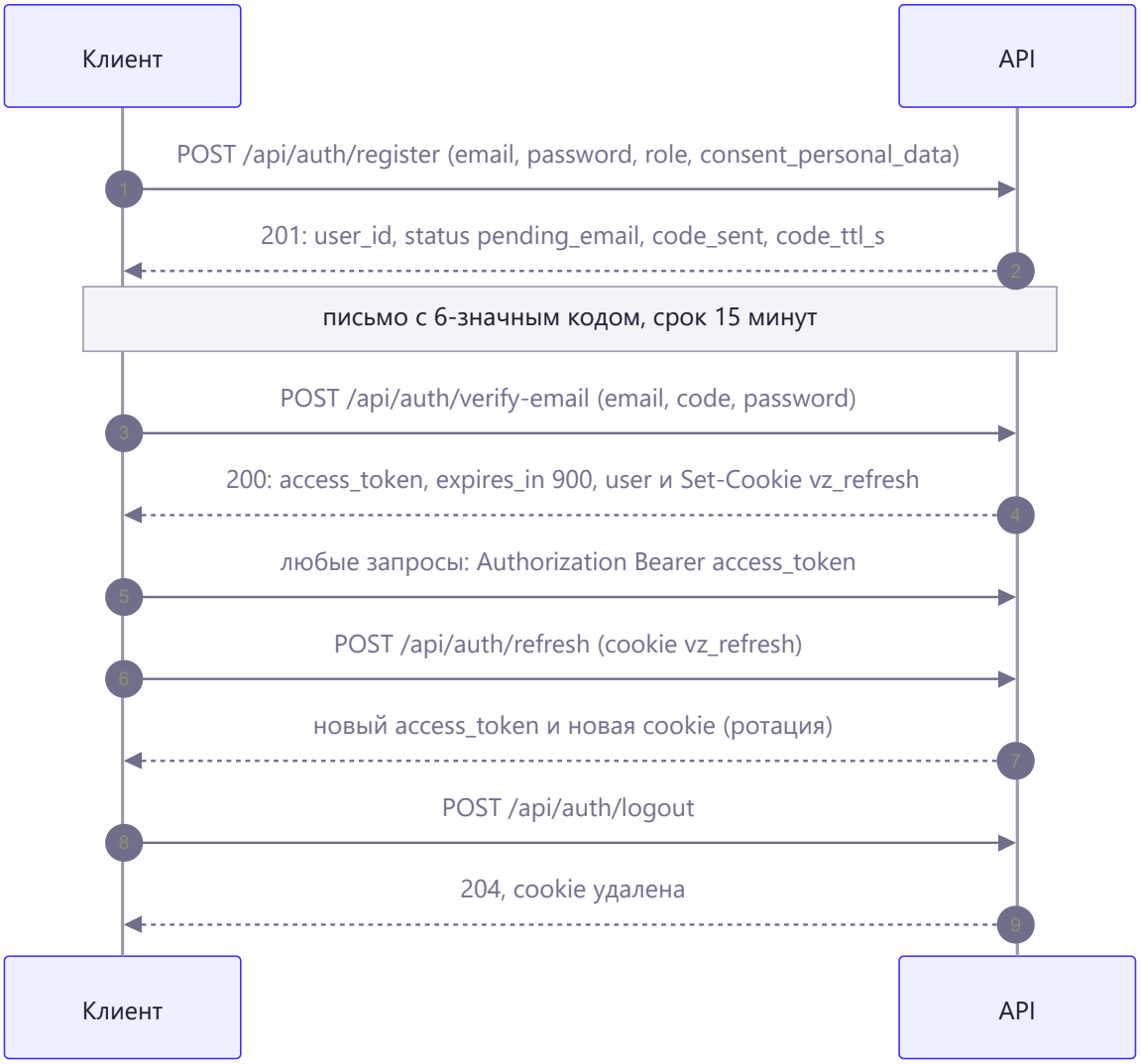
# 3. Типы TypeScript для фронтенда (из запущенного бэкенда или из файла)
cd frontend
npm run gen:api
OPENAPI_FILE=../backend/openapi.json npm run gen:api
```

Примечание: страницы `/api/docs` и `/api/redoc` работают без интернета: скрипты и стили Swagger UI (Apache-2.0) и ReDoc (MIT) лежат в `backend/app/static/docs` и отдаются самим бэкендом по `/api/docs-assets/*`; внешних CDN и шрифтов Google нет.

Документация реестра ФСП (предлагаемый контракт) — отдельный OpenAPI сервиса `fsp-mock` (`http://localhost:8001/docs`), см. FSP_INTEGRATION.md.

2. Аутентификация

Своя аутентификация в FastAPI: email и пароль с подтверждением кодом из письма, JWT-доступ и refresh-токен в httpOnly-cookie. Роли: `candidate` («Ищу работу»), `employer` («Нанимаю»), `methodist`, `admin`. Самостоятельно регистрируются только первые две; методист и администратор создаются из БД или сидом.



Элемент	Правила
Регистрация	`POST /api/auth/register`: `email`, `password`, `role` (`candidate` или `employer`), `consent_personal_data` (обязательно `true`), необязательно `consent_text_version`. Аккаунт создаётся в статусе `pending_email`; согласие на обработку ПДн пишется в журнал отдельной записью с версией текста. Повтор для неподтверждённого email обновляет пароль и роль и отправляет новый код
Требования к паролю	8–128 символов, хотя бы одна буква и одна цифра. Хранится как argon2id
Код из письма	6 цифр, действует 15 минут; повторная отправка не раньше чем через 60 с и не больше 5 писем в час; на один код — 5 неверных попыток
Подтверждение	`POST /api/auth/verify-email` принимает код **и пароль** из регистрации (код доказывает владение ящиком, пароль — что активирует тот, кто задал аккаунт). После успеха создаются профиль кандидата или компания работодателя (по домену почты; личные домены — своя компания с пометкой «личный email»), выдаются токены
Access-токен	JWT HS256, срок 15 минут (`expires_in: 900`); claims `sub` (id пользователя), `role`, `type: access`, `iss: vzaimno`, `iat`, `exp`, `jti`. Передаётся `Authorization: Bearer <токен>`; клиент держит его в памяти. Роль и статус аккаунта сервер проверяет **по БД**, а не по токену: блокировка и смена роли действуют сразу
Refresh-токен	Случайные 256 бит в cookie `vz_refresh` (httpOnly, `Path=/api/auth`, `SameSite=Lax`, `Secure` в проде), срок 30 дней. В БД хранится только sha256. Каждый `POST /api/auth/refresh` выдаёт новый токен той же цепочки, старый помечается использованным; повторное предъявление использованного токена = подозрение на кражу: цепочка завершается (`401 refresh_reused`). Повтор в пределах 10 секунд — гонка вкладок (`409 refresh_superseded`, цепочка жива)

Элемент	Правила
Вход	`POST /api/auth/login`; ответ одинаков для неизвестного email и неверного пароля (`401 invalid_credentials`); `403 email_not_verified` — нужно ввести код; `403 account_blocked`
Восстановление пароля	`POST /api/auth/password-reset/request` (ответ не выдаёт, есть ли аккаунт) и `…/confirm` (код + новый пароль; все прежние сессии завершаются)
Вход через ФСП ID	`GET /api/fsp/login`, `GET /api/fsp/callback` (адрес возврата из ФСП ID — страница фронтенда `/candidate/fsp/callback`; переход браузера на `GET /api/fsp/callback` отвечает 303 на неё), демо-вход — раздел «ФСП»
Ответы `/api/auth/*`	Заголовок `Cache-Control: no-store`

3. Формат ошибок и коды

Любая ошибка возвращается в одном формате с русским текстом для пользователя:

```
{
  "code": "invalid_credentials",
  "message": "Неверный email или пароль",
  "details": null
}
```

`code` — стабильный машинный код (`snake_case`), по нему клиент выбирает экран; `message` — готовая фраза; `details` — подробности (объект или список), например:

- `422 validation_error`: `details` — список `{"field": "password", "message": "Минимум 8 символов", "type": "string_too_short"}`;
- `429`: `details.retry_after_s`, заголовок `Retry-After`;
- `409` от машины состояний контакта: `contact_rejected`, `details.reason` и списки `errors`, `warnings`, `notes` с кодами правил;
- ошибки правил теста: `details.available_at` — когда действие станет доступно.

Необработанное исключение — `500 internal_error` в том же формате (тело читается браузером благодаря CORS-заголовкам).

HTTP	Смысл	Типичные коды
400	Некорректный запрос, нарушено правило формы	`code_invalid`, `code_expired`, `code_attempts_exceeded`, `consent_required`, `salary_invalid`, `salary_required`, `grade_required`, `unknown_specialization`, `derived_specialization`, `contact_invalid`, `fsp_state_invalid`, `fsp_state_mismatch`, `fsp_email_missing`, `role_not_allowed`
401	Нет входа или токен недействителен	`not_authenticated`, `token_invalid`, `token_expired`, `invalid_credentials`, `refresh_missing`, `refresh_invalid`, `refresh_expired`, `refresh_reused`. Заголовок `WWW-Authenticate: Bearer`
403	Недостаточно прав	`role_forbidden` (в `details` — требуемые роли), `account_blocked`, `email_not_verified`, `fsp_demo_disabled`, `fsp_state_foreign`
404	Не найдено **или чужое** (запись другого пользователя неотличима от несуществующей, чтобы нельзя было перебирать идентификаторы)	`not_found`, `need_not_found`, `invitation_not_found`, `session_not_found`, `candidate_not_found`, `profile_not_disclosed`

HTTP	Смысл	Типичные коды
409	Конфликт состояния и бизнес-правил	`email_taken`, `already_verified`, `consent_outdated`, `active_session_exists`, `survey_required`, `specialization_not_testable`, `extension_pending`, `contact_rejected`, `company_profile_incomplete`, `pair_active`, `shortlist_full`, `rollback_too_old`, `vacancy_exists`, `application_exists`, `refresh_superseded`, `fsp_id_taken`, `fsp_already_linked`
413	Слишком большой файл	`resume_too_large` (PDF до 5 МБ)
422	Ошибка валидации полей	`validation_error`, `brief_invalid`, `category_invalid`, `resume_empty`
429	Слишком часто	`rate_limited`, `resend_too_soon`, `resend_hourly_limit`; `Retry-After` в секундах
501	Раздел в разработке	`not_implemented`
503	Внешняя зависимость недоступна	`mail_unavailable`, `fsp_registry_unavailable`, `fsp_id_unavailable`, `task_unavailable`

Полный список кодов раздела указан в описании соответствующего эндпоинта в Swagger UI.

4. Ограничение частоты запросов

Токен-корзина в памяти процесса (`N/секунд`): корзина вмещает `N` токенов и равномерно пополняется за период. Превышение — `429` с `Retry-After`. Лимиты безопасности настраиваются переменными `VZ_RL_*` (`.env.example`), лимиты разделов — в коде и конфигах.

Область	Лимит	Ключ
Вход	5 за 300 с по email; 30 за 300 с по IP	email, IP
Регистрация	10 в час	IP
Ввод кода (подтверждение email, новый пароль)	10 за 900 с	email
Повторная отправка кода, запрос восстановления	20 в час	IP
Обновление сессии	60 в минуту	IP
Начало и завершение входа через ФСП ID, демо-вход	30/300 с, 30/300 с, 20/300 с	IP
«Обновить достижения ФСП»	6 за 600 с	аккаунт
Шаги опроса	60 в минуту	аккаунт
Создание сессии теста	10 в час	аккаунт
`next`, ответы, события теста	120, 60, 120 в минуту	аккаунт
Оспорить флаг	10 в час	аккаунт
Загрузка резюме	10 в час	аккаунт
PDF-профиль, превью «так вас видят», поиск компаний для блок-листа	30 за 600 с, 120 и 60 в минуту	аккаунт
Страница сверки PDF `/api/v/{code}`	120 в минуту	IP
Ответы кандидата и действия работодателя с приглашениями и откликами; отклики; ответы на задачи; тест вебхука	30, 10, 20 и 5 в минуту (`config/contact.yaml → http_limits`)	аккаунт или компания
Подбор (бриф, версии, банк) и отправка приглашений	30 в минуту (`config/contact.yaml → quotas.rate_limit_per_minute`)	аккаунт
Калибровка (лаборатория)	5 за 300 с	аккаунт

Корзины живут в памяти процесса: при нескольких воркерах лимит действует на воркер (для продакшена их выносят в Redis или PostgreSQL).

5. Разделы и эндпоинты

Всего 242 эндпоинта в 25 разделах (список построен автоматически из OpenAPI приложения). Колонка «Кто» — требуемая роль (`require_roles`); «любой вошедший» — любой авторизованный пользователь, «без входа» — публичный маршрут (`refresh` и `logout` опираются на cookie).

Служебное

Метод	Путь	Кто	Назначение
`GET`	`/api/health`	без входа	Готовность сервиса
`GET`	`/api/health/pool`	без входа	Готовность пула заданий
`GET`	`/api/health/live`	без входа	Процесс жив

Аутентификация

Метод	Путь	Кто	Назначение
`POST`	`/api/auth/register`	без входа	Регистрация по email
`POST`	`/api/auth/verify-email`	без входа	Подтверждение email кодом
`POST`	`/api/auth/resend-code`	без входа	Отправить код ещё раз
`POST`	`/api/auth/login`	без входа	Вход по email и паролю
`POST`	`/api/auth/password-reset/request`	без входа	Восстановление пароля: отправить код
`POST`	`/api/auth/password-reset/confirm`	без входа	Восстановление пароля: новый пароль по коду
`POST`	`/api/auth/refresh`	без входа	Обновить сессию
`POST`	`/api/auth/logout`	без входа	Выйти
`GET`	`/api/auth/me`	любой вошедший	Текущий пользователь

Согласия

Метод	Путь	Кто	Назначение
`GET`	`/api/consents/documents`	без входа	Тексты согласий
`GET`	`/api/consents`	любой вошедший	Мой журнал согласий
`POST`	`/api/consents`	любой вошедший	Дать согласие
`POST`	`/api/consents/{consent_id}/revoke`	любой вошедший	Отозвать согласие

Опрос

Метод	Путь	Кто	Назначение
`GET`	`/api/survey/form`	кандидат	Анкета опроса
`GET`	`/api/survey/current`	кандидат	Текущая анкета
`PUT`	`/api/survey/steps/industries`	кандидат	Шаг 1: сфера бизнеса
`PUT`	`/api/survey/steps/actions`	кандидат	Шаг 2: что делали руками
`PUT`	`/api/survey/steps/stack`	кандидат	Шаг 3: технологии и стаж
`POST`	`/api/survey/hint`	кандидат	Подсказка по специализации
`PUT`	`/api/survey/steps/specialization`	кандидат	Шаг 4: специализация
`GET`	`/api/survey/history`	кандидат	История опросов

Тестирование

Метод	Путь	Кто	Назначение
`GET`	`/api/tests/briefing`	кандидат	Выбор грейда и брифинг
`POST`	`/api/tests/sessions`	кандидат	Начать тест
`GET`	`/api/tests/sessions/{session_id}`	кандидат	Состояние сессии
`GET`	`/api/tests/sessions/{session_id}/next`	кандидат	Текущее задание
`POST`	`/api/tests/sessions/{session_id}/answer`	кандидат	Ответить на задание
`POST`	`/api/tests/sessions/{session_id}/events`	кандидат	События страницы теста
`POST`	`/api/tests/sessions/{session_id}/extension`	кандидат	Добор вверх или вниз
`GET`	`/api/tests/sessions/{session_id}/result`	кандидат	Результат теста
`GET`	`/api/tests/sessions/{session_id}/review`	кандидат	Разбор ошибок
`GET`	`/api/tests/qualification`	кандидат	Мой грейд
`GET`	`/api/tests/qualification/{specialization}`	кандидат	Мой грейд по специализации
`POST`	`/api/tests/qualification/{specialization}/downgrade`	кандидат	Перейти на грейд ниже
`POST`	`/api/tests/flags/{flag_id}/dispute`	кандидат	Оспорить флаг

Кандидат

Метод	Путь	Кто	Назначение
`GET`	`/api/candidates/me`	кандидат	Мой профиль
`PUT`	`/api/candidates/me`	кандидат	Сохранить профиль
`DELETE`	`/api/candidates/me`	кандидат	Удалить аккаунт (право на удаление, 152-ФЗ): тело `{"password"}` или `{"confirm": "УДАЛИТЬ"}` для аккаунта без пароля; необратимо, все сессии завершаются; `403 password_invalid`, `429` — 5 попыток в час
`GET`	`/api/candidates/me/guardian`	кандидат	Согласие законного представителя
`POST`	`/api/candidates/me/guardian`	кандидат	Запросить согласие законного представителя
`GET`	`/api/candidates/me/resume`	кандидат	Черновик распознанного резюме
`POST`	`/api/candidates/me/resume`	кандидат	Загрузить резюме (PDF)
`DELETE`	`/api/candidates/me/resume`	кандидат	Удалить резюме
`POST`	`/api/candidates/me/resume/confirm`	кандидат	Подтвердить поля из резюме
`GET`	`/api/candidates/me/profile.pdf`	кандидат	Скачать PDF-профиль
`GET`	`/api/candidates/me/profile-pdf`	кандидат	Ссылка сверки PDF-профиля
`GET`	`/api/candidates/disclosed/{ref}/profile.pdf`	работодатель	Полный PDF-профиль кандидата для работодателя
`GET`	`/api/candidates/me/privacy`	кандидат	Приватность и режим поиска

Метод	Путь	Кто	Назначение
`PUT`	`/api/candidates/me/privacy`	кандидат	Сохранить настройки приватности и поиска
`GET`	`/api/candidates/me/preview`	кандидат	Так вас видят
`POST`	`/api/candidates/me/preview`	кандидат	Так вас видят — с черновиком настроек
`GET`	`/api/candidates/me/blocklist/search`	кандидат	Найти компанию для блок-листа
`GET`	`/api/candidates/me/consents`	кандидат	Журнал согласий
`GET`	`/api/candidates/me/visibility-stats`	кандидат	Как меня видят

Компании

Метод	Путь	Кто	Назначение
`GET`	`/api/companies/me`	работодатель	Профиль моей компании
`PUT`	`/api/companies/me`	работодатель	Изменить профиль компании
`DELETE`	`/api/companies/me`	работодатель	Удалить аккаунт работодателя (тело как у кандидата); если он последний в компании — ждущие приглашения отзываются, брифы и вакансии закрываются, контакты связи и вебхук ATS стираются
`GET`	`/api/companies/me/verification`	работодатель	Метка «Проверенная компания»
`POST`	`/api/companies/me/verification`	работодатель	Подать реквизиты на проверку
`DELETE`	`/api/companies/me/verification`	работодатель	Убрать метку
`GET`	`/api/companies/me/preview`	работодатель	Так видит кандидат
`GET`	`/api/companies/me/webhooks`	работодатель	Журнал доставок вебхуков
`POST`	`/api/companies/me/webhooks/test`	работодатель	Проверить вебхук
`POST`	`/api/companies/me/webhooks/rotate-secret`	работодатель	Новый секрет подписи
`GET`	`/api/companies/{company_id}/reliability`	любой вошедший	Надёжность компании
`GET`	`/api/companies/me/ats`	работодатель	Настройки ATS компании
`PUT`	`/api/companies/me/ats`	работодатель	Сохранить настройки ATS
`POST`	`/api/companies/me/ats/check`	работодатель	Проверить подключение (эмулятор записывает запрос в журнал)
`GET`	`/api/companies/me/ats/log`	работодатель	«Журнал ATS»

Метод	Путь	Кто	Назначение
`GET`	`/api/companies/me/ats/keys`	работодатель	API-ключи компании
`POST`	`/api/companies/me/ats/keys`	работодатель	Создать API-ключ
`POST`	`/api/companies/me/ats/keys/{key_pk}/rotate`	работодатель	Перевыпустить ключ
`DELETE`	`/api/companies/me/ats/keys/{key_pk}`	работодатель	Отозвать ключ

Потребности и подборка

Метод	Путь	Кто	Назначение
`POST`	`/api/needs/parse`	работодатель	Разобрать бриф (живая карточка)
`GET`	`/api/needs/market-salary`	работодатель	Рыночная вилка по открытым данным (подсказка, gross/net)
`POST`	`/api/needs`	работодатель	Создать бриф
`GET`	`/api/needs`	работодатель	Мои брифы
`GET`	`/api/needs/{need_id}`	работодатель	Бриф
`PATCH`	`/api/needs/{need_id}`	работодатель	Изменить бриф
`GET`	`/api/needs/{need_id}/categories`	работодатель	Категории и охват вилки
`GET`	`/api/needs/{need_id}/check`	работодатель	Как мы проверим
`POST`	`/api/needs/{need_id}/versions`	работодатель	Собрать новую версию подборки
`GET`	`/api/needs/{need_id}/versions`	работодатель	Версии подборки
`GET`	`/api/needs/{need_id}/selection`	работодатель	Текущая подборка
`GET`	`/api/needs/{need_id}/versions/{n}`	работодатель	Версия подборки
`POST`	`/api/needs/{need_id}/rollback`	работодатель	Откатиться к версии
`GET`	`/api/needs/{need_id}/versions/{n}/neighbors/{rank}`	работодатель	Почему выше соседа
`GET`	`/api/needs/{need_id}/shortlist`	работодатель	Шортлист брифа
`PUT`	`/api/needs/{need_id}/shortlist/{ref}`	работодатель	Закрепить кандидата

Метод	Путь	Кто	Назначение
`DELETE`	`/api/needs/{need_id}/shortlist/{ref}`	работодатель	Открепить кандидата
`GET`	`/api/needs/{need_id}/candidates/{ref}`	работодатель	Карточка кандидата (Р6)
`GET`	`/api/bank`	работодатель	Банк кандидатов
`GET`	`/api/bank/candidates/{ref}`	работодатель	Карточка кандидата из банка
`GET`	`/api/bank/summary`	работодатель	Сводка «специализация × грейд» одним запросом для карты талантов: число подтверждённых кандидатов, доля с ФСП, ожидания P25/P50/P75 «на руки» (появляются при N ≥ `privacy_min_n` = 10, округлены до ступени), «ждут теста» по направлениям; свой лимит запросов
`GET`	`/api/skills`	любой вошедший	Справочник навыков для фильтров (`testable` , модули теста, специализации); `?testable=true` , `?spec=`

Приглашения

Метод	Путь	Кто	Назначение
`POST`	`/api/invitations`	работодатель	Пригласить кандидата
`GET`	`/api/invitations`	работодатель	Воронка приглашений
`POST`	`/api/invitations/batch`	работодатель	Пригласить пачку из шортлиста
`POST`	`/api/invitations/preview`	работодатель	Превью «так увидит кандидат»
`GET`	`/api/invitations/inbox`	кандидат	Мои приглашения
`GET`	`/api/invitations/inbox/{invitation_id}`	кандидат	Приглашение (кандидат)
`GET`	`/api/invitations/{invitation_id}`	работодатель	Приглашение (работодатель)
`POST`	`/api/invitations/{invitation_id}/view`	кандидат	Отметить просмотренным
`POST`	`/api/invitations/{invitation_id}/accept`	кандидат	Ответить «Да»
`POST`	`/api/invitations/{invitation_id}/decline`	кандидат	Ответить «Нет»
`POST`	`/api/invitations/{invitation_id}/counter`	кандидат	Ответить «Да, если…»
`POST`	`/api/invitations/{invitation_id}/dispute`	кандидат	«Со мной не связались»
`POST`	`/api/invitations/{invitation_id}/check`	кандидат	«Условия совпали?»
`POST`	`/api/invitations/{invitation_id}/revoke-consent`	кандидат	Отозвать согласие на передачу контактов
`POST`	`/api/invitations/{invitation_id}/grant-consent`	кандидат	Подтвердить согласие после встречного
`POST`	`/api/invitations/{invitation_id}/report`	кандидат	Пожаловаться на приглашение
`POST`	`/api/invitations/{invitation_id}/withdraw`	работодатель	Отозвать приглашение
`POST`	`/api/invitations/{invitation_id}/counter/accept`	работодатель	Согласиться на встречное условие
`POST`	`/api/invitations/{invitation_id}/counter/decline`	работодатель	Отказать во встречном условии
`POST`	`/api/invitations/{invitation_id}/contacted`	работодатель	«Связались»
`POST`	`/api/invitations/{invitation_id}/interview`	работодатель	«Собеседование»
`POST`	`/api/invitations/{invitation_id}/outcome`	работодатель	Итог: найм или «не сложилось»

Отклики

Метод	Путь	Кто	Назначение
`POST`	`/api/applications`	кандидат	Откликнуться на вакансию
`GET`	`/api/applications`	работодатель	Отклики на мои вакансии
`GET`	`/api/applications/mine`	кандидат	Мои отклики
`GET`	`/api/applications/mine/{application_id}`	кандидат	Мой отклик
`POST`	`/api/applications/{application_id}/withdraw`	кандидат	Отозвать отклик
`POST`	`/api/applications/{application_id}/revoke-consent`	кандидат	Отозвать согласие после отказа
`GET`	`/api/applications/{application_id}`	работодатель	Отклик (работодатель)
`POST`	`/api/applications/{application_id}/interest`	работодатель	«Есть интерес»
`POST`	`/api/applications/{application_id}/reject`	работодатель	Отказать с причиной

Вакансии

Метод	Путь	Кто	Назначение
`POST`	`/api/vacancies`	работодатель	Опубликовать бриф как вакансию
`GET`	`/api/vacancies`	кандидат	Вакансии для меня
`GET`	`/api/vacancies/mine`	работодатель	Мои вакансии
`PATCH`	`/api/vacancies/{vacancy_id}`	работодатель	Изменить вакансию
`GET`	`/api/vacancies/{vacancy_id}`	кандидат	Вакансия (кандидат)
`POST`	`/api/vacancies/{vacancy_id}/close`	работодатель	Снять вакансию с публикации
`POST`	`/api/vacancies/{vacancy_id}/report`	кандидат	Пожаловаться на вакансию

Регулярные задачи

Метод	Путь	Кто	Назначение
`GET`	`/api/tasks/catalog`	работодатель	Каталог семейств (тип A)
`GET`	`/api/tasks`	работодатель	Мои задачи
`POST`	`/api/tasks`	работодатель	Создать задачу
`POST`	`/api/tasks/{task_id}/pause`	работодатель	Поставить задачу на паузу
`POST`	`/api/tasks/{task_id}/activate`	работодатель	Возобновить задачу
`POST`	`/api/tasks/{task_id}/close`	работодатель	Закрыть задачу
`GET`	`/api/tasks/{task_id}/submissions`	работодатель	Решения по задаче
`POST`	`/api/tasks/assignments/{assignment_id}/review`	работодатель	Оценить решение (тип B)
`POST`	`/api/tasks/check-sets`	работодатель	Проверочный набор по вакансии
`GET`	`/api/tasks/check-sets/{need_id}`	работодатель	Результаты проверочного набора
`GET`	`/api/tasks/me`	кандидат	Мои задачи (кандидат)
`POST`	`/api/tasks/assignments/{assignment_id}/start`	кандидат	Открыть задачу
`POST`	`/api/tasks/assignments/{assignment_id}/submit`	кандидат	Отправить решение
`POST`	`/api/tasks/assignments/{assignment_id}/skip`	кандидат	Пропустить задачу

ФСП

Метод	Путь	Кто	Назначение
GET	/api/fsp/availability	без входа	Доступен ли ФСП ID
GET	/api/fsp/login	без входа или кандидат	Начать вход через ФСП ID
GET	/api/fsp/callback	без входа или кандидат	Завершить вход через ФСП ID
GET	/api/fsp/demo/athletes	без входа	Демо-спортсмены для «демо-входа ФСП»
POST	/api/fsp/demo-login	без входа или кандидат	Демо-вход ФСП
GET	/api/fsp/me	кандидат	Мои данные ФСП
DELETE	/api/fsp/me	кандидат	Отвязать ФСП ID
GET	/api/fsp/me/achievements	кандидат	Предпросмотр достижений ФСП
PATCH	/api/fsp/me/results/{result_id}	кандидат	Показывать или скрыть результат
POST	/api/fsp/me/enrichment	кандидат	Согласие на обогащение профиля данными ФСП
POST	/api/fsp/me/refresh	кандидат	Обновить данные из реестра ФСП
GET	/api/fsp/dashboard	администратор, методист	Кадровый резерв ФСП: агрегаты
GET	/api/fsp/dashboard/export.csv	администратор, методист	Кадровый резерв ФСП: выгрузка CSV

Проверка профиля

Метод	Путь	Кто	Назначение
GET	/api/v/{code}	без входа	Сверить PDF-профиль

Уведомления

Метод	Путь	Кто	Назначение
GET	/api/notifications	любой вошедший	Мои уведомления
GET	/api/notifications/unread	любой вошедший	Счётчики непрочитанных
POST	/api/notifications/read-all	любой вошедший	Отметить все прочитанными
POST	/api/notifications/{notification_id}/read	любой вошедший	Отметить прочитанным

Лаборатория

Метод	Путь	Кто	Назначение
GET	/api/lab/sessions	методист, админ	Сессии теста
GET	/api/lab/sessions/{session_id}/trace	методист, админ	Трасса сессии
GET	/api/lab/bank	методист, админ	Банк заданий
GET	/api/lab/bank/{family_id}/clones	методист, админ	Клоны рядом
GET	/api/lab/flags	методист, админ	Очередь флагов
POST	/api/lab/flags/{flag_id}/resolve	методист, админ	Решение по флагу
GET	/api/lab/validation	методист, админ	Валидация
POST	/api/lab/calibrate	методист, админ	Калибровка заданий
GET	/api/lab/reconciliations	администратор, методист	Лента сверок «двойника в буфере»

Метод	Путь	Кто	Назначение
`GET`	`/api/lab/reconciliations/{reconciliation_id}`	администратор, методист	Акт сверки
`POST`	`/api/lab/reconciliations/{reconciliation_id}/review`	администратор, методист	Решение по сверке
`GET`	`/api/lab/sessions/{session_id}/verification-kit`	администратор, методист	«Подтверждение за 5 минут» по сессии
`GET`	`/api/lab/twins/coverage`	администратор, методист	Покрытие банка двойниками
`POST`	`/api/lab/twins/precompute`	администратор, методист	Посчитать двойники пула
`GET`	`/api/lab/quality`	администратор, методист	Качество заданий: DIF, дрейф, экспозиция
`GET`	`/api/lab/validation/report`	администратор, методист	Отчёт последней валидации

Администрирование

Маршруты `/api/admin/*` — роль «администратор» или «методист» (вход жюри «Лаборатория»; методист видит только свои загрузки); маршруты `/api/demo/*` существуют только при `VZ_DEMO_MODE=true` (иначе `404`) и на боевом стенде должны быть выключены. Если стенд всё же в `VZ_ENV=prod` с демо (стенд жюри), общие кнопки (часы, сброс, автоответчик, режим жюри) доступны только роли `admin` или заголовку `X-Demo-Key` (`VZ_DEMO_CONTROL_KEY`): `403 demo_controls_restricted` остальным; вне prod — любой вошедший.

Метод	Путь	Кто	Назначение
`GET`	`/api/demo/state`	без входа	Состояние демо-режима
`GET`	`/api/demo/accounts`	без входа	Демо-аккаунты
`POST`	`/api/demo/clock`	любой вошедший; в prod — админ или `X-Demo-Key`	Сдвинуть часы демо
`POST`	`/api/demo/reset`	то же	Сбросить демо-мир (режим жюри сохраняет значение)
`POST`	`/api/demo/autoresponder`	то же	Автоответчик синтетических кандидатов
`GET`	`/api/demo/synthetic`	работодатель	Какие кандидаты синтетические
`GET`	`/api/demo/webhook-sink`	без входа	Последние события вебхуков ATS
`POST`	`/api/demo/webhook-sink`	без входа	Принять вебхук ATS
`GET`	`/api/admin/import`	администратор, методист	Список импортов жюри
`POST`	`/api/admin/import`	администратор, методист	Импорт кандидатов и брифов жюри
`POST`	`/api/admin/import/simple`	администратор, методист	Импорт жюри в простом формате (вакансии, кандидаты, разметка — CSV или JSON)
`DELETE`	`/api/admin/import/{batch_id}`	администратор, методист	Удалить импорт жюри
`POST`	`/api/admin/eval/rank`	администратор, методист	Ранжирование по брифу (топ-N с компонентами Score)
`POST`	`/api/admin/eval/pairs`	администратор, методист	Метрики по размеченным парам (P@5, P@10, nDCG@10)
`GET`	`/api/admin/validation`	администратор, методист	Последний прогон валидации
`GET`	`/api/admin/check/repo`	администратор, методист	Наш набор пар: метрики и список брифов

Метод	Путь	Кто	Назначение
`GET`	`/api/admin/check/repo/briefs/{brief_id}`	администратор, методист	Наш набор пар: бриф и топ для ручной проверки
`GET`	`/api/admin/check/repo/tops.csv`	администратор, методист	Наш набор пар: топы по всем брифам в CSV
`GET`	`/api/admin/check/batches/{batch_id}`	администратор, методист	Своя загрузка: метрики и список брифов
`GET`	`/api/admin/check/batches/{batch_id}/briefs/{brief_id}`	администратор, методист	Своя загрузка: бриф и топ
`GET`	`/api/admin/check/batches/{batch_id}/tops.csv`	администратор, методист	Своя загрузка: топы по всем брифам в CSV
`GET`	`/api/admin/check/templates/{name}`	администратор, методист	Шаблон простого формата для жюри

Администрирование и демо-режим: подробнее

- **`/api/admin/*`** (роли `admin` и `methodist`): страница `/lab/check` построена на `/api/admin/check/*` и `POST /api/admin/import/simple` (три таблицы: вакансии, кандидаты, разметка `rel` 0–3, CSV или JSON; шаблоны — `GET /api/admin/check/templates/{name}`; ошибка в таблице — `400` с номером строки и полем). `POST /api/admin/import` принимает набор размеченных пар «бриф × кандидат» (формат `backend/eval/pairs_test.jsonl`, схема `backend/eval/pairs_test.schema.json`; отдельно можно передать `briefs` и `candidates`); кандидат с `features: null` — без теста: он попадает в группу «категория по опросу, не подтверждена» и ранжируется ниже подтверждённых. Импортированные данные **не попадают в банк платформы**. `POST /api/admin/eval/rank` строит топ-N по брифу (`brief_id` из импорта или `text` с вилкой) тем же движком, что подборка работодателя, с компонентами Score, вкладами и «за»/«против»; `?format=csv` отдаёт тот же топ в CSV. `POST /api/admin/eval/pairs` считает P@5, P@10, nDCG@10 со средним по брифам и 95 % бутстреп-доверительным интервалом (тот же код, что `make validate`); `GET /api/admin/validation` отдаёт последний прогон валидации.
- **`/api/demo/*`** существует только при `VZ_DEMO_MODE=true`, иначе `404`: `POST /api/demo/clock` (`{"preset": "plus_1d"}`, «+1 день», «+3 дня», сброс; сдвигает сроки приглашений и откликов и свежесть тестов), `POST /api/demo/reset` (быстрый сброс к состоянию seed), `POST /api/demo/autoresponder` (автоответчик синтетических кандидатов отвечает на приглашения за 30–90 с), `POST /api/demo/jury-mode` и `GET /api/demo/jury/hint` («режим жюри»: подсказка к текущему заданию, сессия помечается `demo_assisted` и исключается из метрик и калибровки), `GET /api/demo/webhook-sink` и `POST /api/demo/webhook-sink` (встроенный приёмник вебхуков ATS, последние 20 событий), `GET /api/demo/state`, `GET /api/demo/accounts` (логины без паролей, открытые маршруты). На боевом стенде демо-режим выключают.

Чат

Переписка кандидата и работодателя открывается только после взаимного «да» и раскрытия контактов; `kind` — `invitation` или `application`, `ref_id` — идентификатор приглашения или отклика. Только текст, лимиты длины и частоты — в `config/`; чужие переписки не отдаются.

Метод	Путь	Кто	Назначение
`GET`	`/api/chat/threads`	любой вошедший	Мои переписки
`GET`	`/api/chat/unread`	любой вошедший	Непрочитанные сообщения (счётчик)
`GET`	`/api/chat/{kind}/{ref_id}`	любой вошедший	Переписка
`POST`	`/api/chat/{kind}/{ref_id}/messages`	любой вошедший	Написать сообщение

Интеграция ATS

Pull-API для системы найма работодателя: заголовок `X-API-Key` (ключ создаётся через `POST /api/companies/me/ats/keys`, хранится только хэш). Схема, вебхуки и соответствие статусов — в ATS_INTEGRATION.md. Настройки и журнал ATS компании — в разделе «Компании».

Метод	Путь	Кто	Назначение
`GET`	`/api/ats/v1/me`	ключ компании (X-API-Key)	Проверка ключа
`GET`	`/api/ats/v1/vacancies`	ключ компании (X-API-Key)	Вакансии
`GET`	`/api/ats/v1/invitations`	ключ компании (X-API-Key)	Приглашения со статусами
`GET`	`/api/ats/v1/invitations/{invitation_id}`	ключ компании (X-API-Key)	Приглашение
`GET`	`/api/ats/v1/applications`	ключ компании (X-API-Key)	Отклики со статусами
`GET`	`/api/ats/v1/applications/{application_id}`	ключ компании (X-API-Key)	Отклик
`POST`	`/api/ats/v1/invitations/{invitation_id}/status`	ключ компании (X-API-Key)	Статус из ATS: приглашение
`POST`	`/api/ats/v1/applications/{application_id}/status`	ключ компании (X-API-Key)	Статус из ATS: отклик

Законный представитель

Запрос согласия для кандидатов 15–17 лет — `POST /api/candidates/me/guardian` (раздел «Кандидат»); подтверждение по одноразовой ссылке из письма. См. SECURITY.md.

Метод	Путь	Кто	Назначение
`POST`	`/api/guardian/confirm`	без входа	Подтвердить согласие законного представителя

Антиподсказчик

«Подтверждение за 5 минут» для работодателя после раскрытия контакта; сверки двойника и акты — в разделе «Лаборатория». См. TESTING_MECHANISM.md.

Метод	Путь	Кто	Назначение
`GET`	`/api/invitations/{invitation_id}/verification-kit`	работодатель	«Подтверждение за 5 минут» для работодателя

Режим жюри и песочница «Попробуй списать»

Работает только при `VZ_DEMO_MODE=true`, иначе `404`. Позволяет жюри пройти тест самому одноразовым кандидатом, войти демо-работодателем и проверить «Антиподсказчика» на мини-тесте.

Метод	Путь	Кто	Назначение
`POST`	`/api/jury/cheat-lab`	без входа (только DEMO_MODE)	Песочница жюри: начать мини-тест
`POST`	`/api/jury/cheat-lab/{test_id}/answers`	без входа (только DEMO_MODE)	Песочница жюри: отправить ответы
`GET`	`/api/jury/cheat-lab/{test_id}/result`	без входа (только DEMO_MODE)	Песочница жюри: результат
`POST`	`/api/demo/jury-mode`	то же	Режим жюри
`GET`	`/api/demo/jury/hint`	кандидат	Подсказка жюри к заданию
`GET`	`/api/demo/jury/paths`	без входа	Что доступно на странице входа жюри
`POST`	`/api/demo/jury/start`	без входа	Пройти тест самому: одноразовый кандидат

Метод	Путь	Кто	Назначение
`POST`	`/api/demo/jury/persona`	без входа	Войти как демо-работодатель или жюри лаборатории

Доверие и безопасность

Модерация: жалобы кандидатов, автосигналы по компаниям, отзыв и восстановление метки «Проверенная компания». Доступ — администратор или методист. См. TRUST_AND_SAFETY.md.

Метод	Путь	Кто	Назначение
`GET`	`/api/admin/trust/complaints`	администратор, методист	Очередь жалоб
`POST`	`/api/admin/trust/complaints/{complaint_id}/resolve`	администратор, методист	Разобрать жалобу
`GET`	`/api/admin/trust/flags`	администратор, методист	Автосигналы по компаниям
`POST`	`/api/admin/trust/companies/{company_id}/revoke`	администратор, методист	Отозвать метку «Проверенная компания»
`POST`	`/api/admin/trust/companies/{company_id}/restore`	администратор, методист	Снять отзыв метки

Сигнал интереса

Кандидат отмечает компанию, которая ему интересна: 2 сигнала за 30 дней, каждый живёт 60 дней, отзыв слот не возвращает; работодатель видит бейдж «Взаимный интерес» только у тех, чью карточку уже открыл или закрепил, в Score сигнал не входит. Нужно отдельное согласие `interest_signal`. Правила — MATCHING.md п. 9. Отметка работодателя «Есть интерес» по отклику (`POST /api/applications/{id}/interest`) — в разделе «Отклики».

Метод	Путь	Кто	Назначение
`GET`	`/api/interest/me`	кандидат	Мои сигналы интереса
`POST`	`/api/interest`	кандидат	Отправить сигнал интереса
`POST`	`/api/interest/{signal_id}/revoke`	кандидат	Отозвать сигнал

Паспорт навыков

Цифровой паспорт навыков: Verifiable Credential с подписью Ed25519, QR на `/v/{код}`, список отзыва Bitstring Status List. Выпуск и отзыв — кандидат; проверка публичная, с ограничением частоты. Подробно — CREDENTIALS.md.

Метод	Путь	Кто	Назначение
`GET`	`/api/credentials/me`	кандидат	Мои паспорта и что можно выпустить
`POST`	`/api/credentials/me`	кандидат	Выпустить паспорт
`GET`	`/api/credentials/me/{credential_id}/file`	кандидат	Скачать паспорт (JSON)
`POST`	`/api/credentials/me/{credential_id}/revoke`	кандидат	Отозвать паспорт
`GET`	`/api/credentials/status/{list_id}`	без входа	Список отзыва (Bitstring Status List)
`GET`	`/api/credentials/verify/{code}`	без входа	Проверить паспорт по коду из QR
`POST`	`/api/credentials/verify`	без входа	Проверить загруженный файл паспорта

6. Сквозные соглашения

Тема	Правило
Идентификаторы	UUID в строковом виде. **Кандидаты в API работодателя** — только непрозрачная ссылка `ref` (и псевдоним «К-4821»): внутренний id кандидата, id попыток теста, задач и записей ФСП наружу не уходят
Время	Даты в ответах — ISO 8601 в UTC; тексты для людей — относительные («через 3 дня»), считаются по общим часам платформы
Проверка владельца	Любая запись проверяется на принадлежность текущему пользователю или его компании; чужая даёт `404`
Согласия	Действия, требующие согласия (`test_logging` при старте теста, `contact_disclosure` при «Да» и отклике, `blind_card` , `fsp_enrichment`), принимают отдельное булево поле; устаревший текст согласия — `409 consent_outdated`
Пагинация и версии	Подборка и банк: `page` (по 10 строк, до 100) и `hidden_page` («Скрыто фильтрами»); вкладка `tab=overall` («Общий топ») или `tab=category` с `category=backend_python:middle` ; версии подборки — номер `n` ; сравнение версий — `compare_to` . Списки уведомлений и лаборатории — `limit`
Идемпотентность	`POST /api/tests/sessions/{id}/answer` : повтор того же `seq` ничего не меняет (`duplicate: true`). Получение текущего задания (`next`) возвращает уже выданный экземпляр
Приватность	В ответах работодателю до раскрытия контактов нет полей ПДн ни на какой глубине; ключи заданий и теги дистракторов в ответы теста не попадают
CORS	Разрешённые источники — `VZ_CORS_ORIGINS` ; `allow_credentials` включён (cookie); заголовок `Retry-After` доступен клиенту
Заголовки ответа	`X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , `Referrer-Policy: strict-origin-when-cross-origin`
Язык	Все сообщения для пользователя — русские; идентификаторы и коды — английские

7. Примеры (сценарии curl)

```
API=http://localhost:8000/api

# Регистрация кандидата и подтверждение (письмо с кодом — в Mailpit, http://localhost:8025)
curl -s -X POST $API/auth/register -H 'Content-Type: application/json' \
  -d '{"email":"anna@example.ru","password":"Secret123","role":"candidate","consent_personal_data":true}'
curl -s -c cookies.txt -X POST $API/auth/verify-email -H 'Content-Type: application/json' \
  -d '{"email":"anna@example.ru","code":"123456","password":"Secret123"}'
# → {"access_token":"…","token_type":"bearer","expires_in":900,"user":{…}}
TOKEN=...   # подставьте access_token

# Опрос и тест
curl -s $API/survey/form -H "Authorization: Bearer $TOKEN"
curl -s -X POST $API/tests/sessions -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json'
\
  -d '{"specialization":"backend_python","grade":"middle","consent_test_logging":true}'
curl -s $API/tests/sessions/$SESSION/next -H "Authorization: Bearer $TOKEN"
curl -s -X POST $API/tests/sessions/$SESSION/answer -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"seq":1,"answer":"42"}'
curl -s $API/tests/sessions/$SESSION/result -H "Authorization: Bearer $TOKEN"

# Обновление сессии (cookie vz_refresh)
curl -s -b cookies.txt -c cookies.txt -X POST $API/auth/refresh

# Работодатель: бриф и подборка (вилка обязательна)
curl -s -X POST $API/needs -H "Authorization: Bearer $EMP" -H 'Content-Type: application/json' \
  -d '{"text":"Ищем backend-разработчика на Python, Middle, FastAPI и PostgreSQL обязательны, Kafka будет
плюсом","salary_from":220000,"salary_to":300000,"salary_net":true}'
curl -s -X POST $API/needs/$NEED/versions -H "Authorization: Bearer $EMP" -H 'Content-Type:
```

```
application/json' -d '{}'
curl -s "$API/needs/$NEED/selection?tab=overall&page=1" -H "Authorization: Bearer $EMP"
```

```
# Жюри: свои данные и ранжирование (токен администратора; пароли демо-аккаунтов:
backend/var/demo_accounts.md)
PAIRS=$(python -c "import json; print(json.dumps([json.loads(l) for l in
open('backend/eval/pairs_test.jsonl', encoding='utf-8')][:300]))")
curl -s -X POST $API/admin/import -H "Authorization: Bearer $ADMIN" -H 'Content-Type: application/json' \
  -d "{\"name\": \"жюри\", \"pairs\": $PAIRS}"
curl -s -X POST "$API/admin/eval/rank?format=csv" -H "Authorization: Bearer $ADMIN" -H 'Content-Type:
application/json' \
  -d '{"brief_id": "b01", "top": 10}'
```

Полный сквозной сценарий (приглашение → «Да, если…» → контакты) — в диаграмме ARCHITECTURE.md п. 6.3.

8. Статус и то, чего ещё нет

Что	Состояние
ATS: забор изменений и обратная синхронизация этапов	Готово: `/api/ats/v1/*`, адаптер «Хантфлоу» (эмулятор; боевой режим не проверялся на живом сервисе) — ATS_INTEGRATION.md; исходящие вебхуки работают (`/api/companies/me/webhooks*`)
«Сигнал интереса»	Готово: раздел «Сигнал интереса» выше
События «Антиподсказчика»	Готово: `POST /api/tests/sessions/{id}/events` принимает `blur`, `focus`, `paste`, `copy`, `cut`, `dragstart`, `self_paste`, `copy_bypass`, `reconnect`; хуки `anticheat_hooks.py` подключены (подмена буфера выдаётся в `ItemOut.clip`, улики и акты сверки — в «Лаборатории»)
Направления QA и DevOps в тесте	Готово: обе специализации `active` в `config/taxonomy.yaml`, тест доступен
Удаление аккаунта	Готово: `DELETE /api/candidates/me`, `DELETE /api/companies/me`; отвязка ФСП ID — `DELETE /api/fsp/me`, резюме — `DELETE /api/candidates/me/resume`

Сторонние компоненты и лицензии

Список библиотек, контейнерных образов, шрифтов и ресурсов, из которых собрано «Взаимно», с версиями и лицензиями. Версии Python-пакетов извлечены скриптом из `backend/uv.lock` (зафиксированное разрешение зависимостей), версии JavaScript-пакетов — из `frontend/package-lock.json`, образы — из `docker-compose.yml` и `backend/Dockerfile`; лицензии Python-пакетов — из метаданных PyPI для зафиксированных версий (с проверкой по установленным пакетам). Состояние на 2026-10-09 (версии повторно сверены с `backend/uv.lock` и `frontend/package-lock.json`: расхождений нет). Все компоненты имеют открытые лицензии; собственный код проекта — «Взаимно» (хакатон ЛЦТ-2026, спецтрек ФСП).

Содержание: 1. Бэкенд (Python) · 2. Фронтенд (JavaScript) · 3. Образы и системные компоненты · 4. Шрифты · 5. Иллюстрации и данные · 6. Замечания по лицензиям · 7. Как обновить список

1. Бэкенд (Python)

Требование проекта: Python не ниже 3.11 (`requires-python`). В Docker-образе — Python 3.12 (`python:3.12-slim`), зависимости ставятся строго по `uv.lock` (`uv sync --frozen --no-dev`). Для `numpy` и `scipy` lock содержит две ветки разрешения (Python 3.11 и Python 3.12 и выше); в образе действует вторая.

1.1. Прямые зависимости (`backend/pyproject.toml`)

Пакет	Версия	Лицензия	Для чего
`alembic`	1.20.0	MIT	Миграции схемы БД
`argon2-cffi`	25.1.0	MIT	Хеширование паролей argon2id
`email-validator`	2.3.0	Unlicense	Проверка адресов email
`fastapi`	0.142.2	MIT	HTTP-фреймворк API
`httpx`	0.28.1	BSD-3-Clause	HTTP-клиент: реестр ФСП, OIDC, LLM, вебхуки ATS
`cryptography`	50.0.2	Apache-2.0 OR BSD-3-Clause	Fernet: шифрование ключей заданий (`app/services/testing/keystore.py`); объявлена напрямую
`numpy`	2.4.6 (Python 3.11)	BSD-3-Clause AND 0BSD AND MIT AND Zlib AND CC0-1.0	Расчёты IRT/CAT (сетка EAP, калибровка)
`numpy`	2.5.3 (Python ≥ 3.12, образ Docker)	BSD-3-Clause AND 0BSD AND MIT AND Zlib AND CC0-1.0	Расчёты IRT/CAT (сетка EAP, калибровка)
`pdfminer.six`	20260107	MIT	Исключения разбора PDF (`PDFSyntaxError`); объявлена напрямую
`pdfplumber`	0.11.10	MIT	Извлечение текста из PDF-резюме
`pgvector`	0.5.0	MIT	Клиент pgvector (расширение в БД включено; в коде пока не импортируется)
`psycopg`	3.3.6	LGPL-3.0-only	Драйвер PostgreSQL (psycopg 3, extra `binary`)
`pyahocorasick`	2.3.1	BSD-3-Clause (и Public Domain)	Поиск словарных фраз (навыки, роли) в тексте брифа и резюме
`pydantic`	2.13.5	MIT	Модели и валидация DTO
`pydantic-settings`	2.15.0	MIT	Настройки `VZ_*` из окружения

Пакет	Версия	Лицензия	Для чего
`pyjwt`	2.15.1	MIT	Access-токены JWT (extra `crypto` : cryptography)
`python-multipart`	0.0.32	Apache-2.0	Приём загрузки файла (резюме)
`pyyaml`	6.0.3	MIT	Чтение `config/*.yaml`
`qrcode`	8.2	BSD	QR-код сверки в PDF-профиле (extra `pil` : Pillow)
`rapidfuzz`	3.14.6	MIT	Нечёткое сравнение: навыки с опечатками, проверка цитат LLM
`reportlab`	5.0.1	BSD (лицензия ReportLab Inc.)	Генерация PDF-профиля
`sqlalchemy`	2.1.3	MIT	ORM и доступ к PostgreSQL
`starlette`	1.7.0	BSD-3-Clause	ASGI-основа FastAPI: статика документации, ответы, конкурентность (`app/main.py` , `app/api/demo.py`); объявлена напрямую
`uvicorn`	0.54.0	BSD-3-Clause	ASGI-сервер (extra `standard` : httptools, uvloop, watchfiles, websockets, python-dotenv)

1.2. Транзитивные зависимости времени выполнения

Все пакеты, которые попадают в образ, кроме прямых (включая extras: `uvicorn[standard]` , `psycopg[binary]` , `pyjwt[crypto]` , `qrcode[pil]`). Колонка «Нужен для» — прямые зависимости, которые их тянут.

Пакет	Версия	Лицензия	Нужен для
`annotated-doc`	0.0.5	MIT	`fastapi`
`annotated-types`	0.8.0	MIT	`fastapi` , `pydantic` , `pydantic-settings`
`anyio`	4.15.1	MIT	`fastapi` , `httpx` , `uvicorn`
`argon2-cffi-bindings`	26.1.0	MIT	`argon2-cffi`
`certifi`	2026.7.22	MPL-2.0	`httpx`
`cffi`	2.1.1	MIT-0	`argon2-cffi` , `pdfplumber` , `pyjwt`
`charset-normalizer`	3.5.2	MIT	`pdfplumber` , `reportlab`
`click`	8.5.0	BSD-3-Clause	`uvicorn`
`colorama`	0.4.6	BSD	`qrcode`
`cryptography`	50.0.2	Apache-2.0 OR BSD-3-Clause	прямая; также `pdfplumber` , `pyjwt`
`dnspython`	2.8.0	ISC	`email-validator`
`h11`	0.16.0	MIT	`httpx` , `uvicorn`
`httpcore`	1.0.9	BSD-3-Clause	`httpx`
`httptools`	0.8.0	MIT	`uvicorn`
`idna`	3.20	BSD-3-Clause	`email-validator` , `fastapi` , `httpx` , `uvicorn`
`mako`	1.4.3	MIT	`alembic`
`markupsafe`	3.0.4	BSD-3-Clause	`alembic`
`opentelemetry-api`	1.45.0	Apache-2.0	`fastapi`

Пакет	Версия	Лицензия	Нужен для
`pdfminer-six`	20260107	MIT	прямая (`pdfminer.six`); также `pdfplumber`
`pillow`	12.3.0	MIT-CMU	`pdfplumber` , `qrcode` , `reportlab`
`psycopg-binary`	3.3.6	LGPL-3.0-only	`psycopg`
`pycparser`	3.0	BSD-3-Clause	`argon2-cffi` , `pdfplumber` , `pyjwt`
`pydantic-core`	2.46.5	MIT	`fastapi` , `pydantic` , `pydantic-settings`
`pypdfium2`	5.14.0	BSD-3-Clause или Apache-2.0 (плюс лицензии вложенных библиотек PDFium)	`pdfplumber`
`python-dotenv`	1.2.4	BSD-3-Clause	`pydantic-settings` , `uvicorn`
`starlette`	1.7.0	BSD-3-Clause	прямая; также `fastapi`
`typing-extensions`	4.16.0	PSF-2.0	`alembic` , `fastapi` , `httpx` , `psycopg` , `pydantic` , `pydantic-settings` , `sqlalchemy` , `uvicorn`
`typing-inspection`	0.4.4	MIT	`fastapi` , `pydantic` , `pydantic-settings`
`tzdata`	2026.5	Apache-2.0	`psycopg`
`uvloop`	0.23.0	Apache-2.0 или MIT	`uvicorn`
`watchfiles`	1.3.0	MIT	`uvicorn`
`websockets`	17.2	BSD-3-Clause	`uvicorn`

`colorama` ставится только на Windows (зависимость `qrcode`); `uvloop` , `httptools` — на Linux и macOS.

1.3. Инструменты разработки (в образ не входят)

Пакет	Версия	Лицензия	Тип
`ast-serialize`	0.12.1	MIT	транзитивная (mypy)
`iniconfig`	2.3.0	MIT	транзитивная
`librt`	0.16.0	MIT	транзитивная (mypy)
`mypy`	2.4.0	MIT	прямая (группа `dev`): `mypy --strict` на ядрах, `make typecheck`
`mypy-extensions`	1.1.0	MIT	транзитивная (mypy)
`packaging`	26.3	Apache-2.0 OR BSD-2-Clause	транзитивная
`pathspec`	1.1.1	MPL-2.0	транзитивная (mypy); слабый копилефт, только dev, не изменялась
`pluggy`	1.6.0	MIT	транзитивная
`pygments`	2.21.0	BSD-2-Clause	транзитивная
`pytest`	9.1.1	MIT	прямая
`pytest-asyncio`	1.4.0	Apache-2.0	прямая
`scipy`	1.17.1 (Python 3.11), 1.18.1 (Python ≥ 3.12)	BSD	прямая (группа `dev`): только `tests/test_matching_scoring.py` , в образ не входит
`ruff`	0.16.10	MIT	прямая

2. Фронтенд (JavaScript)

`frontend/package.json` (версия приложения 0.1.0), версии — из `package-lock.json` (lockfileVersion 3); сборка `npm run build` (TypeScript и Vite). Окружение сборки: Node.js 22 LTS (проверено на 22.20.0) и npm 11.

2.1. Зависимости, попадающие в сборку (`dependencies`)

Пакет	Версия	Лицензия	Диапазон в `package.json`
`@tanstack/react-query`	5.104.1	MIT	`^5.62.0`
`clsx`	2.1.1	MIT	`^2.1.1`
`openapi-fetch`	0.17.0	MIT	`^0.17.0`
`react`	18.3.1	MIT	`^18.3.1`
`react-dom`	18.3.1	MIT	`^18.3.1`
`react-router-dom`	7.18.4	MIT	`^7.1.0`
`tailwind-merge`	3.7.0	MIT	`^3.0.0`

Всего пакетов, попадающих в сборку, с транзитивными: 15. Лицензии: MIT — 15.

2.2. Инструменты сборки и контроля качества (`devDependencies`)

Пакет	Версия	Лицензия	Диапазон в `package.json`
`@eslint/js`	9.39.5	MIT	`^9.17.0`
`@types/node`	22.20.5	MIT	`^22.10.0`
`@types/react`	18.3.31	MIT	`^18.3.12`
`@types/react-dom`	18.3.7	MIT	`^18.3.1`
`@vitejs/plugin-react`	5.2.0	MIT	`^5.0.0`
`autoprefixer`	10.6.1	MIT	`^10.4.20`
`eslint`	9.39.5	MIT	`^9.17.0`
`eslint-plugin-react-hooks`	5.2.0	MIT	`^5.1.0`
`eslint-plugin-react-refresh`	0.4.26	MIT	`^0.4.16`
`globals`	15.15.0	MIT	`^15.14.0`
`openapi-typescript`	7.13.0	MIT	`^7.13.0`
`postcss`	8.5.29	MIT	`^8.4.49`
`sharp`	0.34.5	Apache-2.0	`^0.34.0`
`tailwindcss`	3.4.19	MIT	`^3.4.17`
`typescript`	5.9.3	Apache-2.0	`~5.9.0`
`typescript-eslint`	8.71.0	MIT	`^8.18.0`
`vite`	7.3.6	MIT	`^7.0.0`

Всего dev-пакетов с транзитивными: 336. Сводка лицензий dev-пакетов:

Лицензия	Пакетов
MIT	261
Apache-2.0	30
ISC	18

Лицензия	Пакетов
LGPL-3.0-or-later	10
BSD-2-Clause	6
Apache-2.0 AND LGPL-3.0-or-later	3
BSD-3-Clause	2
Apache-2.0 AND LGPL-3.0-or-later AND MIT	1
BlueOak-1.0.0	1
Python-2.0	1
CC-BY-4.0	1
0BSD	1
(MIT OR CC0-1.0)	1

2.3. Отдельные пакеты разработки (в образ и сборку не входят)

Каждый лежит в своём каталоге со своим `package-lock.json`, чтобы не добавлять зависимости в основной `frontend/package.json`.

Пакет	Версия	Лицензия	Где и зачем
`@playwright/test`	1.57.0	Apache-2.0	`frontend/e2e`: сквозные тесты (ENGINEERING.md)
`playwright`, `playwright-core`	1.57.0	Apache-2.0	то же (транзитивные)
`@axe-core/playwright`	4.11.0	MPL-2.0	`frontend/e2e`: проверка доступности (ACCESSIBILITY.md); только `devDependency`, в сборку не попадает
`axe-core`	4.11.4	MPL-2.0	то же (транзитивная); слабый копилефт на уровне файлов, не изменялся
`typescript`	5.9.3	Apache-2.0	`frontend/e2e`
`mermaid`	11.12.0	MIT	`docs/pdf-tools`: отрисовка диаграмм для PDF документации
`playwright-core`	1.56.1	Apache-2.0	`docs/pdf-tools`: печать HTML в PDF через Chrome или Edge

Для сборки PDF (`docs/build_pdf.py`) используются ещё `markdown-it-py` (MIT) и `pypdf` (BSD-3-Clause) — ставятся вручную, в `pyproject.toml` не входят.

3. Образы и системные компоненты

Компонент	Версия (источник)	Лицензия	Назначение
PostgreSQL 16 с расширением pgvector	`pgvector/pgvector:0.8.7-pg16` (`docker-compose.yml`, `docker-compose.prod.yml`)	PostgreSQL License (PostgreSQL и pgvector)	База данных
PostgreSQL 16 (Keycloak)	`postgres:16.15-alpine` (`docker-compose.prod.yml`, профиль `fsp`)	PostgreSQL License	БД Keycloak на боевой схеме
Keycloak	`quay.io/keycloak/keycloak:26.0.8` (профиль `fsp`; локально `start-dev`, в prod `start`)	Apache-2.0	ФСП ID на демо-стенде
Mailpit	`axllent/mailpit:v1.31.4` (`docker-compose.yml`)	MIT	Перехват писем в разработке

Компонент	Версия (источник)	Лицензия	Назначение
Python	`python:3.12-slim` (`backend/Dockerfile` , `services/fsp-mock/Dockerfile`)	PSF-2.0 (CPython); базовый образ Debian содержит пакеты под своими лицензиями	Среда выполнения бэкенда и `fsp-mock`
uv	`ghcr.io/astral-sh/uv:0.11.12` (`backend/Dockerfile`)	MIT или Apache-2.0	Установка зависимостей по `uv.lock`
nginx	`nginx:1.27.5-alpine` (`frontend/Dockerfile` , образ `web`)	BSD-2-Clause	Статика фронтенда, SPA-fallback, прокси `/api`
Node.js (образы)	`node:22.12-alpine` (сборка фронтенда, `frontend/Dockerfile`), `node:22.12-slim` (исполнение JS-семейств при генерации пула, `backend/Dockerfile`)	MIT	Сборка Vite; ключи JavaScript-заданий
Caddy 2	`caddy:2.10.2-alpine` (`docker-compose.prod.yml` , DEPLOY.md)	Apache-2.0	Обратный прокси, TLS
Swagger UI (swagger-ui-dist)	5.33.1, файлы в `backend/app/static/docs`	Apache-2.0 (лицензия и NOTICE рядом)	Интерактивная документация `/api/docs` , без внешних CDN
ReDoc	2.5.4 (`redoc.standalone.js`), файлы в `backend/app/static/docs`	MIT (лицензия рядом)	Документация `/api/redoc` , без внешних CDN и шрифтов
Node.js	22 LTS на машине сборки и при генерации пула клонов (семейства JavaScript исполняются `node -e`)	MIT-подобная (Node.js license)	Сборка фронтенда, ключи JS-заданий
SQLite	Встроен в Python (`sqlite3`)	Public Domain	Исполнение SQL-семейств при генерации ключей заданий

Образы compose закреплены по версиям (БД, Caddy, Keycloak, Mailpit); плавающим остаётся только `python:3.12-slim` в Dockerfile — для воспроизводимого релиза его нужно закрепить по digest.

4. Шрифты

Все шрифты распространяются по SIL Open Font License 1.1 (OFL) и размещены в репозитории вместе с текстом лицензии; внешних обращений к Google Fonts и CDN нет.

Шрифт	Версия	Где используется	Файлы
Inter	4.001 (Inter Project Authors)	PDF-профиль (бэкенд, reportlab)	`backend/app/assets/fonts/Inter-*.ttf` , `Inter-OFL.txt`
JetBrains Mono	2.304 (JetBrains Mono Project Authors)	PDF-профиль: числа, код сверки, выписка ФСП	`backend/app/assets/fonts/JetBrains Mono-*.ttf` , `JetBrainsMono-OFL.txt`
Onest	woff2 из Google Fonts (Onest Project Authors); версия файла в метаданных не указана	Интерфейс (основной)	`frontend/public/fonts/onest-*.woff2` , `OFL-Onest.txt`
JetBrains Mono	woff2 (JetBrains Mono Project Authors)	Интерфейс: числа, коды	`frontend/public/fonts/jetbrains-mono-*.woff2` , `OFL-JetBrainsMono.txt`

5. Иллюстрации и данные

Ресурс	Происхождение и права
3D-иллюстрации интерфейса (`frontend/public/3d/*.webp`)	Собственные: отрисованы локальным скриптом на `numpy` и `Pillow` (трассировка полей расстояний), сторонних моделей и сервисов нет
Иконка `favicon.svg`	Собственная
Данные ФСП в демо	Синтетические: вымышленные спортсмены, команды и номера ЕКП (после года стоит «99»), фамилии собраны из ИТ-терминов
Демо-компании и кандидаты	Вымышленные; домены в зоне `.example` ; реальных брендов и ФИО нет
Справочники (навыки, города, имена для разбора резюме, матрицы весов)	Собственные: `config/*.yaml` и `backend/app/services/resume/names_data.py`
Банк заданий	Собственные генераторы (`backend/app/testing/bank/families`); эталонный код и ключи порождаются исполнением
Агрегаты рынка (`data/market/` , `config/market.yaml`): квантили зарплат, частоты и соседи навыков, алиасы	Производные от открытых вакансий портала «Работа в России» (`opendata.trudvsem.ru` , открытые данные Роструда), выгрузка 2026-10-08. Хранятся только агрегаты; сырые тексты, контакты и данные компаний не сохраняются и не распространяются. Отдельного текста лицензии на странице API не найдено — условия нужно перепроверить перед использованием вне хакатона; подробности и ограничения — `data/market/README.md` . hh.ru не используется: анонимный доступ к `/vacancies` закрыт (403). ESCO не подключался
Эмбеддинги, предобученные модели, внешние датасеты	Не используются (кроме агрегатов рынка выше)

6. Замечания по лицензиям

- **LGPL-3.0:** `psycopg` **и** `psycopg-binary` (драйвер PostgreSQL; бинарная сборка включает libpq). Используются как неизменённые библиотеки, подключаемые динамически, что допускается лицензией. Лицензия отличается от MIT, BSD и Apache: при распространении образов нужно сохранить уведомления и обеспечить доступ к исходникам библиотеки (они публичны).
- **MPL-2.0:** `certifi` (набор корневых сертификатов) — слабый копилефт на уровне файлов; библиотека не изменялась.
- **MIT-CMU:** `pillow` , **PSF-2.0:** `typing-extensions` , **ISC:** `dnspython` , **Unlicense:** `email-validator` — разрешительные лицензии.
- `pypdfium2` включает бинарную сборку PDFium (BSD-3-Clause и Apache-2.0) и её собственные зависимости под своими лицензиями; `numpy` содержит части под 0BSD, MIT, Zlib и CC0-1.0.
- **Фронтенд, dev-зависимости:** пакеты `@img/sharp-libvips-*` под LGPL-3.0-or-later (бинарная libvips для `sharp`) нужны только скрипту оптимизации изображений `npm run optimize:3d` и в сборку (`dist/`) не попадают.
- **Прямые импорты и** `pyproject.toml` **:** `cryptography` , `pdfminer.six` и `starlette` объявлены явно (раньше приходили транзитивно); `scipy` перенесён в группу `dev` , потому что нужен только тестам. Пакет `pgvector` объявлен намеренно: расширение в БД уже включено, клиент понадобится для эмбеддингов (план NLP), в коде пока не импортируется.
- Образы `latest` и плавающие теги не воспроизводимы; закрепить перед релизом.
- Уязвимости: CI гоняет `pip-audit` по прод-зависимостям и `npm audit --omit=dev` (ENGINEERING.md); автоматической проверки лицензий нет — список выше собран вручную по метаданным.
- **MPL-2.0 в dev-инструментах:** `axe-core` , `@axe-core/playwright` и `pathspec` — слабый копилефт на уровне файлов. Они не входят ни в сборку, ни в образы и не изменялись; правило «MIT/BSD/Apache» для зависимостей, попадающих в продукт, соблюдается.
- Исходный код проекта распространяется по лицензии MIT (LICENSE).

7. Как обновить список

```
# Python: версии и зависимости из lock
cd backend
uv lock --check                    # lock актуален
uv tree --locked                   # дерево зависимостей
uv run python -c "import tomllib; d = tomllib.load(open('uv.lock','rb')); [print(p['name'], p['version'])
for p in d['package']]"

# JavaScript
cd ../frontend
npm ci
npm ls --all --omit=dev            # дерево зависимостей сборки
npm ls --all                       # вместе с dev
```

Лицензия конкретного Python-пакета: `uv run python -c "from importlib.metadata import metadata; m = metadata('fastapi'); print(m['License-Expression'] or m['License'])"`. Для JavaScript лицензия указана в поле `license` записи пакета в `package-lock.json`.

Сборка, развёртывание и запуск

Пошаговая инструкция: локальный запуск для разработки и проверки, сборка образов, развёртывание на VPS в РФ с HTTPS через Caddy, переменные окружения, требования к ресурсам, обслуживание. Команды сверены с `docker-compose.yml`, `backend/Dockerfile`, `Makefile`, `.env.example` и скриптами `backend/scripts`. Файлы раздела 5 (`docker-compose.prod.yml`, `deploy/Caddyfile`) лежат в репозитории: compose проверен `docker compose config`, Caddyfile — `caddy validate`; локальная схема (п. 2.3) собрана и проверена с нуля в отдельном проекте compose, замеры — в п. 2.5.

Содержание: 1. Требования · 2. Локальный запуск · 3. Переменные окружения · 4. Сборка образов · 5. VPS в РФ, HTTPS через Caddy · 6. Обслуживание · 7. Диагностика · 8. В разработке

1. Требования и ресурсы

Что	Требование
ПО на машине разработки	Docker Engine 24+ с Compose v2.17+ (в `docker-compose.yml` используется `additional_contexts`), uv 0.11+, Node.js 22 LTS и npm (фронтенд, а также ключи JavaScript-заданий при генерации пула), GNU make (на Windows — команды PowerShell из п. 2.2)
Python	3.11 и выше для локального запуска (`requires-python`); в образе — 3.12
Порты на хосте (локально)	80 (интерфейс, `VZ_WEB_PORT`), 5432 (PostgreSQL), 8000 (бэкенд), 8001 (`fsp-mock`), 8025 и 1025 (Mailpit), 8080 (Keycloak, профиль `fsp`), 5173 (Vite) — переопределяются `VZ_*_PORT`
Память под ограничения контейнеров	Локальный compose: `db` 1 ГБ + `backend` 2 ГБ + `pool` 1 ГБ (только на время генерации пула) + `web` 128 МБ + `fsp-mock` 256 МБ + `mailpit` 128 МБ; в покое около 0,5 ГБ, под нагрузкой 100 пользователей около 2 ГБ (п. 2.5). С Keycloak (+768 МБ). Прод-шаблон: `db` 1 ГБ, `backend` 2 ГБ, `web` 128 МБ, `caddy` 128 МБ
Рекомендуемый VPS	4 vCPU, 8 ГБ ОЗУ для 4 процессов бэкенда (стенд жюри ~8 CPU / 32–64 ГБ — с запасом; при 2 vCPU и 4 ГБ — `VZ_WORKERS=1` – `2`). Исходно: 2 vCPU, 4 ГБ ОЗУ (сборка образов и пик Argon2: каждое хеширование занимает 64 МБ, одновременно не больше 4), 20–40 ГБ SSD. С собственным Keycloak — 6–8 ГБ ОЗУ. Это оценка по лимитам контейнеров; нагрузочное измерение на VPS не проводилось
Диск	Образы порядка 1–2 ГБ (оценка); данные PostgreSQL: пул клонов (111 семейств × 200 = около 22 тысяч экземпляров) и демо-мир — десятки мегабайт (оценка)
Домен и сеть (прод)	Доменное имя с A-записью на VPS; открытые порты 80 и 443 (выпуск сертификата Caddy и HTTPS); исходящий доступ к SMTP-серверу
Размещение	VPS с центром обработки данных **в РФ** (требование локализации ПДн, см. SECURITY.md)

2. Локальный запуск

2.1. Разработка: бэкенд на хосте (с `make`)

```
git clone <репозиторий> vzaimno && cd vzaimno
cp .env.example .env            # необязательно: значения по умолчанию подходят для локальной работы

make up-infra                   # PostgreSQL, Mailpit, fsp-mock (docker compose up -d db mailpit fsp-
mock)
make migrate                    # alembic upgrade head
make gen-pool                   # пул одноразовых клонов заданий (идемпотентно; нужен node для JS-
семейств)
make seed                       # демо-мир: ~300 кандидатов с настоящими тестами, персоны, компании
(пароли в backend/var/demo_accounts.md)
make dev                        # бэкенд с автоперезагрузкой в DEMO_MODE: http://localhost:8000/api/docs
```

```
cd frontend && npm ci && npm run dev    # интерфейс: http://localhost:5173 (Vite проксирует /api на
:8000)
```

Адрес	Что
http://localhost:8000/api/docs, `/api/redoc`, `/api/openapi.json`	Документация API (API.md)
http://localhost:8000/api/health	Готовность (БД)
http://localhost:8025	Mailpit: письма с кодами подтверждения
http://localhost:8001/docs	Демо-реестр ФСП (`fsp-mock`)
http://localhost:5173	Frontend (Vite, разработка); в контейнерной схеме интерфейс — http://localhost

Минимальная проверка без `make seed`: зарегистрироваться (`POST /api/auth/register`), прочитать код из Mailpit, подтвердить email, пройти опрос и тест (нужен пул из `make gen-pool`).

Первый запуск `make gen-pool` и `make seed` занимает несколько минут (генерация 200 клонов на семейство; `seed` дополнительно проводит настоящие тестовые сессии синтетических кандидатов). Повторные запуски идемпотентны: доливают недостающее.

Вход через ФСП ID (Keycloak). По умолчанию демо обходится без него (встроенный снимок реестра и «демо-вход ФСП»). Чтобы поднять настоящий Keycloak с realm `fsp` и 30 демо-пользователями:

```
docker compose --profile fsp up -d keycloak        # http://localhost:8080 (admin / admin); пароль демо-
спортсменов: Demo-FSP-2026
```

Для бэкенда в контейнере переменные `VZ_FSP_OIDC_*` уже заданы в `docker-compose.yml` . Для локального `make dev` задайте `VZ_FSP_OIDC_CLIENT_SECRET=vzaimno-demo-secret` (в `backend/.env` или в окружении); если вы копировали `.env.example` в `backend/.env` , очистите `VZ_FSP_OIDC_INTERNAL_URL` : адрес `keycloak:8080` виден только внутри сети docker. Подробности — FSP_INTEGRATION.md и `deploy/keycloak/README.md` .

2.2. Без `make` (Windows, PowerShell)

```
docker compose up -d db mailpit fsp-mock                       # make up-infra
cd backend
uv run alembic upgrade head                                    # make migrate
uv run python scripts/generate_pool.py                         # make gen-pool
$env:PYTHONUTF8 = "1"; uv run python scripts/seed_demo.py      # make seed
$env:VZ_DEMO_MODE = "true"; uv run uvicorn app.main:app --reload --port 8000   # make dev
uv run pytest -q                                               # make test
uv run ruff check .                                            # make lint
uv run python -m eval.validate                                 # make validate
```

2.3. Всё в контейнерах (стенд для жюри, из чистого клона)

Нужен только Docker (Compose v2.17+); `make` необязателен (эквиваленты в комментариях).

```
git clone <репозиторий> vzaimno && cd vzaimno
docker compose up -d --build                                   # make up
curl -s http://localhost/api/health                            # ждите "pool_ready":true (PowerShell:
curl.exe); make demo ждёт сам
docker compose exec -T backend python scripts/seed_demo.py     # make demo (сам ждёт готовности пула)
docker compose exec -T backend cat var/demo_accounts.md        # make creds: логины и пароли демо-
аккаунтов
```

Сервис	Что делает	Адрес на хосте
`web`	nginx: статика Vite (SPA-fallback, gzip, `immutable` для хешированных файлов, заголовки безопасности) и прокси `/api` в `backend`	http://localhost (`VZ_WEB_PORT`)
`backend`	FastAPI; при старте `alembic upgrade head` , затем `uvicorn --workers $VZ_WORKERS` (по умолчанию 4)	http://localhost:8000
`pool`	одноразовая задача: `scripts/generate_pool.py` (пул заданий: быстрый режим — 111 семейств × 40 = 4 440 экземпляров, боевой — × 200 = 22 200; идемпотентно; несколько процессов по числу ядер), затем предрасчёт двойников (ждёт конца seed); в образе есть Node.js для JS-семейств	—
`db`	PostgreSQL 16 + pgvector (`max_connections=200`)	127.0.0.1:5432
`mailpit` , `fsp-mock`	письма с кодами, мок реестра ФСП	:8025, :8001
`keycloak`	только `--profile fsp`	:8080

Что важно:

- Локальный compose — стенд для жюри: `VZ_DEMO_MODE` по умолчанию `true` , секреты демонстрационные (в `docker-compose.prod.yml` обязательны свои).
- `pool` стартует, когда `backend` стал healthy; тест кандидата заработает, когда пул готов (`pool_ready` в `/api/health` ; `docker compose logs -f pool` ; `docker compose ps -a pool` — Exited (0) после предрасчёта двойников).
- **Быстрый режим для проверки (локальный compose по умолчанию).** `VZ_POOL_PER_FAMILY=40` в `docker-compose.yml` : 40 одноразовых клонов на семейство вместо боевых 200. Расчёт: сессия берёт из семейства не больше одного клона (≤ 20 заданий на сессию), порог долива — четверть пула (10), поэтому 40 клонов обслуживают десятки прохождений одного семейства подряд, а дальше пул доливается в фоне по ходу выдачи (так же, как в бою; seed демо доливает синхронно). Полный пул: `VZ_POOL_PER_FAMILY=200 docker compose up -d --build` (или строка в `.env`). `docker-compose.prod.yml` и `deploy/bootstrap.sh` переменную не задают — боевой пул 200. Готовность (`pool_ready`) не зависит от размера пула: не менее 80 % семейств имеют ≥ 3 свободных клона. Число процессов генерации — `VZ_POOL_WORKERS` (0 — по ядрам, не больше 8).
- Клиентский IP за nginx: nginx перезаписывает `X-Forwarded-For` адресом клиента (`VZ_REAL_IP_FROM` — кому nginx верит), бэкенд доверяет частным подсетям docker (`VZ_FORWARDED_ALLOW_IPS`).
- `restart: unless-stopped` поднимет упавший контейнер сам; healthcheck есть у всех сервисов, кроме одноразового `pool` .
- Другие порты (80 занят): `VZ_WEB_PORT=8088 VZ_PUBLIC_BASE_URL=http://localhost:8088 VZ_CORS_ORIGINS=http://localhost:8088 docker compose up -d --build` . Заняты ещё 8000/5432/8025/1025/8001 (другой стек, локальная БД) — перенесите и их: `VZ_BACKEND_PORT` , `VZ_DB_PORT` , `VZ_MAILPIT_UI_PORT` , `VZ_MAILPIT_SMTP_PORT` , `VZ_FSP_MOCK_PORT` (переменные окружения или строки в `.env` ; в PowerShell — `$env:VZ_WEB_PORT=8088`).
- Несколько процессов бэкенда и демо-режим: смещение демо-часов (`/api/demo/clock`) хранится в БД и подхватывается всеми процессами раз в секунду; счётчики лимитов запросов ведутся в памяти каждого процесса (эффективный лимит до `VZ_WORKERS` раз выше), буфер прямых POST на `/api/demo/webhook-sink` — тоже по процессу. Для строгих лимитов и показа вебхуков — `VZ_WORKERS=1` .
- `docker compose down` сохраняет данные в томе, `docker compose down -v` стирает БД и демо-мир. Сброс демо к seed без пересборки: `make reset-demo-stack` .

2.4. Тесты и валидация

```
make test        # cd backend && uv run pytest -q: более 1500 тестов; для тестов API нужен PostgreSQL
(docker compose up -d db), без него они пропускаются
make lint        # ruff check .
```

```
make validate    # симуляция и подбор → docs/VALIDATION.md, backend/eval/out/validation.json (порядка
1,5–5 минут)
```

Для тестов создаётся отдельная база `<имя>_<pid>_<rand>` на сервере из `VZ_TEST_DATABASE_URL` и удаляется после прогона.

2.5. Замеры контейнерного стенда

Окружение: Windows 11, Docker Desktop (WSL2), 20 логических ядер, 11 ГБ памяти, отдельный проект compose `vzcheck` на других портах; базовые образы (python, node, nginx, postgres) уже были в кэше Docker (на пустом кэше добавится скачивание порядка 1–2 ГБ). Нагрузка — LOAD_TEST.md.

Сборка `docker compose up -d --build` (backend, pool, web, fsp-mock и старт до `backend` healthy) — ≈ 20–60 с при тёплом кэше слоёв.

Быстрый режим (по умолчанию в локальном compose), замер 10.10.2026, чистый клон, новый том БД:

Шаг	Время
`pool` : 111 × 40 = 4 440 экземпляров в 8 процессах, 0 ошибок; `pool_ready`	≈ 70 с генерации; `pool_ready` через ≈ 105 с от `up` (последовательно те же 4 440 — ≈ 300 с)
`seed_demo.py` после `pool_ready` (300 кандидатов с настоящими тестами, персоны, 6 компаний, сценарий; пул доливается по ходу)	≈ 2 минуты
От клона до готового демо	**≈ 3 мин 50 с** (сборка с кэшем ≈ 30 с, пул ≈ 75 с, seed ≈ 2 мин)
Предрасчёт двойников (в контейнере `pool` после конца seed, до 8 потоков)	ещё ≈ 5–6 минут в фоне (всего ≈ 9,5 мин от `up`); тест и демо от него не зависят

Полный пул (`VZ_POOL_PER_FAMILY=200` , как в проде), тот же стенд:

Шаг	Время (до → после распараллеливания)
`pool` : 111 × 200 = 22 200 экземпляров, 0 ошибок; `pool_ready`	≈ 12 минут → ≈ 5,3 минуты (8 процессов; в основном тяжёлые семейства `algo.*` и JS с запуском кода)
`seed_demo.py` после `pool_ready`	≈ 70 с
От клона до готового демо	около 14–15 минут → ≈ 7 минут
Предрасчёт двойников (200 на семейство)	десятки минут → заметно короче (все семейства в общем пуле потоков), но остаётся фоновой задачей

До правок быстрого режима: пул 12 мин + seed 1 мин + сборка ≈ 1 мин = 14–15 минут, а предрасчёт двойников шёл десятки минут. Генерация детерминирована: результат зависит от семейства и seed, а не от числа процессов (тест `test_parallel_generation_equals_sequential`). На слабой машине (4 ядра) быстрый режим займёт дольше: ориентировочно 3–5 минут на пул и столько же на seed (оценка, не измерено).

Ресурсы (`docker stats`), в покое: backend ≈ 70 МБ на процесс, db ≈ 180–300 МБ, web ≈ 16 МБ, fsp-mock ≈ 37 МБ, mailpit ≈ 10–20 МБ. Под нагрузкой — LOAD_TEST.md. Образы: backend ≈ 800 МБ, web ≈ 77 МБ.

3. Переменные окружения

Бэкенд читает только переменные с префиксом `VZ_` (`app/core/settings.py`); полный список с пояснениями — `.env.example` . Продуктовые веса, пороги и квоты — не здесь, а в `config/*.yaml` . `docker compose` читает `.env` из корня репозитория (для подстановок), а сам процесс бэкенда — `.env` **из текущего каталога запуска** (для `make` это `backend/`); в контейнере значения передаёт compose.

Группа	Переменные (по умолчанию)	Заметки
Общее	`VZ_ENV` (`dev`; `test`, `prod`), `VZ_LOG_LEVEL` (`INFO`), `VZ_DEMO_MODE` (`false`), `VZ_CONFIG_DIR`	В `prod` бэкенд не стартует с dev-секретами, секретами короче 32 символов и без `VZ_COOKIE_SECURE=true`. `VZ_DEMO_MODE` включает `/api/demo/*`, автоответчик, «+1 день»
База данных	`VZ_DB_USER`, `VZ_DB_PASSWORD`, `VZ_DB_NAME` (`vzaimno`), `VZ_DB_PORT` (5432), `VZ_DATABASE_URL` (`postgresql+psycopg://vzaimno:vzaimno@localhost:5432/vzaimno`), `VZ_DB_POOL_SIZE` (10), `VZ_DB_MAX_OVERFLOW` (10), `VZ_TEST_DATABASE_URL`	Драйвер — psycopg 3 (схема `postgresql+psycopg`). В проде пароль БД поменять
Адреса	`VZ_PUBLIC_BASE_URL` (`http://localhost:5173`), `VZ_CORS_ORIGINS`, `VZ_BACKEND_PORT` (8000)	`VZ_PUBLIC_BASE_URL` — адрес фронтенда в ссылках писем и в QR сверки PDF
Секреты	`VZ_JWT_SECRET`, `VZ_APP_SECRET`, `VZ_PSEUDONYM_SECRET`	**Обязательно сменить в проде** (≥ 32 символов; `python -c "import secrets; print(secrets.token_urlsafe(48))"`). Смена `VZ_APP_SECRET` делает пул клонов нерасшифровываемым
Сессии	`VZ_ACCESS_TOKEN_TTL_S` (900), `VZ_REFRESH_TOKEN_TTL_S` (2 592 000), `VZ_REFRESH_REUSE_GRACE_S` (10), `VZ_REFRESH_COOKIE_NAME` (`vz_refresh`), `VZ_REFRESH_COOKIE_PATH` (`/api/auth`), `VZ_COOKIE_SECURE` (`false`; в проде `true`), `VZ_COOKIE_SAMESITE` (`lax`), `VZ_COOKIE_DOMAIN`	
Пароли	`VZ_PASSWORD_MIN_LENGTH` (8), `VZ_PASSWORD_MAX_LENGTH` (128), `VZ_ARGON2_TIME_COST` (3), `VZ_ARGON2_MEMORY_KIB` (65536), `VZ_ARGON2_PARALLELISM` (4), `VZ_ARGON2_MAX_CONCURRENT` (4)	Память под хеширование: `MEMORY_KIB × MAX_CONCURRENT`
Коды из писем	`VZ_EMAIL_CODE_TTL_S` (900), `VZ_EMAIL_CODE_RESEND_COOLDOWN_S` (60), `VZ_EMAIL_CODE_MAX_SENDS_PER_HOUR` (5), `VZ_EMAIL_CODE_MAX_ATTEMPTS` (5)	
Ограничение частоты	`VZ_RL_LOGIN_EMAIL` (`5/300`), `VZ_RL_LOGIN_IP` (`30/300`), `VZ_RL_VERIFY_EMAIL`, `VZ_RL_REGISTER_IP`, `VZ_RL_RESEND_IP`, `VZ_RL_REFRESH_IP`	Формат `N/секунд`
Почта	`VZ_MAIL_BACKEND` (`smtp` или `console`), `VZ_SMTP_HOST` (`localhost`), `VZ_SMTP_PORT` (1025), `VZ_SMTP_SECURITY` (`none`, `starttls`, `ssl`), `VZ_SMTP_USERNAME`, `VZ_SMTP_PASSWORD`, `VZ_SMTP_TIMEOUT_S`, `VZ_MAIL_FROM`	Локально — Mailpit. В проде — реальный SMTP по 587/465 (порт 25 на VPS часто закрыт), SPF, DKIM, DMARC на своём домене
ФСП ID и реестр	`VZ_FSP_OIDC_ISSUER`, `VZ_FSP_OIDC_INTERNAL_URL`, `VZ_FSP_OIDC_CLIENT_ID` (`vzaimno`), `VZ_FSP_OIDC_CLIENT_SECRET`, `VZ_FSP_OIDC_REDIRECT_URI`, `VZ_FSP_SCOPES`, `VZ_FSP_REGISTRY_URL`, `VZ_FSP_PROVIDER` (`auto`; `http`, `inprocess`), `VZ_FSP_REGISTRY_TOKEN`, `VZ_FSP_TOKEN_URL`, `VZ_FSP_REGISTRY_SCOPE`, `VZ_FSP_TIMEOUT_S` (5), `VZ_FSP_DEMO_LOGIN_ENABLED` (`true`), `VZ_FSP_STATE_TTL_S` (600), `VZ_FSP_RL_*`	Переход на настоящий ФСП ID — FSP_INTEGRATION.md п. 9. Keycloak: `VZ_KEYCLOAK_ADMIN`, `VZ_KEYCLOAK_ADMIN_PASSWORD`, `VZ_KEYCLOAK_PORT`, `VZ_FSP_KEYCLOAK_PUBLIC_URL`, `VZ_PUBLIC_DOMAIN`
Резюме и PDF	`VZ_RESUME_MAX_BYTES` (5 МБ), `VZ_RESUME_MAX_PAGES` (10), `VZ_RESUME_MIN_TEXT_CHARS` (40), `VZ_RESUME_RL_UPLOAD`, `VZ_PROFILE_RL_PDF`, `VZ_PROFILE_RL_PREVIEW`, `VZ_PROFILE_RL_SEARCH`, `VZ_PROFILE_RL_VERIFY`	
LLM (необязательно)	`VZ_LLM_PROVIDER` (`none`; `auto`, `yandexgpt`, `gigachat`), `VZ_LLM_MODE` (`none`, `live`, `record`, `replay`), `VZ_LLM_CASSETTE_PATH`, `VZ_LLM_TIMEOUT_S` (20), `VZ_LLM_MIN_QUOTE_RATIO` (90), `VZ_LLM_YANDEX_API_KEY`, `VZ_LLM_YANDEX_FOLDER_ID`, `VZ_LLM_GIGACHAT_AUTH_KEY` и др.	По умолчанию выключена. Включать только с реальными ключами; ПДн маскируются до вызова
За обратным прокси	`VZ_FORWARDED_ALLOW_IPS` (или `FORWARDED_ALLOW_IPS`, как у uvicorn; по умолчанию `127.0.0.1`)	**Обязательна за Caddy**: адреса или подсеть прокси (п. 5.3); `*` в prod запрещено

Группа	Переменные (по умолчанию)	Заметки
Демо на боевом стенде	`VZ_DEMO_CONTROLS_ADMIN_ONLY` (по умолчанию: `true` в `VZ_ENV=prod`), `VZ_DEMO_CONTROL_KEY`	При `VZ_DEMO_MODE=true` общие кнопки демо (часы, сброс, автоответчик, режим жюри) в prod — только админ или заголовок `X-Demo-Key`; вне prod — любая роль
Планировщик	`VZ_SCHEDULER_ENABLED` (по умолчанию — `config/scheduler.yaml`)	Фоновые задачи внутри бэкенда (п. 6); в тестах не запускается

4. Сборка образов

```
# Бэкенд: python:3.12-slim, uv 0.11.12, зависимости строго по uv.lock, процесс от непривилегированного пользователя
docker compose build backend
# вручную (config/*.yaml лежит вне контекста backend и передаётся отдельным контекстом «config»):
cd backend && docker build --build-context config=../config -t vzaimno-backend .

# Мок реестра ФСП
docker build -t vzaimno/fsp-mock services/fsp-mock

# Фронтенд: multi-stage (node 22 → nginx 1.27), статика и прокси /api внутри образа
docker compose build web
```

В образ бэкенда входят `app/`, `alembic/`, `alembic.ini`, `config/*.yaml`, `eval/` (нужен демо-модулям), `scripts/` (пул, seed) и бинарник Node.js (песочница JS-заданий); тесты не входят. Healthcheck образа — `GET /api/health/live`.

5. Развёртывание на VPS в РФ с HTTPS через Caddy

Схема: наружу открыты только 80 и 443 (Caddy); бэкенд и БД доступны лишь внутри сети compose (БД не публикуется вообще). Caddy терминирует TLS, проксирует `/api/*` в бэкенд, остальное — в контейнер `web` (nginx со статикой, SPA-fallback и кэшем), сам получает и продлевает сертификаты (ACME).

Стенд жюри одной командой (чистый Ubuntu 22.04/24.04 или Debian 12, 4 vCPU / 8 ГБ рекомендуется, открыты 80 и 443):

```
git clone <репозиторий> /opt/vzaimno && cd /opt/vzaimno && sudo bash deploy/bootstrap.sh
```

`deploy/bootstrap.sh` ставит Docker (если его нет), добавляет swap при ОЗУ меньше 6 ГБ, генерирует `.env` с секретами, поднимает `docker-compose.prod.yml`, ждёт бэкенд и пул заданий, засевает демо-мир и проверяет `https://<домен>/api/health`. Без `--domain` адрес стенда — `<ip-через-дефисы>.sslip.io` (публичный wildcard-DNS; Caddy выпускает на него сертификат Let's Encrypt, покупать домен не нужно). Без SMTP письма не уходят, а в демо-режиме регистрация подтверждается автоматически. Повторный запуск (`git pull && sudo bash deploy/bootstrap.sh`) безопасен: `.env` не перезаписывается. Параметры — `bash deploy/bootstrap.sh --help`. Ниже — те же шаги вручную.

5.1. Подготовка сервера

1. Арендовать VPS в дата-центре РФ (параметры — п. 1), установить Debian или Ubuntu LTS, обновить систему.
2. Создать DNS-запись `A` для домена (например, `vzaimno.example`) на адрес VPS; подождать распространения.
3. Установить Docker Engine и плагин Compose v2 (по инструкции дистрибутива или проекта Docker).
4. Открыть в межсетевом экране только 22 (SSH), 80 и 443. Помните: порты, опубликованные Docker, **обходят `ufw`** — поэтому в прод-схеме БД и бэкенд не публикуются вовсе.

5. Настроить доступ по SSH-ключам, отключить вход по паролю.

5.2. Код, секреты, фронтенд

```
git clone <репозиторий> /opt/vzaimno && cd /opt/vzaimno

# Секреты: три разных значения, каждое не короче 32 символов
python3 -c "import secrets; print(secrets.token_urlsafe(48))"      # выполнить трижды

cat > .env <<'EOF'
VZ_PUBLIC_DOMAIN=vzaimno.example
VZ_DB_PASSWORD=<длинный случайный пароль>
VZ_JWT_SECRET=<секрет 1>
VZ_APP_SECRET=<секрет 2>
VZ_PSEUDONYM_SECRET=<секрет 3>
VZ_SMTP_HOST=smtp.<ваш-провайдер>.ru
VZ_SMTP_PORT=587
VZ_SMTP_SECURITY=starttls
VZ_SMTP_USERNAME=<логин>
VZ_SMTP_PASSWORD=<пароль>
VZ_MAIL_FROM=Взаимно <no-reply@vzaimno.example>
# Стенд для жюри: VZ_DEMO_MODE=true, VZ_FSP_DEMO_LOGIN_ENABLED=true; боевой режим — оставить false
VZ_DEMO_MODE=false
EOF
chmod 600 .env

# Фронтенд собирается в образе web при `docker compose -f docker-compose.prod.yml up -d --build`: Node.js
на VPS не нужен
```

`.env` хранить вне git и вместе с резервной копией БД: без `VZ_APP_SECRET` зашифрованные ключи клонов не прочитать.

5.3. Файлы развёртывания

В репозитории лежат готовые файлы (compose проверен `docker compose config`, Caddyfile — `caddy validate`; запуск с настоящим доменом и сертификатом не выполнялся):

- `docker-compose.prod.yml` — боевая схема: `db`, `backend`, `pool` (одноразовая генерация пула), `web`, `caddy`; профиль `fsp` добавляет свой ФСП ID (`keycloak-db` + `keycloak`);
- `deploy/Caddyfile` — HTTPS, заголовки безопасности, проксирование `/api/*` в backend и остального в `web`;
- `deploy/docker-compose.db-loopback.yml` — временная публикация БД на `127.0.0.1` для генерации пула с хоста.

Запуск: `docker compose -f docker-compose.prod.yml up -d --build`. Что отличает схему от локального `docker-compose.yml`:

Мера	Как сделано
БД не публикуется	У `db` нет `ports`: PostgreSQL виден только соседним контейнерам. Для обслуживания — `docker compose -f docker-compose.prod.yml exec db psql -U vzaimno`; для генерации пула с хоста — временный `deploy/docker-compose.db-loopback.yml` (только `127.0.0.1`, после работы пересоздать `db` без него) или `exec` внутри контейнера
Бэкенд не публикуется	`expose: 8000`, наружу только `caddy` (80, 443, 443/udp)
Версии закреплены	`pgvector/pgvector:0.8.7-pg16`, `caddy:2.10.2-alpine`, `postgres:16.15-alpine`, `quay.io/keycloak/keycloak:26.0.8` — вместо плавающих `pg16` / `latest`. Обновление — осознанная правка версии в файле. Mailpit в прод-схему не входит (локальный compose закрепляет `axllent/mailpit:v1.31.4`). Для жёсткого закрепления можно дописать `@sha256:<digest>` к тегу (`docker buildx imagetools inspect <образ>`)

Мера	Как сделано
Keycloak не `start-dev`	В профиле `fsp` — production-режим `start` (с `--hostname`, `--proxy-headers=xforwarded`, своей PostgreSQL). `start-dev` остался только в локальном compose. Обязательные переменные профиля: `VZ_KEYCLOAK_DB_PASSWORD`, `VZ_KEYCLOAK_ADMIN`, `VZ_KEYCLOAK_ADMIN_PASSWORD`, `VZ_FSP_KEYCLOAK_PUBLIC_URL` (`https://sso.<домен>`); для доступа браузера раскомментируйте блок `{$VZ_KEYCLOAK_DOMAIN}` в `deploy/Caddyfile` (админ-консоль наружу не отдавать)
Клиентский IP за Caddy	Подсеть compose зафиксирована (`172.29.80.0/24`), бэкенд доверяет `X-Forwarded-For` только от неё (`VZ_FORWARDED_ALLOW_IPS`), Caddy перезаписывает заголовок адресом клиента, uvicorn запущен с `--no-proxy-headers` (см. ниже)
Демо-кнопки	`VZ_DEMO_CONTROL_KEY` (необязателен): в prod при `VZ_DEMO_MODE=true` часы, сброс, автоответчик и переключатель режима жюри — только роль `admin` или заголовок `X-Demo-Key`

Замечания:

- `VZ_FORWARDED_ALLOW_IPS` (читается и `FORWARDED_ALLOW_IPS`, как у uvicorn): список адресов и подсетей через запятую, по умолчанию `127.0.0.1`. Приложение (`app/core/netutil.py`) верит заголовку `X-Forwarded-For` только от этих соседей и читает цепочку справа налево. Caddy стоит в соседнем контейнере, поэтому без этой переменной все пользователи выглядят как один IP прокси и **делят общие лимиты** (вход, регистрация, обновление сессии); в prod значение `*` запрещено — иначе любой, кто дотянется до порта бэкенда, подставлял бы себе IP. Если в compose используется другая подсеть — поменяйте её в `networks.default.ipam` и в этой переменной одновременно.
- `VZ_PUBLIC_BASE_URL` должен совпадать с публичным адресом: он попадает в письма, QR и адрес возврата из ФСП ID.
- **SPA-маршруты**: `try_files … /index.html` нужен для адресов вида `/candidate/fsp/callback`.
- **Реестр ФСП**: по умолчанию `VZ_FSP_PROVIDER=inprocess` (встроенный демо-снимок, `fsp-mock` не нужен). Для боевого режима — переменные из FSP_INTEGRATION.md п. 9 (`VZ_FSP_PROVIDER=http`, адреса и учётные данные ФСП ID): добавьте их в `environment` сервиса `backend` (пустых значений не задавайте).
- Mailpit и `fsp-mock` из локального compose в прод-схему **не входят**.

Пересборка образа бэкенда (после `git pull`, смены зависимостей или кода; образ собирается заново, старый контейнер не «подхватывает» новые разделы API):

```
docker compose -f docker-compose.prod.yml build --no-cache backend     # зависимости строго по uv.lock
docker compose -f docker-compose.prod.yml up -d backend                # миграции применяются при старте
docker compose -f docker-compose.prod.yml ps                           # backend должен стать healthy
curl -s https://vzaimno.example/api/openapi.json | head -c 200         # новая схема: проверьте, что есть
нужные маршруты
```

Локально (`docker-compose.yml`): `docker compose build --no-cache backend && docker compose up -d backend`; если на `:8000` отвечает старый образ без новых разделов (например, `GET /api/bank/summary` даёт 404), причина — устаревший образ, а не код: пересоберите его, как выше.

5.4. Запуск, миграции, пул клонов, демо-мир

```
docker compose -f docker-compose.prod.yml up -d --build        # миграции применяются при старте backend
docker compose -f docker-compose.prod.yml ps                   # backend должен стать healthy
curl -s https://vzaimno.example/api/health                     # {"status":"ok","db":"ok",...}
```

Пул клонов обязателен: без него тест не стартует. Его создаёт одноразовый сервис `pool` (в образе есть `scripts/` и Node.js): он стартует сам после `backend` и занимает ≈ 5 минут на 20 логических ядрах ноутбука (8 процессов; до распараллеливания ≈ 12 минут), на 2–4 vCPU заметно дольше (оценка, не измерено; `VZ_POOL_WORKERS` по умолчанию 4 в prod-compose) (индикатор «Готовим одноразовые задания» на /jury; готовность — `"pool_ready":true` в `/api/health`) (`docker compose -f docker-compose.prod.yml logs -f pool`). Повтор или долив: `docker compose -f docker-compose.prod.yml run --rm pool`; состояние: `... run --`

`rm pool python scripts/generate_pool.py --stats`. `VZ_APP_SECRET` у `pool` и `backend` один (ключи шифруются им). Туннель к БД для этого больше не нужен; `deploy/docker-compose.db-loopback.yml` оставлен для обслуживания.

Демо-мир для стенда жюри (`VZ_DEMO_MODE=true` в `.env`): `seed_demo.py` отказывается работать при `VZ_ENV=prod`, поэтому его запускают с машины с туннелем с локальным `VZ_ENV=dev` и тем же `VZ_APP_SECRET` (пароли демо-аккаунтов выводятся из него); через туннель наполнение идёт медленнее, чем локально:

```
cd backend
VZ_ENV=dev uv run python scripts/seed_demo.py              # пароли — в backend/var/demo_accounts.md (в
.gitignore)
```

Боевой режим (`VZ_DEMO_MODE=false`): `seed_demo.py` не запускать, маршруты `/api/demo/*` отвечают 404.

5.5. Проверка после развёртывания

Проверка	Ожидание
`https://<домен>/`	Открывается фронтенд по HTTPS (сертификат Caddy)
`https://<домен>/api/health`	`status: ok` , `db: ok`
`https://<домен>/api/docs`	Swagger UI
Регистрация нового пользователя	Письмо с кодом приходит (реальный SMTP), `Set-Cookie: vz_refresh` с флагами `HttpOnly; Secure; Path=/api/auth`
Серия неверных входов	После 5 попыток за 300 с — 429 (проверяет, что клиентский IP определяется верно: 429 получает тот, кто ошибался, а не все)
`GET /api/fsp/availability`	Корректный `registry_label`
Тест	Опрос → `POST /api/tests/sessions` → задания выдаются (пул создан)
Журнал	`docker compose -f docker-compose.prod.yml logs backend` без ошибок

6. Обслуживание

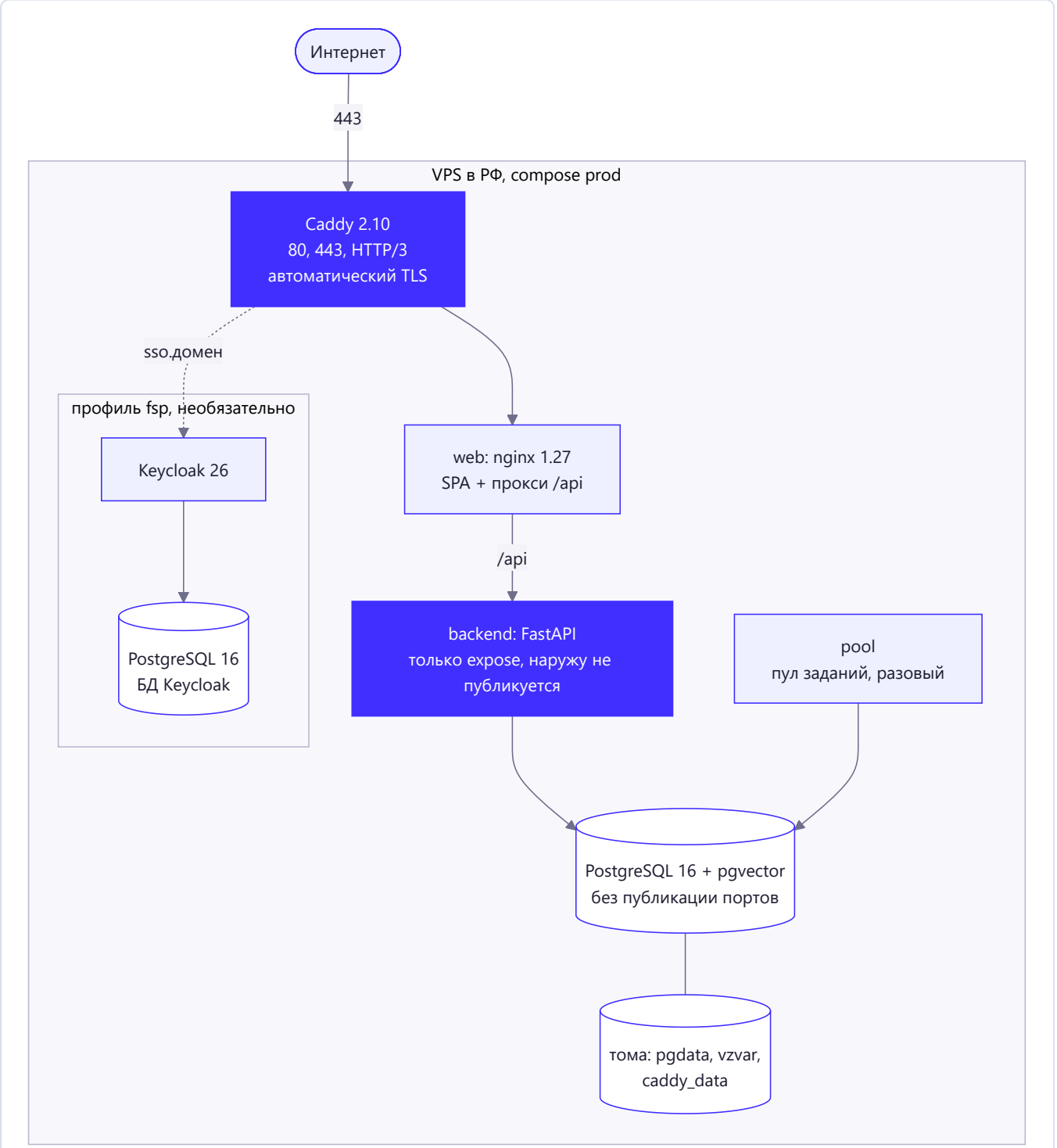
Задача	Команда
Резервная копия БД	`docker compose -f docker-compose.prod.yml exec -T db pg_dump -U vzaimno vzaimno | gzip > backup-$(date +%F).sql.gz` ; копии и `.env` хранить на отдельном носителе в РФ
Восстановление	`gunzip -c backup.sql.gz | docker compose -f docker-compose.prod.yml exec -T db psql -U vzaimno vzaimno` в пустую БД
Обновление версии	`git pull && cd frontend && npm ci && npm run build && cd .. && docker compose -f docker-compose.prod.yml up -d --build` (миграции применяются автоматически); при новых семействах — повторить `generate_pool.py`
Откат	`git checkout <прежний тег>` и тот же `up -d --build` ; миграции Alembic вниз (`alembic downgrade`) выполняются только вручную и осознанно
Долив пула	Идёт автоматически в фоне после выдачи заданий (когда свободных меньше 50); вручную — `generate_pool.py`
Калибровка сложности	Ночью — планировщик (`config/scheduler.yaml` , 00:00 UTC = 03:00 МСК); вручную — `POST /api/lab/calibrate` (роль методист)
Фоновые задачи	Встроенный планировщик (asyncio в процессе бэкенда): раз в минуту — сроки приглашений и откликов и повтор вебхуков, ночью — калибровка с CUSUM и чистка по срокам хранения. Итог суточных задач — в `audit_log` (`scheduler.calibration` , `scheduler.retention`); отключить — `VZ_SCHEDULER_ENABLED=false`
Логи	`docker compose -f docker-compose.prod.yml logs -f backend` ; уровень — `VZ_LOG_LEVEL`

Задача	Команда
Здоровье	`GET /api/health` (БД), `GET /api/health/live` (процесс); у контейнеров есть healthcheck
Смена секретов	`VZ_JWT_SECRET` — пользователи выйдут; `VZ_APP_SECRET` — пул клонов придётся пересоздать; `VZ_PSEUDONYM_SECRET` — новые псевдонимы для новых показов

7. Диагностика

Симптом	Причина и решение
Бэкенд падает при старте: «VZ_..._SECRET обязательно задать в prod» или «не короче 32 символов»	В `VZ_ENV=prod` запрещены dev-секреты; задайте три секрета в `.env`
«В prod refresh-cookie должна быть Secure»	Задайте `VZ_COOKIE_SECURE=true` (в `docker-compose.prod.yml` уже есть)
429 у всех пользователей, в логах один IP	Не задан `VZ_FORWARDED_ALLOW_IPS` за прокси (п. 5.3)
Пользователь не остаётся залогиненным	Нет HTTPS при `VZ_COOKIE_SECURE=true`, неверный `VZ_PUBLIC_DOMAIN` или `VZ_CORS_ORIGINS`
Письма не приходят	Порт 25 закрыт у провайдера: используйте 587 или 465; проверьте SPF, DKIM, DMARC; смотрите `logs backend` на «SMTP: письмо ... не отправлено»; ответ 503 `mail_unavailable`
Тест не стартует, «банк исчерпан» или нет заданий	Не создан пул: `generate_pool.py` (п. 5.4)
«Не удалось расшифровать ключ экземпляра»	Пул создан с другим `VZ_APP_SECRET`: верните прежний секрет либо удалите невыданные экземпляры и создайте пул заново
`generate_pool.py`: «node не найден»	Нужен Node.js в `PATH` для семейств Frontend
Windows: ошибки кодировки в консоли	`PYTHONUTF8=1` (`make` выставляет сам)
`additional_contexts` не поддерживается	Обновите Docker Compose до v2.17+

Схема боевого развёртывания



8. В разработке

- Мониторинг, алерты и автоматическое резервное копирование.
- Закрепление образов по digest (сейчас — по версиям; `python:3.12-slim` в Dockerfile остаётся плавающим тегом минорной линии).

Интеграция с ATS работодателя и чат после «Взаимно»

Документ описывает, как платформа встраивается в существующий процесс найма (по ответу организаторов: «синхронизация вакансий, откликов и статусов с внутренней системой найма через API»), и как это можно проверить без живой коммерческой ATS. Чат после «Взаимно» — в конце.

Решение (DECISIONS 29): боевых подключений к реальным ATS нет; вместо них — проверяемая интеграция из четырёх частей, все они покрыты тестами (`backend/tests/test_api_ats.py` , `test_api_chat.py`).

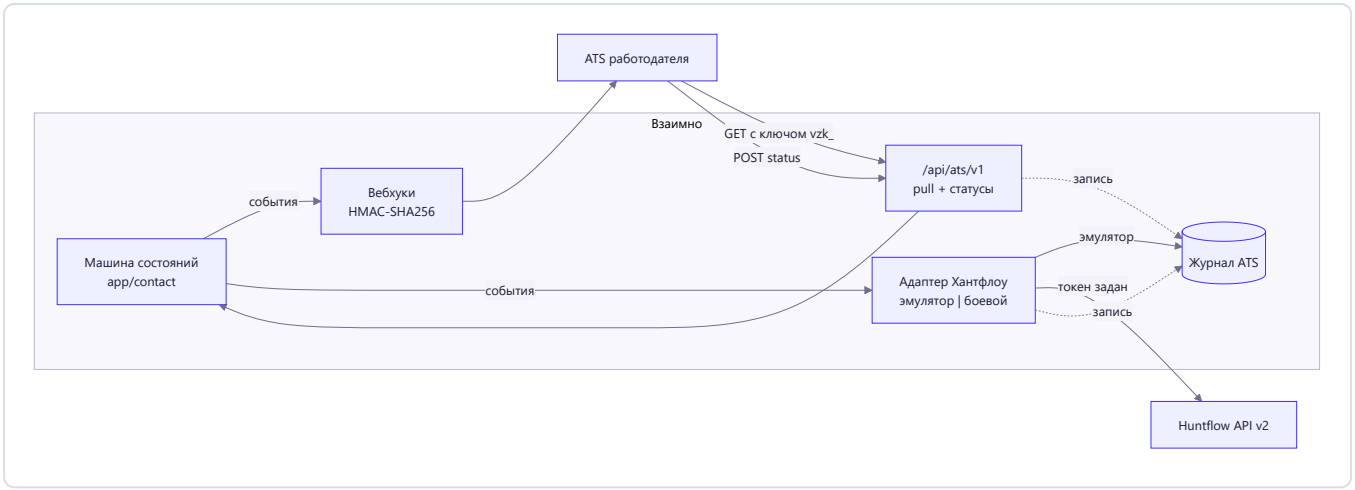
Часть	Что делает	Где
Исходящие вебхуки	Подписанные события (HMAC-SHA256) при смене статуса приглашения и отклика	`app/services/contact/webhooks.py` , `/api/companies/me/webhooks*` (без изменений)
Pull-API	ATS забирает вакансии, приглашения и отклики с курсором и фильтром `updated_since`	`/api/ats/v1/*` , `app/api/ats.py` , `app/services/ats/pull.py`
Обратная синхронизация	ATS передаёт статусы («связались», «интервью», «оффер», «нанят», «отказ») — они идут через нашу машину состояний	`POST /api/ats/v1/{invitations,applications}/{id}/status` , `app/services/ats/status.py`
Адаптер «Хантфлоу»	Наши события → кандидат, вакансия, смена этапа в формате Huntflow API v2; эмулятор и боевой режим	`app/services/ats/huntflow.py` , `sync.py`

1. Что реально, что эмулировано

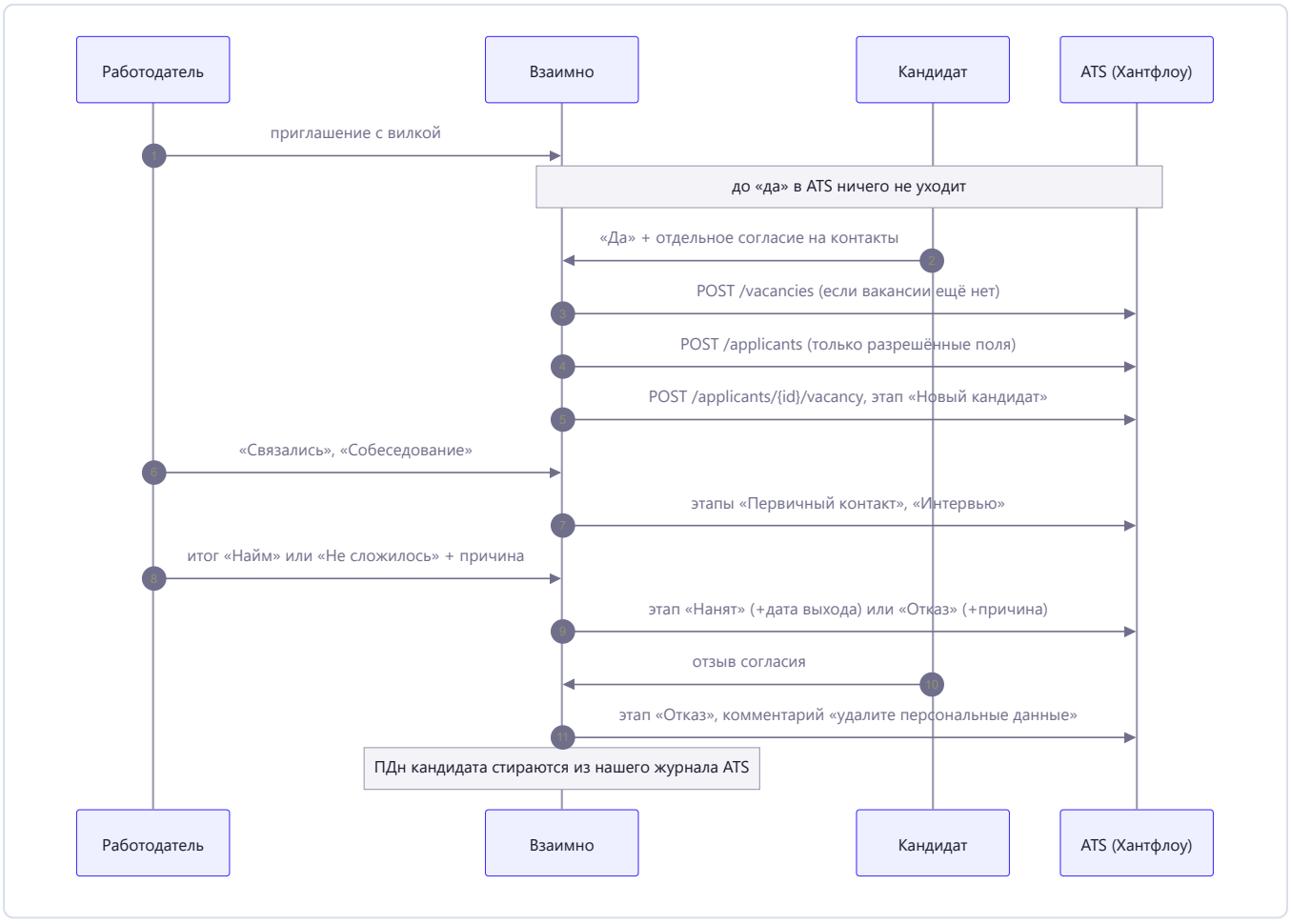
Что	Статус
Ключи компании, области доступа, pull-API, приём статусов, идемпотентность	Реально работает, проверено тестами и вручную через HTTP
Вебхуки	Реально работают (доставка, повторы, подпись, журнал)
Адаптер «Хантфлоу», эмулятор	Реально работает: запросы строятся полностью, записываются в «Журнал ATS» ровно в том виде, в каком ушли бы в Хантфлоу; ответы — синтетические и детерминированные
Адаптер «Хантфлоу», боевой режим	Код отправки есть и проверен на подменённом транспорте (заголовок, пути, id этапов из справочника ATS, сбой не ломает действие). **Против живого Хантфлоу не запускался**: поля сверены с публичной документацией Huntflow API v2, но реальный ответ и токен не проверены. Для включения нужны только адрес API и токен
Повторы неудачных боевых запросов	Нет: сбой пишется в журнал, повторной отправки очереди нет (у вебхуков она есть)
Другие ATS (Potok, E-Staff и т. п.)	Не реализованы: подключаются через pull-API и вебхуки

2. Концепция

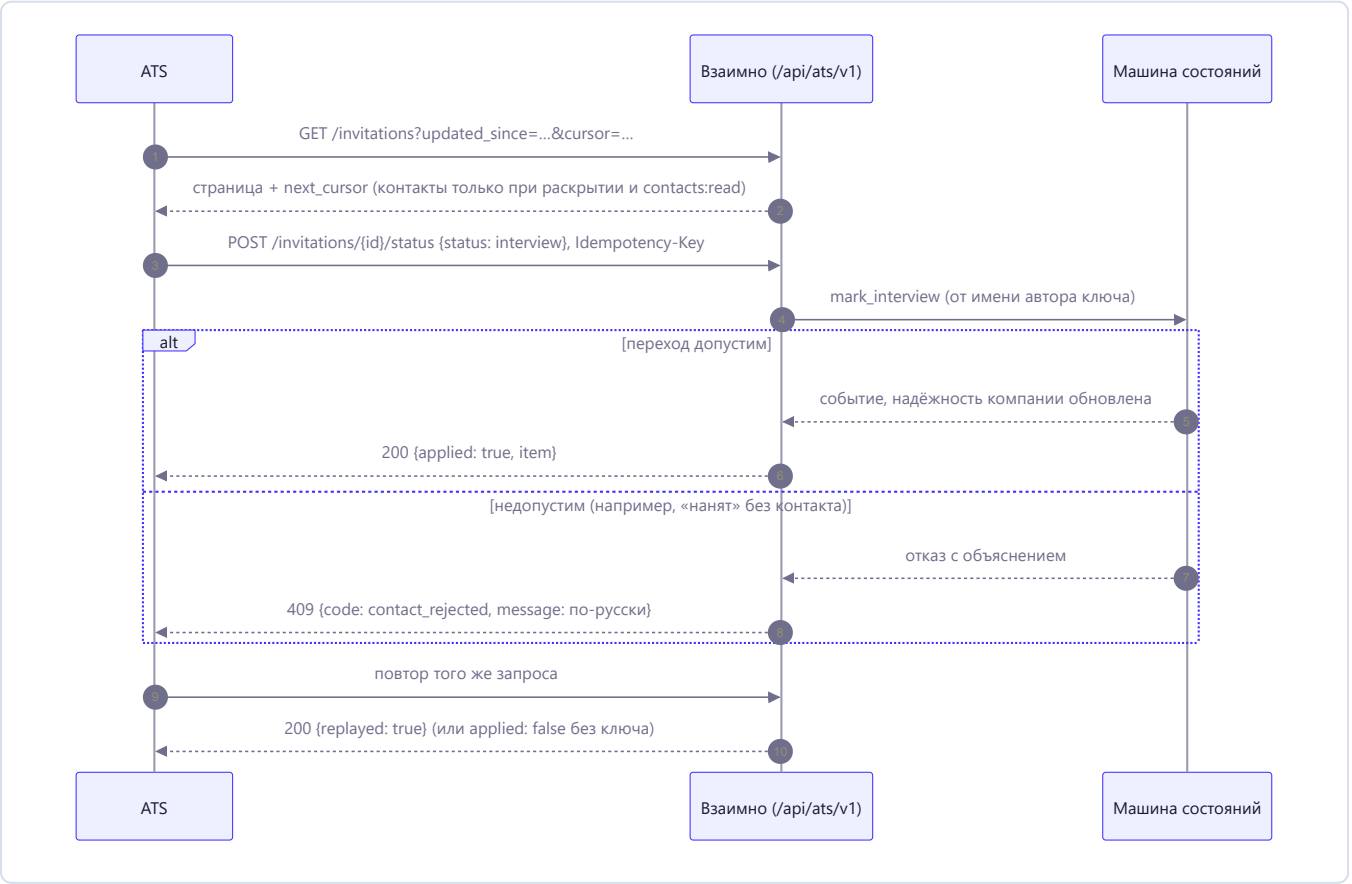
Работодатель в профиле компании выбирает тип интеграции: **Нет / Вебхук / Хантфлоу** — и выпускает API-ключи для pull-API. Принцип один: наружу уходит только то, что кандидат уже разрешил. До взаимного «да» в ATS нет ни имени, ни контактов кандидата, есть только псевдоним «К-4821» в pull-API.



Последовательность: приглашение → контакты → этапы в ATS



Последовательность: статус из ATS



3. Безопасность

- **Ключи компании** (выбрано вместо OAuth2 client credentials: интеграция «сервер—сервер», ключ с областями проще выдать и отозвать из профиля, чем держать OAuth-сервер). Формат `vzk_<key_id>_<секрет>`; в БД — только sha256 секрета; показывается один раз при создании и перевыпуске; перевыпуск и отзыв действуют сразу; не больше 5 действующих ключей (`config/contact.yaml → ats`). Любая ошибка ключа — одинаковый 401. Лимит запросов — на ключ.
- **Области доступа:** `vacancies:read`, `invitations:read`, `applications:read`, `contacts:read`, `status:write`. Без нужной области — 403 `ats_scope_missing`.
- **ПДн только после раскрытия.** Блок `candidate.contacts` приходит, только если у ключа есть `contacts:read` **и** у пары действует согласие на раскрытие (`ContactDisclosure.revoked_at IS NULL`), и содержит только поля, которые разрешил кандидат. В остальных случаях — `null`. Внутренних id кандидата в ответах нет никогда (проверяется тестами: `assert_no_pii`). После отзыва согласия запись «обновляется» (`updated_at`), и ATS видит пропажу контактов при следующем забирании.
- **Токен Хантфлоу** хранится зашифрованным (Fernet, ключ из `VZ_APP_SECRET`, как ключи экземпляров теста), наружу не возвращается (только последние 4 знака), в журнал не попадает. Адрес API проходит ту же SSRF-проверку, что и URL вебхука.
- **Вебхуки:** подпись `X-Vzaimno-Signature: sha256=<HMAC-SHA256(секрет, тело)>`, идемпотентность `X-Vzaimno-Event-Id`; контакты в событии — только при согласии.
- **152-ФЗ:** при отзыве согласия адаптер отправляет этап «Отказ» с просьбой удалить данные, а имя, телефон, email и Telegram стираются из «Журнала ATS» (в самой ATS удаление остаётся за работодателем).
- **Аудит:** создание, перевыпуск и отзыв ключей, смена настроек, принятые статусы — в `audit_log` (без ПДн).

4. Pull-API `/api/ats/v1`

Заголовок: `Authorization: Bearer vzk_…`. Схема — в `/api/docs` (тег «Интеграция ATS»).

Метод и путь	Область	Описание
`GET /me`	любая из `vacancies:read`	Проверка ключа: компания, области
`GET /vacancies`	`vacancies:read`	Опубликованные и закрытые вакансии
`GET /invitations`, `GET /invitations/{id}`	`invitations:read` (+ `contacts:read`)	Приглашения со статусом, этапом, итогом, условиями (хеш sha256)
`GET /applications`, `GET /applications/{id}`	`applications:read` (+ `contacts:read`)	Отклики кандидатов
`POST /invitations/{id}/status`	`status:write`	Статус из ATS
`POST /applications/{id}/status`	`status:write`	Статус отклика из ATS

Пагинация и синхронизация. Параметры `updated_since` (ISO 8601, включительно), `cursor`, `limit` (по умолчанию 50, максимум 100), у приглашений и откликов ещё `status`. Порядок — по `(updated_at, id)`; ответ: `items`, `next_cursor` (`null` — конец), `server_time`. Курсор непрозрачный; повтор с тем же курсором возвращает те же записи. Рекомендуемый цикл: хранить `server_time` предыдущего прохода, передавать как `updated_since`, идти по `next_cursor`; запись, изменённая в ту же секунду, не потеряется (фильтр включительный), дубли отбрасывать по `id`.

Идемпотентность записи. Заголовок `Idempotency-Key` (до 80 знаков): повтор того же запроса возвращает прежний ответ с `replayed: true`; тот же ключ для другого статуса — 409 `ats_idempotency_conflict`. Без ключа повтор уже достигнутого состояния — 200 с `applied: false`.

Соответствие статусов ATS и нашей машины состояний

Статус из ATS	Приглашение	Условие	Ошибка 409, если условие нарушено
`contacted`	`mark_contacted` («Связались», в срок — плюс к надёжности)	приглашение принято, контакты открыты	«Контакты кандидата ещё не открыты»
`interview`	`mark_interview` (этап «собеседование»; контакт отмечается вместе с ним)	принято	сообщение машины состояний
`offer`	этап `offer` (в машине состояний этапа нет, хранится как `stage`)	принято и был контакт	`offer_requires_contact`
`hired`	`close` / `hired`	был контакт	«Сначала отметьте «Связались» или «Собеседование»…»
`rejected` + `reason`	`close` / `not_matched`	причина: `skills_missing` (+ `skill_ids`), `position_closed`, `chose_other`	400 `ats_reason_required` без причины

Для откликов: `viewed` → просмотр, `interested` → «есть интерес» (при необходимости с просмотром), `rejected` + `reason` (`skills_missing`, `salary`, `position_closed`, `chose_other`, `other`) → отказ. Переходы после финального статуса (например, `rejected` по уже закрытому найму) отклоняются кодом 409 с русским объяснением. Надёжность компании пересчитывается теми же побочными эффектами, что при ручной отметке в кабинете. Событие записывается от имени автора ключа.

5. Адаптер «Хантфлоу» и соответствие полей

Формы запросов — по публичной документации Huntflow API v2 (`https://api.huntflow.ru/v2`, заголовок `Authorization: Bearer <токен>`). Адрес и номер организации задаются в профиле; в эмуляторе номер по

умолчанию — 1.

Наша сущность	Запрос Хантфлоу	Поля
Вакансия (опубликованная, бриф или приглашение)	`POST /accounts/{id}/vacancies`	`position` ← название, `company` ← название компании, `body` ← описание, `money` ← «220000-280000 руб. на руки»
Кандидат после «да»	`POST /accounts/{id}/applicants`	`first_name`, `last_name`, `middle_name` ← ФИО (3 слова — «Фамилия Имя Отчество», 2 слова — «Имя Фамилия»); `phone`, `email`; `social: [{social_type: TELEGRAM, value}]`; `position`, `money`; `externals[0].data.body` — «Источник: платформа «Взаимно»...». Только поля, разрешённые кандидатом
Первичное добавление и смена этапа	`POST /accounts/{id}/applicants/{applicant_id}/vacancy`	`vacancy`, `status` (id этапа), `comment`, `rejection_reason` (id причины), `employment_date`
Подключение	`GET /accounts`	«Проверить подключение»
Этапы воронки (боевой режим)	`GET /accounts/{id}/vacancies/statuses`	сопоставление по названию из `config/contact.yaml → ats.huntflow.stage_names`

Этапы: приглашение принято → «Новый кандидат»; «Связались» → «Первичный контакт»; «Собеседование» → «Интервью»; «Найм» → «Нанят» (+дата выхода); «Не сложилось» → «Отказ» с причиной; отзыв согласия → «Отказ». Отклики: «есть интерес» → «Первичный контакт», отказ → «Отказ». Если в организации нет этапа с нужным названием — запись «Журнала ATS» со статусом `failed` и понятной причиной.

Идемпотентность исходящих запросов — по ключам в журнале (`hf:applicant:…`, `hf:move:<id события>`): повтор события не создаёт дублей.

Эмулятор включён, пока токен не задан: журнал содержит запрос целиком (метод, URL, заголовки без секрета, тело) и синтетический ответ с детерминированным `id` — следующие запросы ссылаются на него так же, как на настоящий. В профиле компании: «Журнал ATS» (раскрывается до тела запроса), «Проверить подключение», выбор типа, токен.

Откат кандидата к «слепому» состоянию в ATS невозможен технически; поэтому в ATS кандидат создаётся только после раскрытия.

6. Настройки

`config/contact.yaml → ats` : размер страницы, префикс и длина ключей, лимит ключей и запросов, размер журнала, области, типы, названия этапов Хантфлоу и id этапов эмулятора, допустимые статусы. Таблицы: `company_api_keys`, `ats_settings`, `ats_log`, `chat_messages` (миграция `0010_chat_ats`).

Чат после «Взаимно»

Переписка по приглашению или отклику (ответ организаторов: «списаться на платформе удобнее»; решение DECISIONS 29).

Состояние контакта	Кандидат	Работодатель
Нет «взаимного да» (приглашение не принято, согласия нет)	403 `chat_unavailable`	403 `chat_unavailable`
Принято (отклик: «есть интерес») и согласие действует	читает и пишет	читает и пишет
Контакт завершён («Завершено»), согласие действует	только чтение	только чтение

Состояние контакта	Кандидат	Работодатель
Согласие отозвано	история только для чтения	403 `chat_closed`, переписки нет в списке
Не участник	404	404

- Только текст, длина до `chat.max_len` (1000), не чаще `chat.rate_per_minute` (10) в минуту, не больше `chat.thread_max_messages` (500) в переписке (`config/contact.yaml → chat`).
- Уведомление в кабинете и письмо собеседнику (письмо — не чаще раза в 30 минут на переписку); **текста сообщения в уведомлении и письме нет**.
- Непрочитанные: `GET /api/chat/unread`, `GET /api/chat/threads`; открытие переписки отмечает сообщения собеседника прочитанными.
- Аудит: `chat.message_sent` (длина, без текста).
- DEMO_MODE: синтетический кандидат отвечает заготовкой из `chat.demo_replies` (детерминированно).
- Эндпоинты: `GET /api/chat/{kind}/{id}` (`after` — курсор по `seq`), `POST /api/chat/{kind}/{id}/messages`, `kind` = `invitation` | `application`. Интерфейс: панель на экране «Взаимно» кандидата и в карточке приглашения работодателя (после раскрытия), опрос раз в 8 секунд.

Доверие и безопасность: честные работодатели и защита кандидатов

Документ описывает, как платформа защищается от фиктивных вакансий и недобросовестных работодателей и что в этой части **реализовано**, а что пока **концепция**. Ответы организаторов: подтверждение компании может быть добровольной меткой и не должно блокировать основной сценарий; защита от фиктивных вакансий выходит за рамки MVP и засчитывается как концепция. Поэтому всё ниже построено на дешёвых сигналах, а решение всегда принимает человек.

Все числа, пороги и подписи — в `config/trust.yaml` и `config/contact.yaml` (в коде их нет).

1. Принципы

1. **Метка добровольная.** «Проверенная компания» никогда не требуется для приглашений, вакансий и квоты. Отсутствие метки — не вывод о компании и не штраф.
2. **Сигналы подсказывают, не наказывают.** Автоматически меняется только квота (мягко, обратимо); метку снимает и жалобы разбирает методист или администратор.
3. **Кандидат защищён.** Жалоба анонимна для компании: компания о ней не уведомляется и не видит автора.
4. **Всё объяснимо.** Работодатель видит, какие проверки не прошли, а причина сниженной квоты написана словами в его профиле.

2. Метка «Проверенная компания» (реализовано)

Компания вводит ИНН и ОГРН (`POST /api/companies/me/verification` , экран «Профиль компании»). Проверки (`app/trust/verification.py`):

Проверка	Как
Контрольная сумма ИНН (10 цифр — юрлицо, 12 — ИП)	Алгоритм ФНС: взвешенная сумма по модулю 11 и 10 (`app/trust/checksums.py`)
Контрольная сумма ОГРН (13 цифр) / ОГРНИП (15)	Остаток от деления первых 12 (14) цифр на 11 (13), последняя цифра
Компания найдена в реестре и ИНН с ОГРН относятся к одной записи	Учебный реестр ЕГРЮЛ
Статус «действующая»	Поле реестра
Название совпадает	Слова названия профиля против названия в реестре без «ООО/АО/ИП» и кавычек, порог `name_match_min_ratio`
ИНН не занят другой компанией	Один ИНН — одна проверенная компания (защита от выдачи себя за другого)
Домен корпоративной почты совпадает с доменом из реестра	Личные домены (gmail и т. п.) уровень «домен» не дают

Уровни: **«Реквизиты проверены»** (все проверки, кроме домена) и **«Домен подтверждён»** (плюс домен). Кандидат видит метку в приглашении, в списке и карточке вакансии и в «так видит кандидат»; реквизиты и детали проверок ему не показываются.

Реестр в MVP **учебный** (`app/trust/data/mock_egrul.json` , вымышленные компании); интерфейс `EgrulProvider` — точка подключения реального сервиса ФНС без правок остального кода. Переименование компании пересчитывает метку (нельзя получить метку и сменить название). Компания может сама убрать метку (`DELETE`). Методист или администратор отзывает метку с причиной (`POST`

`/api/admin/trust/companies/{id}/revoke`); пока отзыв не снят, подать реквизиты заново нельзя, но остальная работа компании не ограничена.

3. Жалобы кандидатов и модерация (реализовано)

- **Где пожаловаться:** на приглашение (`POST /api/invitations/{id}/report`) и на вакансию (`POST /api/vacancies/{id}/report`). Причины (`config/contact.yaml → reasons.complaint`): фиктивная вакансия, просят деньги или документы, спам, условия не соответствуют действительности, **сбор резюме без вакансии**, **зарплата не та**, **не отвечают после «да»**, другое. Одна жалоба от кандидата на объект.
- **Очередь модерации:** методист или администратор видит жалобы (`GET /api/admin/trust/complaints`, экран «Жалобы» в лаборатории), с числом разных кандидатов, пожаловавшихся на ту же компанию, и текущей меткой. Решение: **подтверждена** или **отклонена** (с заметкой).
- **Последствия:**
  - от **трёх разных кандидатов** за 90 дней открытых или подтверждённых жалоб — квота приглашений компании ×0,5 (`complaints.quota_threshold`, `quota_factor`); причина «жалобы кандидатов» показана компании в профиле. Отклонённые жалобы не считаются, поэтому квота возвращается сразу после разбора;
  - подтверждённая жалоба по серьёзной причине (фиктивная вакансия, просьба денег, сбор резюме) **снимает метку** «Проверенная компания»;
  - жалоба влияет на квоту вместе с надёжностью R: берётся меньший множитель, а не произведение.
- Старые жалобы (до миграции `0011_trust_minors`) перенесены в очередь автоматически.

4. Что уже защищало платформу (используется как есть)

Механизм	Что делает
Обязательная вилка и правила (`config/contact.yaml → salary`)	Нижняя граница не меньше минимума, `to/from` > 1,6 — предупреждение, > 2,0 — блокировка: вилка «от 10 до 1 000 000» невозможна
Условия неизменяемы	Каждая версия условий приглашения хранится со `sha256`; кандидат отвечает на конкретные условия, подмена после ответа видна
Квоты	60 новых приглашений за 7 дней на домен (на аккаунт для личной почты), 20 активных на бриф, лимит кандидата в неделю: массовая рассылка в пустоту не окупается, истёкшие без просмотра возвращают слот
Стоп-слова и ссылки	Плашка «Требует осторожности» у кандидата для просьб денег, документов, ссылок-сокращателей и контактов в тексте приглашения (`app/contact/antifraud.py`)
Надёжность R	Формула из журнала событий (ответ в срок, контакт после «да», «условия совпали»); расхождение условий у двух разных кандидатов за 90 дней — бейдж «есть расхождения условий» и квота ×0,5
«Условия совпали?» через 10 дней после «да»	Кандидат сообщает «зарплата ниже» или «вакансии нет» — это событие надёжности
Контакты кандидата	Раскрываются только после взаимного «да» и отдельного согласия: «сбор данных» без согласия невозможен

Сравнение зарплаты приглашения с вилкой брифа раньше не проверялось: работодатель мог указать в приглашении любую допустимую вилку. Теперь это автосигнал (п. 5), а не блокировка: у честного работодателя бриф и приглашение могут расходиться (новое решение по бюджету).

5. Автосигналы для модератора (реализовано)

`GET /api/admin/trust/flags`, вкладка «Автосигналы». Считаются на лету (`app/trust/flags.py`, пороги в `config/trust.yaml → flags`), ничего не блокируют:

Сигнал	Условие
Много приглашений, ни одного принятого	20 и более приглашений за 30 дней и ни одного «да»
Вакансия висит без откликов	Опубликована более 30 дней назад, откликов и принятых приглашений нет
Зарплата в приглашении ниже брифа	Нижняя граница первой версии условий ниже 0,9 от нижней границы брифа (на одном базисе «на руки / до НДФЛ»), не менее двух приглашений
Расхождение условий	Не менее двух сообщений кандидатов «условия не совпали» за окно (журнал надёжности R)

6. Что реализовано и что в плане

Область	Реализовано	В плане (после MVP)
Проверка компании	Контрольные суммы ИНН/ОГРН, учебный реестр, уровни метки, отзыв, пересчёт при переименовании	Реальный ЕГРЮЛ/ФНС-API; подтверждение права подписи (доверенность, выписка); проверка принадлежности домена (DNS-запись или письмо на `admin@домен`)
Жалобы	Приглашения и вакансии, очередь, решения, влияние на квоту и метку	Апелляция компании; обратная связь заявителю о результате; классификатор по тексту жалоб
Автосигналы	4 эвристики, подсказки модератору	Сигналы по скорости рассылки, повторяющимся текстам вакансий, перепродаже контактов; ранжирование очереди по риску
Фиктивные вакансии	Метка, квоты, вилка, стоп-слова, жалобы, висящие вакансии	Верификация вакансии через ATS (вакансия существует в системе найма — см. ATS_INTEGRATION.md); срок жизни вакансии с подтверждением актуальности; публичный «светофор» репутации
Наказание	Мягкое снижение квоты, снятие метки	Блокировка аккаунта по решению модератора с обжалованием; чёрный список реквизитов

7. Ограничения (честно)

- Учебный реестр не проверяет реальные компании: метка в демо демонстрирует механику и интерфейс.
- Совпадение домена не доказывает, что сотрудник вправе действовать от имени компании.
- Квота по жалобам — защита от массовой рассылки, но не от одного точечного обмана; для него нужны разбор человеком и санкции вне MVP.
- Порог «три разных кандидата» не защищает от сговора жалобщиков: поэтому жалобы только снижают квоту, а решающее слово — у модератора.

Модуль несовершеннолетних (согласие законного представителя) описан в SECURITY.md, п. 1.4.

Безопасность и персональные данные

Что и как защищено в «Взаимно»: соответствие 152-ФЗ на техническом уровне, роли и разграничение доступа, защита паролей и токенов, ограничение частоты запросов, приватность данных кандидата до раскрытия, шлюз LLM. Все утверждения сверены с кодом (`backend/app/core` , `app/api` , `app/contact` , `app/llm`) и тестами. Разделы «Известные ограничения» и «В разработке» — честный перечень того, что ещё не закрыто.

Документ описывает **техническую реализацию** требований. Организационные меры оператора (уведомление Роскомнадзора, политика обработки ПДн, приказы, договоры с хостингом и почтовым провайдером) находятся вне кода и должны быть выполнены организацией, развёртывающей платформу.

Содержание: 1. 152-ФЗ · 2. Роли и доступ · 3. Пароли и токены · 4. Ограничение частоты · 5. ПДн до раскрытия · 6. LLM-шлюз · 7. Секреты и инфраструктура · 8. Целостность теста · 9. Угрозы · 10. Ограничения

1. Персональные данные и 152-ФЗ

1.1. Какие данные обрабатываются

Группа	Состав	Где хранится	Кто видит
Учётные	email, хеш пароля (argon2id), роль, статус	`users`	Владелец; администратор БД
Профиль кандидата	ФИО, контакты, текущий работодатель, вуз, город, минимум зарплаты, настройки приватности	`candidate_profiles`	Владелец; работодатель — **только после взаимного «да»**, только разрешённые кандидатом поля
Опрос, тест	Ответы опроса, ответы теста, время ответов, события страницы (уход со вкладки, вставка)	`survey_responses` , `test_responses` , `session_events` , `flags`	Владелец; методист; работодатель — только итоговая категория и полоса
Резюме	Распознанные поля и текст резюме (файл PDF **не сохраняется**: разбирается в памяти, хранятся текст и sha256)	`candidate_profiles`	Владелец; работодатель — после «да» и только при разрешённом поле «Резюме»
ФСП	Привязка `fsp_id` (= `sub`), ответ реестра, нормализованные результаты	`fsp_links` , `fsp_achievements`	Владелец; работодатель — обобщённо до раскрытия, выписка после
Журналы	Согласия (с IP и user-agent), аудит действий	`consents` , `audit_log`	Владелец (журнал согласий); администратор
Не собираются	Биометрия (камера, микрофон), геолокация, возраст и пол в явном виде, паспортные данные	—	—

1.2. Требования и их реализация

Требование	Реализация
Согласие, раздельное и конкретное (ст. 9)	Шесть **отдельных** целей (`ConsentPurpose`): `processing` (обработка ПДн, при регистрации, обязательное), `blind_card` (показ слепой карточки работодателям), `fsp_enrichment` (данные ФСП), `contact_disclosure` (передача контактов **конкретной компании**), `test_logging` (журналирование событий теста), `calibration` (обезличенные ответы для калибровки заданий). Каждое согласие — своя запись `consents` с `purpose` , `text_version` , `granted_at` , `revoked_at` , `scope` , `ip` , `user_agent` . Тексты — `GET /api/consents/documents`
Версионирование текстов	Человек соглашается с тем текстом, который видел; устаревшая версия → `409 consent_outdated` . Старые согласия хранят свою версию

Требование	Реализация
Отзыв без потери доказательств	Запись не удаляется, ставится `revoked_at`; повторное согласие — новая запись. Отзыв цели без контекста гасит все действующие записи цели. Отзыв `contact_disclosure` гасит раскрытие у компании: контакты снова скрыты, приглашение переходит в `consent_revoked`, пара получает паузу 14 дней. Отзыв `fsp_enrichment` удаляет сохранённые данные реестра
Согласие на передачу контактов — контекстное	Выдаётся только в приглашении или отклике (`POST /accept`, `POST /api/applications`, `POST /grant-consent`), не через общий журнал (`400 consent_contextual`)
Журнал	`GET /api/consents`, `GET /api/candidates/me/consents` — все согласия, включая отозванные. Аудит значимых действий (вход, выдача токенов, согласия, раскрытия, вход через ФСП ID) — `audit_log`; в `payload` нет ПДн, только идентификаторы и коды причин
Минимизация и цель (ст. 5)	Работодатель получает слепую проекцию (раздел 5); поля раскрытия выбирает кандидат; ПДн реестра ФСП работодателю не передаются; из ФСП ID сохраняются только служебные claims
Локализация хранения (ст. 18, ч. 5)	Архитектура не содержит зарубежных зависимостей на пути ПДн: база данных, бэкенд, Keycloak и фронтенд разворачиваются на **серверах в РФ** (требование к развёртыванию, см. DEPLOY.md); шрифты и 3D-иллюстрации самохостятся, внешних CDN, счётчиков и аналитики во фронтенде нет; LLM по умолчанию выключена, поддерживаются российские провайдеры (YandexGPT, GigaChat) с маскированием ПДн; для SMTP следует выбирать провайдера с инфраструктурой в РФ. Исключение: страницы `/api/docs` и `/api/redoc` подгружают скрипты с `cdn.jsdelivr.net` (ПДн туда не передаются; на проде рекомендуется отключить документацию или раздавать статику локально)
Автоматизированные решения (ст. 16)	Платформа не принимает юридически значимых решений о человеке без участия человека: Score — ранжирование для работодателя, решение о приглашении и найме принимает человек; грейд — «решение с уровнем доверия», при подозрении сессия уходит методисту; автоматического бана нет; LLM не участвует в тесте, ключах, Score, ранжировании и вердиктах. Кандидат видит свои флаги и может оспорить (`POST /api/tests/flags/{id}/dispute`)
Права субъекта (ст. 14)	Доступ и исправление: профиль, настройки, журнал согласий; отзыв согласий — в журнале; отвязка ФСП ID с удалением данных — `DELETE /api/fsp/me`; удаление резюме — `DELETE /api/candidates/me/resume`. **Удаление аккаунта целиком** — `DELETE /api/candidates/me` (кандидат) и `DELETE /api/companies/me` (работодатель) с подтверждением паролем (для аккаунта без пароля — слово «УДАЛИТЬ»), п. 1.3
Меры защиты (ст. 19)	Argon2id, TLS на периметре, роли и контроль владельца записи, шифрование ключей заданий, HMAC-псевдонимы, rate limit, аудит, непривилегированный контейнер (разделы 2–8)
Несовершеннолетние	Младше 15 лет регистрация отклоняется; 15–17 лет — профиль не виден работодателям, пока законный представитель не подтвердит согласие по ссылке из письма; после этого — только стажировка или лёгкий труд на неполный день; год рождения из реестра ФСП важнее заявленного; п. 1.4
Биометрия	Камеры, микрофона и сканера процессов в тесте нет сознательно: это биометрия и инвазивный контроль
Сроки хранения	Ночная задача планировщика (`config/privacy.yaml`, `config/scheduler.yaml`): неподтверждённые аккаунты удаляются через 7 дней, просроченные коды и токены чистятся, IP в журнале аудита (180 дней) и в записях согласий (365 дней) обезличиваются; п. 1.3

1.3. Удаление аккаунта и сроки хранения

Удаление по запросу (`app/services/account/erasure.py`, `config/privacy.yaml`). `DELETE /api/candidates/me` и `DELETE /api/companies/me` принимают `{"password": "…"}`; аккаунт без пароля (вход через ФСП ID без заданного пароля) подтверждается словом «УДАЛИТЬ» в `confirm`. Неверный пароль — `403 password_invalid`, аккаунт не тронут; попытки считаются лимитом (5 в час на аккаунт), поэтому удаление нельзя использовать для подбора пароля. Что происходит:

1. Строка `users` удаляется одним запросом. Все внешние ключи на `users` и `companies` объявлены `ON DELETE CASCADE` или `SET NULL` (проверяет тест `test_every_foreign_key_to_users_and_companies_cascades_or_nulls`), поэтому база сама убирает профиль и резюме, ответы опроса, признаки подбора, сессии теста и ответы, согласия, привязку ФСП ID и данные реестра, приглашения, отклики, раскрытия контактов, уведомления, коды и refresh-токены — все сессии на всех устройствах перестают работать, cookie очищается.

2. Что не связано внешним ключом, чистится явно: тексты обратной связи по приглашениям и откликам кандидата, контакты кандидата в копиях вебхуков ATS (журнал доставок компании), IP и ссылки на человека в записях журнала аудита.
3. Остаётся только то, что не относится к человеку: запись журнала аудита «аккаунт удалён» (роль, дата, способ — без ПДн), счётчики надёжности компаний (`reliability_events` без ссылки на кандидата и приглашение), откалиброванные параметры заданий.
4. Работодатель: удаляется его аккаунт. Если он был последним в компании — ждущие приглашения отзываются («позиция закрыта», кандидат получает уведомление), брифы и вакансии закрываются, контакты связи и вебхук ATS стираются; название компании (не ПДн) остаётся в истории приглашений кандидатов.
5. На почту уходит письмо-подтверждение; адрес нигде не сохраняется. Тот же email можно зарегистрировать заново.

Сроки хранения (`config/privacy.yaml → retention` ; ночная задача `retention` встроенного планировщика, см. «Планировщик» в ARCHITECTURE.md): неподтверждённые аккаунты старше 7 дней удаляются целиком; сожжённые коды подтверждения (2 дня), просроченные и отозванные refresh-токены (30 дней), брошенные входы через ФСП ID (1 день) — чистятся; IP в журнале аудита (180 дней) и IP/user-agent в записях согласий (365 дней) обезличиваются (поле → `NULL` , сама запись и версия текста остаются). Сроки хранения основных данных активных аккаунтов определяет оператор в политике обработки (организационная мера).

Минимизация данных реестра ФСП. Из ответа реестра сохраняется только то, что платформа использует: `sub` и год рождения спортсмена (политика по несовершеннолетним), результаты и разряды без номера приказа и без `sub` других участников команд. ФИО, дата рождения целиком, регион и организация не сохраняются (`app/services/fsp_link/minimize.py` , `config/privacy.yaml → fsp_registry`).

1.4. Несовершеннолетние 15–17 лет: согласие законного представителя

Ответы организаторов: с 15 лет допустим только лёгкий труд, согласие законного представителя на обработку ПДн нужно с 15 до 17 лет включительно. Все пороги — `config/contact.yaml → minors` , в коде чисел нет.

Что делает платформа:

1. **Возраст.** Хранится только год рождения (`candidate_profiles.birth_year`); возраст считается осторожно, «минимально возможный» = текущий год − год рождения − 1 (при сомнении считаем моложе). Год спрашивается при регистрации кандидата (`birth_year` в `POST /api/auth/register`) и в профиле (`PUT /api/candidates/me`). Если год рождения подтверждён реестром ФСП, заявленный не заменяет его.
2. **Младше `registration_min_age` (15 лет).** Регистрация отклоняется кодом `422 age_too_young` с объяснением; аккаунт и профиль не создаются, ничего не сохраняется. То же при попытке указать такой год в профиле.
3. **15–17 лет (`guardian_consent_below_age = 18`).** Кандидат указывает ФИО и email представителя (`POST /api/candidates/me/guardian`). На этот адрес уходит письмо (в разработке видно в Mailpit) со ссылкой на `/guardian-confirm?token=…` . Токен одноразовый, действует 7 дней (`token_ttl_hours`), в базе лежит только его SHA-256; повтор письма на тот же адрес — не чаще раза в 10 минут. Представитель подтверждает согласие без входа (`POST /api/guardian/confirm` , ограничение частоты `rl_guardian_ip`). Email представителя не может совпадать с адресом кандидата. В письме email кандидата показан с маской.
4. **До подтверждения** профиль не попадает в подборки и банк (`age_ok` в признаках подбора), прямое приглашение невозможно (`candidate_unavailable` , причина работодателю не раскрывается), в «Приватности» кандидат видит причину «нужно согласие законного представителя».
5. **После подтверждения** кандидат виден только брифам с флагом «стажировка / лёгкий труд». В его слепой карточке есть плашка «Кандидату 15–17 лет: только стажировка или лёгкий труд, неполный день» (возраст и год рождения не раскрываются никогда); приглашение не на стажировку отклоняется с `minor_internship_only` и понятным текстом.
6. **Смена представителя** сбрасывает прежнее согласие. Достижение 18 лет (расчёт по году) снимает ограничения; при смене года рождения на совершеннолетний данные представителя стираются.

7. **Аудит.** Запрос письма (`guardian.requested`) и подтверждение (`guardian.confirmed`) пишутся в журнал аудита без ПДн.

Что остаётся организационным (не в коде): проверка того, что указанный адрес действительно принадлежит родителю или опекуну, платформа технически не проверяет (email не доказывает родство) — это допущение MVP; промышленный режим потребует подписанного согласия (УКЭП или Госуслуги) и отзыва согласия представителем без обращения в поддержку. Отзыв сейчас — через удаление аккаунта (п. 1.3) или обращение к оператору.

2. Роли и разграничение доступа

Четыре роли (`candidate` , `employer` , `methodist` , `admin`). Роль и статус аккаунта проверяются **по БД на каждом запросе**: блокировка и смена роли действуют сразу, не дожидаясь истечения токена. Доступ к маршрутам задаётся зависимостью `require_roles(…)` ; нарушение — `403 role_forbidden` .

Область	Кандидат	Работодатель	Методист / админ
Профиль, опрос, тест, ФСП, согласия, задачи (`/api/survey` , `/tests` , `/candidates/me` , `/fsp/me` , `/tasks/me`)	свои данные	—	—
Бриф, подборка, банк, шортлист, приглашения, вакансии, задачи компании, профиль компании	—	данные **своей компании**	—
Ответы на приглашения, отклики, просмотр вакансий	да	—	—
Надёжность любой компании, уведомления, журнал согласий	да	да	да
Лаборатория (`/api/lab/*`): банк и клоны, трасса, флаги, калибровка, валидация	—	—	да
Администрирование (`/api/admin/*` : импорт жюри, ранжирование, метрики, валидация)	—	—	только `admin`
Демо-режим (`/api/demo/*` , только при `VZ_DEMO_MODE=true` , иначе 404)	вне `VZ_ENV=prod` — любая роль для часов, сброса, автоответчика и режима жюри; в prod эти общие кнопки — только `admin` (или заголовок `X-Demo-Key`), п. 10	то же	то же

Защита от IDOR (`app/core/access.py`): запись, принадлежащая другому пользователю, **неотличима от несуществующей — 404, а не 403**, иначе по кодам ответа можно было бы перебирать чужие идентификаторы. `ensure_owner` / `get_owned_or_404` сравнивают владельца записи с `user_id` или `company_id` пользователя; некорректный UUID тоже даёт 404. Для работодателя кандидаты адресуются только непрозрачной ссылкой `ref` , которая вычисляется HMAC и разрешается **только в рамках его компании** (`candidate_aliases`): угадать `ref` другой компании нельзя. Дополнительные проверки: пригласить можно только того, чью карточку открывали или кто в шортлисте; полный PDF-профиль работодателю — только при действующем раскрытии (иначе 404); кандидату доступны только его собственные сессии теста, флаги и разбор.

Проверяется тестами `tests/test_platform_access.py` , `tests/test_platform_security.py` , `tests/test_platform_password.py` , `tests/test_platform_auth.py` , `tests/test_api_contact_privacy.py` , `tests/test_api_needs_privacy.py` , `tests/test_api_profile_privacy.py` .

3. Пароли и токены

3.1. Пароли

Мера	Параметры
Хеширование	**argon2id** (`argon2-cffi`): время 3, память 64 МиБ, параллелизм 4 (`VZ_ARGON2_*`); автоматический перехеш при изменении параметров
Политика	8–128 символов, хотя бы одна буква и одна цифра
Защита от перебора и перерасхода памяти	Не больше 4 хешированый одновременно (`VZ_ARGON2_MAX_CONCURRENT`): иначе пачка входов выела бы память контейнера (по 64 МБ на хеш)
Защита от выяснения аккаунтов	Для неизвестного email пароль всё равно хешируется (одинаковое время ответа); ответ входа одинаков для неизвестного email и неверного пароля; статус аккаунта проверяется **после** пароля; ответ восстановления пароля не выдаёт, есть ли аккаунт
Восстановление	Код на email (6 цифр, 15 минут, 5 попыток), после смены пароля **все сессии пользователя завершаются**

3.2. Токены и коды

Элемент	Реализация
Access-токен	JWT (PyJWT), HS256, срок 15 минут; обязательные claims `exp` , `iat` , `sub` , `type` , `iss` ; алгоритм задан списком (защита от `alg=none` и подмены алгоритма); тип `access` проверяется. Хранится в памяти клиента, не в cookie и не в `localStorage`
Refresh-токен	Непрозрачные 256 бит (`secrets.token_urlsafe(32)`), в БД только **sha256**. Cookie `vz_refresh` : `HttpOnly` , `Path=/api/auth` , `SameSite=Lax` (`Secure` обязателен в проде)
Ротация и обнаружение кражи	Каждый `refresh` выдаёт новый токен той же цепочки (`family_id`), старый помечается использованным. Повторное предъявление использованного токена — цепочка отзывается (`401 refresh_reused`); повтор в пределах 10 с трактуется как гонка вкладок (`409 refresh_superseded`). `logout` отзывает всю цепочку устройства. Access-токен после выхода живёт до истечения (до 15 минут), но сервер сверяет статус аккаунта по БД
Коды email	6 цифр от `secrets.randbelow` ; в БД — **HMAC-SHA256** от пары « `user_id` , код» ключом `VZ_APP_SECRET` (утечка таблицы не раскрывает коды: перебор 10^6 без секрета не сходится); сравнение `hmac.compare_digest` ; новый код гасит предыдущие; 5 попыток на код; повторная отправка не чаще раза в 60 с и 5 писем в час
CSRF	Состояние меняется только запросами с заголовком `Authorization: Bearer` , которого межсайтовая форма отправить не может; единственная cookie (`vz_refresh`) ограничена путём `/api/auth` и `SameSite=Lax` . Страница сверки `/api/v/{code}` и `GET` -маршруты состояния не меняют
Заголовки	`X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , `Referrer-Policy: strict-origin-when-cross-origin` , `Cache-Control: no-store` на `/api/auth/*` . HSTS и TLS задаёт Caddy (см. DEPLOY.md); CSP интерфейса — `frontend/csp.conf` , подключается nginx образа `web` (ENGINEERING.md): `default-src 'self'` , скрипты только `'self'` , `'unsafe-inline'` лишь для стилей (React и Tailwind задают встроенные), `object-src 'none'` , `frame-ancestors 'none'` ; на `/api/` политика не накладывается (Swagger UI бэкенда использует встроенные скрипты)
Вход через ФСП ID	OIDC authorization code + PKCE S256; `state` (в БД только sha256, одноразовый), `nonce` , привязка к браузеру cookie `vz_fsp_flow` ; проверка подписи id_token по JWKS, `iss` , `aud` , `nonce` , срока; HS-алгоритмы и `none` запрещены; email из ФСП ID не даёт права на чужой аккаунт; защита от «предзахвата» аккаунта (см. FSP_INTEGRATION.md)

4. Ограничение частоты запросов

Токен-корзина (`app/core/rate_limit.py`): ключ — email, IP или аккаунт; превышение — `429` с `Retry-After` . Лимит проверяется **до** проверки пароля: перебор упирается в 429 независимо от результата.

Область	Лимит
Вход	5 за 300 с на email, 30 за 300 с на IP

Область	Лимит
Регистрация	10 в час на IP
Ввод кода, новый пароль	10 за 900 с на email
Повторная отправка кода, запрос восстановления	20 в час на IP
Обновление сессии	60 в минуту на IP
Вход через ФСП ID, демо-вход	30/300 с, 30/300 с, 20/300 с на IP
Тест: создание сессии, `next`, ответ, события	10 в час, 120, 60, 120 в минуту на аккаунт
Резюме, PDF, сверка по QR	10 в час; 30 за 600 с; 120 в минуту (QR — на IP)
Подбор, приглашения, отклики, задачи, вебхук	30, 30, 10, 20, 5 в минуту

Полная таблица — в API.md. Квоты бизнес-уровня (антифейк): не больше 20 активных приглашений на бриф, 60 новых на домен за 7 дней (для личных почтовых доменов — на аккаунт), пачка до 10, лимит кандидата 3/5/10/∞ в неделю, пауза пары, стоп-слова («предоплата», паспорт/СНИЛС до собеседования, контакты в тексте, короткие ссылки) с плашкой «Требует осторожности».

Ограничение: корзины лежат в памяти процесса. Один процесс uvicorn — лимит точный; при нескольких воркерах он действует на воркер.

Клиентский IP за обратным прокси. Приложение само решает, чему верить (`app/core/netutil.py`, `app/core/rate_limit.client_ip`): заголовок `X-Forwarded-For` принимается **только если TCP-сосед — доверенный прокси** из `VZ_FORWARDED_ALLOW_IPS` (то же имя, что у uvicorn: читается и `FORWARDED_ALLOW_IPS`; список адресов и подсетей через запятую, по умолчанию `127.0.0.1`). Цепочка читается справа налево: первый недоверенный адрес — настоящий клиент, всё левее подделывается клиентом и игнорируется; мусор в заголовке — откат к адресу соседа. `*` разрешён только вне prod. Uvicorn запускается с `--no-proxy-headers`, чтобы правило было одно. В схеме с Caddy (docker-compose.prod.yml): подсеть compose зафиксирована (`172.29.80.0/24`) и указана в `VZ_FORWARDED_ALLOW_IPS`, Caddy перезаписывает `X-Forwarded-For` адресом клиента (`header_up X-Forwarded-For {remote_host}`), бэкенд наружу не публикуется. Без этой настройки все пользователи за прокси выглядели бы как один IP и делили лимиты, а с `*` любой, кто дотянулся бы до порта, выбирал бы себе IP сам. Проверяется `tests/test_hardening_api.py`.

5. ПДн кандидата до раскрытия

Работодатель до взаимного «да» (или отклика) получает **слепую проекцию** `CandidateBlind`; контакты и разрешённые поля — `CandidateDisclosed` только при действующей записи `ContactDisclosure` этой пары. Решает не вызывающий код, а согласие: `services.contact.profiles` выбирает проекцию по действующему раскрытию.

Мера	Реализация
Белый список полей	Матрица `config/contact.yaml → visibility`: поле, которого в ней нет, не попадает ни в одну проекцию. Возраст, пол, дата рождения, внутренний id — «никогда» в обеих проекциях; конфиг, нарушающий это, не загружается
Вычищение ПДн в глубину	В слепой проекции ПДн-ключи (`full_name`, `email`, `phone`, `telegram`, `resume`, `city`, `fsp_id`, `ekp`, `team`, `protocol_url`, … — список `pii_fields`) удаляются на любой вложенности
Псевдоним и ссылка	«К-4821» = `HMAC(VZ_PSEUDONYM_SECRET, компания ‖ кандидат)`; `ref` — отдельный HMAC; свои у каждой компании: две компании не сопоставят карточки одного человека
Доказательства	Внутренние id попыток теста, задач, записей ФСП наружу не уходят: непрозрачные HMAC-ссылки в рамках компании; номера ЕКП, команды и ссылки на протоколы — только после раскрытия
Обобщение	Тест — полоса «топ-10/25/50 %» и интервал с шагом 10 баллов; ФСП — дисциплина, уровень, год, полоса места; зарплата — «в вилке ✓ / выше» (превышение округляется до полосы 5 %, чтобы точный минимум не восстанавливался из Score); охват вилки — при N ≥ 10 и ступенями не меньше 3 человек

Мера	Реализация
Свободный текст	Комментарии кандидата, уходящие до раскрытия, проверяются на контакты и ссылки
PDF-профиль	Полная версия работодателю — только при действующем раскрытии и с теми полями, что разрешил кандидат (иначе 404); слепая версия — псевдоним вместо имени. Страница сверки `/api/v/{code}` — публичная, без ПДн, `X-Robots-Tag: noindex`, `Cache-Control: no-store`; код — HMAC содержимого листа, изменённый PDF не пройдёт сверку
Вебхуки ATS	Подпись HMAC-SHA256 (`X-Vzaimno-Signature`), идемпотентный `X-Vzaimno-Event-Id`; контакты кандидата в событии — только при действующем согласии; защита от SSRF: URL проверяется при сохранении и перед каждой доставкой, внутренние, частные, link-local адреса, метаданные облака и служебные зоны запрещены
Контрактные тесты	«0 ПДн-ключей в ответах работодателю до согласия» — `tests/test_contact_privacy.py`, `tests/test_api_contact_privacy.py`, `tests/test_api_needs_privacy.py`, `tests/test_api_profile_privacy.py`

6. LLM-шлюз с маскированием

LLM **выключена по умолчанию** (`VZ_LLM_PROVIDER=none`). Допустима только через шлюз `app/llm/gateway.py`, и только в разборе брифа и резюме; в тесте, ключах, Score, ранжировании, объяснениях и вердиктах она архитектурно запрещена (тест `test_ranking_path_never_imports_llm`).

Мера	Реализация
Маскирование ПДн **до** вызова	`app/llm/masking.py`: email, телефоны, ссылки на профили и Telegram, строки с датой рождения и возрастом, ФИО (найденные правилами) заменяются на `*`; **длина текста сохраняется**, поэтому смещения цитат совпадают с исходным текстом
Только добавление, только с цитатой	Каждое поле от модели обязано прийти с цитатой, найденной в тексте (`rapidfuzz.partial_ratio ≥ 90`); поле без цитаты отбрасывается. LLM не может изменить результат правил, только дополнить
Детерминированный fallback	Цепочка YandexGPT → GigaChat → «без LLM»; сбой, таймаут (общий бюджет 20 с) или пустой ответ — результат правил не меняется
Кэш и воспроизводимость	Кэш по `sha256` замаскированного текста; режимы `record` и `replay` (кассета JSON без сети и ключей): в кассете нет текста запроса и ПДн, только ключ sha256 и ответ модели на замаскированный текст
Секреты	Ключи провайдеров — `SecretStr` из окружения; в логи не пишутся
Юрисдикция	Поддерживаются российские провайдеры; GigaChat — с проверкой TLS (корень НУЦ Минцифры из поставки, `VZ_LLM_GIGACHAT_VERIFY=bundled`)

7. Секреты и инфраструктура

Тема	Реализация и требования
Секреты	`VZ_JWT_SECRET` (подпись access-JWT), `VZ_APP_SECRET` (HMAC кодов email, ключ шифрования клонов, секретные seed клонов, подпись демо-токена ФСП), `VZ_PSEUDONYM_SECRET` (псевдонимы и `ref`). В `VZ_ENV=prod` бэкенд **не стартует** с dev-значениями, секретами короче 32 символов и без `VZ_COOKIE_SECURE=true`. Генерация: `python -c "import secrets; print(secrets.token_urlsafe(48))"`
Последствия смены секретов	`VZ_JWT_SECRET` — все пользователи выходят; `VZ_APP_SECRET` — **пул клонов перестаёт расшифровываться** (его нужно пересоздать), действующие коды email теряют силу; `VZ_PSEUDONYM_SECRET` — новые псевдонимы и ссылки (выданные ранее сохранены в `candidate_aliases`). Секреты хранить в менеджере секретов или защищённом `.env` (файл `.env` в git не попадает)
Ключи заданий	`item_instances.key_enc` — Fernet (AES-128-CBC + HMAC), ключ шифрования выводится из `VZ_APP_SECRET` отдельной меткой; дамп БД без секрета ключи не раскрывает. Seed боевых клонов — `HMAC(VZ_APP_SECRET, семейство, номер)` в `[2^50, 2^62)`
Контейнер	`python:3.12-slim`, процесс от непривилегированного пользователя (uid 10001, без shell), зависимости строго по `uv.lock`, healthcheck, лимиты памяти. Прод-схема (`docker-compose.prod.yml`): БД и бэкенд наружу не публикуются, образы закреплены по версиям, Keycloak — в режиме `start` (не `start-dev`) со своей БД

Тема	Реализация и требования
Сеть	В проде наружу публикуется только HTTPS (Caddy, порты 80 и 443). Порты PostgreSQL (5432), Mailpit (8025, 1025), `fsp-mock` (8001), Keycloak (8080) **не публиковать**: в `docker-compose.yml` они открыты для удобства разработки
Keycloak	Профиль `fsp` запускает `start-dev` — только для демо. В проде ФСП ID — внешний сервис ФСП; собственный Keycloak, если нужен, запускается в режиме `start` с TLS, настоящим hostname и закрытой админкой
Реестр ФСП	В демо — статический bearer-токен (`FSP_MOCK_TOKENS` , сравнение за постоянное время); в проде — client credentials сервисного аккаунта
Загрузка резюме	Только PDF, до 5 МБ и 10 страниц, разбор в памяти, файл не сохраняется
Логи	`audit_log` без ПДн; почтовый бэкенд `console` пишет письма (с кодами) в лог — только для разработки
Зависимости	Версии зафиксированы `uv.lock` и `package-lock.json` ; список — THIRD_PARTY.md. Автоматический аудит уязвимостей зависимостей в CI — в разработке

8. Целостность теста и ключи заданий

Защита результатов теста от подделки описана в TESTING_MECHANISM.md. С точки зрения безопасности данных важно:

- условие задания отдаётся без ключа и тегов дистракторов; разбор с ключом — только по своему экземпляру и после решения;
- журнал событий страницы теста пишется **только с согласием** `test_logging` ; камеры и микрофона нет;
- флаги целостности работодателю не показываются; кандидат видит свои без источника и может оспорить; решение по сессии на проверке принимает методист, автоматического бана нет;
- «Антиподсказчик» формулирует улики фактами, без обвинений.

9. Модель угроз (кратко)

Угроза	Меры
Подбор паролей, перебор кодов	Лимиты до проверки пароля, argon2id, 5 попыток на код, блокировка аккаунта админом
Перебор чужих идентификаторов (IDOR)	404 вместо 403, `ref` по HMAC в рамках компании, проверка владельца на каждой записи
Кража refresh-токена	httpOnly, хеш в БД, ротация, обнаружение повторного использования, завершение цепочки
Утечка БД	Хеши паролей argon2id, HMAC кодов, sha256 refresh, зашифрованные ключи клонов; ПДн открытым текстом — в профиле (поля раскрытия), поэтому доступ к БД и бэкапам ограничивается организационно
Деанонимизация кандидата	Слепая проекция, обобщение, HMAC-псевдонимы, округление зарплатных сигналов, ограничение охвата, вычищение ПДн-ключей
Накрутка и спам работодателей	Квоты, лимит кандидата, пауза пары, стоп-слова, надёжность R с видимой разбивкой, жалобы
SSRF через вебхук ATS	Проверка URL и разрешённых адресов при сохранении и доставке
Слив заданий	Одноразовые клоны, секретные seed, шифрование ключей, детектор «чужого ключа», CUSUM
Утечка ПДн в LLM	Маскирование до вызова, выключено по умолчанию, кассета без текста запроса
Захват аккаунта через ФСП ID	PKCE, `state` привязан к браузеру и аккаунту, подтверждение email, запрет входа в чужой аккаунт по совпавшему email
SQL-инъекции	Запросы строит SQLAlchemy с параметрами; ручные `text()` -запросы используют связанные параметры

Угроза	Меры
HTML-инъекции в письмах и PDF	Динамические значения в HTML-письмах экранируются (`html.escape`); PDF рисуется векторно через reportlab, пользовательский HTML не вставляется

10. Известные ограничения и «в разработке»

Тема	Состояние
Согласие законного представителя 15–17 лет	Реализовано (п. 1.4); ограничение MVP — адрес представителя не сверяется с документом, отзыв согласия представителем — через поддержку или удаление аккаунта
Rate limit при нескольких воркерах	Корзины в памяти процесса; общая корзина (Redis или PostgreSQL) — в разработке
CSP интерфейса	Реализовано: `frontend/csp.conf` (см. строку «Заголовки» и ENGINEERING.md); остаток — `'unsafe-inline'` для стилей и отсутствие CSP на `/api/docs` ; HSTS и TLS — на Caddy
Subresource Integrity	Не нужна: внешних скриптов и стилей нет, всё отдаётся с того же origin
Аудит зависимостей	Реализован в CI: `pip-audit` и `npm audit --omit=dev` (ENGINEERING.md)
`/api/docs` , `/api/redoc`	Ассеты Swagger UI и ReDoc лежат в `backend/app/static/docs` и отдаются самим бэкенд-ом (`/api/docs-assets` , лицензии рядом): ни CDN, ни Google Fonts
Регистрация показывает занятость email	`409 email_taken` для подтверждённого адреса (вход и восстановление пароля занятость не выдают)
Access-токен не отзывается до истечения	До 15 минут; статус и роль проверяются по БД, блокировка действует сразу
Секреты вебхуков ATS	Ключ HMAC компании хранится в БД открыто (нужен для подписи); ротация — `POST /api/companies/me/webhooks/rotate-secret`
Вебхуки: смена DNS между проверкой и соединением (TOCTOU)	Остаточный риск, принят для MVP
Хранение ответа реестра ФСП	Только при согласии и только нужные поля: `sub` и год рождения спортсмена, результаты без `sub` других участников команд, разряды без номера приказа (`app/services/fsp_link/minimize.py` ; старые строки дочищает ночная задача)
Резервное копирование и мониторинг	Не входят в репозиторий; настраиваются при развёртывании
Кандидат без доступа к почте	Единственный канал подтверждения — email; код приходит только на него
Демо-режим на боевом стенде	`VZ_DEMO_MODE=true` открывает `/api/demo/*` : «режим жюри» показывает верные ответы на задания, `GET /api/demo/accounts` перечисляет демо-логины (без паролей), пароли демо-аккаунтов выводятся из `VZ_APP_SECRET` . Общие кнопки (часы, сброс, автоответчик, переключатель режима жюри) в `VZ_ENV=prod` доступны только роли `admin` или заголовку `X-Demo-Key` (`VZ_DEMO_CONTROL_KEY`); явная настройка — `VZ_DEMO_CONTROLS_ADMIN_ONLY` . Включать `VZ_DEMO_MODE` **только** на стенде для жюри; в боевом режиме держать `false` (тогда `/api/demo/*` — 404).

Использование LLM

Короткий ответ: **платформа полностью работает без LLM и без ключей** (так работает демо-стенд жюри). Языковая модель — необязательная надстройка в двух местах разбора текста и один офлайн-инструмент авторов банка заданий. В тесте, оценке, ранжировании и объяснениях её нет и быть не может.

Организаторы разрешили облачные LLM (в том числе зарубежные; предпочтительны открытые модели) при условии, что команда документирует, какие модели, как и зачем используются, и берёт на себя риски 152-ФЗ, если ПДн уходят в модель. GPU на стенде жюри нет, поэтому локального инференса в поставке нет: модель — всегда внешний API.

1. Где LLM используется

#	Место	Что делает модель	Режим по умолчанию	Если модели нет
1	Разбор брифа работодателя (`app/services/matching/llm_brief.py` , задача `brief_skills`)	Только **добавляет** навыки, которые пропустили правила разбора; вид навыка всегда «желательный»	Выключено	Бриф разбирают правила; результат тот же, что в демо
2	Разбор резюме кандидата (`app/services/resume/parser.py` , задача `resume`)	Только **добавляет** навыки, роли, soft skills, заявленный грейд, которых не нашли правила	Выключено	Работают правила
3	Офлайн, у авторов банка: черновики генераторов семейств заданий	«LLM пишет генератор, а не задание» (см. §4)	Не часть продукта	—

Общие гарантии для пунктов 1 и 2 (шлюз `app/llm/gateway.py`):

- **Только добавление и только с цитатой.** Каждое поле от модели обязано прийти с дословной цитатой из текста (`rapidfuzz.partial_ratio ≥ 90` , `VZ_LLM_MIN_QUOTE_RATIO`). Поле без цитаты, с чужим названием или вне справочника навыков отбрасывается. Результат правил модель изменить не может.
- **Чипы помечены источником**: «правило», «вручную» или имя провайдера; для резюме все элементы — серые «заявлено», подтверждением не считаются.
- **Детерминированный fallback.** Цепочка провайдеров → «без LLM»: сбой, таймаут (общий бюджет 20 с на разбор), невалидный JSON или промах кассеты дают тот же результат, что и выключенная модель. Исключения провайдера в разбор не пробрасываются.
- **Воспроизводимость.** Кэш по `sha256(версия промпта + задача + замаскированный текст)` . Режимы `VZ_LLM_MODE` : `record` (провайдеры плюс запись ответов в JSON-кассету), `replay` (только кассета, без сети и ключей — демо с LLM воспроизводимо). В кассете нет текста запроса, только ключ `sha256` и ответ модели на замаскированный текст.
- **Температура 0** у всех провайдеров (у GigaChat 0,01: строгий 0 он не принимает).

2. Где LLM запрещена и почему

Запрещена в: выдаче и ключах заданий, сложности `b` , адаптивном выборе и остановке теста, определении грейда и категории, Score, ранжировании подборки, объяснениях «почему выше», вердиктах «Антиподсказчика», раскрытии контактов.

Причины:

1. **Сопоставимость.** Результат теста должен быть сравним между людьми и между попытками. Задание «под каждого» от модели не имеет известной сложности, а ключи модели ошибаются: в симуляции каждые 5 %

битых ключей заметно снижают долю верных решений (VALIDATION.md §2.5; долю ошибок у реальных моделей мы не измеряли). Ключи считает исполнение кода генератора, не модель.

2. **Воспроизводимость и аудит.** Подборка и объяснения обязаны быть повторяемыми и проверяемыми (в Score «сумма вкладов = итог» проверяется тестом); недетерминированная модель в этой цепочке разрушает проверяемость.
3. **152-ФЗ, ст. 16.** Решение о человеке не принимается автоматически без человека; модель не должна влиять на то, кого показывают работодателю.
4. **Резюме не влияет ни на категорию, ни на Score** (правило продукта): даже то, что модель извлекла из резюме, остаётся серым «заявлено».

Как это защищено в коде и тестах:

- Модули Score, ранжирования, объяснений не импортируют `app.llm`: `tests/test_api_needs_review.py::test_ranking_path_never_imports_llm`; в `app/testing` и `app/services/testing` импортов `app.llm` нет.
- **Поведенческий тест** `tests/test_llm_boundaries.py`: шлюз подменён на «включённый» с записью каждого обращения (и провайдеры тоже перехвачены). Полная сессия адаптивного теста с результатом, сборка версии подборки, выдача, карточка кандидата — **0 обращений**. Единственное допустимое обращение на стороне работодателя — `brief_skills` при разборе брифа.

3. Какие модели

Шлюз поддерживает три вида провайдеров; цепочка при `VZ_LLM_PROVIDER=auto` идёт в порядке ниже, неподключённые пропускаются.

Провайдер	Значение `VZ_LLM_PROVIDER`	Модель по умолчанию	Для чего	Юрисдикция
YandexGPT (Foundation Models API)	`yandexgpt`	`yandexgpt-lite/latest`	Основной для боевой установки: данные остаются в РФ	РФ
GigaChat	`gigachat`	`GigaChat` (scope `GIGACHAT_API_PERS`)	Проверен вживую (§9). TLS проверяется по корневому сертификату НУЦ Минцифры из поставки (`VZ_LLM_GIGACHAT_VERIFY=bundled`, умолчание)	РФ
OpenAI-совместимый API	`openai_compat`	`qwen/qwen3-235b-a22b-2507` (меняется `VZ_LLM_OPENAI_MODEL`)	**Открытые модели** (Qwen, Llama, DeepSeek, gpt-oss) у облачных хостингов — OpenRouter, Together, Fireworks и им подобных; собственный vLLM/Ollama, если появится GPU; при необходимости зарубежные API	Зависит от хостинга

Зачем именно эти: задача модели узкая (извлечь названия технологий с цитатой из короткого текста), поэтому нужна не мощная, а дешёвая и быстрая модель с хорошим русским; точность держит проверка цитаты, а не размер модели. Открытая модель через OpenAI-совместимый адрес даёт предпочтительный организаторами вариант без GPU на нашей стороне и без привязки к вендору.

TLS для GigaChat

Серверы Сбера (`ngw.devices.sberbank.ru:9443`, `gigachat.devices.sberbank.ru`) подписаны цепочкой НУЦ Минцифры, которой нет в стандартных хранилищах. В поставке лежит корневой сертификат `backend/app/llm/certs/russian_trusted_root_ca.pem` («Russian Trusted Root CA», действует до 27.02.2032), проверка включена всегда:

- SHA-256: `D2:6D:2D:02:31:B7:C3:9F:92:CC:73:85:12:BA:54:10:35:19:E4:40:5D:68:B5:BD:70:3E:97:88:CA:8E:CF:31`

(SHA-1 `8F:F9:15:CC:AB:7B:C1:6F:8C:5C:80:99:D5:3E:0E:11:5B:3A:EC:2F`). Файл скачан с официального CDN госуслуг (`gu-st.ru`, страница gosuslugi.ru/crt) двумя путями (отдельный файл корня и официальный комплект `russiantrustedca.pem`) — отпечатки совпали; отпечаток закреплён тестом `tests/test_llm_gigachat.py`. Сам текст отпечатка на странице госуслуг читается только скриптом, поэтому сверка с «опубликованным значением» сделана криптографически: подчинённый сертификат подписан этим корнем, а цепочка живых серверов Сбера (`openssl s_client -CAfile`) проверяется с кодом 0 для обоих хостов.

- `VZ_LLM_GIGACHAT_VERIFY`: `bundled` (умолчание) — сертификат из поставки; путь к PEM — свой; `true` — системное хранилище; `false` — без проверки, **только для разработки**, в боевой установке не использовать.

Модели, доступные личному scope `GIGACHAT_API_PERS` (список `GET /api/v1/models`, 09.10.2026): `GigaChat`, `GigaChat-2`, `GigaChat-2-Pro`, `GigaChat-2-Max`, `GigaChat-Pro`, `GigaChat-Max`, `GigaChat-Plus`, а также `-preview`-версии; эмбеддинги нам не нужны. Выбор модели — `VZ_LLM_GIGACHAT_MODEL`; по результатам §9 оставлена `GigaChat`.

4. Офлайн-использование: «LLM пишет генератор, а не задание»

Авторы банка заданий могут использовать LLM как помощника при написании **кода семейства** (шаблон текста, генератор параметров, эталонный решатель). Это инструмент разработки, не часть продукта и не вызывается рантаймом. Правила:

- Ключи ответов считает **исполнение** решателя, а не модель; черновик проходит автоматические гейты банка (`app/testing/bank/base.py`, тесты `tests/test_bank_*`: идемпотентность клонов, проверка ключей исполнением, отсутствие утечек) и ревью человека.
- Затем pretest с весом 0 и калибровка по ответам людей; без измеренной сложности семейство в оценку не входит.
- Автоматического конвейера «фабрика семейств» в репозитории **нет** (в TESTING_MECHANISM.md §13 он числится в плане): черновики генераторов писались с помощью ассистента и прошли ту же проверку исполнением и ревью. Никаких ПДн в этом процессе нет.

5. Политика ПДн

Что **не уходит** в модель никогда: контакты и ПДн кандидата из профиля, результаты тестов, ответы на задания, Score, данные ФСП, содержимое подборок и приглашений. Эти данные в шлюз не передаются: он вызывается только с текстом брифа (задача `brief_skills`) и текстом резюме (задача `resume`).

Что уходит и как защищено:

Текст	Что в нём	Меры
Бриф работодателя	Описание вакансии: стек, вилка, формат. ПДн кандидатов нет; возможны имена контактных лиц компании	Маскирование тем же модулем; только задача `brief_skills`
Резюме кандидата	Может содержать ПДн	**Маскирование до вызова** (`app/llm/masking.py`): email, телефоны, ссылки на профили и Telegram, строки с датой рождения/возрастом, ФИО, найденные правилами, заменяются на `*` с сохранением длины (смещения цитат совпадают с исходным текстом); `extra_pii_spans` добавляет найденные ФИО. Текст обрезается до `VZ_LLM_MAX_INPUT_CHARS` (12 000)

Остаточный риск, который берёт на себя команда: маскирование основано на правилах, поэтому ФИО, не распознанные правилами, и косвенные идентификаторы (названия мест работы, должности с датами) в замаскированном тексте могут остаться. Поэтому:

1. **По умолчанию LLM выключена**; в демо-стенде и у жюри текст резюме и брифа никуда не уходит.
2. **Российские провайдеры (YandexGPT, GigaChat) — для боевой установки с резюме**: данные не покидают РФ (ст. 18 ч. 5 152-ФЗ).
3. **Зарубежный/внешний OpenAI-совместимый адрес по умолчанию получает только задачу `brief_skills`** (`VZ_LLM_OPENAI_TASKS=brief_skills`). Задача `resume` уйдёт туда только если оператор явно добавит её в `VZ_LLM_OPENAI_TASKS` ; тогда он обязан оценить трансграничную передачу (ст. 12) и получить согласие кандидата. Это защищено тестом (`test_openai_compat_provider_and_task_restriction`).
4. **Известное ограничение:** отдельной цели «обработка резюме языковой моделью» в журнале согласий (`ConsentPurpose`) пока нет. Поэтому для боевой установки с включённым разбором резюме моделью её нужно добавить (отдельная галочка с версией текста) или ограничиться уведомлением в согласии на обработку; в демо и у жюри функция выключена. Секреты ключей — `SecretStr` , в логи не пишутся.

6. Как включить

Всё через окружение (`.env.example` , раздел «LLM»). Без ключей и без изменений ничего не вызывается.

```
# Российский провайдер (боевая установка)
VZ_LLM_PROVIDER=yandexgpt
VZ_LLM_YANDEX_API_KEY=...        # ключ сервисного аккаунта
VZ_LLM_YANDEX_FOLDER_ID=...

# Открытая модель через OpenAI-совместимый хостинг (брифы; резюме не уходит)
VZ_LLM_PROVIDER=openai_compat
VZ_LLM_OPENAI_BASE_URL=https://openrouter.ai/api/v1
VZ_LLM_OPENAI_API_KEY=...
VZ_LLM_OPENAI_MODEL=qwen/qwen3-235b-a22b-2507

# Цепочка: YandexGPT -> GigaChat -> OpenAI-совместимый -> без LLM
VZ_LLM_PROVIDER=auto

# Демо с «включённой» LLM без сети и ключей (кассета записывается один раз через VZ_LLM_MODE=record)
VZ_LLM_MODE=replay
VZ_LLM_CASSETTE_PATH=/app/data/llm-cassette.json
```

Признак для проверяющего: в ответе разбора брифа `llm_used` и поле `source` чипов («YandexGPT» / «GigaChat» / `openai_compat`); при выключенной модели `llm_used=false` , все чипы «правило» или «вручную».

7. Стоимость и ресурсы

Нагрузка на модель мала: один вызов на разбор брифа или резюме, кэш по `sha256` убирает повторы, правила отрабатывают первыми. Порядок величин для оценки бюджета (тарифы провайдеров меняются, актуальные проверять перед включением):

Операция	Токены (вход / выход)	Комментарий
Бриф	≈ 0,5–1 тыс. / ≈ 0,2 тыс.	Один вызов на версию текста
Резюме	≈ 1–3 тыс. / ≈ 0,3 тыс.	Один вызов на загрузку, вход ограничен 12 000 символов

Для десятков разборов в день, как у демо и пилота, это единицы рублей в сутки у российских провайдеров и копейки у открытых моделей на хостингах. Лимиты запросов к API кандидата и работодателя ограничивают число вызовов сверху. GPU, собственного инференса и дополнительных сервисов не требуется; без ключей затрат нет.

8. Как проверить

```
cd backend
uv run pytest -q tests/test_llm_boundaries.py tests/test_resume_llm.py
tests/test_api_needs_review.py::test_ranking_path_never_imports_llm
```

Тесты проверяют: умолчание «выключено» (ни провайдеров, ни обращений), 0 обращений в сессии теста и ранжировании, ограничение задач зарубежного провайдера и маскирование ПДн в запросе, деградацию в «без LLM» при ошибке провайдера, цитатную проверку и кассету.

9. Результаты на отложенной выборке брифов (GigaChat, 09.10.2026)

Выборка: 62 синтетических брифа `backend/eval/brief_heldout.jsonl` (ПДн нет). Запуск: `uv run python -m eval.llm_brief_eval --mode record|replay|live`. «Правила» — `parse_brief`; «+LLM» — те же правила плюс `llm_brief.augment` (LLM добавляет «желательные» навыки с цитатой). Модель основного прогона и кассеты — `GigaChat` (scope `GIGACHAT_API_PERS`), температура 0,01, `VZ_LLM_MIN_QUOTE_RATIO=90`.

Метрика	Только правила	Правила + LLM
F1 навыков (все)	0,975 (P 0,976 / R 0,973)	0,975 (P 0,976 / R 0,973)
F1 must	0,842 (P 0,984 / R 0,736)	0,842 (без изменений)
Специализация top-1	0,984	0,984
Грейд	1,000	1,000
Вилка (точно) / основание	0,952 / 0,972	0,952 / 0,972
Формат / город	1,000 / 0,984	1,000 / 0,984

Честный вывод: на этой выборке LLM ничего не улучшает. Правила уже находят 97 % навыков; из 348 полей, предложенных моделью, 321 уже найдено правилами, 7 отсутствуют в справочнике навыков, 20 отброшено охранником цитаты, ни одного нового навыка не добавлено. Из 10 навыков эталона, которые пропустили правила, модель не предложила ни одного. Сама по себе модель (без правил) даёт F1 0,908 (P 0,987 / R 0,840): надёжна по точности, но слабее словаря по полноте. LLM-путь остаётся страховкой на лексику вне словаря, а не источником качества, и по умолчанию выключен.

Другие модели того же scope (живые прогоны без кассеты, те же 62 брифа):

Модель	F1 навыков с LLM	Добавлено навыков (верных)	F1 одной модели	Задержка mean / p95, с	Токены вход / выход
GigaChat (запись кассеты)	0,975	0	0,908	1,0 / н.д.	5 082 / 8 288
GigaChat (повторный живой прогон)	0,975	0	0,914	1,1 / 1,8	5 116 / 8 377
GigaChat-2	0,975	0	0,912	1,0 / 1,7	5 151 / 8 422
GigaChat-Pro	0,973	5 (2)	0,928	2,7 / 4,3	4 825 / 8 081
GigaChat-2-Pro	0,973	5 (2)	0,930	2,6 / 3,8	4 974 / 8 065
GigaChat-2-Max	0,975	4 (2)	0,924	3,2 / 5,5	9 603 / 8 333
GigaChat-Max	0,975	4 (2)	0,924	3,2 / 5,8	9 829 / 8 264

Повторный прогон `GigaChat` дал чуть другое поле предложений (F1 одной модели 0,914 против 0,908 при записи): даже при температуре 0,01 ответы не бит-в-бит, поэтому воспроизводимость обеспечивает кассета, а не повтор вызова. Старшие модели добавляют 4–5 навыков, из них верны 2: полнота растёт на 0,6 п.п., точность

падает на 0,5–0,8 п.п., F1 не растёт, а задержка в 3 раза выше. Поэтому берём самую быструю `GigaChat`. (Pro-модели один раз вернули невалидный JSON — это штатный fallback «без LLM» для этого брифа.)

Охранник цитаты (`VZ_LLM_MIN_QUOTE_RATIO`, кассета `GigaChat`, повтор без сети):

Порог partial_ratio	Отброшено полей из 348	F1 одной модели (P / R)
100 (дословно)	39	0,878 (0,987 / 0,791)
90 (умолчание)	20	0,908 (0,987 / 0,840)
70	8	0,922 (0,985 / 0,866)
50	3	0,928 (0,985 / 0,877)

Отброшенные поля — в основном перифразы цитаты («питон» вместо «python»), а не выдумки: точность модели почти не зависит от порога. Значит, на этой выборке порог 90 стоит полноты, а не защищает точность; оставляем 90 как защиту от галлюцинаций на более длинных и шумных текстах (там цитата — единственная проверка), решение о снижении — после измерения на реальных брифах пилота.

Надёжность и стоимость. Сбоев и таймаутов нет: 62 запроса из 62 получили ответ (доля fallback 0,000); токен OAuth запрашивается раз в ~25 минут. Задержка `GigaChat`: mean 1,0–1,1 с, p95 1,8 с, максимум 2,3 с (в бюджете 20 с). Ограничение личного scope: два одновременных прогона по 62 запроса дали 429 (`HTTPStatusError`) на 7 и 18 запросах — шлюз штатно ушёл в «без LLM» (результат правил), но для боевой нагрузки нужен платный scope или очередь запросов; последовательные прогоны шли без единого сбоя. Расход: 5 082 входных + 8 288 выходных токенов на 62 брифа, то есть ≈ 82 + 134 токена на бриф; весь прогон — около 13,4 тыс. токенов (тариф Сбера для личного scope проверять в кабинете; порядок — копейки на прогон).

Что уходит в модель. Только текст брифа работодателя (после маскирования `app/llm/masking.py`) и фиксированная инструкция. Резюме, контакты, результаты тестов, Score не отправляются: задача `resume` идёт к GigaChat только если оператор сам включит разбор резюме (по умолчанию выключен, §5). Брифы выборки синтетические; реальные брифы — тексты работодателя без ПДн кандидатов (152-ФЗ: оператор данных работодателя — он сам; возможные имена контактных лиц компании маскируются правилами `masking.py` лишь частично). Ключ авторизации хранится только в окружении/ `.env` (вне git), в логи и кассеты не попадает.

Воспроизведение без ключа и сети. Кассета ответов `backend/eval/llm_cassette_brief.json` (62 записи: ключ `sha256` и ответ модели; текст запроса не хранится) лежит в репозитории:

```
cd backend
uv run python -m eval.llm_brief_eval --mode replay            # метрики таблицы выше, детерминированно
# стенд: шлюз берёт ответы из кассеты, ключи не нужны
VZ_LLM_PROVIDER=gigachat VZ_LLM_MODE=replay VZ_LLM_CASSETTE_PATH=eval/llm_cassette_brief.json
# перезапись кассеты вживую (нужен VZ_LLM_GIGACHAT_AUTH_KEY)
uv run python -m eval.llm_brief_eval --mode record
```

Промах кассеты (новый текст брифа) — «без LLM»: результат правил. Тесты: `tests/test_llm_gigachat.py` (отпечаток сертификата, форма OAuth/чат-запросов, воспроизведение кассеты), границы — `tests/test_llm_boundaries.py` прогонялись и с живой конфигурацией (`VZ_LLM_PROVIDER=gigachat`, ключ в окружении): тест, оценка и ранжирование — 0 обращений к шлюзу и провайдерам.

Нагрузочный тест

Цель: подтвердить, что стенд на одном сервере (организаторы: ~8 CPU, 32–64 ГБ, без GPU) выдерживает десятки одновременных пользователей с приемлемым временем основных операций (жёсткого SLA нет), и найти узкие места.

Метод

- Инструмент: Locust 2.46 (MIT), сценарий `loadtest/locustfile.py`. Запуск против контейнерного стенда через `web` (nginx → backend), то есть через тот же путь, что у браузера.
- Смесь виртуальных пользователей (веса 5 : 2 : 3):
  - **кандидат**: вход → профиль → опрос (4 шага) → запуск теста → ответы на все задания адаптивной сессии с паузами «читает задание» → результат → квалификация; после теста — просмотр (профиль, приглашения, вакансии);
  - **работодатель**: вход → разбор брифа (`parse`) → создание брифа → категории и охват вилки → проверка навыков → сборка подборки → подборка → карточка кандидата → шорт-лист → приглашение; банк кандидатов и сводка, список брифов и воронка приглашений;
  - **гость**: главная и её ассеты, SPA-маршруты, `/api/health` , `/api/fsp/availability` , `/api/docs` .
- Паузы между действиями 1–3 с (кандидат), 2–6 с (работодатель), 1–5 с (гость). У каждого виртуального пользователя свой `X-Forwarded-For` (лимиты входа считаются по IP, а все запросы идут с одной машины): для этого на стенде `VZ_REAL_IP_FROM=172.16.0.0/12` .
- Сбой = 5xx, таймаут или неожиданный 4xx. 409/422/429 на приглашение (лимиты и «активная пара») и 409/422 на ответ в тесте считаются штатными.
- Аккаунты: `backend/scripts/loadtest_accounts.py` создаёт подтверждённых кандидатов `lt0001@loadtest.example` и HR-аккаунты `lthr001@…` при демо-компаниях (лимит запросов подбора считается на аккаунт, поэтому у виртуальных HR свои логины). Для каждого прогона берётся свежая партия кандидатов (`--offset`), потому что повторный тест на том же аккаунте ограничен.
- Длительность прогона 3 минуты, разгон 5–10 пользователей в секунду.

Как повторить

```
docker compose up -d --build && make demo                  # стенд и демо-мир (см. DEPLOY.md)
# для нагрузки с хоста nginx должен верить X-Forwarded-For от хоста (только для теста):
VZ_REAL_IP_FROM=172.16.0.0/12 docker compose up -d web
make loadtest-accounts                                     # аккаунты → loadtest/accounts.json (в
.gitignore)
make loadtest USERS=50 RUNTIME=3m                          # отчёт: loadtest/out/u50.html и u50_stats.csv
docker compose exec -T backend python scripts/loadtest_accounts.py --candidates 300 --offset 1000 --out
var/loadtest_accounts.json   # свежая партия
```

`make loadtest` запускает Locust через `uvx` (нужен uv); без него — `pip install -r loadtest/requirements.txt` и `locust -f loadtest/locustfile.py --headless -u 50 -r 5 -t 3m --host http://localhost` .

Окружение замеров

Windows 11, Docker Desktop (WSL2), 20 логических ядер, 11 ГБ памяти для Docker; Locust на том же хосте (он тоже потребляет CPU, поэтому цифры слегка занижены относительно отдельной нагрузочной машины). Отдельный

проект compose, чистая БД с демо-миром (300 синтетических кандидатов, пул 18 200 экземпляров). Конфигурация стенда: 4 процесса uvicorn (`VZ_WORKERS=4`), пул БД 8+4 на процесс, PostgreSQL `max_connections=200` , `shared_buffers=256MB` .

Результаты (итоговая конфигурация, 4 процесса)

Пользователей	Запросов	Сбоев	RPS (в среднем)	p50	p95	p99
50	13 075	0	≈ 73	26 мс	180 мс	400 мс
100	22 263	0	≈ 125	74 мс	480 мс	1,1 с

p50 / p95 / p99, мс, по основным операциям:

Операция	50: p50	50: p95	50: p99	100: p50	100: p95	100: p99
`GET /` (index.html)	3	9	13	3	14	34
`POST /api/auth/login` (Argon2)	180	320	430	390	930	1100
`GET /api/auth/me`	14	42	67	46	170	360
`PUT /api/survey/steps/stack`	66	120	230	78	220	260
`POST /api/tests/sessions` (старт теста)	78	150	240	110	330	590
`GET /api/tests/sessions/{id}/next`	70	290	370	140	430	870
`POST /api/tests/sessions/{id}/answer`	48	140	270	86	320	510
`GET /api/tests/sessions/{id}/result`	37	130	140	63	170	260
`GET /api/invitations/inbox` (кандидат)	18	50	88	60	220	410
`POST /api/needs/parse`	26	82	100	72	240	440
`POST /api/needs/{id}/versions` (сборка подборки)	220	440	510	470	1100	1400
`GET /api/needs/{id}/selection`	63	220	330	210	700	1100
`POST /api/invitations`	140	380	640	430	1200	1500
`GET /api/needs`	53	260	320	110	380	470
`GET /api/bank`	160	390	470	430	1100	1700
`GET /api/invitations` (воронка работодателя), **до исправления**	590	1400	1700	1900	4900	6500
`GET /api/invitations` (воронка работодателя), **после исправления**	не измерялось	—	—	65	150	200

Ресурсы (`docker stats` , 4-секундная выборка за прогон):

Сервис	50 польз.: CPU ср. / макс.	50: память макс.	100 польз.: CPU ср. / макс.	100: память макс.
backend (4 процесса)	141 % / 354 %	1,0 ГБ	353 % / 859 %	1,4 ГБ
db	38 % / 77 %	305 МБ	78 % / 137 %	378 МБ
web (nginx)	6 % / 14 %	17 МБ	14 % / 49 %	20 МБ
fsp-mock	3 % / 36 %	47 МБ	8 % / 84 %	37 МБ

100 % = одно ядро. На 8-ядерном сервере 100 пользователей занимают около 4,5 ядер из 8.

Что нашли и исправили

Прогон	Наблюдение	Причина	Исправление
1 процесс, 50 польз.	177 ответов 429 на вход	все запросы шли с одним IP: nginx отдавал бэкенду только адрес прокси	`X-Forwarded-For` теперь идёт от настоящего клиента (`real_ip` в nginx, доверие по `VZ_REAL_IP_FROM`); в тесте у каждого пользователя свой адрес
1 процесс, 50 польз.	8 ответов 500 на `GET/POST /api/invitations`	**deadlock** в PostgreSQL: `refresh_rows` блокировал ожидающие приглашения `FOR UPDATE` в произвольном порядке, а чтения списков идут параллельно (и с фоновым sweep)	строки блокируются в порядке id; блокировка берётся только если срок/напоминание действительно наступили (пробный прогон без блокировки) — `app/services/contact/store.py`
1 процесс, 50 польз.	429 `rate_limited` на `/api/needs/*`	14 виртуальных HR на 6 аккаунтах, лимит 30 запросов/мин на аккаунт	не дефект: у тестовых HR свои аккаунты (40)
1 процесс, 100 польз.	p50 ≈ 1 с даже у `/api/health` , 10 тыс. запросов за 3 мин	один процесс Python упирается в одно ядро (GIL), CPU 120–290 % не помогает	`VZ_WORKERS=4` (uvicorn), пул БД 8+4 на процесс, `max_connections=200` ; часы демо синхронизируются между процессами через БД
4 процесса	`GET /api/needs` 440 мс на 90 брифов	N+1: два запроса на каждый бриф (183 запроса)	один запрос брифов + один запрос версий + один группировки закреплённых — `app/services/matching/needs.py` , тест `tests/test_list_needs_batched.py`

Сводка по этапам (100 пользователей): 1 процесс — p50 1000 мс, p95 2100 мс, p99 3200 мс, 10 тыс. запросов; 4 процесса — p50 74 мс, p95 480 мс, p99 1100 мс, 22 тыс. запросов (рост пропускной способности вдвое при вдвое меньших паузах ожидания клиентов).

Воронка приглашений: пакетная загрузка и постраничность

Что	Было	Стало
Запросов на `GET /api/invitations`	≈ 5 на приглашение (условия, события, раскрытие, проверка исхода, псевдоним) + чтение всего списка	константа на страницу: 19 запросов при 2 и при 6 приглашениях (тест), из них ≈ 12 — общие (аутентификация, квота, компания)
Размер ответа	все приглашения компании	страница `limit` (по умолчанию 50, максимум 200) с курсором `next_cursor` по ключу `(created_at, id)`
Счётчики колонок, воронка, причины отказов, средняя позиция, мини-воронки по брифам	считались по загруженным строкам	агрегатом SQL по **всем** приглашениям (один запрос), не зависят от страницы
Входящие кандидата (`/inbox`), отклики (`/applications/mine` , список работодателя)	≈ 3–5 запросов на строку, компания и R читались на каждую строку	условия, события, раскрытия, вакансии, псевдонимы, компании и события надёжности грузятся пакетом; `/inbox` принимает необязательные `limit` и `cursor` , `new_count` — агрегат
Применение сроков при чтении	пробный `to_domain` на каждую ожидающую строку	то же, но по пакету из 4 запросов на список

Где: `app/services/contact/store.py` (`prefetched` , `refresh_rows`), `invitations.py` (`employer_page` , `employer_cards` , `kanban` , `candidate_cards`), `applications.py` , `identity.aliases` , `reliability.events_by_company` . Тест: `tests/test_list_invitations_batched.py` (число запросов не растёт с длиной списка для воронки, входящих, откликов; курсор не теряет и не дублирует строки; счётчики по всем строкам). Фронтенд: `useKanban` — бесконечный запрос (кнопка «Показать ещё» на странице «Приглашения»), заголовки колонок берут `count` из агрегата.

Перепрогон после исправления (2026-10-08, 100 пользователей, 3 минуты, отдельный проект compose `vzlt` на портах 8100/8180/5543, свежие образы, демо-мир 315 кандидатов, пул ≈ 12 900 экземпляров, 4 процесса uvicorn; Locust на том же хосте): **24 739 запросов, 0 сбоев, ≈ 140 RPS, p50 31 мс, p95 170 мс, p99 300 мс** в целом. Воронка работодателя `GET /api/invitations` (197 запросов): **p50 65 мс, p95 150 мс, p99 200 мс** против 1900 / 4900 / 6500 мс до исправления (≈ 30-кратное снижение медианы). Остальное в том же диапазоне, что в таблице выше: `POST /api/auth/login` 160 / 260 / 630, `GET /api/invitations/inbox` 25 / 86 / 130, `GET /api/needs` 30 / 91 / 120, `GET /api/bank` 140 / 330 / 400, `POST /api/needs/{id}/versions` 220 / 400 / 460, `POST /api/invitations` 140 / 320 / 410 (p50 / p95 / p99, мс). Прогон при 50 пользователях после исправления не повторялся; замеры ресурсов (`docker stats`) в этом прогоне не снимались. Отчёт: `loadtest/out/u100.html`.

Выводы

- 50 одновременных пользователей: 0 сбоев, медиана 26 мс, p99 400 мс, включая воронку приглашений после исправления. 100 одновременных: 0 сбоев, медиана 74 мс, p95 0,5 с, p99 1,1 с. Запас по CPU на сервере класса 8 CPU примерно двукратный.
- Самые тяжёлые операции: сборка подборки (`versions` , 220–470 мс медиана — Score по всему пулу, его и ожидаем), вход (Argon2, 180–390 мс — сознательная цена защиты паролей), банк и воронка приглашений.
- **Было узким местом: `GET /api/invitations` (воронка работодателя)** — список не постраничный, ≈ 5 запросов на каждое приглашение; у компании с сотнями приглашений 100 пользователей давали p50 1,9 с, p99 6,5 с. **Исправлено и перемерено**: p50 65 мс, p99 200 мс при 100 пользователях (см. раздел «Воронка приглашений»).
- Лимиты запросов (`RateLimiter`) и буфер `/api/demo/webhook-sink` хранятся в памяти процесса: при `VZ_WORKERS=4` эффективные лимиты до 4 раз мягче. Часы демо (`/api/demo/clock`) синхронизируются через БД раз в секунду.
- Фоновый планировщик защищён рекомендательной блокировкой PostgreSQL, несколько процессов не дублируют задачи.
- Если контейнер упал, `restart: unless-stopped` поднимет его; при падении одного из 4 процессов uvicorn остальные продолжают работать.

Что не проверено

- Нагрузка с отдельной машины и по сети; сервер класса 8 CPU / 32+ ГБ (замеры на ноутбуке с 20 логическими ядрами и 11 ГБ).
- Более 100 одновременных пользователей и длительные (часы) прогоны; рост БД после тысяч брифов и приглашений.
- Боевая схема с Caddy и HTTPS (`docker-compose.prod.yml`): собран и проверен только конфиг; цепочка Caddy → nginx → backend для клиентского IP (`VZ_REAL_IP_FROM=172.29.80.0/24`) не прогонялась.
- Синхронизация демо-часов между процессами проверена только чтением кода и тестами модуля, живая проверка кнопкой «+1 день» при 4 процессах не выполнена.

Цифровой паспорт навыков

Результат теста и достижения ФСП кандидат может получить в виде **подписанного файла**: его можно показать любому работодателю или положить в портфолио, а подлинность проверяется без входа на платформу. Решение 31.1 в журнале решений.

> **Издатель — демонстрационный.** Паспорт выдаёт платформа «Взаимно», а не ФСП. Это не официальный сертификат ФСП. Надпись «Демонстрационный издатель» есть в самом файле, в API и на странице проверки. Как ФСП может стать настоящим издателем — раздел 6.

1. Что используется (стандарты)

Слой	Стандарт	Как у нас
Модель данных	W3C Verifiable Credentials Data Model 2.0	контекст `https://www.w3.org/ns/credentials/v2`, `validFrom` / `validUntil`
Форма «значка»	1EdTech Open Badges 3.0	тип `OpenBadgeCredential`, `credentialSubject` типа `AchievementSubject`, `Achievement`, `Result` / `ResultDescription`, `Evidence`
Подпись	W3C Data Integrity, криптосьют `eddsa-jcs-2022`	Ed25519 (RFC 8032) поверх канонизации JCS (RFC 8785); `proofValue` — multibase base58btc
Канонизация	JSON Canonicalization Scheme, RFC 8785	собственная реализация `app/credentials/jcs.py`, проверена по контрольным примерам RFC (числа по ECMAScript, порядок ключей по UTF-16)
Идентификатор издателя	`did:web`	`did:web:<домен платформы>`, ключ — в `/.well-known/did.json` (`Multikey`, `publicKeyMultibase` вида `z6Mk…`)
Отзыв	W3C Bitstring Status List	список `/api/credentials/status/1`, `statusPurpose: revocation`, сжатие GZIP + base64url, 131 072 бита

Новых зависимостей нет: `cryptography` уже в проекте, JCS и base58 написаны и покрыты тестами (около 150 строк).

2. Какие паспорта бывают

1. **Категория** («Backend (Python) · Middle»): специализация и грейд, балл на шкале платформы, стандартная погрешность измерения (SEM, баллы), дата подтверждения тестом, ссылка на методику (`evidence`). Выпускается только для категории, подтверждённой тестом и видимой в подборке (не на проверке у методиста).
2. **Достижение ФСП**: обобщённая запись из реестра (демонстрационные данные мока). Только поля из `config/credentials.yaml → fsp.allowed_fields`: дисциплина, уровень, год, итог, полоса места («топ-3»), роль по протоколу (личный зачёт, капитан, в команде; функция в команде из заявки участника в паспорт не попадает — в протоколах ФСП её нет). Вид спорта указан по Всероссийскому реестру видов спорта: «спортивное программирование», код **1010001311Я** (`fieldOfStudy` и `tag: ["ВРВС:1010001311Я"]` достижения Open Badges 3.0; значение — из `config/fsp.yaml → sport`). Номер ЕКП, команда, точное место, ссылка на протокол, ФИО и ФСП ID **не попадают** в файл. Выпускается только при привязанном ФСП ID и согласии на обогащение; скрытые кандидатом результаты недоступны.

Резюме и самоописание в паспорт не попадают и на категорию не влияют.

3. Персональные данные

- Субъект по умолчанию — **псевдонимный идентификатор** `urn:vzaimno:subject:<24 hex>` = HMAC(секрет платформы, id пользователя). Без секрета он не связывается с человеком, при смене секрета связь рвётся.
- Имя добавляется **только если кандидат отметил это при выпуске** (и только если ФИО заполнено в профиле).
- Страница проверки по коду из QR имя не показывает никогда. При проверке загруженного файла имя показывается, если оно в файле: файл уже у проверяющего.
- В реестре выдачи (таблица `issued_credentials`) хранится подписанный документ; при удалении аккаунта записи удаляются каскадом, и такой файл перестаёт проходить проверку («в реестре выдачи записи нет»).
- Работодатель после раскрытия контактов видит на карточке только «Паспорт проверен ✓», названия действующих паспортов и издателя. Баллы и имя из паспорта он не получает: они в файле, которым кандидат делится сам.

4. Что делает кандидат

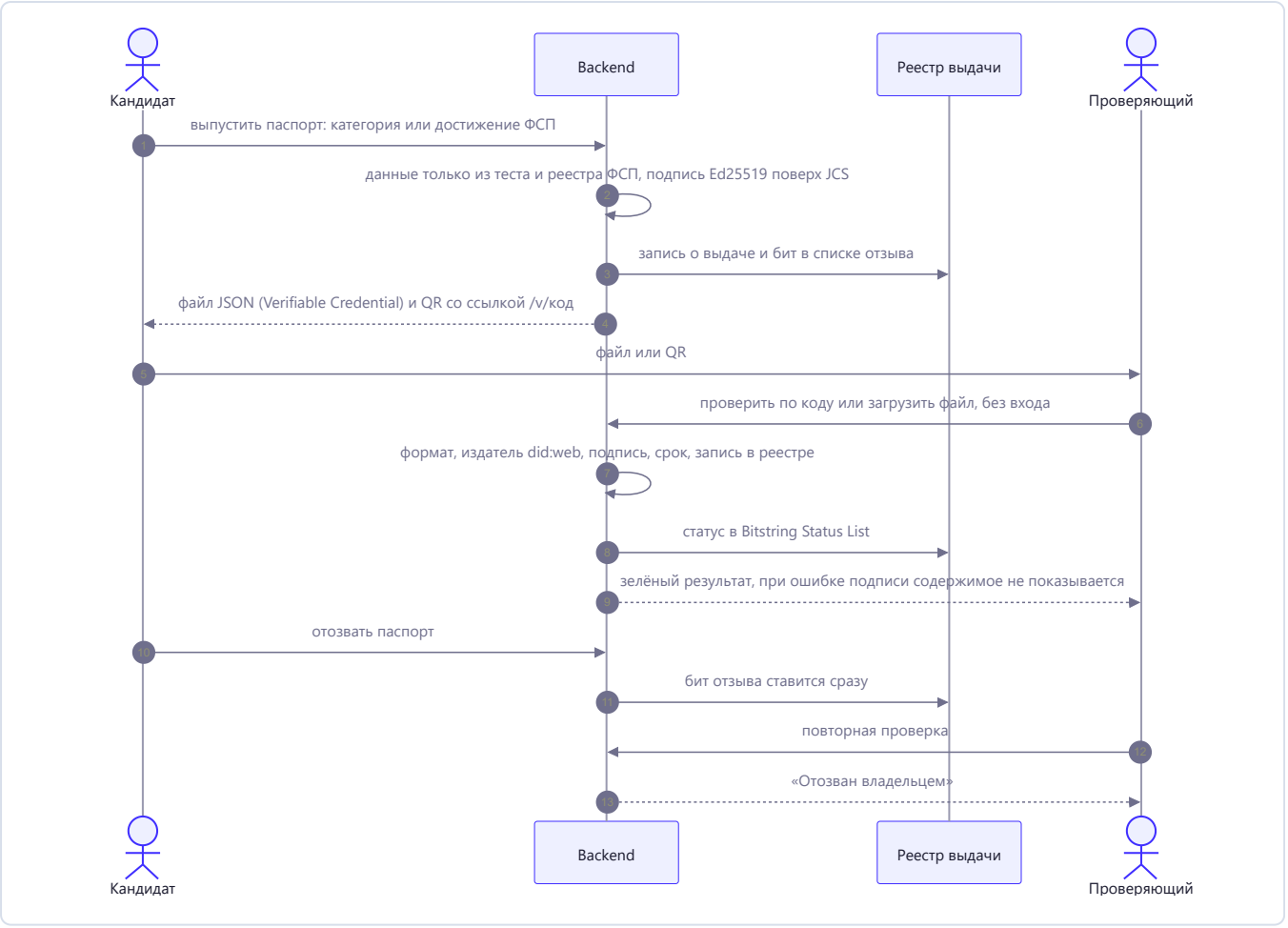
Профиль → «Паспорт навыков»:

1. Выпустить паспорт по категории или по достижению ФСП (с именем или без).
2. Скачать JSON; рядом — QR со ссылкой `/v/<код>`.
3. Отозвать: бит в списке отзыва ставится сразу, любая проверка покажет «отозван».

Срок действия — 365 дней (`config/credentials.yaml → validity_days`): категория актуальна ограниченно, как и в PDF-профиле.

5. Проверка

Выпуск, проверка и отзыв целиком:



Страница `/v/<код>` (QR) и блок «Проверить файл» на той же странице; публично, без входа, с ограничением частоты запросов. Проверяются по отдельности:

Проверка	Красный результат
Формат (Verifiable Credential)	«Это не файл паспорта навыков»
Издатель (`did:web` платформы и её ключ)	«Издатель неизвестен платформе»
Подпись Ed25519	«Подпись не совпадает: содержимое файла изменено после выдачи»
Срок действия	«Срок действия истёк»
Запись в реестре выдачи	«В реестре выдачи записи нет»
Отзыв	«Отозван владельцем: паспорт больше не действует»

Если подпись не прошла, содержимое файла на странице не показывается. Демонстрация: скачать паспорт, изменить в файле один байт (например, `542` на `543`), загрузить — красный крест. Тесты: `backend/tests/test_credentials_core.py` (JCS, base58, подпись, подделка, список отзыва, `did.json`, минимизация ПДн) и `test_credentials_api.py` (выпуск, скачивание, проверка, подделка по байту, срок, отзыв, удалённая запись, имя по выбору, доступ, карточка работодателя).

6. Что настоящее, а что демонстрация

	Сейчас	Для настоящего издателя
Формат и подпись	настоящие: любой валидатор VC/Data Integrity проверит подпись	то же

	Сейчас	Для настоящего издателя
Издатель	`did:web` платформы, ключ создан у нас; в файле явно «демонстрационный издатель»	издатель `did:web:<домен ФСП>`, ключ в HSM/KMS ФСП
Данные ФСП	демо-мок реестра	настоящий реестр по контракту из FSP_INTEGRATION.md
Что заверено	платформа заверяет, что результат теста получен на ней, и факт выписки из реестра	ФСП заверяет само достижение
Реестр выдачи и отзыв	у платформы	у ФСП: тот же Bitstring Status List по адресу ФСП

Как ФСП станет настоящим издателем (дорожная карта ФСП ID).

1. ФСП публикует `did.json` на своём домене и выдаёт платформе делегированный ключ (или подписывает паспорта у себя по запросу платформы).
2. В `issuer` паспорта ставится идентификатор ФСП; поле `issuer.name` — реальное название. Отметка «демо» убирается конфигом, формат файла не меняется.
3. Достижения ФСП подписывает сам ФСП при внесении результата в реестр; платформа подписывает только категорию теста. Работодатель проверяет оба паспорта одним и тем же инструментом.
4. Привязка к человеку через `sub` ФСП ID вместо псевдонима платформы (по согласию). Остальное — как для всех паспортов.

7. Настройка и ключ

- `VZ_CREDENTIALS_KEY_PEM` — PEM (PKCS#8) приватного ключа Ed25519 (переводы строк как `\n` допустимы) **или** `VZ_CREDENTIALS_KEY_FILE` — путь к файлу. Ключ в git не хранится.
- dev: если ничего не задано, ключ создаётся при первом запуске в `backend/var/credentials-ed25519.pem` (каталог вне git). Тесты используют эфемерный ключ.
- prod без ключа: выдача и проверка отвечают 503 «не настроен ключ подписи», запасной ключ не подставляется.
- `VZ_CREDENTIALS_DID_DOMAIN` — домен для `did:web` (по умолчанию хост из `VZ_PUBLIC_BASE_URL`; порт кодируется как `%3A`). Адрес `/.well-known/did.json` должен отдаваться на этом домене: в `deploy/Caddyfile` и прокси Vite он направлен на бэкенд.
- Смена ключа делает ранее выпущенные паспорта непроверяемыми: перед ротацией — перевыпуск или публикация старого ключа в `did.json` вторым `verificationMethod`.

8. Ограничения

- Проверка в реестре выдачи обязательна: файл, выпущенный другим стендом, будет красным, даже если подпись верна для того же ключа.
- Отзыв проверяется только онлайн (список отзыва у платформы); офлайн-проверка подписи без обращения к платформе возможна по `did.json`, но статус отзыва без сети неизвестен.
- Данные ФСП демонстрационные.

Инженерные практики

Что именно проверяется автоматически и как это запустить. Всё ниже — факты репозитория на дату сдачи, без обещаний.

Что гоняет CI

Файл `.github/workflows/ci.yml`. Каждый PR и каждый пуш в `main` запускают семь независимых заданий:

Задание	Что делает
`backend-lint`	`ruff check` (правила E, F, I, B, UP); `mypy --strict` на ядрах (список в `pyproject.toml`, раздел «Типы»); `mypy` по всему `app` для информации (долг считаем числом)
`backend-tests`	`pytest` на PostgreSQL 16 + pgvector 0.8.7 (service container), три шарда: `api`, `engine` (антиподсказчик, банк, CAT, QA, таксономия), `rest`. Node 22 нужен JS-семействам банка заданий. Вывод `--durations=15` показывает самые долгие тесты
`migrations`	`alembic upgrade head`, затем `downgrade base`, снова `upgrade head`, затем `alembic check`: модели и миграции совпадают, неописанных изменений схемы нет
`frontend`	`npm ci`, `tsc -b --noEmit`, `eslint`, `vite build`, `npm audit --omit=dev --audit-level=high`
`e2e`	поднимает БД, пул заданий, демо-мир, бэкенд (DEMO_MODE) и Vite, гонит Playwright, выкладывает отчёт и логи артефактом
`docker`	сборка образов `backend` (он же `pool`), `web` (nginx + интерфейс), `fsp-mock`; кэш GitHub Actions
`dependency-audit`	`pip-audit` по прод-зависимостям из `uv.lock`

Файл проверен `actionlint` (замечаний нет). Установленные версии зависимостей фиксируют `uv.lock` и `package-lock.json`; CI ставит их строго по локам (`UV_FROZEN`, `npm ci`).

Тесты

`cd backend && uv run pytest -q` — 2297 тестов (в последнем полном прогоне 2276 прошли, 21 пропущен, 0 упало). Полный прогон занимает около 22 минут на ноутбуке разработчика, поэтому CI делит набор на три шарда. Распределение по областям (по префиксу файлов):

Область	Тестов	Что покрыто
Банк заданий (`test_bank_*`)	698	контракт семейств, генераторы, эталонные решения, гейты качества
HTTP API (`test_api_*`)	294	воронка кандидата и работодателя, приглашения, чат, ATS, демо-контур
Подбор (`test_matching_*`)	248	разбор брифа, Score, объяснения, версии подборки, покрытие
Антиподсказчик (`test_anticheat_*`)	240	двойники, улики, песочница жюри, пороги ложных срабатываний
Контакты (`test_contact_*`)	199	жизненный цикл приглашения, таймеры, квоты, приватность
ФСП (`test_fsp_*`)	109	модель достижений, VSS, адаптер, мок реестра
Платформа (`test_platform_*`)	109	аутентификация, пароли, безопасность, схема OpenAPI, модели
IRT/CAT (`test_cat_*`)	99	движок, калибровка, детекторы, экспресс-режим, сессии
Оценка на синтетике (`test_eval_*`)	52	популяция с латентной истиной, метрики, спецификация валидации
Матрица доступа (`test_security_matrix.py`)	14	см. раздел «Безопасность»

Остальное — тестовые файлы по резюме, PDF, планировщику, LLM-границам, массовым запросам и т. п. Контрольные примеры из спецификаций оформлены обязательными тестами. Каждый тест с БД работает в откатываемой транзакции в отдельной базе на запуск (параллельные запуски не мешают друг другу), а время в тестах задаёт замороженные часы `frozen_clock`.

Сквозные тесты (Playwright)

Каталог `frontend/e2e` — отдельный пакет, чтобы не добавлять зависимости в основной `package.json` и не попадать в образ интерфейса. `@playwright/test` распространяется по лицензии Apache-2.0. Пять сценариев идут по порядку номеров, потому что второй и четвёртый опираются на данные предыдущих:

1. `01-jury-express`: `/jury`, экспресс-тест, ответы на задания, экран результата.
2. `02-candidate-journey`: регистрация (в DEMO_MODE аккаунт активен без письма), опрос из четырёх шагов, выбор грейда, старт теста.
3. `03-employer-flow`: вход работодателя, бриф с вилкой, подборка, панель доказательств (ровно три «за» и один «против»), отправка приглашения.
4. `04-mutual-chat`: кандидат принимает приглашение, даёт согласие на раскрытие контактов, видит «Взаимно» и контакт компании, пишет в чат.
5. `05-sandbox-twin`: `/try`, копирование условия подменяется двойником (в буфере другое значение, счётчик «копировали»), итог сверки.

Запуск: `make e2e` (нужна БД с демо-миром: `make up-infra migrate gen-pool seed`). Скрипт `frontend/e2e/run.mjs` сам поднимает бэкенд на :8128 и Vite на :5185 и гасит их после прогона. Перед прогоном `global-setup.ts` сбрасывает демо-мир через `POST /api/demo/reset`, поэтому тесты повторяемы. Логины демо-аккаунтов берутся из `backend/var/demo_accounts.md` (в git не попадает). Весь набор также проверен на production-сборке `vite build` с политикой CSP из `frontend/csp.conf`: нарушений политики нет.

Ограничение: лимит попыток входа (5 за 5 минут на email) учитывается и в e2e. На одном и том же бэкенде более пяти прогонов подряд за пять минут упрутся в него; перезапуск бэкенда сбрасывает счётчики.

Типы

`mypy` стоит в dev-группе (лицензия MIT).

- `uv run python scripts/typecheck_strict.py` (или `make typecheck`) запускает `mypy --strict` на 25 файлах ядер: IRT/CAT-движок и сессии (`app/testing`), скоринг, объяснения и подборка (`app/matching`), VSS и модели ФСП, ядро безопасности (`app/core`: пароли и токены, лимитер, доступ, аудит, часы, конфиги, согласия). Список лежит в `pyproject.toml`, раздел `[tool.vzaimno.typecheck_strict]`. Ошибок нет, в CI гейт блокирующий.
- `uv run mypy` проверяет весь `app` с умеренными настройками. Было 344 ошибки на 62 файлах, после правок осталось 258 (на момент сдачи 290: за это время в других ветках работ добавился код). Остаток — в банке заданий, демо-сервисах, разборе брифа и сервисных слоях; типовые причины: повторное использование имени переменной с другим типом, `Optional` без проверки, `Result.rowcount` у SQLAlchemy. В CI этот шаг информативный: число ошибок видно в логе, но сборку он не роняет.

Статический анализ и стиль

`ruff` (E, F, I, B, UP, длина строки 110) без замечаний. Фронтенд: `tsc -b --noEmit` и `eslint` (typescript-eslint, react-hooks, react-refresh) без замечаний.

Безопасность

Проверено на уровне OWASP Top-10, коротко:

- A01, контроль доступа. Точечные проверки чужих ресурсов есть в `test_api_chat`, `test_api_ats`, `test_api_needs_privacy`, `test_api_contact_*` и `test_api_testing_integrity`. Сквозная матрица `tests/test_security_matrix.py` добавлена в рамках аудита: (1) по схеме OpenAPI каждый защищённый маршрут без токена не отвечает 2xx и 5xx; (2) работодатель другой компании не читает и не меняет чужое приглашение и чужой чат; (3) чужой кандидат не отвечает на приглашение, не открывает чужую сессию теста, не шлёт события и ответы; (4) границы ролей (кандидат, работодатель, методист, администратор). Уязвимых мест не найдено.
- A02, криптография. Argon2id (время 3, память 64 МиБ, параллелизм 4); JWT с явным списком алгоритмов при проверке; короткий access (15 минут) и refresh-токен в `HttpOnly`-cookie с ротацией и защитой от повторного использования; в `VZ_ENV=prod` запрещены секреты по умолчанию и короткие секреты.
- A03, инъекции. Сырой SQL один в коде приложения с параметрами; единственная сборка условия из строки — `postgresql_where` индекса из констант-перечислений статусов. Параметризация везде, где значения приходят извне.
- A04/A05, конфигурация. Лимиты запросов на вход, регистрацию, подтверждение, повторную отправку и обновление токена (`VZ_RL_*`); CORS только из явного списка origin; заголовки `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy` в бэкенде и в nginx; HSTS в Caddy (прод). Добавлено в рамках аудита: `Content-Security-Policy` для статики интерфейса (`frontend/csp.conf`), политика проверена полным e2e-прогоном.
- A06, зависимости. `pip-audit` по прод-зависимостям бэкенда: уязвимостей нет. `npm audit --omit=dev`: уязвимостей нет. Полный `npm audit` показывает уязвимости только в инструментах сборки (`sharp`, `braces`, `postcss-selector-parser`); в образ интерфейса они не попадают.
- A07, аутентификация. Подтверждение email кодом с ограничением попыток и частоты отправки; ограничение попыток входа по email и по IP.
- A08/A09, целостность и журналирование. Вебхуки ATS подписаны HMAC-SHA256 и идемпотентны; журнал аудита ключевых действий.
- Секреты в репозитории. Поиск по шаблонам ключей и токенов в отслеживаемых файлах: реальных секретов нет. Демонстрационные значения лежат в `services/fsp-mock/fsp_mock/keycloak.py`, `services/fsp-mock/fsp_mock/main.py` и `deploy/keycloak/realm-fsp.json` и помечены как демо; `.env` и `backend/var/` в `.gitignore`.

Что сознательно не закрыто: `Strict-Transport-Security` отдаёт только Caddy (боевой стенд за ним), локальный nginx работает по HTTP; автоматический динамический скан (OWASP ZAP) не запускалась.

Детерминизм и конфигурация

- Случайность только через явный `random.Random(seed)` или `numpy.random.default_rng(seed)`; глобального `random` нет. Секретные seed экземпляров заданий получаются HMAC от `VZ_APP_SECRET`.
- Все веса, пороги, квоты и справочники лежат в `config/*.yaml` (14 файлов, около 3000 строк). В коде магических чисел нет; настройки инфраструктуры и безопасности — переменные `VZ_*` (`.env.example`).
- LLM не вызывается в рантайме теста и ранжирования; проверка границы — `tests/test_llm_boundaries.py`.
- Резюме и самоописание не влияют на категорию и Score, отсутствие истории ФСП не штрафуется, контакты видны работодателю только после раскрытия (разные DTO `CandidateBlind` и `CandidateDisclosed`): каждое правило закреплено тестами.

Миграции

Alembic, 13 ревизий (`backend/alembic/versions`). Цикл `upgrade, downgrade, upgrade` и отсутствие дрейфа между моделями и схемой проверяются в CI. В `alembic/env.py` добавлен импорт демо-моделей (`app.services.demo.models`): без него `alembic check` считал бы демо-таблицы лишними.

Доступность и адаптивность

Автоматическая проверка: axe-core (WCAG 2.1 A/AA, теги `wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa`) через `@axe-core/playwright`.

Лицензия инструмента

`@axe-core/playwright` и `axe-core` распространяются по MPL-2.0 (слабый копилефт на уровне файлов). Правило проекта для новых зависимостей — MIT/BSD/Apache. Мы подключили пакет только как `devDependency` отдельного пакета `frontend/e2e`: он не попадает в сборку интерфейса и не поставляется пользователям, исходники axe не изменяются. Если команда считает MPL недопустимой даже для dev-инструмента, достаточно удалить пакет из `frontend/e2e/package.json` и файл `tests/06-a11y.spec.ts` — на продукт это не влияет.

Что проверяется

Экран	Маршрут
Лендинг, вход, экспресс-вход жюри	`/`, `/auth/login`, `/jury`
Плеер теста	`/candidate/test/:id` (в spec — выбор грейда, если идущего теста нет; сам плеер — в `audit/audit.mjs`)
Результат теста	`/candidate/result/:id`
Кабинет работодателя	потребности, подборка, карта талантов, банк, приглашения, вакансии
Лаборатория	акт сверки `/lab/reconciliations/:id`

Запуск (стенд: бэкенд в DEMO_MODE и Vite; сброс демо-мира не выполняется):

```
cd frontend/e2e
npm ci
E2E_BASE_URL=http://localhost:5185 npx playwright test -c playwright.a11y.config.ts
node audit/audit.mjs after      # скриншоты 390/768/1440 + axe + проверка горизонтального скролла
node audit/drawer-shot.mjs after
```

Тест `tests/06-a11y.spec.ts` падает при любом нарушении уровня serious или critical. Скриншоты — `frontend/.screens/responsive/{before,after}/` (в .gitignore).

Результаты: было → стало (serious/critical, узлов)

Все нарушения до правок — `color-contrast`.

Экран	Было	Стало
Лендинг	3	0
Вход	1	0
Жюри	4	0
Плеер теста	2	0
Результат	не измерялось; позже найден `link-in-text-block` (1)	0
Потребности	15	0

Экран	Было	Стало
Подборка	7	0
Карта талантов	33	0
Банк	5	0
Приглашения	11	0
Вакансии	6	0
Акт сверки (лаборатория)	27	0
Итого	**114** `color-contrast` + 1 `link-in-text-block` = **115**	**0**

Итог посчитан по этой таблице. Если в других местах встречается другое число (в сообщении коммита — 130), верна таблица: она суммирует узлы по экранам, перечисленным здесь.

Что исправлено

- Токен `ink-3` (второстепенный серый текст) `#7A7B82` → `#686970`: контраст на белом 4,2 → 5,5, на панели `#F6F6F8` не ниже 4,9 (причина почти всех нарушений: подписи, навигация, подсказки).
- Тепловая шкала `heat-4` `#7A6CFF` → `#6A5CFF`: белая цифра в ячейке карты талантов 3,9 → 4,6.
- Номер места в таблице подборки: `steel` (1,7:1) → `#85868D` (3,6:1, крупный текст требует 3:1).
- Ссылка внутри абзаца на экране результата отличается от текста подчёркиванием, а не только цветом.

Остаётся

- В проекте есть ещё около 40 ссылок вида `text-brand ... hover:underline`. Axe флагует `link-in-text-block` только там, где ссылка стоит внутри абзаца и контраст с окружающим текстом ниже 3:1; на проверенных экранах такой случай один (исправлен). На остальных экранах стоит проверить.
- Проверены 13 экранов из 30+: не покрыты регистрация, профиль кандидата, опрос, вакансии кандидата, лаборатория кроме акта сверки.
- Автоматика ловит около трети проблем WCAG. Ручная проверка клавиатурой и скринридером выборочная: диалоги (`Drawer`) запирают фокус и закрываются по Esc, навигация имеет `aria-label` , кнопки иконок подписаны.
- Правила `color-contrast` для текста поверх изображений и градиентов axe не вычисляет (помечает как «incomplete»); такие места (hero-панель) проверяются глазами.

Адаптивность кабинета работодателя

Горизонтального скролла страницы нет на 390, 768 и 1440 px (измеряется в `audit.mjs` : `scrollWidth - innerWidth`). Навигация ниже `lg` — раскрывающееся меню; приглашения — колонки одна под другой; карта талантов — сетка с сужаемыми колонками; таблицы подборки и банка — строки-карточки.

Карточка кандидата (панель доказательств) на подборке и в банке: от `xl` (1280 px) — липкая панель справа, ниже — выдвижная панель `Drawer` поверх страницы (компонент `EvidenceSlot`). Раньше на телефоне и планшете карточка оказывалась под длинной таблицей, и страница автоматически прокручивалась вниз.